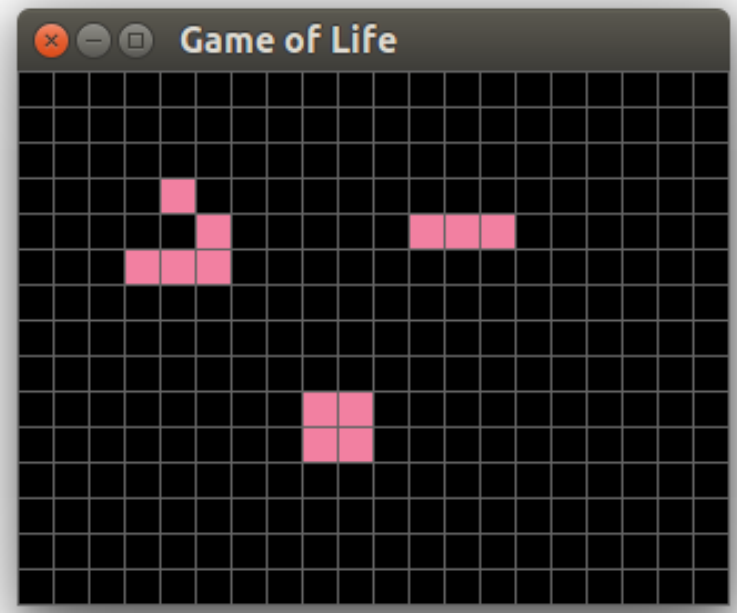
Introduction to programming with Scala

Study Guide 2

Second study period: Module 8 – 14



Björn Regnell, Hans Middelkoop (translation)

H-ICT-SE-2-25 2026
Object Oriented Software Development
The Hague University of Applied Science

Compilation date: March 8, 2026

Editor: Björn Regnell

Contributors in alphabetical order: Anders Buhl, André Philipsson Eriksson, Anna Axelsson, Anna Palmqvist Sjövall, Annie Predel, Anton Andersson, Benjamin Lindberg, Björn Regnell, Casper Schreiter, Cecilia Lindskog, Dag Hemberg, Elias Åradsson, Elliot Bräck, Elsa Cervetti Ogestad, Emelie Engström, Emil Wihlander, Erik Bjäreholt, Erik Grampp, Evelyn Beck, Felix Ohrgren, Fredrik Danebjer, Fritjof Bengtsson, Gustav Cedersjö, Henrik Olsson, Hjalmar Rutberg, Hussein Taher, Jakob Hök, Jakob Sinclair, Johan Ravnborg, Johannes Nydahl, Jonas Danebjer, Jos Rosenqvist, Lovisa Löfgren, Maj Stenmark, Maria Kulesh, Melker Widén, Måns Magnusson, Nicholas Boyd Isacsson, Niklas Sandén, Oliver Levay, Oliver Persson, Oscar Sigurdsson, Oskar Berg, Oskar Widmark, Patrik Persson, Per Holm, Philip Sadrian, Sandra Nilsson, Sebastian Hegardt, Simon Persson, Stefan Jonsson, Theodor Lundqvist, Tim Borglund, Tom Postema, Valthor Halldorsson, Viktor Claesson, Wilhelm Wanecek, William Karlsson.

Home: https://lunduniversity.github.io/pgk

Repo: https://github.com/lunduniversity/introprog

This compendium is on-going work.
Contributions are welcome!
Contact: bjorn.regnell@cs.lth.se

Versions:
Scala 3.7.2
JDK 21
introprog-scalalib 1.4.0

You can use this work if you respect this **LICENSE**: CC BY-SA 4.0
http://creativecommons.org/licenses/by-sa/4.0/
Do *not* distribute solutions to lab assignments and projects.

Copyright © 2015-2026.
Björn Regnell, Dept. of Computer Science, LTH, Lund University.

Progress Protocol

Completed Exercises

Each lab comes with an exercise containing tasks that serve as preparation for the lab. You need to master at least the basic tasks in order to complete the lab within a reasonable time. If you feel you need more practice on the basics, then also do the extra tasks. If you want to deepen your understanding, do the advanced tasks which are at a more advanced level. Check off below which exercises you have done, so it will be easier for your supervisor to tailor the discussion to the knowledge you have acquired so far.

Exercise	Basic	Extra	Advanced

Approved Mandatory Components

To pass the lab assignments and the project assignment you must solve the sub-tasks and discuss your solutions with a supervisor. This discussion is your opportunity to get feedback on your solutions. Take advantage of it! Make sure the supervisor notes below when you have been approved on the respective mandatory components. Keep this sheet until you have received your final grade in the course.

Name: .....

Signature: .....

Lab	compl+date, pass+date	Supv. signature + printed name
`kojo`	.....	.....
`irritext`	.....	.....
`blockmole`	.....	.....
`blockbattle0`	.....	.....
`blockbattle1`	.....	.....
`shuffle`	.....	.....
`life`	.....	.....
`words`	.....	.....
`snake0`	.....	.....
`snake1`	.....	.....
Project Assignment	.....	.....

☐ `bank` *If custom defined, provide brief description here:*

☐ `music`

☐ `photo`

☐ *custom defined*

Oral Exam

☐ approved	.....	.....

Preface

This compendium contains exercises and lab work as well as exercise solutions for the second study period in LTH's introductory programming course for the Master's program in Computer Engineering.
It is important that you use your learning from the first study period about what you need to practice more and immediately make a plan for how you can consolidate your understanding of the concepts from the first study period, so that you can keep up during the upcoming study period.
The overall goal for the second study period is that you should be able to create your own programs that solve more extensive problems than before, by combining several abstraction mechanisms and concepts from study period 1. We also introduce new abstraction mechanisms (e.g., inheritance) and new language constructs (e.g., pattern matching). Study period 2 ends with a mandatory individual project. At the end of the course, you will also take an oral exam that verifies that you have the knowledge required for the subsequent course. You are also offered an optional written exam that provides the opportunity for a higher grade.
The compendium is distributed as open source. It may be freely used as long as acknowledgment is given and any changes are published under the same license as the original material.
In the course's repo github.com/lunduniversity/introprog there are instructions on how you can contribute to the course material.
Welcome to the second half!

Lund, March 8, 2026, Björn Regnell

Contents

Progress Protocol iii

Preface v

I Module overview 1

II Modules 7

8 Nested and Generic Structures 9

8.1 Theory . . . 10

8.1.1 This Week's Lab: `life` . . . 10
8.1.2 What is a matrix? . . . 10
8.1.3 Indexing in a matrix . . . 10
8.1.4 How to create matrices? . . . 11
8.1.5 How to index in matrices? . . . 11
8.1.6 How to index in matrices? . . . 11
8.1.7 Updating a mutable nested structure . . . 12
8.1.8 Updating a mutable nested structure . . . 12
8.1.9 Some different ways to create mutable matrices . . . 13
8.1.10 Example of creating an immutable nested structure . . . 13
8.1.11 Example of creating an immutable nested structure . . . 14
8.1.12 Updating an immutable nested structure . . . 14
8.1.13 Updating an immutable nested structure . . . 14
8.1.14 Iterate over nested structure . . . 15
8.1.15 Iterate over nested structure . . . 15
8.1.16 Exercise Example: Yatzy . . . 16
8.1.17 Exercise Example: Yatzy . . . 16
8.1.18 Iterate over nested structure: for-loop . . . 16
8.1.19 Iterate over nested structure: for-loop . . . 17
8.1.20 Nested for-expressions . . . 17
8.1.21 Nested for-expressions . . . 18
8.1.22 Nested map-expressions . . . 18
8.1.23 Nested map-expressions . . . 18
8.1.24 Pitfall: equality of array . . . 18
8.1.25 Check equality between two integer matrices (reinventing the wheel) . . . 19
8.1.26 Do NOT use `sameElements` on nested arrays . . . 19
8.1.27 Checking content equality between nested arrays . . . 20
8.1.28 Practice matrices with this week's exercises: . . . 20

8.1.29 Example: Non-generic case class with integer matrix 20
8.1.30 Example: Generic case class with general matrix 21
8.1.31 What is a type parameter? 21
8.1.32 Example: Generic function 21
8.1.33 Example: Generic case class 22
8.1.34 Pitfall: Type erasure 22
8.1.35 Debugging – Appendix D 23
8.1.36 The first bug 23
8.1.37 Different kinds of errors? 24
8.1.38 When are errors detected? 24
8.1.39 Preventing errors 24
8.1.40 Finding the cause of errors: debugging 25
8.1.41 Fixing errors 25
8.1.42 Using a debugger 25
8.2 Exercise `Excercise08` 26
8.2.1 Basic Tasks; preparation for the lab 26
8.2.2 Extra Tasks; additional practice 30
8.2.3 Advanced Tasks; challenges 32
8.3 Lab: `life` 34
8.3.1 Background 34
8.3.2 Mandatory requirements 35
8.3.3 Optional requirements – choose at least one 36
8.3.4 Tips and suggestions 37

9 Sets and Maps **41**
9.1 Theory 42
9.1.1 Hierarchy of collection types in `scala.collection` v2.13 . . . 42
9.1.2 The method `iterator` gives a "one-time iterator" 42
9.1.3 More specific collection types in `scala.collection` 42
9.1.4 Some immutable and mutable sequence collections 43
9.1.5 Some useful methods on collections 43
9.1.6 Modify in place in a mutable collection 44
9.1.7 Review: What is a sequence? 44
9.1.8 A string is also an `IndexedSeq[Char]` 45
9.1.9 Convert between different collection types 45
9.1.10 What is a set? 45
9.1.11 Immutable set 46
9.1.12 The mystery of the vanished elements 46
9.1.13 Mutable set 46
9.1.14 Special methods on mutable sets 47
9.1.15 What is a key-value table? 47
9.1.16 The amazing key-value table `Map` 48
9.1.17 Immutable key-value table 48
9.1.18 More examples of key-value table 48
9.1.19 From sequence of pairs to table 49
9.1.20 Exercise: Implement a `Multimap` 49
9.1.21 Exercise: `Multimap` 50
9.1.22 Solution: `Multimap` 50
9.1.23 Special methods on mutable map 50
9.1.24 Further reading: groupMap, groupMapReduce 51
9.1.25 Exercise: Mutable locally, return immutable 51

9.1.26 Exercise: Mutable locally, return immutable 51
9.1.27 Solution: Mutable locally, return immutable 52
9.1.28 The method `sliding` 52
9.1.29 Methods zipWithIndex, groupBy 53
9.1.30 More useful collection methods 53
9.1.31 Serialization and Deserialization 53
9.1.32 Reading text from file and webpage 54
9.1.33 Serialization in the module `introprog.IO` 54
9.2 Exercise `Excercise09` 55
9.2.1 Basic Tasks; preparation for the lab 55
9.2.2 Extra Tasks; additional practice 59
9.2.3 Advanced Tasks; challenges 60
9.3 Lab: `words` 62
9.3.1 Background 62
9.3.2 Mandatory tasks 62
9.3.3 Control questions 68
9.3.4 Optional tasks 68

10 Inheritance and Composition 71
10.1 Theory 72
10.1.1 What is inheritance? 72
10.1.2 Why is inheritance needed? 72
10.1.3 Class diagram with UML (Unified Modeling Language) 73
10.1.4 Example: Robot as base type for two subtypes 73
10.1.5 Alternative to inheritance: composition 73
10.1.6 Example of composition in the snake lab 74
10.1.7 The need for a common base type 74
10.1.8 Why isn't the common member visible in a union type? 74
10.1.9 Create a common base type with inheritance 75
10.1.10 Create a common base type with `trait` and `extends` 75
10.1.11 A common base type with common parts 76
10.1.12 Place common parts in the base type 76
10.1.13 Scala's type hierarchy 77
10.1.14 Implicit supertypes of your own classes 77
10.1.15 What is a trait? 78
10.1.16 What is a trait used for? 78
10.1.17 A trait can have abstract members 79
10.1.18 A trait can have parameters 79
10.1.19 Abstract and concrete members 80
10.1.20 Avoid code duplication using inheritance 80
10.1.21 Why can code duplication cause problems? 81
10.1.22 Subtyping polymorphism and dynamic binding 81
10.1.23 Example: Overriding and `override` 81
10.1.24 A final member cannot be overridden 82
10.1.25 Protected gives visibility limited to subtypes 82
10.1.26 Filename rules and conventions 83
10.1.27 Classes, inheritance and class parameters 83
10.1.28 Static and dynamic type 84
10.1.29 Mixin 84
10.1.30 Runtime type with mixin 85
10.1.31 Anonymous class 85

10.1.32 How to prevent subtyping? . . . 86
10.1.33 Sealed types with `sealed` . . . 86
10.1.34 Open class signals encouraged subtyping . . . 87
10.1.35 Trait or abstract class? . . . 87
10.1.36 A trait cannot forward parameters . . . 88
10.1.37 Members, inheritance, and shadowing . . . 88
10.1.38 Deep dive: Detailed override rules . . . 89
10.1.39 Deep dive: override of `var` with `var` . . . 89
10.1.40 Deep dive: override of `def` with `var` . . . 90
10.1.41 Distinguishing mine and yours with `super` . . . 90
10.1.42 Terminology and keywords in inheritance . . . 91
10.1.43 What is an algebraic data type? . . . 91
10.1.44 A case class is a product. . . . 91
10.1.45 Algebraic data type, combined product and sum . . . 92
10.1.46 Algebraic data type with type parameter . . . 92
10.1.47 Algebraic data type with type parameter . . . 93
10.2 Exercise `Excercise10` . . . 94
10.2.1 Basic Tasks; preparation for the lab . . . 94
10.2.2 Extra Tasks; additional practice . . . 99
10.2.3 Advanced Tasks; challenges . . . 100
10.3 Team Lab: `snake0` . . . 105
10.3.1 Background . . . 106
10.3.2 Mandatory functional requirements . . . 106
10.3.3 Mandatory design requirements . . . 107
10.3.4 Optional requirements – each person must choose at least one 111
10.3.5 Tips and suggestions . . . 112

11 Variance and Context Parameters 117
11.1 Theory . . . 118
11.1.1 Type Parameter, Generic Structure, Type Constructor . . . 118
11.1.2 Different Ways to Restrict Generic Types . . . 118
11.1.3 Upper and Lower Type Bounds . . . 119
11.1.4 Example of Upper and Lower Type Bounds . . . 119
11.1.5 What is Variance? . . . 120
11.1.6 Why is Variance Needed? . . . 120
11.1.7 Covariance . . . 120
11.1.8 Contravariance . . . 121
11.1.9 Contravariance . . . 121
11.1.10 Variance Problems – Thanks Compiler! . . . 122
11.1.11 When to Use Which Kind of Variance? . . . 123
11.1.12 Invariance with Mutability . . . 123
11.1.13 Wildcard Types: Warning about Unbounded Types . . . 123
11.1.14 More about Variance for the Curious . . . 124
11.1.15 Self Type + Anonymous Class to ”Inject” Dependencies . . . 124
11.1.16 What is the Advantage of Self Types Instead of Inheritance? . 124
11.1.17 Type Unions with the Or-Operator . . . 125
11.1.18 Intersection Type with And-Operator . . . 125
11.1.19 Transparent Types . . . 126
11.1.20 What is a Good API? . . . 126
11.1.21 API Design with Scala . . . 127
11.1.22 Context is Crucial When You Code! . . . 127

11.1.23 Review: Default Arguments . . . 127
11.1.24 Review: Curried Parameter Lists . . . 128
11.1.25 Given Values + Context Parameter . . . 128
11.1.26 Wouldn't a Global Variable Work as Well? . . . 129
11.1.27 Importing a Context Parameter . . . 129
11.1.28 Summon Value with `summon` . . . 130
11.1.29 Order of Priority When Summoning Given Values . . . 130
11.1.30 Ad hoc Polymorphism . . . 131
11.1.31 How to Make the Type Class `Parser` Work for More Types? . . . 131
11.1.32 Context Parameter Names Can Be Omitted . . . 132
11.1.33 Context Bound . . . 132
11.1.34 Even More Convenient Type Class with Extension Method . . . 132
11.1.35 Sorting Collections with Given Ordering . . . 133
11.1.36 Sorting Collections with Even Easier Given Ordering . . . 133
11.1.37 Suggested Use of Context Parameters in the Snake Lab . . . 133
11.2 Exercise `Excercise11` . . . 135
11.2.1 Basic Tasks; preparation for the lab . . . 135
11.2.2 Extra Tasks; additional practice . . . 136
11.2.3 Advanced Tasks; challenges . . . 137
11.3 Lab: `snake1` . . . 142
11.3.1 Group Lab Presentation . . . 142

12 Advanced, Project 143
12.1 Project assignment . . . 144
12.1.1 About your final project assignment . . . 144
12.1.2 Project assignments . . . 144
12.1.3 Creating documentation . . . 144
12.1.4 Documentation comments . . . 145
12.2 Exercise `Excercise12` . . . 146
12.2.1 Tasks on searching and sorting . . . 146
12.2.2 Tasks on threads and concurrent execution . . . 153
12.3 Project Assignment: `bank` . . . 162
12.3.1 Focus . . . 162
12.3.2 Background . . . 162
12.3.3 Requirements . . . 162
12.3.4 Design . . . 164
12.3.5 Tips . . . 165
12.3.6 Mandatory tasks . . . 166
12.3.7 Optional extra tasks . . . 167
12.3.8 Example of history file . . . 167
12.3.9 Example of program execution . . . 167
12.4 Project Assignment: `music` . . . 170
12.4.1 Background . . . 170
12.4.2 Domain model . . . 170
12.4.3 Optional tasks . . . 177
12.5 Project Assignment: `photo` . . . 178
12.5.1 Background . . . 178
12.5.2 Preparations . . . 178
12.5.3 Matrix with values of type `Short` . . . 178
12.5.4 User Interface . . . 179
12.5.5 Filters . . . 181

12.5.6 Optional extra tasks 186

13 Revision **189**
13.1 Tips 190
13.1.1 Summary FAQ 190
13.1.2 More FAQ 190
13.1.3 Review: What is an algorithm? 190
13.1.4 Review: Rules of thumb/tips when choosing abstraction 191
13.1.5 Review: Tips on choice of collection 191
13.2 Exercise `Excercise13` 193

14 ORAL EXAM **195**

III Appendix **197**

A Kojo **199**
A.1 What is Kojo? 199
A.2 Using the graphics library in Kojo 200
A.3 Kojo Desktop 201
A.4 Kojo in the Browser 201
A.5 More about Kojo 201

B Terminal Window **205**
B.1 What is a terminal window? 205
B.2 What is a path? 207
B.3 Some important terminal commands 208

C Edit, compile and execute **209**
C.1 What is an editor? 209
C.1.1 Choose editor 210
C.2 What is a compiler? 211
C.3 Java JDK 212
C.3.1 Check if you have JDK installed 212
C.3.2 Install JDK 213
C.4 Scala 213
C.4.1 Install Scala 213
C.4.2 Scala Read-Evaluate-Print-Loop (REPL) 213
C.4.3 Compile and run with Scala Command Line Interface 214

D Fixing bugs **217**
D.1 What is a bug? 217
D.1.1 Different types of errors 218
D.2 Preventing errors 221
D.3 What is debugging? 223
D.3.1 How is the fault found? 223
D.4 Fixing errors 223
D.5 Using a debugger 224

E Documentation **227**
E.1 What does a documentation tool do? . . . 228
E.2 scaladoc . . . 228
E.2.1 Using documentation from scaladoc . . . 228
E.2.2 Writing documentation comments . . . 231
E.2.3 Generating documentation . . . 232

F Build Tools **233**
F.1 What Does a Build Tool Do? . . . 233
F.2 Scala Command Line Interface `scala-cli` . . . 235
F.2.1 Example Usage of Scala CLI . . . 235
F.2.2 Basic Build Functions in Scala CLI . . . 236
F.2.3 Using Options to Control Scala CLI . . . 236
F.2.4 Generate Documentation with Scala CLI . . . 237
F.2.5 Packaging an Executable File with Scala CLI . . . 237
F.2.6 Options as Usage Directives in Magic Comments . . . 238
F.3 Scala Build Tool `sbt` . . . 238
F.3.1 Installing sbt . . . 238
F.3.2 Using sbt . . . 239

G Version Control and Code Hosting **243**
G.1 What is Version Control? . . . 243
G.2 The Git Version Control Tool . . . 243
G.2.1 Installing git . . . 244
G.2.2 Customizing Git . . . 245
G.2.3 Using git . . . 245
G.3 Code Hosting Sites on the Internet . . . 247

H Integrated Development Environment **249**
H.1 What is an integrated development environment? . . . 249
H.2 Visual Studio Code with the Scala Metals Extension . . . 250
H.2.1 Installing VS Code and Metals . . . 250
H.2.2 Running Programs in VS Code . . . 250
H.2.3 Using the Debugger in VS Code . . . 252
H.3 JetBrains IntelliJ IDEA with Scala Plugin . . . 253
H.3.1 Installing IntelliJ IDEA . . . 254

J Introduction to Java **257**
J.1 Theory . . . 258
J.1.1 Exercise `scalajava` and lab `javatext` . . . 258
J.1.2 Many new features in recent Java versions . . . 258
J.1.3 Try Java in `jshell` . . . 258
J.1.4 Basic similarities and differences Java–Scala . . . 259
J.1.5 Main program in Scala and Java . . . 259
J.1.6 Looping through arguments in a Java main program . . . 260
J.1.7 HIGHSCORE implemented in Java . . . 260
J.1.8 Some things present in Java but not in Scala . . . 261
J.1.9 Some things present in Scala but not yet in Java . . . 261
J.1.10 Limitations of functional programming in Java . . . 262
J.1.11 Basic types in Scala and primitive types in Java . . . 263
J.1.12 Java's switch statement . . . 263

J.1.13 Java switch statement without break 264
J.1.14 Java switch statement with forgotten break 264
J.1.15 Syntax for variable declaration in Scala and Java 266
J.1.16 For-loop in Scala and Java 266
J.1.17 For-loop in Scala with indices 267
J.1.18 For-loop and arrays in Java 267
J.1.19 Implementation of SEQ-COPY in Java with `for` loop 268
J.1.20 Element by element with special for-each in Java 268
J.1.21 Typical design of Java class 268
J.1.22 Static members in Java 269
J.1.23 Example: immutable class in Scala and Java 269
J.1.24 Example: Scala class Complex 270
J.1.25 Example: Corresponding Java class JComplex 270
J.1.26 Example: Using JComplex in Scala code 271
J.1.27 Example: Using JComplex in Java code 271
J.1.28 Example: Mutable class in Scala and Java 272
J.1.29 Scala's "case class goodness" not in Java 274
J.1.30 Review: The primitive type Array in JVM 274
J.1.31 Syntax for Array in Scala and Java 274
J.1.32 Example: Polygon with primitive array in Java 275
J.1.33 Polygon with primitive array in Java: insert last and create more space if needed 275
J.1.34 Polygon with primitive array in Java: insert in the middle at given position 276
J.1.35 Scan files and strings with `java.util.Scanner` 276
J.1.36 Example: Scanner 277
J.1.37 Using Java collections in Scala with `CollectionConverters` . 277
J.1.38 Generic collections in Java 277
J.1.39 About ArrayList in Java 278
J.1.40 Polygon with ArrayList in Java 278
J.1.41 Some important operations on ArrayList<E> 278
J.1.42 Exercise ArrayList: new and add 279
J.1.43 For-each statement in Java: 279
J.1.44 Polygon with ArrayList: methods become simpler 280
J.1.45 Polygon with ArrayList: iterate over all vertices in draw with indexing 280
J.1.46 Polygon with ArrayList: iterate over all vertices in draw with for-each 281
J.1.47 Exercise ArrayList: implement the method hasVertex 281
J.1.48 Solution ArrayList: implement the method hasVertex 281
J.1.49 For-each loop with array 282
J.1.50 Generic classes (e.g. ArrayList) with primitive types 282
J.1.51 Wrapper class `Integer` 282
J.1.52 Wrapper classes in `java.lang` 283
J.1.53 Exercise: primitive versus wrapped types 283
J.1.54 Example: List with integers without autoboxing 283
J.1.55 Special rules for wrapper classes 284
J.1.56 Example: List with integers and autoboxing 284
J.1.57 Pitfalls with autoboxing 284
J.1.58 Reference equality or structural equality in Scala and Java . . 285

J.1.59 Pitfall with collections: contains method requires equals implementation . . . 285
J.1.60 Complete recipe for `equals` . . . 285
J.1.61 Conditional expressions in Java . . . 286
J.1.62 Type test and cast . . . 286
J.1.63 Rules for overriding in Java . . . 286
J.1.64 Catching exceptions in Scala and Java . . . 286
J.1.65 Interface `List` in Java . . . 287
J.1.66 Cannot create generic Array in Java . . . 287
J.1.67 Compare strings in Java . . . 288
J.1.68 Compare strings in Java: example . . . 288
J.2 Exercise `java` . . . 290
J.2.1 Basic Tasks; preparation for the lab . . . 290
J.3 Lab: `java` . . . 310
J.3.1 Requirements . . . 310
J.3.2 Optional extra tasks . . . 311
J.3.3 Inspiration and tips . . . 311

IV Solutions 313

L Solutions to the exercises 315
L.8 Solution `Excercise08` . . . 316
L.8.1 Basic Tasks; preparation for the lab . . . 316
L.8.2 Extra Tasks; additional practice . . . 320
L.8.3 Advanced Tasks; challenges . . . 322
L.9 Solution `Excercise09` . . . 324
L.9.1 Basic Tasks; preparation for the lab . . . 324
L.9.2 Extra Tasks; additional practice . . . 328
L.9.3 Advanced Tasks; challenges . . . 328
L.10 Solution `Excercise10` . . . 329
L.10.1 Basic Tasks; preparation for the lab . . . 329
L.10.2 Extra Tasks; additional practice . . . 334
L.10.3 Advanced Tasks; challenges . . . 335
L.11 Solution `Excercise11` . . . 338
L.11.1 Basic Tasks; preparation for the lab . . . 338
L.11.2 Extra Tasks; additional practice . . . 341
L.11.3 Advanced Tasks; challenges . . . 341
L.12 Solution `Excercise12` . . . 347
L.12.1 Tasks on searching and sorting . . . 347
L.12.2 Tasks on threads and concurrent execution . . . 352
L.12.3 Extra Tasks; additional practice . . . 353
L.13 Solution `Excercise13` . . . 357
L.14 Solution `java` . . . 358
L.14.1 Basic Tasks; preparation for the lab . . . 358

Part I

Module overview

W	*Module*	*Exercise*	*Lab*
W01	Introduction	Excercise01	kojo
W02	Programs and Control Structures	Excercise02	–
W03	Functions and Abstraction	Excercise03	irritext
W04	Objects and Encapsulation	Excercise04	blockmole
W05	Classes and Data Modeling	Excercise05	blockbattle0
W06	Patterns and Exception Handling	Excercise06	blockbattle1
W07	Sequences and Enumerations	Excercise07	shuffle
TP	–	–	–
W08	Nested and Generic Structures	Excercise08	life
W09	Sets and Maps	Excercise09	words
W10	Inheritance and Composition	Excercise10	snake0
W11	Variance and Context Parameters	Excercise11	snake1
W12	Advanced, Project	Excercise12	Project0
W13	Revision	Excercise13	Project1
W14	ORAL EXAM	Excercise14	Munta

W01	**Introduction**	sequence, alternative, repetition, abstraction, edit, compile, execute, parts of computer, virtual machine, literal, value, expression, identifier, variable, type, assignment, name, val, var, def, define and call function, function header, function body, procedure, built-in primitive types, println, Unit type, unit value (), string interpolator s, arithmetic, random numbers, logical expressions, De Morgan's laws, if, true, false, while, for, dod: operating system
W02	**Programs and Control Structures**	main program, program argument, input, scala.io.StdIn.readLine, control structure, iterate over elements in collection, for-expression, yield, map, foreach, collection, sequence, indexing, Array, Vector, range, Range, algorithm, implementation, pseudocode, algorithm examples: SWAP, SUM, MIN-MAX, MIN-INDEX, dod: version control
W03	**Functions and Abstraction**	abstraction, function, parameter, argument, return type, default argument, named arguments, parameter list, function header, function body, apply function to all elements in a collection, curried parameter list, define own control structure, function value, function type, pure function, step function, apply, anonymous functions, lambda, predicate, activation record, call stack, object heap, stack trace, value call, name call, curly braces and colon for single parameter, recursion, scala.util.Random, random seed, dod: typing
W04	**Objects and Encapsulation**	module, singleton object, dot notation, state, member, attribute, method, package, file structure, jar, classpath, documentation, JDK, import, selective import, renaming on import, export, tuple, multiple return values, block, local variable, shadowing, local function, functions are objects with apply method, namespace, visibility, private member, encapsulation, getter and setter, uniform access principle, method overloading, introprog.PixelWindow, initialization, lazy val, type alias, dod: machine code
W05	**Classes and Data Modeling**	application domain, data model, object-orientation, class, instance, Any, isInstanceOf, toString, new, null, this, access rules, private, private[this], class parameter, primary constructor, factory method, alternative constructor, mutable, immutable, case class, companion object, reference equality, structural equality, eq, ==
W06	**Patterns and Exception Handling**	pattern matching, match, Option, throw, try, catch, Try, unapply, sealed, flatten, flatMap, partial functions, collect, wildcard patterns, variable binding in patterns, sequence wildcard, literal patterns, implement equals, hashCode
W07	**Sequences and Enumerations**	overview of Scala's collection library and collection methods, class hierarchy in scala.collection, Iterable, Seq, List, ListBuffer, ArrayBuffer, WrappedArray, sequence algorithm, algorithm: SEQ-COPY, in-place vs copy, algorithm: SEQ-REVERSE, registration, algorithm: SEQ-REGISTER, linear search, algorithm: LINEAR-SEARCH, time complexity, memory complexity, overview of string methods, StringBuilder, order, built-in search methods, find, indexOf, indexWhere, built-in sorting methods, sorted, sortWith, sortBy, repeated parameters

TP	–	
W08	**Nested and Generic Structures**	matrix, nested collection, nested for-statement, type parameter, generic function, generic class, free and bound type parameter, generic data structures, generic collections in Scala
W09	**Sets and Maps**	membership test, set, Set, mutable.Set, key-value table, Map, mutable.Map, hash code, java.util.HashMap, java.util.HashSet, persistence, serialization, text files, Source.fromFile, java.nio.file
W10	**Inheritance and Composition**	inheritance, composition, polymorphism, trait, extends, asInstanceOf, with, mix-in supertype, subtype, base type, override, Scala type hierarchy, Any, AnyRef, Object, AnyVal, Null, Nothing, top type, bottom type, reference types, value types, access rules in inheritance, protected, final, trait, abstract class
W11	**Variance and Context Parameters**	upper and lower type bound, variance, contravariance, covariance, type wildcard, context bound, type constructor, self type, type wildcard, given value (given), context parameter (using), ad hoc polymorphism, type class, api, code readability, reviews
W12	**Advanced, Project**	choose any advanced topic, start project
W13	**Revision**	practice exercises, report project, prepare for oral exam
W14	ORAL EXAM	

Part II

Modules

Chapter 8

Nested and Generic Structures

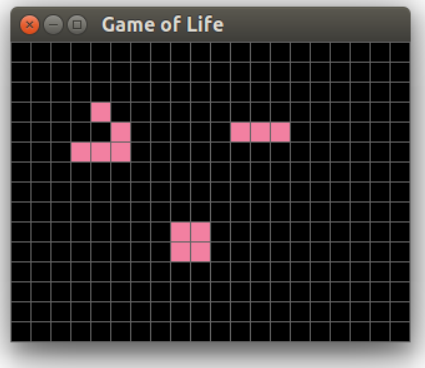
Concepts included in this week's studies:

- ☐ matrix
- ☐ nested collection
- ☐ nested for-statement
- ☐ type parameter
- ☐ generic function
- ☐ generic class
- ☐ free and bound type parameter
- ☐ generic data structures
- ☐ generic collections in Scala

8.1 Theory

8.1.1 This Week's Lab: `life`

- The universe is a binary matrix of **cells** where **living** cells are represented by `true` and **dead** cells by `false`.
- The following rules apply for the **next generation** of cells in the universe:
  - **Survival**: a living cell with 2 or 3 neighbors **continues to live**
  - **Death**: a living cell with fewer than 2 or more than 3 neighbors **dies**
  - **Birth**: a dead cell with exactly three neighbors is born
- Exercise `matrices` task 5: create a generic `case class Matrix[T]`
- In the lab: use `Matrix[Boolean]`



- You will simulate the *Game of Life* in an `introprog.PixelWindow`
- Further reading: https://en.wikipedia.org/wiki/Conway%27s_Game_of_Life

---

8.1.2 What is a matrix?

- A **matrix** in **mathematics** contains **rows** and **columns**[1] with numbers.
- In a **mathematical** matrix all rows have **the same number** of elements and
- also all columns have **the same number** of elements.
- A matrix of dimension 2×5 has $2 \cdot 5 = 10$ elements.
- Example of a mathematical matrix of dimension 2×5:

$$M_{2,5} = \begin{pmatrix} 5 & 2 & 42 & 4 & 5 \\ 3 & 4 & 18 & 6 & 7 \end{pmatrix}$$

8.1.3 Indexing in a matrix

- A matrix of dimension $m \times n$ has $m \cdot n$ elements.
- A matrix $A_{m,n}$ of dimension $m \times n$ is often depicted in mathematics like this:

$$A_{m,n} = \begin{pmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \cdots & a_{m,n} \end{pmatrix}$$

- Matrix indexing in mathematics often starts from 1, but often from 0 in computer programs.

[1] also called *columns*

- What is the index of the number 42 in the matrix $M_{2,5}$ below?
  - In mathematics?
  - In Scala and Java and many other languages?

$$M_{2,5} = \begin{pmatrix} 5 & 2 & 42 & 4 & 5 \\ 3 & 4 & 18 & 6 & 7 \end{pmatrix}$$

8.1.4 How to create matrices?

- In programming, the word **matrix** is often used to denote a **nested structure** in two dimensions. Example:
  - **Immutable** sequences, e.g. `Vector[Vector[Int]]`
    `val xss = Vector(Vector(0, 0, 0), Vector(0, 0, 0))` or simpler:
    `val xss = Vector.fill(2,3)(0)`
  - **Mutable** sequences, e.g. `Array[Array[Int]]`
    `val yss = Array(Array(0, 0, 0), Array(0, 0, 0))` or simpler:
    `val yss = Array.fill(2,3)(0)`

8.1.5 How to index in matrices?

A matrix with array of arrays:

```
scala> val xss = Array(Array(5,2,42,4,5),Array(3,4,18,6,7))
xss: Array[Array[Int]] = Array(Array(5, 2, 42, 4, 5), Array(3, 4, 18, 6, 7))
```

One indexes in a nested sequence with repeated `apply`:

```
scala> xss(0)(2)
res0: ???

scala> xss.apply(0).apply(2)
res1: ???

scala> xss(0)
res2: ???
```

Exercise: What is the type and value at `???` above?

8.1.6 How to index in matrices?

A matrix with array of arrays:

```
scala> val xss = Array(Array(5,2,42,4,5),Array(3,4,18,6,7))
xss: Array[Array[Int]] = Array(Array(5, 2, 42, 4, 5), Array(3, 4, 18, 6, 7))
```

One indexes in a nested sequence with repeated `apply`:

```
scala> xss(0)(2)
res0: Int = 42

scala> xss.apply(0).apply(2)
res1: Int = 42

scala> xss(0)
res2: Array[Int] = Array(5, 2, 42, 4, 5)
```

Exercise: Draw a diagram of the memory that the reference `xss` refers to.

8.1.7 Updating a mutable nested structure

You can modify an array of arrays "in place" via assignment:

```
scala> val xss = Array(Array(5,2,42,4,5),Array(3,4,18,6,7))

scala> xss(0)(0) = 100

scala> xss
res0: ???

scala> xss(0)(2) = xss(0)(2) - 1

scala> xss
res1: ???

scala> xss(1) = Array.fill(5)(-1)

scala> xss
res2: ???
```

8.1.8 Updating a mutable nested structure

You can modify an array of arrays "in place" via assignment:

```
scala> val xss = Array(Array(5,2,42,4,5),Array(3,4,18,6,7))

scala> xss(0)(0) = 100

scala> xss
res0: Array[Array[Int]] = Array(Array(100, 2, 42, 4, 5), Array(3, 4, 18, 6, 7))

scala> xss(0)(2) = xss(0)(2) - 1

scala> xss
res1: Array[Array[Int]] = Array(Array(100, 2, 41, 4, 5), Array(3, 4, 18, 6, 7))

scala> xss(1) = Array.fill(5)(-1)

scala> xss
```

```
res2: Array[Array[Int]] = Array(Array(100, 2, 41, 4, 5), Array(-1,-1,-1,-1,-1))
```

8.1.9 Some different ways to create mutable matrices

The cumbersome, primitive way:

```
scala> val xss = new Array[Array[Int]](2)
xss: Array[Array[Int]] = Array(null, null)

scala> for (i <- xss.indices) {xss(i) = new Array[Int](5)}

scala> xss
res0: Array[Array[Int]] = Array(Array(0, 0, 0, 0, 0), Array(0, 0, 0, 0, 0))

scala> println(xss)
[[I@196a99d0
```

Easier way:

```
scala> val xss = Array.ofDim[Int](2,5)
xss: Array[Array[Int]] = Array(Array(0, 0, 0, 0, 0), Array(0, 0, 0, 0, 0))
```

Easier and clearer way, where the initial value is explicitly given:

```
scala> val xss = Array.fill(2,5)(0)
xss: Array[Array[Int]] = Array(Array(0, 0, 0, 0, 0), Array(0, 0, 0, 0, 0))
```

8.1.10 Example of creating an immutable nested structure

If you can compute the initial value directly, use `Vector.fill`:

```
def fill[A](n1: Int, n2: Int)(elem: => A): Vector[Vector[A]]
```

```
scala> Vector.fill(2,5)(scala.util.Random.nextInt(6) + 1)
res0:
  type???
  value???
```

If you can compute the initial value from the index, use `Vector.tabulate`:

```
def tabulate[A](n1: Int, n2: Int)(f: (Int, Int) => A): Vector[Vector[A]]
```

```
scala> Vector.tabulate(5,2)((x,y) => x + y + 1)
res1:
  type???
  value???
```

8.1.11 Example of creating an immutable nested structure

If you can compute the initial value directly, use `Vector.fill`:
`def fill[A](n1: Int, n2: Int)(elem: => A): Vector[Vector[A]]`

```
scala> Vector.fill(2,5)(scala.util.Random.nextInt(6) + 1)
res0: Vector[Vector[Int]] =
  Vector(Vector(1, 2, 6, 2, 1), Vector(1, 4, 3, 3, 2))
```

If you can compute the initial value from the index, use `Vector.tabulate`:
`def tabulate[A](n1: Int, n2: Int)(f: (Int, Int) => A): Vector[Vector[A]]`

```
scala> Vector.tabulate(5,2)((x,y) => x + y + 1)
res1: Vector[Vector[Int]] =
  Vector(Vector(1,2), Vector(2,3), Vector(3,4), Vector(4,5), Vector(5,  6))
```

8.1.12 Updating an immutable nested structure

Updating a one-dimensional structure with `xs.updated`:
`def updated[A](index: Int, elem: A): Vector[A]`

```
scala> var xs = Vector.tabulate(5)(x => x + 1)
xs: type??? = value???

scala> xs = xs.updated(1, 42)
xs: type??? = value???
```

Updating a nested two-dimensional structure:

```
scala> var xss = Vector.tabulate(2, 5)((x,y) => x + y + 1)
xss:
  type??? =
  value???

scala> xss = xss.updated(0, xss(0).updated(1, 42))
xss:
  type??? =
  value???
```

8.1.13 Updating an immutable nested structure

Updating a one-dimensional structure with `xs.updated`:
`def updated[A](index: Int, elem: A): Vector[A]`

```
scala> var xs = Vector.tabulate(5)(x => x + 1)
xs: Vector[Int] = Vector(1, 2, 3, 4, 5)

scala> xs = xs.updated(1, 42)
xs: Vector[Int] = Vector(1, 42, 3, 4, 5)
```

Updating a nested two-dimensional structure:

```
scala> var xss = Vector.tabulate(2, 5)((x,y) => x + y + 1)
xss: Vector[Vector[Int]] =
  Vector(Vector(1, 2, 3, 4, 5), Vector(2, 3, 4, 5, 6))

scala> xss = xss.updated(0, xss(0).updated(1, 42))
xss:
  Vector[Vector[Int]] =
  Vector(Vector(1, 42, 3, 4, 5), Vector(2, 3, 4, 5, 6))
```

8.1.14 Iterate over nested structure

Handling nested structures often requires algorithms with nested iteration.
Example: iterate with nested **for**-statement to print this matrix
`val xss = Vector.tabulate(2,5)((x,y) => x + y + 1)`

```
scala> for ??? do
         for ??? do
           print(s"x ")
         end for
         println()
       end for

1 2 3 4 5
2 3 4 5 6
```

Exercise:
What should be written at `???` so that all elements are printed as above?

8.1.15 Iterate over nested structure

Handling nested structures often requires algorithms with nested iteration.
Example: iterate with nested **for**-statement to print this matrix
`val xss = Vector.tabulate(2,5)((x,y) => x + y + 1)`

```
scala> for xs <- xss do
         for x <- xs do
           print(s"$x ")
         end for
         println()
       end for

1 2 3 4 5
2 3 4 5 6
```

Exercise: print the matrix with nested `foreach`

```
xss.foreach: xs =>
  xs.foreach(x => print(s"$x "))
  println()
```

8.1.16 Exercise Example: Yatzy

Create a function `roll` that gives the result of n dice rolls:

```
scala> import scala.util.Random

scala> def roll(n: Int): Vector[Int] = ???
```

Create a function `isYatzy` that returns **`true`** if all rolls are the same:

```
scala> def isYatzy(xs: Vector[Int]): Boolean = ???
```

You may assume that xs.length > 0
Tip: use the method xs.forall:
`def forall[A](p: A => Boolean): Boolean`

8.1.17 Exercise Example: Yatzy

Create a function `roll` that gives the result of n dice rolls:

```
scala> import scala.util.Random

scala> def roll(n: Int): Vector[Int] = Vector.fill(n)(Random.nextInt(6) + 1)
```

Create a function `isYatzy` that returns **`true`** if all rolls are the same:

```
scala> def isYatzy(xs: Vector[Int]): Boolean = xs.forall(x => x == xs(0))
```

You may assume that xs.length > 0
Tip: use the method xs.forall:
`def forall[A](p: A => Boolean): Boolean`

8.1.18 Iterate over nested structure: for-loop

Iterate with nested for-loop: (what is the type of xss?)

```
scala> val xss = Vector.fill(100)(roll(5))

scala> for i <- ??? do
         for j <- ??? do
           print(s"($i)($j): ${xss(i)(j)} ")
         end for
         println(s" YATZY: ${isYatzy(xss(i))}")
       end for

(0)(0): 3 (0)(1): 6 (0)(2): 4 (0)(3): 4 (0)(4): 6  YATZY: false
(1)(0): 4 (1)(1): 1 (1)(2): 5 (1)(3): 2 (1)(4): 6  YATZY: false
(2)(0): 1 (2)(1): 3 (2)(2): 5 (2)(3): 6 (2)(4): 2  YATZY: false
(3)(0): 2 (3)(1): 1 (3)(2): 1 (3)(3): 5 (3)(4): 4  YATZY: false
(4)(0): 4 (4)(1): 4 (4)(2): 1 (4)(3): 6 (4)(4): 5  YATZY: false
(5)(0): 3 (5)(1): 3 (5)(2): 2 (5)(3): 3 (5)(4): 6  YATZY: false
```

```
(6)(0): 3 (6)(1): 6 (6)(2): 1 (6)(3): 1 (6)(4): 4  YATZY: false
(7)(0): 6 (7)(1): 2 (7)(2): 4 (7)(3): 4 (7)(4): 3  YATZY: false
(8)(0): 1 (8)(1): 5 (8)(2): 4 (8)(3): 2 (8)(4): 4  YATZY: false
(9)(0): 1 (9)(1): 1 (9)(2): 3 (9)(3): 6 (9)(4): 6  YATZY: false
...
```

8.1.19 Iterate over nested structure: for-loop

Iterate with nested for-loop: (xss is a `Vector[Vector[Int]]`)

```
scala> val xss = Vector.fill(100)(roll(5))

scala> for i <- xss.indices do
         for j <- xss(i).indices do
           print(s"($i)($j): ${xss(i)(j)} ")
         end for
         println(s" YATZY: ${isYatzy(xss(i))}")
       end for

(0)(0): 3 (0)(1): 6 (0)(2): 4 (0)(3): 4 (0)(4): 6  YATZY: false
(1)(0): 4 (1)(1): 1 (1)(2): 5 (1)(3): 2 (1)(4): 6  YATZY: false
(2)(0): 1 (2)(1): 3 (2)(2): 5 (2)(3): 6 (2)(4): 2  YATZY: false
(3)(0): 2 (3)(1): 1 (3)(2): 1 (3)(3): 5 (3)(4): 4  YATZY: false
(4)(0): 4 (4)(1): 4 (4)(2): 1 (4)(3): 6 (4)(4): 5  YATZY: false
(5)(0): 3 (5)(1): 3 (5)(2): 2 (5)(3): 3 (5)(4): 6  YATZY: false
(6)(0): 3 (6)(1): 6 (6)(2): 1 (6)(3): 1 (6)(4): 4  YATZY: false
(7)(0): 6 (7)(1): 2 (7)(2): 4 (7)(3): 4 (7)(4): 3  YATZY: false
(8)(0): 1 (8)(1): 5 (8)(2): 4 (8)(3): 2 (8)(4): 4  YATZY: false
(9)(0): 1 (9)(1): 1 (9)(2): 3 (9)(3): 6 (9)(4): 6  YATZY: false
...
```

8.1.20 Nested for-expressions

Iterate with **nested for-yield**:

```
scala> val xss = for i <- 1 to 2 yield
                   for j <- 1 to 5 yield i + j + 1

val xss: IndexedSeq[IndexedSeq[Int]] =
      ???
```

If you write this way you get a one-dimensional structure:

```
scala> val xs = for { i <- 1 to 2; j <- 1 to 5 } yield i + j + 1
val xs: IndexedSeq[Int] =
    ???
```

8.1.21 Nested for-expressions

Iterate with **nested for-yield**:

```
scala> val xss = for i <- 1 to 2 yield
                   for j <- 1 to 5 yield i + j + 1

val xss: IndexedSeq[IndexedSeq[Int]] =
    Vector(Vector(3, 4, 5, 6, 7), Vector(4, 5, 6, 7, 8))
```

If you write this way you get a one-dimensional structure:

```
scala> val xs = for { i <- 1 to 2; j <- 1 to 5 } yield i + j + 1
val xs: IndexedSeq[Int] =
    Vector(3, 4, 5, 6, 7, 4, 5, 6, 7, 8)
```

8.1.22 Nested map-expressions

Iterate with **nested map-expressions**:

```
scala> val xss = (1 to 2).map(i => (1 to 5).map(j => i + j + 1))
xss: IndexedSeq[IndexedSeq[Int]] =
      ???
```

8.1.23 Nested map-expressions

Iterate with **nested map-expressions**:

```
scala> val xss = (1 to 2).map(i => (1 to 5).map(j => i + j + 1))
xss: IndexedSeq[IndexedSeq[Int]] =
      Vector(Vector(3, 4, 5, 6, 7), Vector(4, 5, 6, 7, 8))
```

8.1.24 Pitfall: equality of array

```
scala> Vector.fill(5, 2)(42) == Vector.fill(5, 2)(42)
val res0: Boolean = true

scala> Array.fill(5, 2)(42) == Array.fill(5, 2)(42)
val res1: Boolean = false  // AAAARRGH!!! :(
```

Primitive arrays have an equals method that gives reference equality, **not** content equality. Consequently, this does not work on nested structures either.

Exercise: implement

```
def isEqual(xss: Array[Array[Int]], yss: Array[Array[Int]]): Boolean = ???
```

8.1.25 Check equality between two integer matrices (reinventing the wheel)

```
def isEqual(xss: Array[Array[Int]], yss: Array[Array[Int]]): Boolean =
  if xss.length != yss.length then false else
    var i = 0
    var foundUnequal = false
    while i < xss.length && !foundUnequal do
      if xss(i).length != yss(i).length then
        foundUnequal = true
      else
        var j = 0
        while j < xss(i).length && !foundUnequal do
          if xss(i)(j) != yss(i)(j) then foundUnequal = true
          j += 1
        end while
      end if
      i += 1
    end while
    !foundUnequal
  end if
end isEqual
```

8.1.26 Do NOT use `sameElements` on nested arrays

In Scala you can use the method `sameElements` to check content equality between two arrays, but it does **NOT** work recursively on nested structures.

```
scala> val xs = Array(1,2,3)
xs: Array[Int] = Array(1, 2, 3)

scala> val ys = Array(1,2,3)
ys: Array[Int] = Array(1, 2, 3)

scala> xs.sameElements(ys)                          // xs, ys not nested
res0: Boolean = true                                // content equal!

scala> Array(Array(1)) sameElements Array(Array(1))
res1: Boolean = false                               //AAAARGH!
```

Instead use:
`java.util.Objects.deepEquals` or `java.util.Arrays.deepEquals`
The latter requires type casting of arguments with: `asInstanceOf[Array[Object]]`

8.1.27 Checking content equality between nested arrays

`java.util.Objects.deepEquals` works **deeply** for arbitrary reference types:

```
scala> java.util.Objects.deepEquals(
          Array(Array("a", Array("b"), 42)),
          Array(Array("a", Array("b"), 42)))
val res0: Boolean = true

scala> java.util.Objects.deepEquals(
          Array(Array("a", Array("b"), 42)),
          Array(Array("a", Array("b"), 43)))
val res1: Boolean = false
```

`java.util.Objects.deepEquals` checks if arguments are arrays and then calls `java.util.Arrays.deepEquals` with type cast:

```
scala> java.util.Arrays.deepEquals(
          Array(Array("a", Array("b"), 42)).asInstanceOf[Array[Object]],
          Array(Array("a", Array("b"), 42)).asInstanceOf[Array[Object]])
val res3: Boolean = true
```

https://stackoverflow.com/questions/63686721/best-replacement-of-deep-method-in-scala-2-13

8.1.28 Practice matrices with this week's exercises:

Exercise `matrices`:

- Practice iterating over nested structures
- Continue working with the Yatzy example
- Practice creating **imperative** algorithms: solve `isYatzy` with **`while`**-loop
- Extra assignment where you build a yatzy game in the terminal
  (could have been an exam question...)

8.1.29 Example: Non-generic case class with integer matrix

A *non-generic* data structure has no unbound type parameters; all types are **concrete** (i.e., specific).

A non-generic case class with a `Vector[Vector[Int]]`:

```
case class Matrix(data: Vector[Vector[Int]]):
  def apply(x: Int, y: Int): Int = data(x)(y)
```

```
scala> Matrix(Vector(Vector(5, 2, 42, 4, 5),Vector(3, 4, 18, 6, 7)))
res0: Matrix =
  Matrix(Vector(Vector(5, 2, 42, 4, 5), Vector(3, 4, 18, 6, 7)))

```

8.1.30 Example: Generic case class with general matrix

A *generic* data structure has at least one **unbound type parameter** that at use should be bound to a **concrete type argument**.

```
case class Matrix[T](data: Vector[Vector[T]]):
  def apply(x: Int, y: Int): T = data(x)(y)
```

`Matrix` in the example above is a **generic** case class where `T` is unbound, because `T` is a type parameter declared inside `[]` **after** the class name but **before** the class parameter list.

Usage where `T` is bound to `Int` via the compiler's type inference:

```
scala> Matrix(Vector(Vector(5, 2, 42, 4, 5),Vector(3, 4, 18, 6, 7)))
res1: Matrix[Int] =
  Matrix(Vector(Vector(5, 2, 42, 4, 5), Vector(3, 4, 18, 6, 7)))

```

8.1.31 What is a type parameter?

- A **type parameter** makes it possible to provide a **type argument**.
- This is called **parametric polymorphism**.
- Example: **generic function**:

```
def tnirp[A](x: A):Unit = println(x.toString.reverse)
```

- A **free** type parameter can be bound to any type.
- The binding of type arguments to type parameters happens at **compile time**.
- A type parameter is **free** if it **has not** been assigned a value, otherwise **bound**.
- Example: **generic class** with **generic methods**:

```
class Cell[A](   // A is here free but must be bound when used
    var value: A):              // A is bound when using Cell
  def update(a: A): Unit = value = a    // A is also bound here when used
  def replaced[B](b: B): Cell[B] = new Cell(b)  // first [B] is free
```

- **Shadowing can occur**: If `replaced` in `Cell` had used the name A for its type parameter it would have **shadowed** the class's type parameter; therefore the new name `B` – it would be confusing to use the same name for different type parameters (compare name shadowing for **local** names in nested blocks).

8.1.32 Example: Generic function

What happens here?

```

scala> def screamBackwards(x: T): String = x.toString.toUpperCase.reverse
1 |def screamBackwards(x: T): String = x.toString.toUpperCase.reverse
  |                       ^
  |                       Not found: type T

scala> def screamBackwards[T](x: T): String = x.toString.toUpperCase.reverse

scala> screamBackwards("cucumber is good")
val res0: String = DOOG SI REBMUCUC
```

If no type parameter is declared within square brackets after the function's name, the compiler does not know what `T` is for a type. But with a type parameter `[T]` after the function name, the compiler interprets the function as **generic** and the type `T` is determined by the argument's type **at call time** and `T` can be bound to any type.

8.1.33 Example: Generic case class

- A generic class has one or more type parameters after the class name:

```
case class Box[A](value: A)
```

- The compiler infers the type of the type parameters based on given values.

```
scala> Box("gurka")
val res1: Box[String] = Box(gurka)
```

- You can also explicitly provide the type parameter:

```
scala> Box[Int](42)
val res2: Box[Int] = Box(42)
```

- If the type does not match, you get help from the compiler to find the error:

```
scala> Box[String](42)
-- Error:
1 |Box[String](42)
  |            ^^ Found:    (42 : Int)  Required: String
```

- A generic class, here `Box`, is also called a **type constructor** since the "finished" type `Box[Int]` is "constructed" at the place where it is used.

8.1.34 Pitfall: Type erasure

Information about types in type parameters is erased before code generation for the JVM for performance reasons and **type parameters are missing at runtime** in the bytecode.

```
scala> def isIntVector[T](xs: Vector[T]) = xs.isInstanceOf[Vector[Int]]
-- Warning:
1 |def isIntVector[T](xs: Vector[T]) = xs.isInstanceOf[Vector[Int]]
  |                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
  |                the type test for Vector[Int] cannot be checked at runtime
def isIntVector[T](xs: Vector[T]): Boolean

scala> isIntVector(Vector("hej"))
res42: Boolean = true  // AAAARGHH!! :(
```

You need to ”unpack” the collection and type test all elements:

```
scala> def isIntVector[T](xs: Vector[T]) = xs.forall(_.isInstanceOf[Int])

scala> isIntVector(Vector("hej"))
res43: Boolean = false  // WORKS :)
```

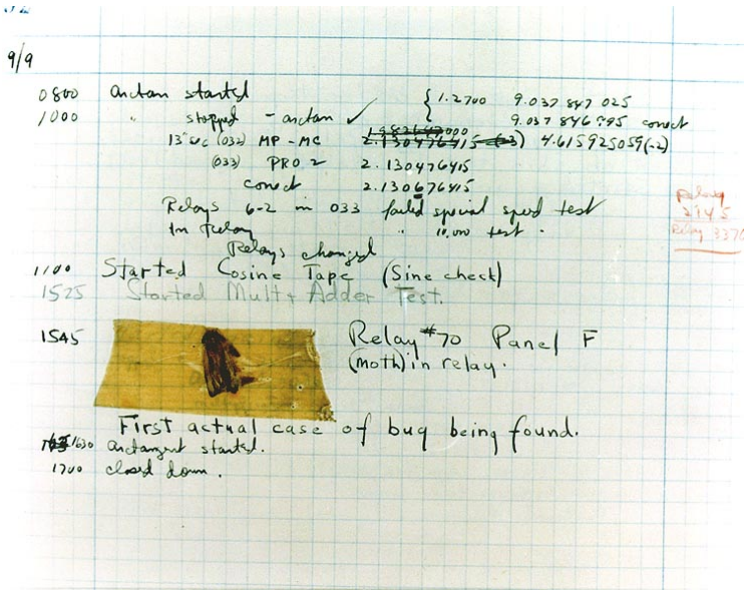
Type checking at runtime is usually better done using **`match`**.

8.1.35 Debugging – Appendix D

- Read about testing and debugging in Appendix D: ”Fixing bugs”
- Practice println-debugging
- Try the debugger in VS code

8.1.36 The first bug

A moth in a relay in the Mark II computer found by Grace Hopper in a log from 1940:



https://en.wikipedia.org/wiki/Debugging

8.1.37 Different kinds of errors?

- Requirement errors
- Design errors
- Implementation errors
- Test errors
- Operator errors
- User errors

8.1.38 When are errors detected?

- During human review
- Compilation errors – thanks compiler!
- Runtime errors
  - Execution yields unwanted result
    - During testing – or is the test case faulty?
    - In production – unhappy users : (
  - Execution hangs (eng. *hang*)
    - infinite loop
    - very slow
    - waiting for input
    - deadlock
  - Execution crashes (eng. *crash*)
    - out of memory
    - null reference
    - exception

8.1.39 Preventing errors

- Create understandable code.
- Think of good names.
- Check parameters and variables.
- Check types.
- Handle missing values.
- Handle exceptions.
- Review code.
- Test code.
- Learn from user experiences.

8.1.40 Finding the cause of errors: debugging

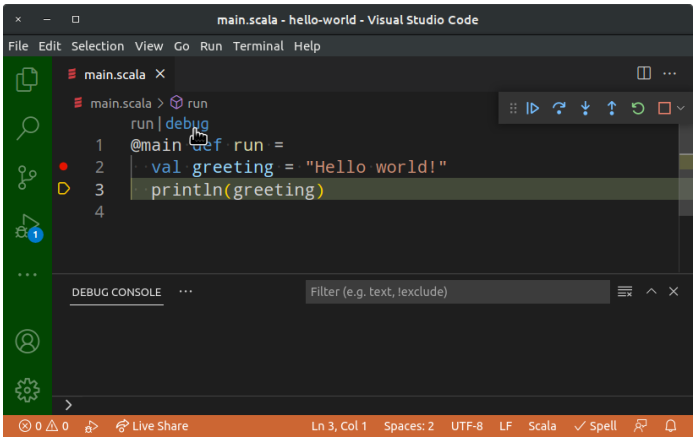
- Reproduce the bug with a minimal test case.
- Formulate and verify hypotheses about the bug.
- Instrumentation with printouts, "println-debugging".

8.1.41 Fixing errors

- The algorithm is fundamentally flawed: create a new algorithm
- Edge cases are not handled correctly.
- A tricky algorithm is especially hard to fix.
- While fixing a bug, one might accidentally create new bugs.
- Execution time grows too fast with increased data volume.

8.1.42 Using a debugger

- Set breakpoints.
- Stepwise execution.
- Inspect variables.



Read more in Appendix H about the debugger in VSCode.

8.2 Exercise `Excercise08`

Goals

- ☐ Be able to create and use matrices with nested structures of `Vector`.
- ☐ Be able to iterate over the elements in a matrix with nested **`for`** statements and **`for-yield`** expressions, as well as nested application of `map` and `foreach`.
- ☐ Be able to create and use functions that take matrices as parameters.
- ☐ Be able to create a simple generic class and simple generic functions using a type parameter.

Preparations

- ☐ Study the concepts in chapter 8

8.2.1 Basic Tasks; preparation for the lab

Task 1. *Match concepts with descriptions.*

Connect each concept with the (simplified) description that best fits:

matrix	1	A	indexable 2D data structure
row vector	2	B	has abstract type parameter, type is general
column vector	3	C	another word for column
column	4	D	concrete type, binds to type parameter at compile time
generic	5	E	compiler deduces type from context
type argument	6	F	matrix of dimension $1imesm$ with m horizontal values
type inference	7	G	matrix of dimension $mimes1$ with m vertical values

Task 2. *Create matrices using nested collections.* In a computer program, it is possible, using collections that contain collections, to create nested structures that can be indexed in two dimensions and thereby represent a **matrix**.[2]

a) Draw the memory situation after the assignment on line 1 below. What type and value does `m` have? What are the dimensions of `m`? How does indexing occur in a computer program compared to mathematics?

```
1 scala> val m = Vector((1 to 5).toVector, (3 to 7).toVector)
2 scala> m.apply(0).apply(1)
3 scala> m(1)
4 scala> m(1)(4)
```

b) What values and types are produced by the expressions on lines 2, 3, and 4 above?

c) It is also possible to create two-dimensional, nested structures where the rows *do not* contain the same number of elements. This is therefore not a true matrix in the strict mathematical sense, but such a structure is often still called a ”matrix”. What type do the variables `m2`, `m3`, `m4`, and `m5` below have?

[2] sv.wikipedia.org/wiki/Matrix

```
scala> val m2 = Vector(Vector(1,2,3),Vector(4,5),Vector(42))
scala> val m3 = Vector(Vector(1,2), Vector(1.0, 2.0, 3.0))
scala> val m4 = m3(1) +: Vector("a") +: m3
scala> val m5 = Vector.fill(42){ m2(1).map(e => (e * math.random()).toInt) }
```

d) Which of the variables `m2`, `m3`, `m4`, and `m5` above represents a true matrix in the mathematical sense? What are its dimensions?

Task 3. *Create and iterate over matrices.* You will create matrices where each row represents 5 throws of a die in the Yatzy game.[3]

a) Define in REPL a function `def throwDie: Int = ???` that returns a random integer between 1 and 6.

b) Create the integer matrix below in REPL. What dimension does this matrix have?

```
scala> val ds1 = for (i <- 1 to 1000) yield
            for (j <- 1 to 5) yield throwDie
```

c) You can also use the variants below to create an integer matrix. Which of the variants `ds1` ... `ds6` do you find easiest to read and understand? Try each variant in REPL and specify the type for `ds1` ... `ds6` as inferred by the compiler.

```
val ds2 = (1 to 1000).map(i => (1 to 5).map(j => throwDie))
val ds3 = (1 to 1000).map(i => Vector.fill(5)(throwDie))
val ds4 = for (i <- 1 to 1000) yield Vector.fill(5)(throwDie)
val ds5 = Vector.fill(1000)(Vector.fill(5)(throwDie))
val ds6 = Vector.fill(1000, 5)(throwDie)
```

d) Define a function
`def roll(n: Int): Vector[Int] = ???`
that produces an integer vector with n random die throws. The throws should be sorted in ascending order; use the collection method `sorted`.

e) Define in REPL a function `isYatzy(xs: Vector[Int]): Boolean = ???` which tests if all elements in an integer vector are the same. Use the collection method `forall`.

f) Create a function
`def diceMatrix(m: Int, n: Int): Vector[Vector[Int]] = ???`
which uses the `roll` function to create a matrix with `m` vectors each having `n` random die throws.

g) Create a function returning a print-friendly string
`def diceMatrixToString(xss: Vector[Vector[Int]]): String = ???`
using `map` and `mkString` which works as below.

```
scala> val dm2s = diceMatrixToString(diceMatrix(4, 5))
val dm2s: String = 1 4 4 6 6
1 1 2 6 6
2 4 4 5 6
1 1 5 6 6

scala> println(dm2s)
1 4 4 6 6
```

[3] sv.wikipedia.org/wiki/Yatzy

```
1 1 2 6 6
2 4 4 5 6
1 1 5 6 6
```

h) Implement the function
`def filterYatzy(xss: Vector[Vector[Int]]): Vector[Vector[Int]]`
which filters out all rows from the matrix `xss` that are yatzy-rows. Use your function `isYatzy` and the collection method `filter`.

```
scala> println(diceMatrixToString(filterYatzy(diceMatrix(10000, 5))))
4 4 4 4 4
6 6 6 6 6
4 4 4 4 4
6 6 6 6 6
4 4 4 4 4
4 4 4 4 4
2 2 2 2 2
```

i) Implement the function
`def yatzyPips(xss: Vector[Vector[Int]]): Vector[Int] = ???`
which should give a vector of the die values that resulted in yatzy for the rolls in the matrix `xss` as in the example below. Use your `filterYatzy`.

```
scala> val dm = Vector(Vector(1,2,3,4,5),Vector(4,4,4,4,4),Vector(3,3,3,3,3))
scala> yatzyPips(dm)
val res42: Vector[Int] = Vector(4, 3)
```

Task 4. *An immutable, generic matrix class for this week's lab* `life`. During this week's lab, you will simulate a simple form of "life" consisting of cells in a grid. For this purpose, we need a matrix class that you will implement step by step in this exercise. Create the case class below using an editor in the file `Matrix.scala`. Test your solution using any IDE such as `scalaide` or `idea`.

```
case class Matrix(data: Vector[Vector[String]]){
  def apply(row: Int, col: Int): String = data(row)(col)
}
object Matrix {
  def fill(dim: (Int, Int))(value: String): Matrix =
    Matrix(Vector.fill(dim._1, dim._2)(value))
}
```

```
scala> val m = Matrix.fill(3,4)("hej")
scala> val e = m(2, 2)
```

a) What type does `m` get above?

b) What type does `e` get above?

c) In how many places must you change the `Matrix` above for it to instead represent a matrix of integers?

d) You should now, with the help of a **type parameter**, make `Matrix` **generic**, so that it becomes a more useful matrix class which can contain elements of any type. Perform the following changes in `Matrix.scala`:

- Add a type parameter `T` inside square brackets after the name `Matrix` everywhere it occurs *except* after the name of the companion object[4].
- Replace `String` with `T` everywhere `String` appears.
- Add a type parameter `T` inside square brackets after `def fill`.

Test your generic class in REPL by creating a boolean matrix:

```
scala> val bm = Matrix.fill(3,4)(false)
scala> val be = bm(0, 0)
```

e) What type does `bm` have above?

f) What type does `be` have above?

g) Add a code line at the beginning of the class body that, using `require`, guarantees that all rows in the matrix are equally long.

h) Add a member `val dim: (Int, Int)` in the class body after the `require` statement which gives a pair (a 2-tuple) with the number of rows and columns in the matrix.

i) Add a method `def updated(row: Int, col: Int)(value: T): Matrix[T]` that returns a new matrix where the element at position `(row, col)` has been updated to `value`.

j) Add a method `def foreachIndex(f: (Int, Int) => Unit): Unit` which applies the function `f` to every index in `data`.

k) Add a method `override def toString` so that an instance of `Matrix` appears as follows:

```
scala> val dm = Matrix.fill(3,4)(42.0)
val dm: Matrix[Double] =
Matrix of dim (3,4):
42.0 42.0 42.0 42.0
42.0 42.0 42.0 42.0
42.0 42.0 42.0 42.0
```

[4] Singleton objects cannot have type parameters, but their members can.

8.2.2 Extra Tasks; additional practice

Task 5. *Imperative matrix algorithms.* Imperative approaches are necessary to master when working with collections and/or languages lacking functional methods and/or functional programming features. By studying imperative solutions to often more concise functional ones, you gain practice creating algorithms using change through assignment during iteration.

a) Implement `isYatzy` from problem 3e again, but now using an imperative approach with a **`while`** loop instead of the functional `forall`. Use a variable `i` to keep track of the index and a variable `foundDiff` to track whether a differing value is found. The function requires about 9 lines, so it's suitable to open an editor to write while figuring out the solution. Start with writing pseudocode, preferably with pencil and paper. Test by pasting it into REPL.

b) An imperative implementation of `diceMatrixToString` from problem 3g using mutable `StringBuilder`[5] is shown below. Explain how the code works. What happens if `xss` is empty? What happens if `xss` contains only empty vectors? Mention one advantage and one disadvantage of using **`val`** `sb: StringBuilder` and `append`, compared to ordinary immutable **`var`** `s: String` and + for appending at the end.

```
def diceMatrixToString(xss: Vector[Vector[Int]]): String =
  val sb = new StringBuilder()
  for(m <- xss.indices) do
    for(n <- xss(m).indices) do
      sb.append(xss(m)(n).toString)
      if n < xss(m).size - 1 then sb.append(" ")
      else if m < xss.size - 1 then sb.append("\n")
    end for
  end for
  sb.toString
```

c) As practice, make an imperative implementation of `filterYatzy` with a **`for`**-**`do`** loop (i.e., without using `filter` and without using **`yield`**).

d) Explain how the functional implementation of `filterYatzy` below works, using a **`for`**-**`yield`**-expression. Do you find your imperative solution easier or harder to read and understand compared to the functional one below?

```
def filterYatzy(xss: Vector[Vector[Int]]): Vector[Vector[Int]] =
  (for i <- xss.indices if isYatzy(xss(i)) yield xss(i)).toVector
```

Task 6. *String table with column headers.*

a) Implement the case class `Table` according to the specification below. You can assume all rows have as many columns as the number of elements in `headings`, and that all headings in `headings` are unique. The parameter `sep` indicates the character used to separate columns. This is also assumed to hold for input files read with `fromFile`.
Tips:

- The value `indexOfHeading` can be created using the method `zipWithIndex`, which works on all sequence collections, and the method `toMap`, which works on

[5] https://www.scala-lang.org/api/2.12.9/scala/collection/mutable/StringBuilder.html

sequences of 2-tuples. First investigate how these methods work in REPL and read their documentation.

- Create an input file to test that `Table` works correctly.

```
case class Table(
  data: Vector[Vector[String]],
  headings: Vector[String],
  sep: Char
):
  /** A 2-tuple with (number of rows, number of columns) in data */
  val dim: (Int, Int) = ???

  /** The element in row r and column c of data, counting from 0 */
  def apply(r: Int, c: Int): String = ???

  /** The row-vector r in data, counting from 0 */
  def row(r: Int): Vector[String]= ???

  /** The column-vector c in data, counting from 0 */
  def col(c: Int): Vector[String] = ???

  /** A map from heading to index counting from 0 */
  lazy val indexOfHeading: Map[String, Int] = ???

  /** The column-vector with heading h in data */
  def col(h: String): Vector[String] = ???

  /** A vector with the distinct, sorted values of col with heading h */
  def values(h: String): Vector[String] = ???

  /** Headings and data with columns separated by sep */
  override lazy val toString: String = ???

object Table:
  /** Creates a new Table from fileName with columns split by sep */
  def fromFile(fileName: String, sep: Char = ';'): Table = ???
```

b) Create a program using `Table` that can be run from the terminal with `scala run infile.csv ';'` which outputs the count of occurrences of different values in each column (i.e., a form of recording).

Task 7. *Create a yatzy game for terminal use.* Build a simplified yatzy game in the terminal where the user can decide which dice to re-throw. Start with something really simple and then build your game by gradually adding more and more features.

8.2.3 Advanced Tasks; challenges

Task 8. *Generic functions.* A generic function has (at least) one type parameter within square brackets after the name, for example `[T]`. This type then appears as the type of (some of) the parameters in the parameter list. The compiler deduces a concrete type at compile time and replaces the type parameter with this concrete type. This way, a function can work for many types.

a) Explain what happens on each line below.

```
scala> def tnirp[T](x: T): Unit = println(x.toString.reverse)
scala> tnirp(42)
scala> tnirp("hej")
scala> case class Cucumber(weight: Int)
scala> tnirp(Cucumber(42))
scala> tnirp[String](42)
scala> tnirp[Double](42)
```

b) You can combine generic functions with functions that take functions as parameters. This is how `map` and `foreach` are implemented. Explain for each line below what happens.

```
scala> def compose[A, B, C](f: A => B, g: B => C)(x: A): C = g(f(x))
scala> def inc(x: Int): Int = x + 1
scala> def half(x: Int): Double = x / 2.0
scala> compose(inc, half)(42)
scala> compose(half, inc)(42)
```

c) What is the error message on the last line above? Change `inc` and/or `half` so that the types fit.

Task 9. *Generic classes.* Classes can also be generic. A generic class has (at least) one type parameter within square brackets after the class name.

a) Test the generic class `Cell[T]` below in REPL. Create instances of `Cell[T]` where the type parameter `T` is bound to different concrete types, and explain what happens.

```
scala> class Cell[T](var value: T):
         override def toString = "Cell(" + value + ")"

scala> new Cell(42)
scala> new Cell("hej")
scala> new Cell(new Cell(math.Pi))
scala> new Cell[String](42)
scala> new Cell[Double](42)
```

b) Add the method `def concat[U](that: Cell[U]):Cell[String]` to the class `Cell` which concatenates the string representations of the two cell values.

```
scala> val a = new Cell("hej")
scala> val b = new Cell(42)
scala> a concat b
```

c) What kinds of cells can you concatenate if you remove the type parameter name `U` in `concat` while using type `Cell[T]` as the parameter type of `that`? What consequences does this have for cells of types other than `Cell[String]`?

Task 10. *Implement more generic methods in* `Matrix[T]`. Build further on problem 4 and implement the specification below. Create your own tests that verify all methods work as expected.

Specification **Matrix[T]**

```
/** An immutable, generic matrix class. */
case class Matrix[T](data: Vector[Vector[T]]):
  require(???)  // guarantee all rows have equal columns

  /** Returns a pair with number of rows and columns. */
  val dim: (Int, Int) = ???

  /** Returns the element at position (row, col). */
  def apply(row: Int, col: Int): T = ???

  /** Returns a new matrix where the element at (row, col) has the value value. */
  def updated(row: Int, col: Int)(value: T): Matrix[T] =  ???

  /** Applies function f to all elements. */
  def foreach(f: T => Unit): Unit = ???

  /** Applies function f to all indices. */
  def foreachIndex(f: (Int, Int) => Unit): Unit = ???

  /** Returns a new matrix with results of element-wise application of f. */
  def map[U](f: T => U): Matrix[U] = ???

  /** Returns a new matrix with results of applying f to each index. */
  def mapIndex[U](f: (Int, Int) => U): Matrix[U] = ???

  /** Returns a print-friendly string representation of the matrix. */
  override def toString = ???

object Matrix:
  /** Returns a matrix with dimension dim where all elements have value value. */
  def fill[T](dim: (Int, Int))(value: T): Matrix[T] = ???
```

8.3 Lab: `life`

Goals

- ☐ Be able to create and use matrices with the help of a generic data type.
- ☐ Be able to iterate over all elements in a matrix.
- ☐ Practice algorithm construction.
- ☐ Practice handling both immutable and mutable objects.
- ☐ Try using a debugger in an integrated development environment (IDE), such as VS Code.

Preparations

- ☐ Do exercise `Excercise08` in chapter 8, especially task 4.
- ☐ Read through the entire lab and study the provided code[6].
- ☐ Read appendix D about debugging.
- ☐ Download the given code via the course's GitHub repository.

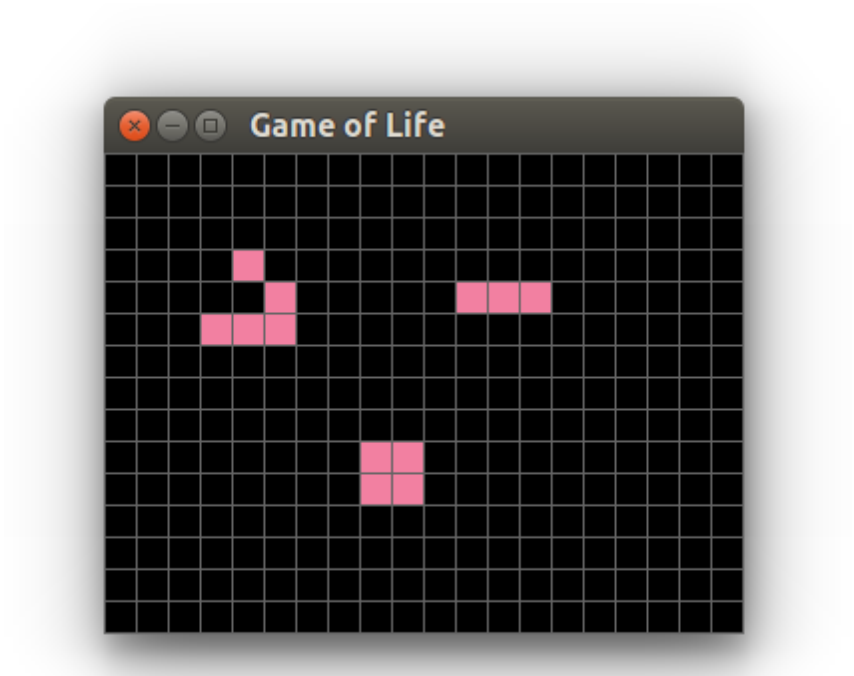


Figure 8.1: A binary, dark data universe of dimension 15×20. The cell colony contains three cell groups: a spaceship of the type *glider*, a *blinker*, and a *block*.

8.3.1 Background

Game of Life simulates a colony of unicellular organisms that live, reproduce, and die in a matrix, according to some simple but well-chosen rules constructed by the mathematician John Horton Conway in the 1970s. The goal of the game is to simulate many generations from a starting configuration, also called a *cell colony*, where the survival of each individual cell depends on its surroundings. The game has no conscious

[6] `https://github.com/lunduniversity/introprog/tree/master/workspace/w08_life`

players, and if the rules are followed, the final result depends solely on the initial configuration.
In *Game of Life*, the universe consists of a matrix with cells that are either alive or dead. Each cell has 8 *neighbors*, consisting of the closest surrounding cells vertically, horizontally, and diagonally. Each cell's state in the next generation is determined by the following rules:

1. **Survival**. If a living cell has two or three neighbors, it survives.
2. **Death**. If a living cell has fewer than two or more than three neighbors, it dies of underpopulation or overpopulation, respectively.
3. **Birth**. If a cell is dead and has exactly three neighbors, it is born and its state changes to living; otherwise it remains dead.

Multiple cell colonies exhibit a "living" behavior where the cell matrix is colonized in interesting ways when a sequence of generations is visualized. This is an example of *emergent* behavior where complex, self-organized structures can arise from simple conditions.
Read more about *Game of Life* on Wikipedia:

- https://en.wikipedia.org/wiki/Conway's_Game_of_Life
- https://sv.wikipedia.org/wiki/Game_of_Life

8.3.2 Mandatory requirements

The following functional requirements must be fulfilled by your program:

- ☐ Living cells shall have the beautiful pink[7] RGB color `(242, 128, 161)`.
- ☐ Dead cells shall be black as space.
- ☐ This dark universe with binary data cells shall be drawn in a grid consisting of thin, stylish lines, as shown in fig. 8.1.
- ☐ Keystrokes and mouse clicks shall function according to the following help text, which shall be printed when the program starts:

```
val help = """
  Welcome to GAME OF LIFE!

  Click on cell to toggle.
  Press ENTER for next generation.
  Press SPACE to toggle play/stop.
  Press R to create random life.
  Press BACKSPACE to clear life.
  Close window to exit.
"""
```

  When *play* is activated with the space key, a continuous simulation of the universe shall continue where each new generation is visualized with an appropriate delay in between generations, until the simulation is stopped, e.g. by pressing the space key again. At each *Enter* press, one subsequent generation is shown and any ongoing simulation is stopped. At mouse click on a cell, the life state shall be toggled from live to dead or vice versa. Pressing R shall produce random life. Pressing the backspace key shall render all universe cells dead.

Your code shall be designed according to these design requirements:

- ☐ All classes and singleton objects shall be in the package `life`.

[7] https://www.dsek.se/aktiva/grafiskprofil/farg.php

- ☐ There shall be an immutable case class `Life` which represents a cell universe using a `Matrix[Boolean]` from task 4 in this week's exercise.
- ☐ There shall be a class `LifeWindow` that visualizes an instance of the class `Life` in an `introprog.PixelWindow` as in fig. 8.1.

8.3.3 Optional requirements – choose at least one

You shall implement at least one (preferably several) of these requirements:

- ☐ Cells shall be colored in different colors according to the rules for the next generation. Survival shall still be beautiful pink and persistent death black. The following colors are suggested but you may choose others if you think they look nicer:

```
val UnderPopulated = java.awt.Color.cyan  // a toxic color
val OverPopulated  = java.awt.Color.red   // red squeezed by crowding
val WillBeBorn     = new java.awt.Color(40, 0, 0)  // soon alive
```

  Also give `LifeWindow` a class parameter `isMultiColor` that allows choosing whether cells shall be multicolored or just pink and black as in the basic requirements.

- ☐ If you press `S` for *Save*, `introprog.Dialog.file("Save Life")` shall be shown and, if the user does not press **Cancel**, the current life shall be saved via `introprog.IO.saveString` to a text file using the `toString` method of `Life`.
- ☐ If you press `O` for *Open*, `introprog.Dialog.file("Open Life")` shall be invoked and a new universe read from a text file according to a suitable format. Reading shall be done with `introprog.IO.loadString` and interpreted into a `Life` instance by a method in the companion object with the following header:

```
def fromString(s: String, rowDelim: String="\n", alive: Char='0'): Life
```

  Test with the file `glider-gun.txt` which should have the following content in the first 11 lines and a total of 32 lines where all lines after the eleventh contain empty life:

```
> head -11 glider-gun.txt
----------------------------------------
-------------------------0----------------
-----------------------0-0----------------
-------------00------00------------00-----
------------0---0----00------------00-----
-00--------0-----0---00-------------------
-00--------0---0-00----0-0----------------
-----------0-----0-------0----------------
------------0---0-------------------------
-------------00-----------------------------
----------------------------------------
```

- ☐ The universe shall be circular, i.e., neighbors at the edge shall be on the opposite side by wrapping the indexing, using modulo arithmetic. Introduce a class parameter `isWrapped` in `Life` and a variable `wrapped: Boolean` in the companion object `Life` that controls whether factory methods create a circular universe or not, so you can easily configure this. *Tip:* You can benefit greatly from using `java.lang.Math.floorMod` in the `apply` method in `Life`; the `floorMod` method handles negative values correctly, see documentation for the `Math` package in JDK8.

- ☐ Read about variants of `Game of Life` on Wikipedia and implement alternative rules which can be selected via configuration through the `args` parameter in `main`.
- ☐ Create a class `LifeStatistics` which, through many simulations, shall find the probability that a random cell colony after n generations is still evolving, respectively is completely dead. No visualization with `PixelWindow` shall be done; only the number of cells alive at generation n and the number of cells changed since generation $n-1$ need to be registered.

8.3.4 Tips and suggestions

1. Here is a suggestion for how you can design the class `Life`:

```
package life

case class Life(cells: Matrix[Boolean]):

  /**Returns true if the cell at location (row, col) is alive, otherwise false.
    * Returns false if the index is outside the bounds of the universe.
    */
  def apply(row: Int, col: Int): Boolean = ???

  /** Sets the status of the cell at location (row, col) to value. */
  def updated(row: Int, col: Int, value: Boolean): Life = ???

  /** Toggles the status of the cell in place (row, col). */
  def toggled(row: Int, col: Int): Life = ???

  /** Counts the number of living neighbors of the cell in (row, col). */
  def nbrOfNeighbours(row: Int, col: Int): Int = ???

  /**Creates a new Life instance with the next generation of the universe.
    * This is done by applying the rule function to the cells.
    */
  def evolved(rule: (Int, Int, Life) => Boolean = Life.defaultRule):Life =
    var nextGeneration = Life.empty(cells.dim)
    cells.foreachIndex( (r,c) =>
      ???
    )
    nextGeneration

  /** Line separated text where 0 is live cell and - is dead cell. */
  override def toString = ???

object Life:
  /** Creating a universe of dead cells. */
  def empty(dim: (Int, Int)): Life = ???

  /** Creates a universe with random life. */
  def random(dim: (Int, Int)): Life = ???

  /** Implements the rules according to Conways Game of Life. */
  def defaultRule(row: Int, col: Int, current: Life): Boolean = ???
```

You will find the method `nbrOfNeighbours` useful when implementing `defaultRule`. When implementing `random`, the method `foreachIndex` in `Matrix[T]` is convenient

to use. If you, as in the suggestion above, let `evolved` take the update rule as a function parameter, it becomes easier to configure the rules in effect and thus easier to create variants of *Game of Life* by introducing new rules in the companion object (see one of the optional tasks with further reference to Wikipedia).

2. Here is a suggestion for how you can design the class `LifeWindow`:

```
package life

import introprog.PixelWindow
import introprog.PixelWindow.Event

object LifeWindow:
  val EventMaxWait = 1 // milliseconds
  var NextGenerationDelay = 200 // milliseconds
  // add more useful constants here, e.g. colors etc.

class LifeWindow(rows: Int, cols: Int):
  import LifeWindow.* // import names from companion object

  var life = Life.empty(rows, cols)
  val window: PixelWindow = ???
  var quit = false
  var play = false

  def drawGrid(): Unit = ???

  def drawCell(row: Int, col: Int): Unit = ???

  def update(newLife: Life): Unit =
    val oldLife = life
    life = newLife
    life.cells.foreachIndex{ ??? }

  def handleKey(key: String): Unit = ???

  def handleClick(pos: (Int, Int)): Unit = ???

  def loopUntilQuit(): Unit = while !quit do
    val t0 = System.currentTimeMillis
    if play then update(life.evolved())
    window.awaitEvent(EventMaxWait)
    while window.lastEventType != PixelWindow.Event.Undefined do
      window.lastEventType match
        case Event.KeyPressed  =>  handleKey(window.lastKey)
        case Event.MousePressed => handleClick(window.lastMousePos)
        case Event.WindowClosed => quit = true
        case _ =>
      window.awaitEvent(EventMaxWait)
    val elapsed = System.currentTimeMillis - t0
    Thread.sleep((NextGenerationDelay - elapsed) max 0)

  def start(): Unit = { drawGrid(); loopUntilQuit() }
```

3. **Test your partial solutions thoroughly**. You avoid hard subsequent errors if you test your partial solutions carefully before proceeding. For example, it is common

to confuse rows and columns, but you will discover this if you test all relevant functions with a rectangular universe where the number of rows and columns differ.

4. **Take advantage of your IDE**. There are many useful features in an integrated development environment (IDE) such as Microsoft VS Code[8] with the Metals extension[9]. Read online about your IDE and learn about things you didn't know that seem useful. Especially look up the list of keyboard shortcuts[10] and learn some shortcuts that can help you speed up common tasks.

5. Study the documentation about the debugger in your IDE.[11] [12]

[8] https://code.visualstudio.com/
[9] https://scalameta.org/metals/docs/editors/vscode
[10] https://code.visualstudio.com/docs/getstarted/keybindings
[11] https://scalameta.org/metals/docs/editors/vscode#running-and-debugging-your-code
[12] https://code.visualstudio.com/docs/editor/debugging

Chapter 9

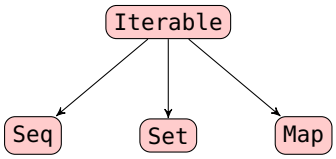
Sets and Maps

Concepts included in this week's studies:

- ☐ membership test
- ☐ set
- ☐ Set
- ☐ mutable.Set
- ☐ key-value table
- ☐ Map
- ☐ mutable.Map
- ☐ hash code
- ☐ java.util.HashMap
- ☐ java.util.HashSet
- ☐ persistence
- ☐ serialization
- ☐ text files
- ☐ Source.fromFile
- ☐ java.nio.file

9.1 Theory

9.1.1 Hierarchy of collection types in `scala.collection` v2.13



`Iterable` has methods implemented using:

```
def foreach[U](f: Elem => U): Unit
def iterator: Iterator[A]
```

`Seq`: ordered in sequence
`Set`: unique elements
`Map`: pairs of (key, value)

The collection **`Vector`** is a `Seq` which is an `Iterable`.
The concrete collections are divided into these packages:

`scala.collection.immutable` where many are **automatically** imported
`scala.collection.mutable` which **must be imported** explicitly
(exception: primitive mutable `scala.Array` is automatically visible)

9.1.2 The method `iterator` gives a "one-time iterator"

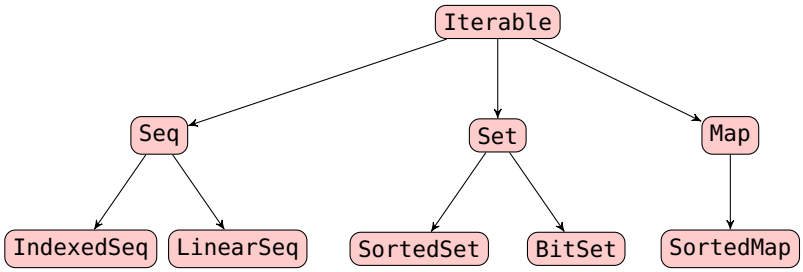
With `iterator` you can iterate with **`while`**, but only **once**; then the iterator is "used up". (But you can request a new one.) Used "under the hood" in the collection library to implement other methods.

```
scala> val xs = Vector(1,2,3,4)
val xs: Vector[Int] = Vector(1, 2, 3, 4)

scala> val it = xs.iterator
val it: Iterator[Int] = <iterator>

scala> while it.hasNext do print(it.next)
1234

scala> it.hasNext
val res0: Boolean = false

scala> it.next
java.util.NoSuchElementException: next on empty iterator
```

Normally you **do not** need to use `iterator`: there are usually ready-made methods that do what you want, for example `foreach`, `map`, `sum`, `min` etc.

9.1.3 More specific collection types in `scala.collection`

There are **more specific subtypes** of `Seq`, `Set` and `Map`:



Vector is an **IndexedSeq** while **List** is an **LinearSeq**.

docs.scala-lang.org/overviews/collections-2.13/overview.html

9.1.4 Some immutable and mutable sequence collections

```
scala.collection.immutable.Seq.
                                    IndexedSeq.
                                                Vector
                                                Range
                                    LinearSeq.
                                                List
                                                Queue
  scala.collection.mutable.Seq.
                                    IndexedSeq.
                                                ArrayBuffer
                                                StringBuilder
                                    LinearSeq.
                                                ListBuffer
                                                Queue
```

Further study: Study collections' performance characteristics here:
docs.scala-lang.org/overviews/collections-2.13/performance-characteristics.html

---

9.1.5 Some useful methods on collections

Iterable	`xs.size`	number of elements
	`xs.head`	first element
	`xs.last`	last element
	`xs.take(n)`	new collection with the first n elements
	`xs.drop(n)`	new collection without the first n elements
	`xs.foreach(f)`	applies `f` to all elements, returns `Unit`
	`xs.map(f)`	applies `f` to all elements, returns new collection
	`xs.filter(p)`	new collection with only the elements where p is true
	`xs.groupBy(f)`	returns a `Map` grouping elements according to f
	`xs.mkString(",")`	a comma-separated string of all elements
	`xs.zip(ys)`	new collection with pairs (x, y); "zip together" xs and ys
	`xs.zipWithIndex`	returns a `Map` with pairs (x, index of x)
	`xs.sliding(n)`	new collection of collections through sliding "window"
Seq	`xs.length`	same as `xs.size`
	`xs :+ x`	new collection with x appended after xs
	`x +: xs`	new collection with x prepended before xs

Try more collection methods from the quick reference.

Memory aid for `+:` and `:+` **Colon on the collection side**
Dig this: https://youtu.be/Lm9JWlEMHjo?si=sNdn_ZDaORlGr3lt

9.1.6 Modify in place in a mutable collection

Some special methods that modify **in place**:

- `xs.addOne(x).addAll(ys)`
- returns `this` so you can chain multiple calls
- `xs.filterInPlace(p)` filters by removing from the existing collection
- `xs.mapInPlace(f)` transforms elements in the existing collection

Example:

```
scala> case class Person(name: String, age: Int)

scala> val xs = collection.mutable.ArrayBuffer.empty[Person]

scala> xs.addOne(Person("Björn", 58)).addAll(Seq(Person("Ken", 25), Person("Kim", 45)))

scala> xs.filterInPlace(_.age > 50).mapInPlace(p => p.copy(name = p.name.reverse))

scala> println(xs)
ArrayBuffer(Person(nröjB,58))
```

9.1.7 Review: What is a sequence?

- A sequence is a **series of elements** that
  - is **numbered** (e.g. from zero), and
  - has a certain **type** (e.g. integer).
- A sequence can contain **duplicates**.
- A sequence can be **empty** and have length zero.
- Example of a non-empty sequence with duplicates:

```
scala> val xs = Vector(42, 0, 42, -9, 0, 13, 7)
val xs: Vector[Int] = Vector(42, 0, 42, -9, 0, 13, 7)
```

- **Indexing** gives an element via its order number:

```
scala> xs(2)
val res0: Int = 42

scala> xs.apply(2)
val res1: Int = 42
```

9.1.8 A string is also an `IndexedSeq[Char]`

Implicit conversion from `String` to `IndexedSeq[Char]` occurs as needed.

```
scala> val x: IndexedSeq[Char] = "hej"
val x: IndexedSeq[Char] = hej
```

This makes **all collection methods on `Seq` also work on strings** and also several other handy string methods are offered **in addition to** those existing in `java.lang.String` via the class `StringOps`.

```
scala> "hej".  //press TAB and see all string methods
JLine: do you wish to see all 248 possibilities (42 lines)?
```

This is a big advantage of Scala compared to many other languages, which have strings that cannot do everything other sequence collections can.

9.1.9 Convert between different collection types

- For commonly used conversions there are methods `toVector`, `toList`, `toArray`, `toBuffer`, `toMap`, `toSeq`, `toIndexedSeq`, `toSet`, `toString`
- The method `to` takes a collection companion object as argument and can be used for conversion to any collection type.
- This requires copying if the underlying representation is different and the collection is mutable.
- Can be used for example to convert between immutable and mutable collections:

```
scala> val ms = Set(1,2,3).to(collection.mutable.Set)
val ms: scala.collection.mutable.Set[Int] = HashSet(1, 2, 3)
```

9.1.10 What is a set?

- A **set** is a collection of **unique** elements of a certain **type**.
- A set cannot contain duplicates:

  ```
  scala> Set(1,1,2,2,3,3,4,4,5,5)
  val res0: Set[Int] = HashSet(5, 1, 2, 3, 4)
  ```

- A set is **not** a sequence: you **cannot** assume the elements are in any particular order, e.g. the order given at construction; a set has no length, but a **size**; the method `size` gives the number of elements but the method `length` is missing.
- A set can be **empty** and has size `0`.
- You can iterate through elements in **some** order (exact order is not defined), for example with `xs.map(f)` or `for (x <- xs) yield f(x)`
- It is **not** possible to index a set with `apply`, which instead provides a **membership test**: `Set(1,2,3).apply(3) == true`
- A set `Set[T]` with elements of type `T` can thus be seen as a **predicate for membership test**: i.e. a function `T => Boolean` which is `true` if the element exists else `false`

9.1.11 Immutable set

- **Create**:

```
scala> var xs = Set("cucumber", "tomato", "banana", "penguin")
```

- **Read**: test membership

```
scala> xs("cucumber")
val res1: Boolean = true
```

- **Update**: add element (nothing happens if already exists)

```
scala> xs = xs + "groundhog" // a new, partially modified
```

- **Remove**: (if exists, else nothing happens)

```
scala> xs = xs - "cucumber" // a new, partially modified
```

`CRUD` = Create, Read, Update, Delete

9.1.12 The mystery of the vanished elements

What happens here?

```
scala> val xs1 = Vector(1,2,3,4,5,6)
scala> xs1.map(_ % 2).count(_ == 0)
val res0: Int = 3                          // number of even numbers
scala> val xs2 = Set(1,2,3,4,5,6)
scala> xs2.map(_ % 2).count(_ == 0)
val res1: Int = 1                          // why?
```

Set properties imply that `xs2.map(_ % 2) == Set(0, 1)`
Always think carefully if you **risk losing duplicates** you actually wanted to keep!
Use `toSeq` on a set if you need sequence properties:

```
scala> xs2.toSeq.map(_ % 2).count(_ == 0)
val res1: Int = 3         // with toSeq it behaves as intended
```

9.1.13 Mutable set

With a **mutable** set you can incrementally extend in place.

```
scala> val set = scala.collection.mutable.Set.empty[Int]

scala> for i <- 1 to 1_000_000 do set.addOne(i)

scala> set.contains(-1)   // same as set(-1) or set.apply(-1)
```

A **set** is **fast** to check if an element **exists or not** in the set. No linear search is needed because the clever implementation of the data structure allows fast lookup of an element.
But in a sequence a linear search is required to test membership:

```
scala> val sequence = (1 to 1_000_000).toVector

scala> sequence.contains(-1)   // requires linear search all the way to the end
```

Exercise: Measure the time difference yourself using:

```
def nanos(b: => Unit) = { val t0 = System.nanoTime; b; System.nanoTime - t0 }
```

9.1.14 Special methods on mutable sets

Mutable sets have methods that modify in place:

```
scala> val s = scala.collection.mutable.Set.empty[Int]

scala> s.addOne(1)     // also known as += if you prefer operator notation
val res0: scala.collection.mutable.Set[Int] = HashSet(1)

scala> s.addOne(2).addOne(3).addOne(3).addOne(42) // addOne returns this
val res1: scala.collection.mutable.Set[Int] = HashSet(1, 2, 3, 42)

scala> res0.eq(res1)  // same instance of mutable.Set (no new created)
val res2: Boolean = true

scala> s.addAll(Vector(3, 4, 5))  // also ++= if you like operator notation
val res3: scala.collection.mutable.Set[Int] = HashSet(1, 2, 3, 4, 5, 42)

scala> s.subtractOne(1).subtractAll(List(1,2,3))  // also -= and --=
val res4: scala.collection.mutable.Set[Int] = HashSet(4, 5, 42)

scala> s.filterInPlace(_ > 4)
val res5: scala.collection.mutable.Set[Int] = HashSet(5, 42)
```

Methods `addOne`, `addAll`, `subtractOne`, `subtractAll`, `filterInPlace` return `this` so you can modify in place with **chained calls** using dot notation.

9.1.15 What is a key-value table?

- A **key-value table** is a collection of **pairs** with (**key**, **value**).
- Such a table can be created from a sequence of pairs `(k, v)`
  where `k` is a key and `v` is a value:

```
scala> val age = Map("Björn" -> 42, "Sandra" -> 35, "Kim" -> 19)
val age: Map[String, Int] = Map(Björn -> 42, Sandra -> 35, Kim -> 19)
```

- `Map` has **two** type parameters: `K` for the key type and `V` for the value type.
- The table's keys form a set given by the method `keySet`;
  keys are **unique**.

- Elements do **not** form a sequence and have no special order;
  a key-value table has no length but a **size**;
  method `size` returns the number of elements.
- A table can be seen as a lookup function (eng. *dictionary*):
  that is a function `K => V` that gives a value given a key.

9.1.16 The amazing key-value table `Map`

- A **key-value table** is a kind of generalized vector where you can "index" with an arbitrary type.
- Also called **hash table**, **dictionary** or **map** (which can easily be confused with the method `map`).
- If you know the key, you can look up the value **quickly**, similar to how an index into a vector with an integer index works fast.
- This data structure is **very useful** and works as a kind of database together with filtering, registration, etc.

9.1.17 Immutable key-value table

- **Create**: give pairs to the `apply` method

```
scala> var birth = Map("C" -> 1972,  "C++" -> 1983, "C#" -> 2000,
  "Scala" -> 2004, "Java" -> 1995, "Javascript" -> 1995, "Python" -> 1991)
```

- **Read**: look up a value by key

```
scala> val year = birth.apply("Scala")
val year: Int = 2004
```

- **Update**: add a pair, replace a pair

```
scala> birth = birth + ("Kotlin" -> 2011)
birth: Map[String, Int] = HashMap(Scala -> 2004, C# -> 2000, Python -> 1991,
Javascript -> 1995, C -> 1972, C++ -> 1983, Kotlin -> 2011, Java -> 1995)
```

- **Remove** a pair by key (if exists, else nothing happens)

```
scala> birth = birth - "Python"
birth: Map[String, Int] = HashMap(Scala -> 2004, C# -> 2000,
Javascript -> 1995, C -> 1972, C++ -> 1983, Kotlin -> 2011, Java -> 1995)
```

9.1.18 More examples of key-value table

Some frequently used methods on tables:

- `xs.keySet` returns a set of all keys

- `xs.map(f)` applies the function f to all pairs (key, value) in **some** order
- `xs.map((k, v) => k -> f(v))` applies function f to all **values**

```
scala> val color = Map("cucumber" -> "green", "tomato"->"red", "eggplant"->"purple")
val color: Map[String, String] =
  Map(cucumber -> green, tomato -> red, eggplant -> purple)

scala> color("cucumber")
val res0: String = green

scala> color.keySet
val res1: Set[String] = Set(cucumber, tomato, eggplant)

scala> val isGreenThing = color.map((k,v) => (k, v == "green"))
val isGreenThing: Map[String,Boolean] =
  Map(cucumber -> true, tomato -> false, eggplant -> false)

scala> val reversedColor = color.map((k, v) => k -> v.reverse)
val reversedColor: Map[String,String] =
  Map(cucumber -> neerg, tomato -> der, eggplant -> elprup)
```

9.1.19 From sequence of pairs to table

```
scala> val xs = Vector(("Kim",42), ("Pam", 42), ("Kim", 50), ("Pam", 50))
val xs: Vector[(String, Int)] =
  Vector((Kim,42), (Pam,42), (Kim,50), (Pam,50))

scala> xs.toMap
val res0: Map[String, Int] =
  Map(Kim -> 50, Pam -> 50) // no duplicate keys

scala> val groupedByName = xs.groupBy(_._1)
groupedByName: Map[String,Vector[(String, Int)]] =
  Map(Kim -> Vector((Kim,42), (Kim,50)), Pam -> Vector((Pam,42), (Pam,50)))

scala> val groupedByAge = xs.groupBy(_._2)
groupedByAge: Map[Int,Vector[(String, Int)]] =
  Map(50 -> Vector((Kim,50), (Pam,50)), 42 -> Vector((Kim,42), (Pam,42)))
```

9.1.20 Exercise: Implement a `Multimap`

- If you add a value to a *regular* `Map`, the value gets replaced:

  ```
  scala> val m = Map(1 -> 2, 1 -> 3, 2 -> 1, 2 -> 2)
  val m: Map[Int, Int] = Map(1 -> 3, 2 -> 2)   // the latest value applies
  ```

  ...but sometimes we want to store all added values.
- A **multimap** is a special key-value table where values are a collection (often a set).
- A multimap collects all values that have the same key.

```
scala> val mm = Multimap(1 -> 2, 1 -> 3, 2 -> 1, 2 -> 2)
val mm: Multimap[Int, Int] = Multimap(1 -> Set(2, 3), 2 -> Set(1, 2))
```

Exercise: Implement a multimap that works as above, using a case class with the attribute `toMap` which is an immutable key-value table where values are a set.
Tip: Use `groupBy`

9.1.21 Exercise: `Multimap`

```
case class Multimap[K, V] private (toMap: Map[K,Set[V]]):
  def apply(k: K): Set[V] = ???

  def +(kv: (K, V)): Multimap[K, V] = kv match
    case (k, v) if toMap.isDefinedAt(k) => ???
    case (k, v) => ???

  override def toString = toMap.mkString("Multimap(", ", ")")

object Multimap:
  def apply[K, V](kvs: (K,V)*): Multimap[K, V] =
    new Multimap(???)
```

9.1.22 Solution: `Multimap`

```
case class Multimap[K, V] private (toMap: Map[K,Set[V]]):
  def apply(k: K): Set[V] = toMap(k)

  def +(kv: (K, V)): Multimap[K, V] = kv match
    case (k, v) if toMap.isDefinedAt(k) => Multimap(toMap.updated(k, toMap(k) + v))
    case (k, v) => Multimap(toMap + (k -> Set(v)))

  override def toString = toMap.mkString("Multimap(", ", ")")

object Multimap:
  def apply[K, V](kvs: (K,V)*): Multimap[K, V] =
    new Multimap(kvs.groupBy(_._1).map((k,xs) => k -> xs.map(_._2).toSet))
```

9.1.23 Special methods on mutable map

Both `Set` and `Map` exist in **mutable** variants with extra methods for in-place updates without creating new collections.

```
scala> import scala.collection.mutable

scala> val mm = mutable.Map.empty[String, String]
val mm: scala.collection.mutable.Map[String, String] = HashMap()
```

```
scala> mm.addOne("hej" -> "svejs")
val res0: scala.collection.mutable.Map[String, String] =
  HashMap(hej -> svejs)

scala> mm.addAll(Seq("abra" -> "kadabra", "ada" -> "lovelace"))
val res1: scala.collection.mutable.Map[String, String] =
  HashMap(hej -> svejs, abra -> kadabra, ada -> lovelace)

scala> mm("abra")
val res2: String = kadabra
```

The method ++= is the same as addAll; often used with operator notation:
`mm ++= Seq("hej" -> "san", "abra" -> "kada", "bra" -> "scala")`

9.1.24 Further reading: groupMap, groupMapReduce

Useful but advanced methods that run in **a single** iteration:

- `xs.groupMap(f)(g)`
  returns Map grouped according to f with values computed according to g
- `xs.groupMapReduce(f)(g)(h)`
  returns Map after a groupMap(f)(g) which is reduced according to h
- Read about `groupMap` here:
  https://www.scala-lang.org/blog/2018/06/13/scala-213-collections.html
- See visualization of `groupMapReduce` here:
  https://superruzafa.github.io/visual-scala-reference/groupMapReduce/

9.1.25 Exercise: Mutable locally, return immutable

If you want to implement an imperative algorithm with a mutable collection:
Prefer to do this **locally** in a **mutable** collection and then return an **immutable** collection, by running e.g. `toSet`, `toMap`, `toVector` on the mutable collection.

Example where the solution benefits from local mutation in place:

```
def rollDiceUntilAllOutcomesExceptOne(sides: Int = 6): (Int, Set[Int]) = ???
```

```
scala> rollDiceUntilAllOutcomesExceptOne()
val res0: (Int, Set[Int]) = (13,HashSet(5, 1, 6, 2, 3))
```

9.1.26 Exercise: Mutable locally, return immutable

```
def rollDiceUntilAllOutcomesExceptOne(sides: Int = 6): (Int, Set[Int]) =
  /*
    let s be an empty mutable integer set
    let n be zero
```

```
    while the size of s is less than sides - 1 do:
      add a new dice roll to s
      update n to count how many random numbers drawn
  */
  (n, s.toSet)   // note toSet which gives an immutable set
```

```
scala> rollDiceUntilAllOutcomesExceptOne()
val res0: (Int, Set[Int]) = (13,HashSet(5, 1, 6, 2, 3))
```

9.1.27 Solution: Mutable locally, return immutable

```
def rollDiceUntilAllOutcomesExceptOne(sides: Int = 6): (Int, Set[Int]) =
  val s = scala.collection.mutable.Set.empty[Int] // mutable locally
  var n = 0
  while s.size < sides - 1 do
    s.addOne(util.Random.nextInt(sides) + 1)
    n += 1
  (n, s.toSet)   // note toSet which gives an immutable set
```

```
scala> rollDiceUntilAllOutcomesExceptOne()
val res0: (Int, Set[Int]) = (13,HashSet(5, 1, 6, 2, 3))
```

In this week's assignments this is used in a so-called **builder**: First build up a mutable structure step by step in `FreqMapBuilder`, and then, when all additions are made, transition to an immutable structure `Map[String, Int]`.

9.1.28 The method `sliding`

The method `sliding(n)` creates, with a "sliding window," a sequence of subsequences of length `n` by "sweeping the window" from start to end:

```
scala> val xs = "five ants are more than four elephants".split(' ').toVector
val xs: Vector[String] = Vector(five, ants, are, more, than, four, elephants)

scala> xs.sliding(2).toVector
val res0: Vector[Vector[String]] =
  Vector(Vector(five, ants), Vector(ants, are), Vector(are, more),
      Vector(more, than), Vector(than, four), Vector(four, elephants))

scala> xs.sliding(3).toVector
val res1: Vector[Vector[String]] =
  Vector(Vector(five, ants, are), Vector(ants, are, more),
    Vector(are, more, than), Vector(more, than, four),
      Vector(than, four, elephants))
```

This method will be useful in this week's lab!
(see more examples in the exercises)

9.1.29 Methods zipWithIndex, groupBy

```
scala> val cards = Vector("Jack", "Queen", "King", "Ace")

scala> val cardIndex = cards.zipWithIndex.toMap
cardIndex: Map[String,Int] = Map(Jack -> 0, Queen -> 1, King -> 2, Ace -> 3)

scala> cardIndex("King") > cardIndex("Jack")
res0: Boolean = true

scala> cardIndex.map(p => p._1 -> (p._2 + 11))

scala> val diceRolls = Vector(1,2,3,4,5,6,2,4,6)

scala> val groupedGreaterThanFour = diceRolls.groupBy(_ > 4)
groupedGreaterThanFour: Map[Boolean,Vector[Int]] =
  Map(false -> Vector(1, 2, 3, 4, 2, 4), true -> Vector(5, 6, 6))

scala> val groupedEqual = diceRolls.groupBy(x => x)
groupedEqual: Map[Int,Vector[Int]] = Map(5 -> Vector(5), 1 -> Vector(1),
  6 -> Vector(6, 6), 2 -> Vector(2, 2), 3 -> Vector(3), 4 -> Vector(4, 4))

scala> val frequency = diceRolls.groupBy(x => x).map((k,v) => k -> v.size)
frequency: Map[Int,Int] = Map(5 -> 1, 1 -> 1, 6 -> 2, 2 -> 2, 3 -> 1, 4 -> 2)
```

9.1.30 More useful collection methods

Example to practice: count letters in words.
Examine what happens in REPL:

```
val words = "six laxars in a salmon box seven seasick sailors"
val splitWords = words.split(' ').toVector
val wordLengths = splitWords.map(_.length)
val wordLengthMap = splitWords.map(s => (s, s.size)).toMap
val groupedByFirstLetter = splitWords.groupBy(s => s(0))
val letters = words.toVector.filter(_ != ' ')
val countX = letters.count(_ == 'x')
val grouped = letters.groupBy(ch => ch)
val count = grouped.map(p => p._1 -> p._2.size)
//same as above but uses "parameter untupling":
val count2 = grouped.map((k,v) => k -> v.size)
val sorted = count.toVector.sortBy(_._2)
val mostCommon = count.maxBy(_._2)
```

9.1.31 Serialization and Deserialization

- To **serialize** means to **encode objects** in memory into a decodable **sequence of symbols**, which can be stored e.g. in a file on your hard drive.

- To **deserialize** means to **decode a sequence of symbols**, e.g. from a file, and **recreate objects** in memory.

9.1.32 Reading text from file and webpage

In the package `scala.io` there is the singleton object `Source` with the methods:

- `fromFile` for reading from a file
- `fromUrl` for reading from a URL (Universal Resource Locator) `http://some.site.domain/page.html`

```
def readFile(filename: String, encoding: String = "UTF-8"): String =
  val s = scala.io.Source.fromFile(filename, encoding)
  try s.mkString finally s.close // ensure closing even on crash

def readLinesFromFile(filename: String, encoding: String = "UTF-8"): Vector[String] =
  val s = scala.io.Source.fromFile(filename, encoding)
  try s.getLines.toVector finally s.close

def readWebpage(url: String, encoding: String = "UTF-8"): String =
  val s = scala.io.Source.fromURL(url, encoding)
  try s.mkString finally s.close

def readLinesWebpage(url: String, encoding: String = "UTF-8"): Vector[String] =
  val s = scala.io.Source.fromURL(url, encoding) // read with given character encoding
  try s.getLines.toVector finally s.close
```

See also this week's exercise. Example of another character encoding: `"ISO-8859-1"`

9.1.33 Serialization in the module `introprog.IO`

- In the course's code library `introprog` there is a singleton object `IO` that gathers handy functions for serialization and deserialization.
- See API documentation here:
  https://fileadmin.cs.lth.se/pgk/api
  Search for IO and click on the singleton object.
- See the source here:
  https://github.com/lunduniversity/introprog-scalalib/blob/master/src/main/scala/introprog/IO.scala
- If you want, you are welcome to use `introprog.IO` instead of `scala.io.Source` in the lab.

9.2 Exercise Excercise09

Goals

- ☐ Be able to create and use tuples as parameters and return values.
- ☐ Know and be able to use basic methods on collections.
- ☐ Be able to create and use both immutable and mutable sets.
- ☐ Understand the differences and similarities between a set and a sequence.
- ☐ Be able to describe how the linear search algorithm works.
- ☐ Be able to create and use both immutable and mutable key-value tables.
- ☐ Be able to use key-value tables to implement registration.
- ☐ Understand similarities and differences between a key-value table and a sequence.
- ☐ Be able to save and read data to/from text files on disk.

Preparations

- ☐ Study the concepts in chapter 9

9.2.1 Basic Tasks; preparation for the lab

Task 1. *Match concepts with description.*

Match each concept with the (simplified) description that fits best:

set	1	A	search sequence until criterion met
key-value table	2	B	decode symbol sequence and recreate object in memory
mapping	3	C	a unique identifier
key	4	D	property of remaining after program ends
persistence	5	E	encode object into decodable sequence of symbols
serialize	6	F	unordered collection of mappings with unique keys
deserialize	7	G	`key -> value`
linear search	8	H	unordered collection with unique elements

Task 2. *What is a set?* Explain what happens below. Why do the elements end up in a ”strange” order? Why does one element ”disappear”?

```
scala> val xs = Vector(1,2,3,1,2,3,4,5,7).toSet
xs: scala.collection.immutable.Set[Int] = Set(5, 1, 2, 7, 3, 4)
scala> xs.foreach(print)
512734
```

Task 3. *Using sets.*

Match each expression on the left with an expression on the right that has the same value:

`Set(1, 2) ++ Set(1, 2)`	1	A	`3`
`(1 to 3).toSet`	2	B	`Set(3)`
`Vector.fill(3)(1).toSet`	3	C	`6`
`Set(1, 2, 3) diff Set(1, 2)`	4	D	`error: ...`
`(1 to 7).toSet.apply(8)`	5	E	`true`
`Set(1, 2, 3).sorted`	6	F	`Set(1, 2) - 2`
`Set(2,4) subsetOf (1 to 7).toSet`	7	G	`Set(1) + 2 + 3`
`Set(1, -1, 2, -2).map(_.abs).sum`	8	H	`false`
`Set(1, 1, 1, 1, 1, 5).sum`	9	I	`Set(1, 2)`

Task 4. *Count unique words using a set.* In this week's lab, we will perform automatic language processing of long texts which we split into words. With the method `s.split(' ').toVector` you can split a string `s` into a sequence of words, where `s` is split into many strings at each space character and all spaces are removed.

a) Use the methods `split` and `toSet` to create an expression that calculates how many unique words there are in the string `hej` below:

```
scala> val hej = "hej hej hemskt mycket hej"
```

b) Sets are fast at checking if an element is in the set, but you cannot expect the elements to be in any particular order. There is a sequence collection method that produces a sequence with unique elements from a sequence and preserves the original order. What is the method called?
Tip: Look in the quick reference or search online. The method works on all collections that are of type `Seq` and has a name that starts with the letters `di`.

Task 5. *Creating 2-tuples with the method `->` which can be read as "maps to".* We have previously seen how two different values can be collected in a 2-tuple, for example `(0, true)`. Pairs can also be created using the method `->` as shown below. Try this in REPL:

```
scala> ("Skåne", "Lund")          // a string pair with a regular 2-tuple
scala> "Skåne" -> "Lund"           // operator notation with ->
scala> "Skåne".->("Lund")         // dot notation with -> (not very common)
```

The method `->` works with all types and is a factory method for pairs. The method name resembles a right arrow and illustrates a mapping from the first to the second value.

a) Does it work on pairs created with `->` to use the methods `_1` and `_2`?

b) Declare a variable `val huvudstad: Vector[(String, String)]` that contains mappings between geographical areas and their capitals according to the table below.

Sweden	Stockholm
Denmark	Copenhagen
Greenland	Nuuk
Skåne	Lund

c) Write an expression that extracts "Lund" from huvudstad.

Task 6. *Linear search for key in sequence of mappings.*

a) Implement the function lookupIndex below using the collection method indexWhere so that a linear search is performed for the index of a pair in the sequence where key is in the first position of the pair.

```
def lookupIndex(xs: Vector[(String, String)])(key: String): Int = ???
```

b) Test your function in REPL by looking up the index for Skåne's capital in the sequence huvudstad from the previous task.

Task 7. *Key-value table.* A key-value table is a smart data structure that allows you to look up the value that a key maps to *without* the need for linear search. The value is retrieved directly in constant time, i.e., the time to look up a value does *not* depend on the number of elements in the collection, but happens with very little delay.
In Scala, key-value tables are called Map with a capital M, and are practical to use in many different contexts. Map exists in both an immutable and a mutable variant. There are easy methods of the form toXXX to convert between a Map and a sequence of pairs of type XXX[(KeyType, ValueType)].

a) Declare the map telnr below in REPL and use apply to find the phone number to Fröken Ur.

b) What is the type of telnr?

c) What is the type of telnr.toVector?

```
val telnr = Map(
  "Anna"     -> 46462229812L,
  "Björn"     -> 46462229009L,
  "Sandra"    -> 46462220368L,
  "Fröken Ur" -> 4690510L,
)
```

A set of Map instances, nested if necessary, can advantageously be used to build an in-memory database where built-in collection methods, e.g. map, filter, and **for-yield** expressions, provide flexible and efficient search capabilities. You will do this in this week's lab.
The collection Map is a generalization of a sequence, where you can "index", not only by an integer, but with any type of value, e.g., a string. The Map data structure is also called an *associative array*[1] and is implemented as a so-called *hash table*[2], but you will have to wait until the advanced course to go through how such a data structure is implemented.

Task 8. *Using key-value table.*

a) Create the variables below in REPL.

```
val follow = for i <- 2 to 16 by 2 yield (i, i + 1)
val xs = follow.toMap
val ys = xs.toVector
```

[1] https://en.wikipedia.org/wiki/Associative_array
[2] https://en.wikipedia.org/wiki/Hash_table

Do the mappings in `ys` appear in the same order as `follow`? Why?

b) With `xs` and `ys` declared in REPL as above, match expressions on the left with the correct result on the right. If you are unsure about complex expressions, try simpler expressions in REPL and examine the value and type of partial results.

`xs(2) + xs(4)`	1	A	`8`
`ys(0)`	2	B	`7`
`xs(0)`	3	C	`(10, 11)`
`(xs + (0 -> 1)).apply(0)`	4	D	`1`
`xs.keySet.apply(2)`	5	E	`(16, 17)`
`xs isDefinedAt 0`	6	F	`-9`
`xs.getOrElse(0, 7)`	7	G	`true`
`xs.maxBy(_._2)`	8	H	`false`
`xs.map(p => p._1 -> -p._2)(8)`	9	I	`NoSuchElementException`

Task 9. *Registration in mutable key-value table.* In this task, you will implement a helper class for registration in a frequency table which you will then use in this week's lab. The class should be called `FreqMapBuilder` that, after repeated calls to the method `add(s: String): Unit`, can create frequency tables of type `Map[String, Int]`, where the key-value pairs in the table indicate the number of occurrences of a certain string. You should start from the code below.
The class uses a mutable table internally. After having added many strings, you can obtain an immutable table for lookup of frequencies for specific strings with the method `toMap`. Read in the quick reference about which extra update methods are offered by `mutable.Map[K, V]`.

```
class FreqMapBuilder:
  private val register = collection.mutable.Map.empty[String, Int]
  def toMap: Map[String, Int] = register.toMap
  def add(s: String): Unit = ???

object FreqMapBuilder:
  def apply(xs: String*): FreqMapBuilder = ???
```

Implement and test `FreqMapBuilder`. *Tip:* You can for example use `mutable.Map`'s methods `addOne` and `getOrElse`.

Task 10. *The method `sliding`.* In this week's lab, you will find the method `sliding` useful, which gives an iterator for special subsequences of a sequence, which can be likened to the "view" in a "sliding window".

a) Run the code below in REPL and describe what happens.

```
scala> val xs = Vector("five", "cucumbers", "is", "more", "than", "four", "toma
scala> xs.sliding(2).toVector
scala> xs.sliding(3).toVector
scala> xs.sliding(10).toVector
```

b) Use `xs.sliding(2)` and convert each element in the result to a pair. Then convert the sequence of pairs into a key-value table. What can the table be used for?

Task 11. *Reading text from file and web servers.* In the lab, you will build tables from data in text format. Then you will find it useful to be able to read text from files and from the web. Test this in REPL:

```
scala> val url = "https://fileadmin.cs.lth.se/pgk/europa.txt"
scala> val xs = io.Source.fromURL(url, "UTF-8").getLines.toVector
scala> val data = xs.map(_.split(';').toVector)
scala> data.head
scala> data.foreach(println)
```

a) Create these tables from the sequence `data`:

```
val populationOf: Map[String, Int]    = ???  // countries' population
val sizeOf:       Map[String, Int]    = ???  // countries' area in km^2
val capitalOf:    Map[String, String] = ???  // countries' capitals
```

Test the tables in REPL.

b) Save the data in a text file `europa.txt`. Read the data from the file with the method
`Source.fromFile(filename, encoding)` in a similar way as with `fromURL` above. If you run in a Linux terminal you can easily download a file as follows (note that the option `-sLO` has a capital letter `O` and not a zero):

```
> curl -sLO https://fileadmin.cs.lth.se/pgk/europa.txt
```

Print all lines in `europa.txt` using `Source.fromFile` in REPL.

9.2.2 Extra Tasks; additional practice

Task 12. *Create a text game using tables.* Make a simple game to practice various facts about Europe's countries and capitals by reading data from the URL:
https://fileadmin.cs.lth.se/pgk/europa.txt
There is text encoded in UTF-8 with the following content (only the first lines are shown):

```
Land;Invånarantal;Storlek(km^2);Huvudstad
Albanien;3581655;28748;Tirana
Andorra;71201;468;Andorra la Vella
Belgien;10584534;30528;Bryssel
Bosnien-Hercegovina;4590310;51129;Sarajevo
Bulgarien;7385367;110910;Sofia
Cypern;854000;9250;Nicosia
Danmark;5475791;43094;Köpenhamn
Estland;1324333;45226;Tallinn
Finland;5315280;338145;Helsingfors
Frankrike;61538322;551695;Paris
Färöarna;48344;139574;Torshamn
Grekland;10964021;131940;Aten
// ... etcetera for all European countries.
```

Let for example the user answer random questions of the type:

- Does Andorra have more inhabitants than Cyprus?

- What is the capital of Bulgaria?
- Does Denmark have a larger area than Finland?

Use immutable tables with suitable keys and values. You can use a set with countries/-capitals that the user has answered correctly so far to prevent these from recurring again.

9.2.3 Advanced Tasks; challenges

Task 13. *Registration with `groupBy`.* We will now use a really clever trick to implement registration using just one line of code with the collection methods `groupBy` and `map`.

a) Read about the method `groupBy` in the quick reference. You find it under the heading *"Methods in trait `Iterable[A]`"* since `groupBy` works on all collections. Test `groupBy` as below and describe what happens.

```
scala> val xs = Vector(1, 1, 2, 2, 4, 4, 4).groupBy(x => x > 2)
scala> val ys = Vector(1, 1, 2, 2, 4, 4, 4).groupBy(x => x)
```

b) Create a function `freq` with the function header below that returns a table with the count of occurrences of different integers in `xs`. Test `freq` on a sequence of 1000 random dice throws and explain how the function `freq` works. *Tip:* Do first `groupBy(???)` then `map(???)`.

```
def freq(xs: Vector[Int]): Map[Int, Int] = ???

def toss(n: Int): Vector[Int] =
  Vector.fill(n)(scala.util.Random.nextInt(6) + 1)
```

Task 14. *Iterate at once with `groupMap` and `groupMapReduce`.* A student at LTH has created the following shopping list:

```
enum Department:
  case Vegetables, Fruit, Dairy, Meat, Grain

case class Grocery(product: String, department: Department, price: Int)
val shoppinglist = List(
    Food("tomato", Department.Vegetables, 10),
    Food("ground beef", Department.Meat, 60),
    Food("cucumber", Department.Vegetables, 5),
    Food("banana", Department.Fruit, 20),
    Food("milk", Department.Dairy, 20),
    Food("sausage", Department.Meat, 50),
    Food("apple", Department.Fruit, 30),
    Food("orange", Department.Fruit, 40),
  )
```

To make a rational division of the list and calculate costs, the student studies the methods `groupMap` and `groupMapReduce` carefully here:
https://superruzafa.github.io/visual-scala-reference/

a) Use the function `groupMap` to split the groceries based on the department where they can be found.

b) Use the function `groupMapReduce` to find out what the total costs will be for each department.

Task 15. *Writing to file.* As an aid when you create your own interesting applications or build on the course's labs and exercises with optional extra tasks, you can use the functions in the singleton object `IO` below, which exists in the course's scala library introprog.[3]
The IO module uses `scala.io.Source` to serialize and deserialize strings to and from ordinary text files. The IO module also uses the package `java.io` to offer functions that make it easy to serialize/deserialize arbitrary objects created by serializable classes to/from binary files. Case classes in Scala become automatically serializable. In the implementation of `IO`, `try ... finally` is used to ensure that files are not left open even if something goes wrong during the read/write process that is handled by the underlying operating system.

a) Compile and run the code below with `introprog` on the classpath, e.g. using `sbt`.

```
import introprog.IO

case class Player(name: String)

@main def run(): Unit =
  println("Test of output/input objects to/from disk:")
  val highscores = Map(Player("Sandra") -> 42, Player("Björn") -> 5)
  IO.saveObject(highscores,"highscores.ser")
  val highscores2 = IO.loadObject[Map[Player, Int]]("highscores.ser")
  val isSameContents = highscores2 == highscores
  val testResult = if (isSameContents) "SUCCESS :)" else "FAILURE :("
  println(testResult)
```

b) Use the `IO` module to save the user's score results in your game about Europe's countries and cities, in the extra task above. The implementation of `introprog.IO` can be found here: `https://github.com/lunduniversity/introprog-scalalib/blob/master/src/main/scala/introprog/IO.scala`

[3]The source code can be found here and also on page **??**:
https://github.com/lunduniversity/introprog-scalalib/blob/master/src/main/scala/introprog/IO.scala

9.3 Lab: `words`

Goals

- ☐ Be able to use key-value tables with the collection `Map`.
- ☐ Be able to use sets with the collection `Set`.
- ☐ Understand the differences and similarities between a sequence and a set.
- ☐ Understand the differences and similarities between a sequence of pairs and a key-value table.
- ☐ Be able to implement algorithms that use nested structures.
- ☐ Be able to combine mutable and immutable collections.

Preparations

- ☐ Do exercise `Excercise09` in section 9.2
- ☐ Read through the entire laboratory assignment.
- ☐ Fetch and read the given code from the course's GitHub page.

9.3.1 Background

This assignment deals with analysis of natural language (eng. *Natural Language Processing, NLP*). Language analysis often relies on statistics about the occurrence of different words in long texts. You will write code that, based on a long text, for example a book, can help you answer this type of questions:

- How common is a certain word in a given text?
- What is the most common word that follows a certain word?
- How can one generate word sequences that resemble the word order in a given text?

To be able to answer such questions, you will create frequency tables and also so called *n-grams*; sequences of n words that occur consecutively in a text. Examples of some 2-grams (also called *bigrams*) found in the previous sentence: (för, att), (att, kunna), (kunna, svara), (svara, på), (på, sådana), and so on.[4]

9.3.2 Mandatory tasks

You shall build your program with an editor, e.g. VS `code`, and compile and run your code in the terminal using `scala-cli` in *watch mode* with the workflow described in appendix F section F.2.1. While you develop your program step by step, you should simultaneously experiment in the REPL to explore how to use collection methods to solve the tasks. Code to start from is here: `https://github.com/lunduniversity/introprog/tree/master/workspace/w09_words`
These incomplete code files are in the package `nlp`:

- `FreqMapBuilder.scala` contains a skeleton for a class to, word by word, build a key-value table that records the number of occurrences of different words. Implementing this was part of the exercise you did earlier this week.

- `Text.scala` contains a skeleton for a class that can do text processing by analyzing words in a text.

[4] You can investigate various n-grams in a large collection of books using Google's n-gram viewer: `https://books.google.com/ngrams/`

- `Main.scala` contains a partial main program that you can use in the later part of the lab.

Task 1. *Create frequency tables.* You shall use `FreqMapBuilder` from this week's exercise to create frequency tables of type `Map[String, Int]`, where the key-value pairs in the table denote the number of occurrences of a certain string.

a) Put the class `FreqMapBuilder` in a package called `nlp` and compile.

```scala
package nlp

class FreqMapBuilder:
  private val register = collection.mutable.Map.empty[String, Int]
  def toMap: Map[String, Int] = register.toMap
  def add(s: String): Unit = ???

object FreqMapBuilder:
  /** Create new FreqMapBuilder and count the strings in xs*/
  def apply(xs: String*): FreqMapBuilder = ???
```

b) Test carefully that your `FreqMapBuilder` works correctly. Example test in REPL:

```
scala> import nlp.*

scala> val fmb = FreqMapBuilder("hej", "on", "dough")
fmb: nlp.FreqMapBuilder = nlp.FreqMapBuilder@458f85ef

scala> fmb.add("hej")

scala> fmb.toMap
res0: Map[String,Int] = Map(on -> 1, hej -> 2, dough -> 1)

scala> (1 to Short.MaxValue).foreach(i => fmb.add(i.toString))

scala> fmb.toMap.size
res1: Int = 32770

scala> fmb.toMap
res2: Map[String,Int] =
  Map(10292 -> 1, 19125 -> 1, 26985 -> 1, 29301 -> 1, 5451 -> 1, 4018 -> 1, 312
```

In the upcoming tasks you shall step by step create and test the case class `Text`.

```scala
package nlp

case class Text(source: String):
  lazy val words: Vector[String] = ???  // dela upp source i ord

  lazy val distinct: Vector[String] = words.distinct

  lazy val wordSet: Set[String] = words.toSet

  lazy val wordsOfLength: Map[Int, Set[String]] = wordSet.groupBy(_.length)

  lazy val wordFreq: Map[String, Int] = ???  // use FreqMapBuilder

  def ngrams(n: Int): Vector[Vector[String]] = ???  // use sliding

  lazy val bigrams: Vector[(String, String)] =
    ngrams(2).map(xs => (xs(0), xs(1)))

  lazy val followFreq: Map[String, Map[String, Int]] = ??? //nested map

  lazy val follows: Map[String, String] =
    followFreq.map( (key, followMap) =>
      val maxByFreq: (String, Int) = followMap.maxBy(_._2)
      val mostCommonFollower: String = maxByFreq._1
      (key, mostCommonFollower)
    )
    // or shorter with the same result: (easier or harder to read)
    //  followFreq.map((k, v) => k -> v.maxBy(_._2)._1)

object Text:
  def fromFile(fileName: String, encoding: String = "UTF-8"): Text =
    val source = scala.io.Source.fromFile(fileName, encoding)
    val txt = try source.mkString finally source.close()
    Text(txt)

  def fromURL(url: String, encoding: String = "UTF-8"): Text =
    val source = scala.io.Source.fromURL(url, encoding)
    val txt = try source.mkString finally source.close()
    Text(txt)
```

Task 2. *Split a string into words*. You shall implement the member `words`. It should contain a vector with all words in `source`, without any other characters than letters. You achieve this by starting from the string `source` and sequentially doing the following:

1. Replace all characters in `source` for which `isLetter` is false with `' '`
2. Split the string from the previous step into an array of strings with `split(' ')`
3. Filter out all empty strings
4. Convert all letters in all strings to lowercase
5. Convert the array into a sequence of type `Vector[String]`.

Test that `words` and the values using `words` work in REPL:

```
scala> val t = Text("Cucumber is not tomato, but cucumber is a vegetable.")

scala> t.words
res1: Vector[String] =
  Vector(cucumber, is, not, tomato, but, cucumber, is, a, vegetable)

scala> t.distinct
res2: Vector[String] =
  Vector(cucumber, is, no, tomato, but, a, vegetable)

scala> t.wordSet
res3: Set[String] = Set(cucumber, is, no, tomato, but, a, vegetable)

scala> t.wordsOfLength(3)
res4: Set[String] = Set(but)
```

Task 3. You shall now create the word frequency table `wordFreq` by registering word occurrences using `FreqMapBuilder`. The table `wordFreq` shall consist of key-value pairs `w -> f` where `f` is the number of times the word `w` occurs in `words`. Test `wordFreq` by downloading the book ”Treasure Island” written by Robert Louis Stevenson[5] and check the frequency for various common words. Which word is the most common? The second most common?

```
scala> val piratbok = Text.fromURL("https://fileadmin.cs.lth.se/pgk/skattkammaron.txt")
val piratbok: nlp.Text = Text(Herr Trelawney, doktor Livesey and the other gentlemen ha

scala> piratbok.words.size
val res0: Int = 69438

scala> piratbok.wordFreq("pirat")
val res1: Int = 7
```

Links to UTF-8 formatted books you can use in your tests:

- ”Treasure Island” by R. L. Stevenson:
  https://fileadmin.cs.lth.se/pgk/skattkammaron.txt
- ”Inferno” by August Strindberg:
  https://fileadmin.cs.lth.se/pgk/inferno.txt
- ”Pride and Prejudice” by Jane Austen:
  https://fileadmin.cs.lth.se/pgk/prideandprejudice.txt
- Project Gutenberg with many freely available books in text format:
  https://www.gutenberg.org/

Task 4. Implement the method `ngrams` that gives a sequence with all word sequences of length n. *Tip:* In this week's exercise, you examined how the method `sliding` works, which you can use to create n-grams. Use `toVector` on the result from `sliding`. Test carefully that `ngrams` and `bigrams` work correctly before proceeding.

[5] Copyright for this book has expired, so you are not infringing copyright (in a legal sense).

```
scala> piratbok.ngrams(3).take(2)
val res1: Vector[Vector[String]] =
  Vector(Vector(herr, trelawney, doktor), Vector(trelawney, doktor, livesey))

scala> piratbok.bigrams.take(2)
val res2: Vector[(String, String)] =
  Vector((herr,trelawney), (trelawney,doktor))
```

Task 5. Implement `followFreq`, which shall contain a key-value table where the value in turn is a frequency table over words that follow the key.
By analyzing all word pairs we can find out which word is the most common to follow a given word. The method `bigrams` gives us all word pairs `(w1, w2)` where `w2` follows `w1`. We can save statistics over following words in a key-value table with mappings `w -> f` where the key `w` is a word and the value `f` is a frequency table of type `Map[String, Int]`. In the frequency table, we store the frequency for all words that follow `w`. You shall thus build a nested table of type `Map[String, Map[String, Int]]`. Draw a picture of the nested structure. ✎
Implement the method followFreq based on the pseudocode below:

```
val result = collection.mutable.Map.empty[String, FreqMapBuilder]
for (key, next) <- bigrams do
  if /* key already defined in result */ then
    /* at "location" result(key): add next to the frequency table */
  else
    /* add (key -> new frequency table with next) to result */
end for
result.map(p => p._1 -> p._2.toMap).toMap // toMap gives immutable Map
```

Make printouts to find out answers to the following questions. Write down your answers and be ready to present them during the control questions (see section 9.3.3). ✎

a) Which words usually follow *han* and *hon* in Stevenson's "Treasure Island"?

b) Which words usually follow *han* and *hon* in Strindberg's "Inferno"?

c) Which words usually follow *he* and *she* in Austen's "Pride and Prejudice"?

Task 6. Create a main program that reports any interesting measures about the words in a text. The program should take the text source as an argument, given as a URL or a filename. Write the main program in the file `Main.scala` in a single object named `Main`. An example of a report your main program can generate is shown below. Here is also an integer argument controlling the length of the top lists.

```
> scala run . -- https://fileadmin.cs.lth.se/pgk/skattkammaron.txt 13

Source: https://fileadmin.cs.lth.se/pgk/skattkammaron.txt

*** Number of words: 69438

*** The 13 most common words and their frequency:
(och,3089), (jag,2007), (att,1594), (det,1382), (en,1262),
(i,1244), (som,1132), (på,1068), (han,1063), (var,990),
(med,854), (den,774), (av,740)

*** The 13 longest words and their length:
```

```
(besättningsmedlemmarnas,23), (befästningsanordningar,22),
(temperamentsuppvisning,22), (undsättningsexpedition,22),
(besättningsmedlemmarna,22), (försiktighetsåtgärder,21),
(undsättningsfartyget,20), (sjukdomsframkallande,20),
(husföreståndarinnans,20), (sjömansterminologin,19),
(parlamentärsflaggan,19), (bregravningsplatsen,19),
(tidvattenströmmarna,19)
```

Example main program that can produce the above output:

```
package nlp

object Main:
  val defaultUrl = "https://fileadmin.cs.lth.se/pgk/skattkammaron.txt"
  val defaultN = 10

  def mostFrequentWords(n: Int, freqMap: Map[String, Int]): Vector[(String, Int)] = ???


/*  |Källa: $from
    |
    |*** Antal ord: ${text.words.size}
    |
    |*** De $n vanligaste orden och deras frekvens:
    |${mostFrequentWords(n, text.wordFreq).mkString(", ")}
    |
    |*** De $n längsta orden och deras längd:
    |$longestWordsWithLength
*/

  def report(text: Text, from: String, n: Int): String =
    val longestWordsWithLength =
      mostFrequentWords(n, text.distinct.map(w => (w, w.length)).toMap).mkString(", ")
    s"""
    |Source: $from
    |
    |*** Number of words: ${text.words.size}
    |
    |*** The $n most common words and their frequency:
    |${mostFrequentWords(n, text.wordFreq).mkString(", ")}
    |
    |*** The $n longest words and their length:
    """.stripMargin

  def main(args: Array[String]): Unit =
    val location = if args.isEmpty then defaultUrl else args(0)
    val n = if args.length < 2 then defaultN else args(1).toInt
    val text =
      if location.startsWith("http") then Text.fromURL(location)
      else Text.fromFile(location)

    println(report(text, location, n))
```

Task 7. Pair up with another student and plan how you together can, using https://fileadmin.cs.lth.se/pgk/muntabot, train for the oral exam where you mutually

act as ”mock examiners”. Make a plan for when you will test each other on which weeks. Show your plan to the instructor and discuss what it means to be a good ”mock examiner”.

9.3.3 Control questions

1. What are your answers to task 5 a) b) c) on page 66?
2. In what order do elements appear when calling `distinct` on a sequence?
3. If you iterate over a set, in what order are the elements processed?
4. Give an example of when it is appropriate to use a set instead of a sequence of distinct values.
5. Are all keys in a key-value table guaranteed to be unique?
6. Are all values in a key-value table guaranteed to be unique?
7. The LTH engineer Oddput Clementin wants to sum the length of each string in a set and writes:

```
scala> Set("hej", "on", "dej").map(_.length).sum
res0: Int = 5
```

   Why does this produce an incorrect result? How can Oddput fix the problem?

9.3.4 Optional tasks

Task 8. Extend the class `Text` and implement the following method that should return a random word from `wordSet`. Each word should be equally probable.

```
def randomWord: String = ???
```

Task 9. With NLP, you can generate random sentences that statistically resemble ”real”, human-created sentences.
Implement the method `randomSeq(firstWord, n)` below in the class `Text`. It should produce a sequence $w_1, w_2, ..., w_n$ where w_1 is `firstWord` and w_{i+1} is a random word drawn among the words that follow w_i. You can achieve this by selecting each following word w_{i+1} from `keys.toVector` for the `followFreq` table associated with w_i. The words should be drawn from the following set with equal probability.

```
def randomSeq(firstWord: String, n: Int): Vector[String] = ???
```

Task 10. To make your computer-generated sentences really resemble human language, we can create the most probable sentences of different lengths from our word frequency analysis.
Add the method `mostCommonSeq` in the class `Text` as follows:

```
def mostCommonSeq(firstWord: String, n: Int): Vector[String] = ???
```

a) Implement the method so that the result is a sequence of `n` words. The sequence should start with `firstWord` and then be followed by the word that is the *most common* word following `firstWord`, and thereafter the most common following word after that, etc. *Tip:* Use a local variable `val result` which is an ArrayBuffer to which you add the following words in a `while` loop.

b) Compare the random sequences with sequences generated with `randomSeq` in task 9. Which sequences most resemble "real" sentences?

Task 11. Use existing collection methods instead of `FreqMapBuilder` to register following words.

a) Explore in REPL how the method `groupBy(x => x)` behaves when applied on a collection of strings. Search for and study the documentation for `groupBy`.

b) Introduce the value `lazy val wordFreq2`. It should yield the same result as `wordFreq` but implemented using `groupBy` and `map` instead of `FreqMapBuilder`.

★ c) Compare performance between `wordFreq2` and `wordFreq`. Which is fastest for large texts? Is there a big difference?

d) Introduce the value `lazy val followsFreq2`. It should yield the same result as `followsFreq` but implemented using `groupBy` and `map` instead of `FreqMapBuilder`. This task is rather tricky. Experiment in REPL and build a solution step by step. *Tip:*

```
bigrams
  .groupBy(???)
  .map(p => p._1 -> p._2.map(???).groupBy(???).map(???))
```

★ e) Compare performance between `followsFreq2` and `followsFreq`. Which is fastest for large texts? Is there a large difference?

Task 12. *Make `FreqMapBuilder` generic.* Generic structures, that is those that have type parameters, are often significantly more useful. If you make `FreqMapBuilder` generic by introducing a type parameter instead of hard-coding the type to `String`, you can use `FreqMapBuilder` with arbitrary element types.

a) Examine `FreqMapBuilder` and identify everything in that class that is specific to the type `String`.

b) Introduce a type parameter `A` inside square brackets after the class name and then use `A` instead of `String` in all methods.

c) Test that your generic frequency table builder works on sequences that contain other than strings.
This works because nothing in `FreqMapBuilder` really assumes that the elements counted are of string type (it suffices that there is a proper `equals` and `hashCode`).

Chapter 10

Inheritance and Composition

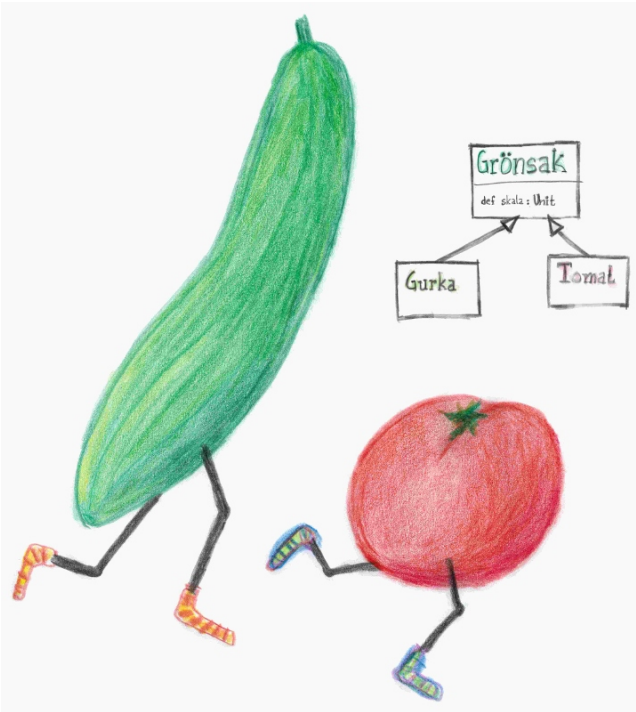
Concepts included in this week's studies:

- ☐ inheritance
- ☐ composition
- ☐ polymorphism
- ☐ trait
- ☐ extends
- ☐ asInstanceOf
- ☐ with
- ☐ mix-in supertype
- ☐ subtype
- ☐ base type
- ☐ override
- ☐ Scala type hierarchy
- ☐ Any
- ☐ AnyRef
- ☐ Object
- ☐ AnyVal
- ☐ Null
- ☐ Nothing
- ☐ top type
- ☐ bottom type
- ☐ reference types
- ☐ value types
- ☐ access rules in inheritance
- ☐ protected
- ☐ final
- ☐ trait
- ☐ abstract class

10.1 Theory

10.1.1 What is inheritance?

Inheritance (eng. *inheritance*) describes the relation
X **is a** *Y*

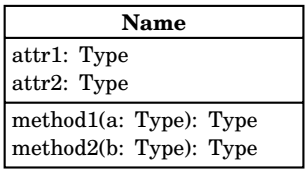


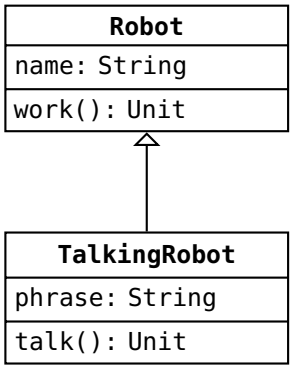
10.1.2 Why is inheritance needed?

- You can use inheritance to separate code into:
  - **general** (common) parts and
  - **specific** (customized) parts.
- You can achieve **controlled flexibility**:
  - Client code can **extend** (eng. *extend*) a given API with its own specific additions.
- You can use inheritance to declare a common **base type** so that variables and generic collections can be given an appropriately specific element type.
  - It is enough to know the base type to be able to access common members for all elements in the collection.
  - Example: All vegetables have a weight.

10.1.3 Class diagram with UML (Unified Modeling Language)

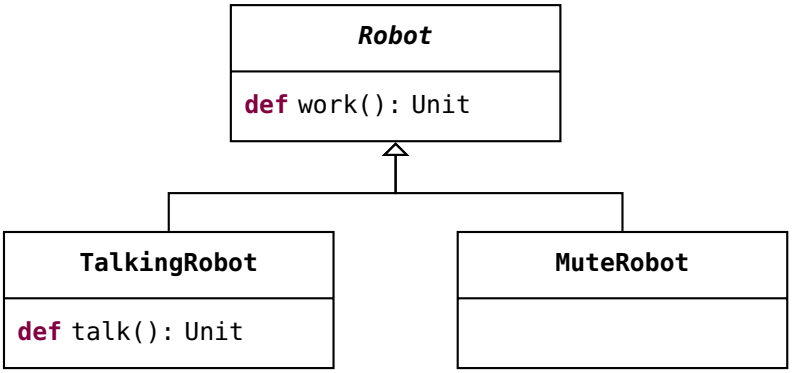
UML Class diagram:





More about UML diagrams in later courses.
en.wikipedia.org/wiki/Class_diagram

10.1.4 Example: Robot as base type for two subtypes

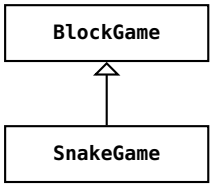


(Sometimes attributes and/or methods are left out in a UML diagram to reduce the amount of detail in the model and provide an overview.)

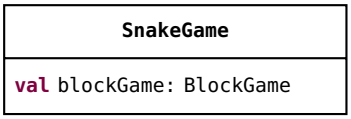
10.1.5 Alternative to inheritance: composition

- As an alternative to the class X inheriting class Y, you can instead use **composition** (eng. *composition*), which means that class X **has an attribute** that **refers** to class Y.

Example of inheritance in the `snake` lab:
SnakeGame **is** a BlockGame



An alternative would be **composition**:
SnakeGame **has** a BlockGame



- Composition where **X has a Y** is often a better alternative than inheritance if:
  1. there is *not* a clear **X-is-a-Y** relation
  2. a **common base type** is *not* needed
  3. it is *not* desirable to inherit and expose *all* of Y's members via X

10.1.6 Example of composition in the snake lab

A `Player` **has a** [1] `snake`.

Player
`val snake: Snake`

```
class Player(val snake: Snake)
```

10.1.7 The need for a common base type

```
scala> case class Cucumber(weight: Int)

scala> case class Tomato(weight: Int)

scala> val gurkor = Vector(Cucumber(200), Cucumber(300))
val gurkor: Vector[Cucumber] = Vector(Cucumber(200), Cucumber(300))

scala> gurkor.map(_.weight)
res0: Vector[Int] = Vector(200, 300)

scala> val vegetables = Vector(Cucumber(200), Tomato(42))
val vegetables: Vector[Cucumber | Tomato] = Vector(Cucumber(200), Tomato(42))

scala> vegetables.map(_.weight)
-- [E008] Not Found Error: -------------------------
1 |vegetables.map(_.weight)
  |              ^^^^^^
  |              value weight is not a member of Cucumber | Tomato
```

How do we get this to work as we want?
→ Create a base type **base type** with a common attribute `weight`!

10.1.8 Why isn't the common member visible in a union type?

[1] Read more about composition and related concepts such as aggregation, association, ownership etc. here:
https://en.wikipedia.org/wiki/Object_composition

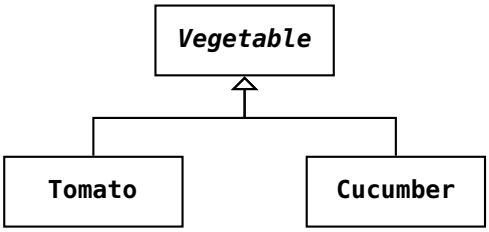
```
scala> val vegetables = Vector(Cucumber(200), Tomato(42))
val vegetables: Vector[Cucumber | Tomato] = Vector(Cucumber@15f11bfb,Tomato@16a499d1)

scala> vegetables.map(_.weight)
-- [E008] Not Found Error: --------------------------
1 |vegetables.map(_.weight)
  |               ^^^^^^
  |               value weight is not a member of Cucumber | Tomato
```

- The types `Cucumber` and `Tomato` are **unrelated**. (`AnyRef` lacks `weight`)
- A member that happens to have the same name does not always have the same meaning.
- In Scala you must explicitly relate the members by **inheritance** of a base type with common member, or you have to **match** on the parts of the union.
- You can offer such matching as an **extension method**:

```
scala> extension (cucOrTom: Cucumber | Tomato) def weight = cucOrTom match
         case c: Cucumber => g.weight
         case t: Tomato => t.weight

scala> val weights = vegetables.map(_.weight)
val weights: Vector[Int] = Vector(200, 42)
```

10.1.9 Create a common base type with inheritance

The type ***`Vegetable`*** is a **base type** in the inheritance hierarchy below:



The arrow ⇧ denotes **inheritance** and reads "**is a**"

The types `Tomato` and `Cucumber` are **subtypes** of the type `Vegetable`.
Base types that should not be instantiated directly are **abstract** (shown in *italic* style).

10.1.10 Create a common base type with `trait` and `extends`

With a **`trait`** `Vegetable`, the classes `Cucumber` and `Tomato` can share a common abstract **base type** by both **subtypes** doing **`extends`** `Vegetable`:

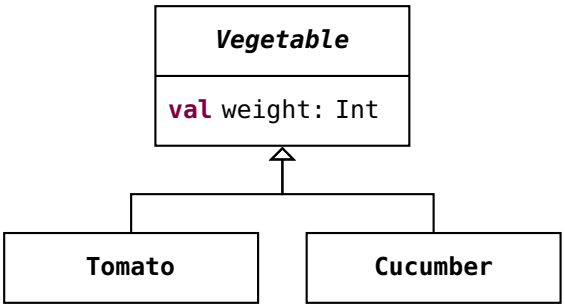
```
scala> trait Vegetable

scala> case class Cucumber(weight: Int) extends Vegetable

scala> case class Tomato(weight: Int) extends Vegetable

scala> val vegetables = Vector(Cucumber(200), Tomato(42))
val vegetables: Vector[Vegetable] = Vector(Cucumber(200), Tomato(42))
```

But it is still **not** what we want:

```
scala> vegetables.map(_.weight)
-- [E008] Not Found Error: -------------------------
1 |vegetables.map(_.weight)
  |               ^^^^^^
  |               value weight is not a member of Vegetable
```

10.1.11 A common base type with common parts

Put common members in the base type:



- All vegetables now have the attribute `val weight`.
- The specific value of the weight is **not** defined in the base type.
- The member `weight` is called **abstract** because it **lacks implementation** and therefore cannot be instantiated directly.

10.1.12 Place common parts in the base type

We include the common attribute `val weight` as an **abstract member** in the base type:

```
trait Vegetable:
  val weight: Int    // implementation missing, no =

case class Cucumber(weight: Int) extends Vegetable
```

```
case class Tomato(weight: Int) extends Vegetable
```

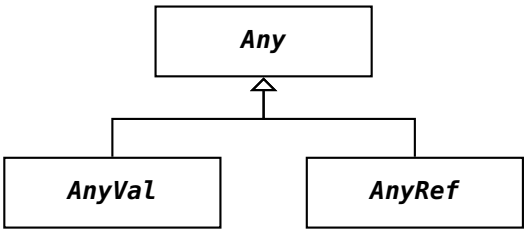
Now you have explicitly told the compiler that you want all vegetables to have a weight:

```
scala> val vegetables = Vector(Cucumber(200), Tomato(42))
val vegetables: Vector[Vegetable] = Vector(Cucumber(200), Tomato(42))

scala> vegetables.map(_.weight)
val res0: Vector[Int] = Vector(200, 42)
```

The abstract member `weight` in the abstract type `Vegetable` is **implemented** in a concrete subclass, here as a class parameter.

10.1.13 Scala's type hierarchy

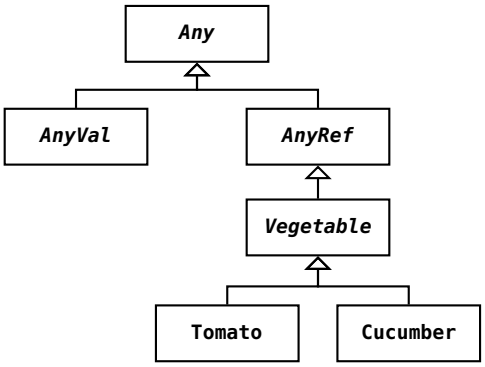
A simplified picture of the top part of Scala's type hierarchy:



- The numeric types `Int`, `Double`, etc., are subtypes of **`AnyVal`** and are called **value types** and are stored in a special, efficient way in memory.
- All your own classes are subtypes of **`AnyRef`** and are called **reference types** and can (directly or indirectly) be constructed with **`new`**.
- `AnyRef` corresponds to **`java.lang.Object`** in the JVM.
- (There is also `Matchable` which is a subtype of `Any` and supertype of `AnyRef` and `AnyVal`. The type `Matchable` is needed to distinguish between types that can be examined with pattern matching and other so-called **opaque types** (advanced topic).)

10.1.14 Implicit supertypes of your own classes

All your own types implicitly belong under Scala's type hierarchy:



10.1.15 What is a trait?

- **Trait** means **property**.
- A trait resembles a class, **but** special rules apply:
  - it **can** contain parts that **lack implementation**
  - it **can be mixed** with several other traits so that different pieces of code can be combined flexibly.
  - it **cannot** be instantiated directly.
  - it **can** have **parameters**[2] just like classes.

10.1.16 What is a trait used for?

A `trait` can be used to create a base type that can be home for common parts of subtypes:

```
trait BaseType { val x = 42 }                  // BaseType has member x
class Subtype1 extends BaseType { val y = 43 } // Subtype1 inherits x, also has y
class Subtype2 extends BaseType { val z = 44 } // Subtype2 inherits x, also has z
```

```
scala> val a = new Subtype1
val a: Subtype1 = Subtype1@51016012

scala> a.x
val res0: Int = 42

scala> a.y
val res1: Int = 43

scala> a.z
-- Error:
  value z is not a member of Subtype1

scala> new BaseType
--Error:
  BaseType is a trait; it cannot be instantiated
```

[2] In old Scala 2 traits cannot have parameters

10.1.17 A trait can have abstract members

```
trait X { val x: Int }   // x is abstract, i.e. lacks implementation
class A extends X { val x = 42 }   // x is given an implementation
class B extends X { val x = 43 }   // x is given another implementation
```

```
scala> val a = new A
val a: A = A@5faeada1

scala> val b = B()    // works without new but then () is needed
val b: B = B@cb51256

scala> val xs = Vector(a,b)
val xs: Vector[X] = Vector(A@5faeada1, B@cb51256)

scala> xs.map(_.x)
val res0: Vector[Int] = Vector(42, 43)

scala> class Y { val y: Int }
-- Error:
  class Y needs to be abstract, since val y: Int in class Y is not defined
```

10.1.18 A trait can have parameters

```
trait X(val x: Int)
class A extends X(42)
class B(y: Int) extends X(y) // the value of y becomes argument to x in X
```

```
scala> val a = A()
val a: A = A@5faeada1

scala> val b = B(43)
val b: B = B@cb51256

scala> val xs = Vector(a,b)
val xs: Vector[X] = Vector(A@5faeada1, B@cb51256)

scala> xs.map(_.x)
val res0: Vector[Int] = Vector(42, 43)

scala> b.y
--  Error:
  value y cannot be accessed as a member of (b : B)
```

How can we make the member y visible?
Add `val` before the parameter name, or use a `case` class.

10.1.19 Abstract and concrete members

```
object exempelVego1:

  trait Grönsak:
    var vikt: Double              // abstrakt medlem, saknar implementation
    var ärSkalad: Boolean = false // konkret medlem, har implementation

    def skala(): Unit             // abstrakt medlem, saknar implementation

  class Gurka(var vikt: Double) extends Grönsak:
    def skala(): Unit =           // implementation specifik för Gurka
      if !ärSkalad then
        println("Skalas med skalare.")
        vikt = 0.99 * vikt
        ärSkalad = true

  class Tomat(var vikt: Double) extends Grönsak:
    def skala(): Unit =           // implementation specifik för Tomat
      if !ärSkalad then
        println("Skållas.")
        vikt = 0.99 * vikt
        ärSkalad = true
```

10.1.20 Avoid code duplication using inheritance

```
object exempelVego2:

  trait Grönsak:  // innehåller gemensamma delar; hjälper oss undvika upprepning
    val skalningsmetod: String      // abstrakt
    val skalfaktor = 0.99           // konkret
    var vikt: Double                // abstrakt
    var ärSkalad: Boolean = false   // konkret

    def skala(): Unit = if !ärSkalad then // konkret
      println(skalningsmetod)
      vikt = skalfaktor * vikt
      ärSkalad = true

  class Gurka(var vikt: Double) extends Grönsak: // det som är speciellt för gurkor
    val skalningsmetod = "Skalas med skalare."

  class Tomat(var vikt: Double) extends Grönsak: // det som är speciellt för tomater
    val skalningsmetod = "Skållas."
```

10.1.21 Why can code duplication cause problems?

- More to write (not a very big problem)
- More lines of code to read and understand
- More lines of code affected when adding features
- More lines of code to maintain:
  - If you fix a bug in one place you must remember to make **exactly the same change** in all places where code duplication occurs → **risk of new bugs**
- Principle in English: `def DRY = "Don't Repeat Yourself!"`

There *are* cases when **code duplication actually is preferable**: e.g. if you want different parts of the code to be **completely independent** from each other.

10.1.22 Subtyping polymorphism and dynamic binding

```
trait Robot { def work(): Unit }

case class CleaningBot(name: String) extends Robot:
  def work(): Unit = println(" Cleaning Cleaning")

case class TalkingBot(name: String) extends Robot:
  def work(): Unit = println(" Talking Talking")
```

Polymorphism means "many forms". The references r and bot below can have different "forms", i.e. they can refer to different kinds of robots.
Dynamic binding means that the runtime type determines which method is executed.

```
scala> def robotDoWork(bot: Robot) = { print(bot); bot.work() }

scala> var r: Robot = CleaningBot("Wall-E")

scala> robotDoWork(r)
CleaningBot(Wall-E) Cleaning Cleaning

scala> r = TalkingBot("C3PO")

scala> robotDoWork(r)
TalkingBot(C3PO) Talking Talking
```

10.1.23 Example: Overriding and `override`

```
object exempelVego3:

  trait Grönsak:
    val skalningsmetod: String      // abstrakt
    val skalfaktor = 0.99           // konkret
    var vikt: Double                // abstrakt
```

```
    var ärSkalad: Boolean = false   // konkret

    def skala(): Unit = if !ärSkalad then
      println(skalningsmetod)
      vikt = skalfaktor * vikt
      ärSkalad = true

  class Gurka(var vikt: Double) extends Grönsak:
//nyckelordet override behövs ej om abstrakt medlem i supertyp
    val skalningsmetod = "Skalas med skalare."

  class Tomat(var vikt: Double) extends Grönsak:
//nyckelordet override behövs ej om abstrakt medlem i supertyp men tillåtet:
    override val skalningsmetod = "Skållas."
//  override val skalningmetod = "Skållas." //kompilatorn hittar stavfelet!
    override val skalfaktor = 0.95  // override måste anges vid ändrad impl.
```

10.1.24 A final member cannot be overridden

```
object exempelVego4:

  trait Grönsak:
    val skalningsmetod: String
    final val skalfaktor = 0.99  // en final medlem kan ej överskuggas
    var vikt: Double
    var ärSkalad: Boolean = false

    def skala(): Unit = if !ärSkalad then
      println(skalningsmetod)
      vikt = skalfaktor * vikt
      ärSkalad = true

  class Gurka(var vikt: Double) extends Grönsak:
    val skalningsmetod = "Skalas med skalare."

  class Tomat(var vikt: Double) extends Grönsak:
    val skalningsmetod = "Skållas."
//  override val skalfaktor = 0.95
//  ger KOMPILERINGSFEL: "cannot override final member"
```

10.1.25 Protected gives visibility limited to subtypes

```
scala> trait Super:
         private val mySecret = 42
         protected val ourSecret = 42

scala> class Sub extends Super { def revealed = mySecret }
```

```
- Error: Not found: mySecret

scala> class Sub extends Super { def revealed = ourSecret }

scala> val s = Sub()
val s: Sub = Sub@2eee9593

scala> s.revealed
val res0: Int = 42

scala> s.mySecret
-- Error:
  value mySecret is not a member of Sub - did you mean s.ourSecret?

scala> s.ourSecret
-- Error:
  Access to protected value ourSecret not permitted because enclosing object
  is not a subclass of trait Super where target is defined
```

10.1.26 Filename rules and conventions

- In several languages, e.g. Java, these rules apply (but **not** in Scala):
  - You can only have **one public class per source file**.
  - The source file must have the **same name** as the public class, e.g. `ClassName.java`
- Scala is more **free**:
  - In Scala you may have **as many** non-private classes/traits/singleton objects in the same source file **as you want**.
  - In Scala you may name source files **independently** of their content.
- These **conventions** are usually used in Scala:
  - If a source file contains only **one single** class/trait/singleton object, give the file the same name as the content, e.g. `ClassName.scala`
  - If a source file contains **several** things, name the file something that reflects all content and use **lowercase first letter**, e.g. `drawing-utils.scala` or `baseTypeName.scala`
- Scala 3 warns when inheriting from a class outside the same source file (more about so-called open classes later).

10.1.27 Classes, inheritance and class parameters

Classes, singleton objects, and traits can inherit other classes and traits with **`extends`**. If the supertype has parameters then the subtype **must** provide arguments after **`extends`**.

```
object personExample1:

  class Person(val name: String)

  class Academic(
```

```
    override val name: String,
    val university: String) extends Person(name)

  class Student(
    override val name: String,
    override val university`: String,
    val program: String) extends Akademic(name, university)

  class Researcher(
    override val name: String,
    override val university: String,
    val titel: String) extends Akademic(name, university)

  @main def Test: Unit =
    val kim = new Student("Kim Robinsson", "Lund", "Data")
    println(s"${kim.name} ${kim.university} ${kim.program}")
```

10.1.28 Static and dynamic type

```
var p: Person = Researcher("Robin Smith", "Lund", "Professor")
```

- The **static type** for `p` is `Person` which allows `p` to be assigned to *other* instances that are of type Person.

  ```
  p = Student("Kim Robinson", "Lund", "Computer Science")
  ```

- By "static type" we mean the type information existing at **compile time**.
- The **dynamic type**, also called the **runtime type** which applies during execution can be more specific: the dynamic type of `p` is now after the assignment Student, but also Academic and Person.
- You can check if the dynamic type for `p` is `SomeType` with `p.isInstanceOf[SomeType]` (but better use `match`)
- You can tell the compiler: **"I guarantee that p is of type `SomeType` so you can cast the static type to `SomeType` and I accept runtime errors if I lie"** by doing a **type cast** with `p.asInstanceOf[SomeType]` (this is very rare in normal Scala code because type test + cast is safer with `match`)

10.1.29 Mixin

You can inherit multiple traits. This is called **mixin** and is done with a comma-separated type list after `extends`.

```
object personExample2:

  trait Person     { val name: String }
  trait Akademic { val university: String }
  trait Graduated { val title: String }
```

```

  class Student(val name: String,
                val university: String,
                val program: String) extends Person, Akademic

  class Researcher(val name: String,
                 val university: String,
                 val title: String) extends Person, Akademic, Graduated

  @main def Test: Unit =
    val f = new Researcher("B. Regnell", "Lunds university", "Professor")
    println(s"${f.titel} ${f.name}, ${f.university}")
```

10.1.30 Runtime type with mixin

Test runtime type with `isInstanceOf[Type]`. Promise the compiler (and take responsibility yourself) that it is a certain runtime type with `asInstanceOf[Type]`. NOTE! Better to use **`match`**.

```
object personExample3:

  trait Person     { val name: String }
  trait Academic { val university: String }
  trait Graduate { val title: String }

  class Student(val name: String,
                val university: String,
                val program: String) extends Person, Academic

  class Researcher(val name: String,
                 val university: String,
                 val title: String) extends Person, Academic, Graduate

  @main def Test: Unit =
    var p: Person = new ("B. Regnell", "Lunds universitet", "Professor")
    if p.isInstanceOf[Academic] then println(p.name + " is academic")
    p = new Student("Kim Robinson", "Lund", "Data")  // Is it possible to access p.program?
    if p.isInstanceOf[Student] then println(p.asInstanceOf[Student].program)
  // The above is preferably done with match
```

10.1.31 Anonymous class

If you have an abstract type with missing implementations you can fill in the missing parts in an extra block which is ”attached” at instantiation:

```
scala> trait Vegetable { val weight: Int }
// defined trait Vegetable

scala> new Vegetable    // or Vegetable()
-- Error:
```

```
1 |new Vegetable
  |    ^^^^^^^
  |    Vegetable is a trait; it cannot be instantiated

scala> new Vegetable { val weight = 42 }
val res0: Vegetable = anon1@4e3f2908
```

You get what is called an **anonymous class**. (In this case a quite strange vegetable which is no specific sort of vegetable, but which still has a weight.)

The simplest (and most pointless) anonymous class is:

```
scala> new {}
val res0: Object = anon1@5bb37371
```

10.1.32 How to prevent subtyping?

You can prevent subtyping in these ways:

- With **final** you create a final class that cannot be inherited:

  ```
  final class Person(name: String)
  ```

  ```
  scala> object Björn extends Person("Björn")
  -- Error:
  1 |object Björn extends Person("Björn")
    |       ^
    |       object Björn cannot extend final class Person
  ```

- With **sealed** you can seal an entire type hierarchy:

  ```
  scala> sealed trait T  // press Alt+TAB to defer evaluation
       | final class A extends T
       | final class B extends T
       |
  scala> new T{}
  -- Error:
  1 |new T{}
  |^
  |Cannot extend sealed trait T in a different source file
  ```

  With a sealed type hierarchy you can only inherit the base type within the same source file.

10.1.33 Sealed types with `sealed`

With a **sealed** you can create a **sealed** enumeration:

```
sealed trait Color(val toInt: Int)
object Color:
  val values = Vector(Spades, Hearts, Diamonds, Clubs)
```

```
  case object Spades  extends Color(0)
  case object Hearts extends Color(1)
  case object Diamonds   extends Color(2)
  case object Clubs  extends Color(3)
```

The keyword **sealed prevents** further subtyping of `Color` in **another source file** and **gives a warning** if a match is incomplete – a tip to avoid runtime errors.

```
scala> Color.values(0) match { case Color.Spades => "hello" }
-- Warning:
1 |Color.values(0) match { case Color.Spades => "hello" }
  |^^^^^^^^^^^^^^
  |match may not be exhaustive.
  |
  |It would fail on pattern case: Hearts, Diamonds, Clubs
val res0: String = hello
```

Better use **enum** so you get both **sealed** and more goodies included!

10.1.34 Open class signals encouraged subtyping

- If you inherit a class you get access to all members that are not private and can change to any implementation as long as the types match.
- This can be risky if the person who wrote the class did not plan for this and document carefully how the class is meant to be used in inheritance.
- By creating **open classes** with the keyword **open** you signal that the class is meant to be a supertype in inheritance.

```
open class Cucumber(val weight: Int, val pris: Double):
  /** Override this if you want a different alternative. */
  def alternative: String = s"Celery works just as well!"
```

```
scala> object BigCucumber extends Cucumber(1000, 1_000_000): // inherit as you pl
         override def alternative = "maybe also works with a concrete pillar"
```

- **open** is required if you want to silence warnings about inheritance from another source file.
- You can also avoid the warning with **import** `scala.language.adhocExtensions`

10.1.35 Trait or abstract class?

The keyword **abstract** is needed in front of **class** if there are abstract members:

```
scala> class X { val x: Int }
1 |class X { val x: Int }
  |      ^
  |      class X needs to be abstract, since val x: Int in class X is not defined

scala> abstract class X { val x: Int }  // works!
```

But isn't a trait just as good? It often works just as well with a trait.

Use a **trait** if...

- ...you're unsure which is best. (You can always change to an abstract class later.)
- ...you want to mix your trait together with other traits.
- ...you want your trait, when mixed in, to be transparent in type inference.

Use an (abstract) **class** if...

- ...you want to restrict mixin – you can only inherit from one class.
- ...you want to forward parameters to the supertype (see next slide).
- ...you want to inherit your class in Java code.
- ...you want to shorten recompilation time during changes in a *very large* codebase.

10.1.36 A trait cannot forward parameters

A trait cannot pass parameters to a supertype (it would otherwise cause tricky problems during mixin):

```
scala> trait X(x: Int)
// defined trait X

scala> trait Y(y: Int) extends X(y)
-- Error:
1 |trait Y(y: Int) extends X(y)
  |                         ^^^^
  |  trait Y may not call constructor of trait X
```

But it works fine with an `abstract class` or a `class` (which **cannot** be mixin):

```
scala> abstract class Y(y: Int) extends X(y)
// defined class Y
```

More details here: dotty.epfl.ch/docs/reference/other-new-features/trait-parameters.html

---

10.1.37 Members, inheritance, and shadowing

Different kinds of members in **Scala** that can be inherited (replaced) with `override` when inheriting:

- `def`
- `val`
- `lazy val`
- `var`

Overall (simplified) rules:

- When you shadow (eng. *override*) a member with another member with the same name in a subtype, this member gets a (new) implementation.

- Singleton objects cannot be inherited (and members in singleton objects therefore cannot be shadowed).
- When designing an object-oriented language, it is important to define sound shadowing rules for inheritance. This is surprisingly tricky.
- All tricky rules for shadowing are hard to memorize, but fortunately the compiler's error messages help you follow the rules :)

10.1.38 Deep dive: Detailed override rules

A member M1 in a supertype may be shadowed by a member M2 in a subtype, according to these rules: (see also https://stackoverflow.com/questions/40057337)

1. M1 and M2 must have the same name and their types must match.
2. **def** may be replaced with: **def**, **val**, **lazy val**, and under special conditions **var** (more about replacing **def** with **var** soon).
3. **val** may be replaced with **val**. If the member in M1 being shadowed is abstract, then **val** may also be replaced with a **lazy val**.
4. An abstract **var** may only be replaced with a **var**. A concrete **var** may not be replaced.
5. **lazy val** may only be replaced with a **lazy val**.
6. If a member in a supertype is abstract you do *not need* to use the **override** keyword in the subtype. (But it's good to do so anyway, so the compiler helps you check that you are actually overriding something.)
7. If a member in a supertype is concrete you *must* use the **override** keyword in the subtype, otherwise you get a compilation error.
8. M1 may not be **final**.
9. M1 may not be **private**, but can be **private**[X] if M2 is also **private**[X], or **private**[Y] if X is (part of) Y.
10. If M1 is **protected** then M2 must also be **protected**.

10.1.39 Deep dive: override of var with var

Special conditions are required to replace a **var** with a **var** when inheriting.

- A concrete member **cannot** be replaced with a **var** — not even a concrete **var**:

```
scala> trait ConcreteVar { var x = 42 }
scala> trait Sub extends ConcreteVar { override var x = 43 }
-- Error:
1 |trait Sub extends ConcreteVar { override var x = 43 }
  |                                            ^
  |             error overriding variable x in trait ConcreteVar of type Int;
  |               variable x of type Int cannot override a mutable variable
```

- An abstract **var** member may be replaced by a **var** if you do **not** write **override**:

```
scala> trait AbstractVar { var x: Int }
scala> trait Sub extends AbstractVar { override var x = 43 }
-- Error:
```

```
1 |trait Sub extends AbstractVar { override var x = 43 }
  |                                                ^
  |             error overriding variable x in trait AbstractVar of type Int;
  |               variable x of type Int cannot override a mutable variable
scala> trait Sub extends AbstractVar { var x = 43 }   // works without override
// defined trait Sub
```

Avoid abstract **var** members — usually better to use abstract **def**.

10.1.40 Deep dive: override of `def` with `var`

- An abstract **def** member may be replaced by a **var** if you do **not** write **override**:

```
scala> trait Super { def x: Int }
scala> trait Sub extends Super { override var x = 43 }
-- Error:
1 |trait Sub extends Super { override var x = 43 }
  |                                                ^
  |                                                setter x_= overrides nothing

scala> trait Sub extends Super { var x = 43 }   // works if no override
```

  The abstract **def** member is then implemented by a concrete getter.
- Essentially, a public **var** member is a combination of a getter and a setter. You can create a concrete getter + setter and explicitly shadow the getter with **override** (note that the setter cannot use **override**, since the superclass does not have a corresponding method to replace — compare the error above):

```
scala> trait Sub2 extends Super:
         private var myPrivateValue = 42
         override def x: Int = myPrivateValue
         def x_(newValue: Int): Unit = myPrivateValue = newValue
```

  The above works fine since we now have a valid combination of getter + setter.

10.1.41 Distinguishing mine and yours with `super`

```
scala> class X { def cucumber = "super pepino" }

scala> class Y extends X:
         override val cucumber = ":("
         val sg = super.cucumber

scala> val y = new Y
y: Y = Y@26ba2a48

scala> y.cucumber
res0: String = :(
```

Super Pepinos to the rescue:

```
scala> y.sg
res1: String = super pepino
```

https://youtu.be/NPhjiXskz34

10.1.42 Terminology and keywords in inheritance

subtype	a type that inherits a supertype
supertype	a type inherited by a subtype
base type	a type that is root in an inheritance tree
abstract member	a member that lacks an implementation
concrete member	a member that does not lack an implementation
abstract type	a type that may have abstract members; cannot be instantiated
concrete type	a type that has no abstract members; can be instantiated
anonymous class	a nameless class created directly at instantiation
`class`	a concrete type that **cannot have abstract members**
`abstract class`	an abstract type that **may have abstract members**
`trait`	is an abstract type that **can be mixed in**
`extends`	precedes a supertype, causing inheritance of supertype's members
`override`	a member shadows (replaces) a member in a supertype
`protected`	makes a member visible in subtypes of this type (cf. `private`)
`final def cucumber`	makes the member cucumber final: prevents shadowing
`final class`	makes the class final: prevents further subtyping
`sealed`	sealed trait/class: only subtypes in this source file; enables pattern matchi
`open class`	indicates it is intended to be inherited, `open` required for inheritance in an
`super.cucumber`	refers to the supertype's member `cucumber` (cf. `this`)

10.1.43 What is an algebraic data type?

Algebraic data type, abbreviation ADT:

- A data type consisting of (a combination of):
- **product** types and **sum** types:
  - A product type is an "and"-type (aka record, struct), example:
    `case class Person(name: String, age: Int)`
    consists of the attributes name **AND** age.
  - A sum type is an "or"-type (aka disjoint union, co-product).
  - Example: `enum Color { case Red, Black}`
    Color can be either Red **OR** Black.
    A `Vegetable` is either a `Cucumber` **OR** a `Tomato`

```
sealed trait Vegetable { val weight: Int }
case class Cucumber(weight: Int) extends Vegetable
case class Tomato(weight: Int) extends Vegetable
```

https://en.wikipedia.org/wiki/Algebraic_data_type

10.1.44 A case class is a product.

The class `Person` below has a name **and** an age.

```
scala> case class Person(name: String, age: Int)

scala> val p = Person("Kim",42)
val p: Person = Person(Kim,42)

scala> p.isInstanceOf[Product]
val res1: Boolean = true

scala> p.product   // Press TAB
productArity          productElement    productElementName
productElementNames   productIterator   productPrefix

scala> p.productElementNames.toVector
val res2: Vector[String] = Vector(name, age)

scala> p.productElement
val res3: Int => Any = Lambda1981/0x00000008408f2840@44498af0

scala> p.productElement(0)
val res4: Any = Kim
```

10.1.45 Algebraic data type, combined product and sum

With trait, case class and object:

```
sealed trait PersonalIdentity  // sum
object PersonalIdentity:
  final case class Id(num: Long, isTemporary: Boolean) extends PersonalIdentity // product
  case object Missing extends PersonalIdentity
```

Equivalent with **enum**:

```
enum PersonalIdentity:
  case Id(num: Long, isTemporary: Boolean)
  case Missing
```

10.1.46 Algebraic data type with type parameter

This ADT has a case that is a **generic** case class:

```
sealed trait Maybe[+A]   // +A makes the type flexible, aka "covariant" more about that later
object Maybe:
  case class Some[A](a: A) extends Maybe[A]
  case object None extends Maybe[Nothing]
```

Equivalent with **enum**: (the compiler fills in **extends** ... automatically)

```
enum Maybe[+A]:
  case Some(a: A)
```

```
  case None
```

Do you recognize this? Compare to the built-in type `Option`

Exercise: implementation of `getOrElse` with extension method

```
extension [A](m: Maybe[A]) def getOrElse(default: A):A = ???
   /* Tip:
      use match */
```

10.1.47 Algebraic data type with type parameter

This ADT has a case that is a **generic** case class:

```
sealed trait Maybe[+A]   // +A makes the type flexible, aka "covariant" more about that late
object Maybe:
  case class Some[A](a: A) extends Maybe[A]
  case object None extends Maybe[Nothing]
```

Equivalent with **enum**: (the compiler fills in **extends** ... automatically)

```
enum Maybe[+A]:
  case Some(a: A)
  case None
```

Do you recognize this? Compare to the built-in type `Option`

Exercise: implementation of `getOrElse` with extension method

```
extension [A](m: Maybe[A]) def getOrElse(default: A):A = m match
  case Maybe.Some(a) => a
  case Maybe.None => default
```

10.2 Exercise `Excercise10`

Goals

- ☐ Be able to declare and use an inheritance hierarchy with multiple levels using the keyword **`extends`**.
- ☐ Be familiar with visibility rules in inheritance and the usefulness of private and protected members and the keywords **`private`** and **`protected`**.
- ☐ Be able to declare and use overridden members and the keyword **`override`**.
- ☐ Be able to declare and use a hierarchy of classes where constructor parameters are passed to superclasses.
- ☐ Be able to declare and use enumerated values with case objects and a common base type.
- ☐ Be familiar with the rules that apply when overriding different kinds of members.
- ☐ Be familiar with the usefulness of final classes and final attributes and the keyword **`final`**.
- ☐ Knowledge of these concepts: base type, supertype, subtype, runtime type, dynamic binding, polymorphism, trait, mix-in, overridden member, anonymous class, protected member, abstract member, abstract class, reference type,

Preparations

- ☐ Study the concepts in chapter 10

10.2.1 Basic Tasks; preparation for the lab

Task 1. *Match concepts with descriptions.*

Connect each concept with the (simplified) description that fits best:

base type	1	A	has supertype `AnyRef`, allocated on heap via reference
super type	2	B	can have many forms, e.g., one of several subtypes
subtype	3	C	class without name, extended with extra implementation
runtime type	4	D	a type that is more specific
dynamic binding	5	E	can have parameters, cannot be instantiated, cannot be mixed in
polymorphism	6	F	has no implementation
trait	7	G	has supertype `AnyVal`, stored directly on stack
mix-in	8	H	add properties with **`with`** and a trait
overridden member	9	I	abstract, can be mixed in, can have parameters
anonymous class	10	J	may be more specific than static type
protected member	11	K	visible only in subtypes
abstract member	12	L	runtime type determines which method executes
abstract class	13	M	subtype member replaces supertype member
sealed type	14	N	most general type in an inheritance hierarchy
reference type	15	O	a type that is more general
value type	16	P	subtyping outside this file is prevented

Task 2. *Common base type.* One often wants to put objects of different types in the same collection.

```
scala> class Cucumber(val weight: Int)
scala> class Tomat(val weight: Int)
scala> val gurkor = Vector(Cucumber(100), Cucumber(200))
scala> val vegetables = Vector(Cucumber(300), Tomat(42))
```

a) If a collection contains objects of several different types, the compiler tries to infer the most specific common type of the objects. What is the type of the value `vegetables` above?

b) Try to find the sum of the weights as below. What error does the second line give? Why?

```
scala> gurkor.map(_.weight).sum     // works
scala> vegetables.map(_.weight).sum  // does not work
```

c) You will now make it possible to access the weight of all vegetables by giving cucumbers and tomatoes a common base type that the different concrete vegetable types extend using the keyword **`extends`**. It is said that the subtypes `Cucumber` and `Tomat` **inherit** the properties of the supertype `Vegetable`.
Create a base type `Vegetable` with an abstract attribute `weight`. Then let the concrete vegetables inherit the base type:

```
scala> trait Vegetable { val weight: Int }
scala> class Cucumber(val weight: Int) extends Vegetable
scala> class Tomat(val weight: Int) extends Vegetable
scala> val gurkor = Vector(Cucumber(100), Cucumber(200))
scala> val vegetables = Vector(Cucumber(300), Tomat(42))
```

When does initialization of the attribute `weight` occur?

d) What is now the type of the variable `vegetables` above?

e) Is it now possible to sum the weights in `vegetables` with the expression below? Why?
`vegetables.map(_.weight).sum`

f) A trait resembles a class, but you cannot instantiate it directly. What error message do you get if you try to create an instance of a trait as below?

```
scala> trait Vegetable { val weight: Int }
scala> new Vegetable
```

g) The trait `Vegetable` has an abstract member `weight`. It is said to be abstract because it lacks implementation – the member has only a name and a type but no value. You can instantiate the abstract trait `Vegetable` if you provide what is "missing", namely a value for `weight`. You can supply the missing parts by "attaching" a block after the type name during instantiation. You then get what is called an **anonymous class**, in this case a rather unusual vegetable that is not a specific kind of vegetable but still has a weight.
What type and string representation does the `anonymousVegetable` below have?

```
scala> val anonymousVegetable = new Vegetable { val weight = 42 }
```

h) What error message do you get if you create an anonymous class `Vegetable` with a body missing the definition of `weight`?

Task 3. *Polymorphism in inheritance, i.e. subtype polymorphism.* Polymorphism means "many shapes". In the context of inheritance this means that several subtypes, for example `Cow` and `Pig`, can be handled jointly as if they were instances of the same supertype, such as `Animal`. Subclasses can implement a method with the same name in different ways. Which method is executed is determined at runtime depending on which subtype is instantiated. In this way, animals can come in many forms.

a) Implement the function `createAnimal` below so that it returns either a new `Cow` or a new `Pig` with equal probability.

```
scala> trait Animal { def makeNoise: Unit }
scala> class Cow   extends Animal { def makeNoise = println("Moo") }
scala> class Pig   extends Animal { def makeNoise = println("Oink oink") }
scala> def createAnimal(): Animal = ???
scala> val farm = Vector.fill(42)(createAnimal())
scala> farm.foreach(_.makeNoise)
```

b) Add an animal of type Horse that makes an appropriate sound and modify `createAnimal` so that it creates cows, pigs, and horses with equal probability.

Task 4. *Various types of integer pairs for the lab `snake0`.* **NOTE! Do this task *before* you look at given code in the lab to avoid spoiling the exercise.**
During this week's lab you will use various types of pairs representing direction and position on a two-dimensional game board, as well as the game board's size. Instead of using a regular 2-tuple for these three different types of pairs, you should create your own specific types all inheriting the base type `Pair[T]`. These types should all reside in the file `pairs.scala` in `package snake`.

```
// this is a sketch of the file pairs.scala
package snake

trait Pair[T]:
  def x: T
  def y: T
  // task a) add the concrete method tuple

// subsequent subtasks implement these subtypes of Pair:
//   case class Dim describes a 2-dimensional surface size
//   case class Pos describes a position on a surface of size Dim
//   enum Dir describes movement towards North, South, East, West
```

The difference between `Pair[T]` and a regular 2-tuple is that the members `x` and `y` are guaranteed to be of the *same* type, whereas a 2-tuple can contain elements of different types.
Figure 10.1 shows an image of the class hierarchy which you will develop step-by-step in subsequent tasks. The advantage of having different types of pairs is that it is more type-safe: the compiler helps us detect if we accidentally confuse, e.g., a position with a direction.

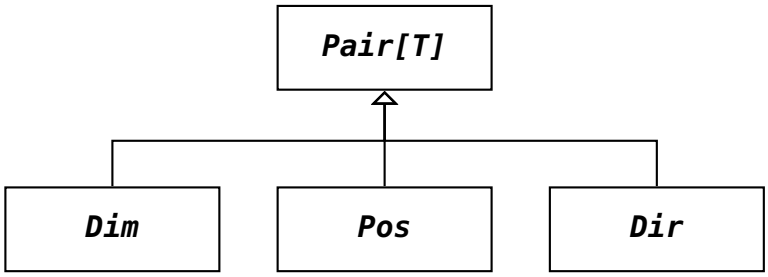


Figure 10.1: Inheritance hierarchy with `Pair[T]` as base type.

a) Open an editor and write the **`trait`** `Pair[T]` in a file `pairs.scala`. Also add a concrete method `tuple` in `Pair[T]` which returns a 2-tuple with the two elements of the pair, so that it is possible to convert `Pair` instances to 2-tuples if needed. Use REPL to test that this works:

```
scala> val p = new Pair[Int] { override val x = 10; override val y = 20 }
p: Pair[Int]{val x: Int; val y: Int} = $anon$1@784223e9

scala> p.tuple
val res0: (Int, Int) = (10,20)
```

b) Does the above code also work without the keyword **`override`** (test in REPL)? Why? When must **`override`** be used? What are the advantages and disadvantages of using **`override`** even when it is not necessary?

c) Create a case class `Dim` which extends `Pair[Int]`. Instances of this class will be used during this week's lab to represent a game board size by letting `x` specify the number of horizontal positions and `y` the number of vertical positions.
Also add a companion object `Dim` with an `apply` method which can create `Dim` instances given a 2-tuple. Test in REPL as below.

```
scala> Dim(50, 60)
val res1: Dim = Dim(50,60)

scala> Dim((60, 50))
val res2: Dim = Dim(60,50)

scala> res2.tuple
val res3: (Int, Int) = (60,50)
```

d) Add a case class `Pos` which extends `Pair[Int]` representing a position with an `x`-coordinate and a `y`-coordinate, both class parameters. The coordinates must be within a game board size given by the class parameter `dim` of type `Dim`. The coordinate positions are integers and counted from `0` up to (but not including) `dim.x` and `dim.y`, respectively.
Make the primary constructor in the case class `Pos` **private**, by writing the keyword **`private`** after the class name but before the parameter list, so that it is not possible to create instances via the primary constructor outside the class body and the companion object.
Implement methods `+` and `-` in the case class `Pos`. Both methods should take a parameter `p` of type `Pair[Int]` and return a new `Pos`, where `p.x` and `p.y` have been added or subtracted from the current position. Note that you should not write **`new`**

when creating a new instance, since these should always be created via the companion object's `apply` method, which is a "smart" factory method ensuring coordinates stay within the game board.
Add a companion object `Pos` with an `apply` method which creates a new `Pos` instance that ensures coordinates are always within `dim`. The arithmetic should be modulo the size `dim`, i.e. a position should never be outside the game board; instead, it wraps around (see example in REPL below).
Tip: Use `java.lang.Math.floorMod` which handles negative arguments to produce a positive result (unlike the modulo operator %).
Also add the factory method `random` which can create new random positions within `dim`. *Tip:* Use `scala.util.Random.nextInt`.
Test that it works according to below:

```
scala> Pos(-1,20,Dim(10,20))
val res4: Pos = Pos(9,0,Dim(10,20))

scala> new Pos(-1,20,Dim(10,20))  // forbidden with private primary constructor
-- Error:
1 |new Pos(-1,20,Dim(10,20))
  |    ^^^
  |constructor Pos cannot be accessed as a member of Pos

scala> Pos(0,0,Dim(5,5)) + Pos(6,12, Dim(5,5))
val res5: Pos = Pos(1,2,Dim(5,5))

scala> Pos(0,0,Dim(5,5)) - Pos(1,2, Dim(5,5))
val res6: Pos = Pos(4,3,Dim(5,5))

scala> for (_ <- 1 to 3) yield Pos.random(Dim(10,10))
val res7: IndexedSeq[Pos] =
  Vector(Pos(8,8,Dim(10,10)), Pos(2,6,Dim(10,10)), Pos(3,7,Dim(10,10)))
```

e) What happens if you forget to write **`new`** when calling the private constructor in your `apply` method? Why does this problem not occur in the `apply` method of `Dim`?

f) Add an **`enum`** `Dir` that extends `Pair[Int]` and has two **`val`** parameters `x` and `y`. Also add four **`case`** values all extending `Dir` representing a single-step movement in the four cardinal directions, by giving the parameters `x` and `y` values of 1, −1 or 0. North should be represented with x-coordinate 0 and y-coordinate −1, etc. Verify in REPL that the enumeration works.
Add an **`export`** that makes it enough to import `snake.*` to get all four directions directly visible (otherwise import of `Dir.*` is needed everywhere direction is used in and outside the `snake` package).

Task 5. *Supertype with parameter.* The Ministry of Education wants to keep track of certain persons with its new data system and therefore creates a class hierarchy as below. Write the code in an editor and test it in REPL with `sbt`.

```
class Person(val name: String)

class Academic(
  name: String,
  val university: String) extends Person(name)
```

```
class Student(
  name: String,
  university: String,
  val program: String) extends Academic(name, university)

class Researcher(
  name: String,
  university: String,
  val title: String) extends Academic(name, university)
```

a) Declare four different `val` variables with suitable names referencing instances of all the different classes above and give the attributes arbitrary initial values through various parameters to the constructors.

b) Write two statements: one that first inserts the instances into a `Vector` and one that then loops through the vector and prints all instances' `toString` and `namn`.

c) The Ministry of Education wants to prevent instantiating objects of types `Person` and `Academic`. This can be achieved by placing the keyword `abstract` before `class`. Update the code accordingly. What is the error message if you try to instantiate an `abstract class`? Is it just as good with a `trait`?

d) The Ministry wants to avoid implementing `toString`. Therefore, convert the types `Student` and `Researcher` to case classes. *Tip:* To avoid a compilation error (which one?), you need to use `override val` in the appropriate places. Create instances of the new case classes `Student` and `Researcher` and print their `toString`.

e) Use abstract attributes instead of parameters for the abstract types, so that you don't need to write `override val` in the class parameters for the concrete case classes. You should also add a case class `NonAcademic` which will be used in various statistical citizen surveys. Also, all persons are given a personal identification number represented as a `Long`. What does the Ministry of Education's code look like now, after all changes? Write a test program that creates some instances and prints their attributes.

10.2.2 Extra Tasks; additional practice

Task 6. *Base type `Shape` and subtypes `Rectangle` and `Circle`.* You are to create a small library for geometric shapes with immutable objects implemented using case classes. The geometric shapes have a common base type called `Shape`. Start from the code below.

```
case class Point(x: Double, y: Double):
  def move(dx: Double, dy: Double): Point = Point(x + dx, y + dy)

trait Shape:
  def pos: Point
  def move(dx: Double, dy: Double): Shape

case class Rectangle(pos: Point, width: Double, height: Double) extends Shape:
  def move(dx: Double, dy: Double): Rectangle = copy(pos = pos.move(dx, dy))

case class Circle(pos: Point, radius: Double) extends Shape:
  def move(dx: Double, dy: Double): Circle = copy(pos = pos.move(dx, dy))
```

a) Instantiate some circles and rectangles and do some relative moves of your instances by calling `move`.

b) Add a concrete method `moveTo` in `Point` which makes an absolute move to the coordinates `x` and `y`. Add an abstract method `moveTo` to `Shape` which is implemented in the subclasses. Test with REPL on some instances of `Rectangle` and `Circle`.

c) Add the method `distanceTo(that: Point): Double` in the case class `Point` which calculates the distance to another point using `math.hypot`. Paste into REPL and test on some instances of `Point`.

d) Add a concrete method `distanceTo(that: Shape): Double` in the trait `Shape` which calculates the distance to the position of another Shape. Test in REPL on some instances of `Rectangle` and `Circle`.

e) Make it so that `distanceTo` can be called with operator notation.

10.2.3 Advanced Tasks; challenges

Task 7. *Mixins.* One can extend a class with multiple traits using a comma-separated list. In this way, members can be distributed in different traits and common code parts can be reused by so-called **mixins**, as the following example shows.
An alternative bird taxonomy divides all birds into two specific categories: Crow (`Crow`) and Duck (`Duck`). Crows can fly but not swim, while ducks can swim and often also fly.

```
trait Flock:
  val sound: String
  def makeNoise: Unit = print(sound * 2)
  val isAbleToSwim: Boolean
  val isAbleToFly: Boolean

trait CanSwim          { val isAbleToSwim = true }
trait CannotSwim       { val isAbleToSwim = false }
trait CanFly           { val isAbleToFly = true }
trait MayBeAbleToFly   { val isAbleToFly = math.random() < 0.8 }

class Crow extends Flock, CanFly, CannotSwim:
  val sound = "caw"

class Duck extends Flock, CanSwim, MayBeAbleToFly:
  val sound = "quack"
  override def makeNoise = print(sound * 4)
```

a) A diligent ornithologist finds 42 birds in a perfect forest where all bird species are equally likely, represented by the vector `flock` below. Write in REPL an expression that finds how many of them are flying ducks by using the methods `filter`, `isInstanceOf`, `isAbleToFly`, and `size`.

```
scala> val flock =
         Vector.fill(42)(if math.random() > 0.5 then new Crow else new Duck)
scala> flock.foreach(_.makeNoise)
scala> val duckCount: Int =
```

b) If all the birds the ornithologist found would make sound exactly once each, how many krax and how many kwak would then be heard? Use the methods `filter` and

size, as well as the predicate isAbleToSwim to calculate the numbers of krax and kwak.

```
scala> val numberOfCrows: Int = ???
scala> val numberOfDucks: Int = ???
```

Task 8. *Final classes.* If you want to prevent a class from being extended, you can make it final by placing the keyword **final** before **class**.

a) Since the classification of birds in the previous task into either Duck or Crow is complete and there are no subtypes of either, it is appropriate to make these classes final. Change this in your code.

b) Try nevertheless to create a subclass SwimCrow **extends** Crow. What is the compiler error if you try to extend a final class?

Task 9. *Access rules in inheritance and the keyword **protected**.* If a member in a supertype is private, it cannot be accessed in a subclass. Sometimes you want the subclass to be able to access a member even if it should be inaccessible in other code.

```
trait Super:
  private val myHome = 42
  protected val ourHome = 42

class Sub extends Super:
  def reveal = myHome
  def cryptic = ourHome * math.Pi
```

a) What is the error message when class Sub tries to access myHome?

b) Declare Sub anew, but now without the forbidden method reveal. What is the error message if you try to access ourHome via an instance of class Sub? Try for example with (**new** Sub).ourHome

c) Is it possible to call the method cryptic on instances of class Sub?

Task 10. *Using **protected**.* The diligent ornithologist from task 7 will ring-tag all 42 birds found in the forest. While at it, the ornithologist also decides to try to find out how much noise is created by each bird species. For this purpose the ornithologist equips a Linux computer on all captured birds. You will help write the program.

a) Introduce a **protected var** soundCount in trait Fyle and write code in suitable places to count how many sounds each bird instance has uttered.

b) Add a method soundCount that returns the number of krax or kwak that a bird has uttered since its creation.

c) Why not use **private** instead of **protected**?

d) Why is it good to make the counter variable inaccessible from the ”outside”?

Task 11. *Mixin of properties.* It turns out that the diligent ornithologist from task 7 on page 100 had slept through a zoology lecture when a freshman at Natfak, and therefore completely missed the fyle category Chick. Help our poor ornithologist so that the bird hierarchy now also includes Chick. A Chick can fly like a Crow but is isSmall whereas a Crow isBig. A Chick chirps twice as many times as a Duck quacks. A Chick MaybeCanSwim with swimming ability probability 0.2. Let the ornithologist

again find 42 random birds in the forest and filter out suitable species. Then examine how they make noise, similarly to subtask 7b.

Task 12. *Inheritance hierarchy with mathematical numbers.* Study the deep inheritance hierarchy in the package `numbers` in the code on the following pages. The package `numbers` models various kinds of numbers in mathematics, aiming to offer a so-called DSL[3], i.e. a domain-specific language[4], here: the domain of mathematical numbers.
You can download the `numbers` code here:
github.com/lunduniversity/introprog/blob/master/compendium/examples/numbers.scala
Note especially the method `reduce` which reduces a number to its simplest form. The method `reduce` is overridden as appropriate with relevant reduction.

a) Draw a diagram of the type hierarchy, e.g., as an upside-down tree with the base type `Number` as root.

b) Write code that uses the various concrete classes in `package numbers`.

```
scala> numbers.  // Press Tab
AbstractComplex   AbstractNatural    AbstractReal   Frac    Nat      Polar
AbstractInteger   AbstractRational   Complex        Integ   Number   Real

scala> numbers.Integ(12)
res0: numbers.Integ = Integ(12)

scala> import numbers.Syntax._
import numbers.Syntax._

scala> 42.j
res1: numbers.Complex = Complex(Real(0),Real(42))

scala> 42.42.j
res2: numbers.Complex = Complex(Real(0),Real(42.42))
```

c) Change the method + in `trait Number` so that it becomes abstract and implement it in all concrete classes.

d) Implement more arithmetic operations and build on the code as you find interesting.

e) Make the `reduce` method in class `AbstractRational` use the Greatest Common Divisor (GCD) algorithm[5] as described here:
www.artima.com/pins1ed/functional-objects.html#6.8
so that numerator and denominator become as small as possible.

```
package numbers

trait Number:
  def reduce: Number = this
  def isZero: Boolean
  def isOne: Boolean
  def +(that: Number): Number = ???

object Number:
```

[3] https://en.wikipedia.org/wiki/Domain-specific_language
[4] https://stackoverflow.com/questions/49216312/what-is-dsl-in-scala
[5] https://sv.wikipedia.org/wiki/St%C3%B6rsta_gemensamma_delare

```
  def Zero = Nat(0)
  def One  = Nat(1)
  def Im(im: BigDecimal) = Complex(Real(0), Real(im))
  def Im(im: Real)       = Complex(Real(0), im)
  def j                  = Complex(Real(0), Real(1))
  def Re(re: BigDecimal) = Complex(Real(re), Real(0))
  def Re(re: Real)       = Complex(re, Real(0))

trait AbstractComplex extends Number:
  def re: AbstractReal
  def im: AbstractReal
  def abs = Real(math.hypot(re.decimal.toDouble,im.decimal.toDouble))
  def fi =  Real(math.atan2(im.decimal.toDouble, re.decimal.toDouble))
  override def isZero: Boolean = re.decimal == 0 && im.decimal == 0
  override def isOne: Boolean = abs.decimal == 1.0
  override def reduce: AbstractComplex = if im.decimal == 0 then re.reduce else this

final case class Complex(re: Real, im: Real) extends AbstractComplex

object Complex:
  def apply(re: BigDecimal, im: BigDecimal) = new Complex(Real(re), Real(im))

final case class Polar(
    override val abs: Real,
    override val fi: Real
  ) extends AbstractComplex:
  override def re = Real(abs.decimal.toDouble * math.cos(fi.decimal.toDouble))
  override def im = Real(abs.decimal.toDouble * math.sin(fi.decimal.toDouble))

object Polar:
  def apply(abs: BigDecimal, fi: BigDecimal) = new Polar(Real(abs), Real(fi))

trait AbstractReal extends AbstractComplex:
  def decimal: BigDecimal
  override def isZero = decimal == 0
  override def isOne = decimal == 1
  override def re = this
  override def im = Number.Zero
  override def reduce:  AbstractReal =
    if decimal == 0 then Number.Zero else if decimal == 1 then Number.One else this

final case class Real(decimal: BigDecimal) extends AbstractReal

trait AbstractRational extends AbstractReal:
  def numerator: AbstractInteger
  def denominator: AbstractInteger
  override def decimal = BigDecimal(numerator.integ)
  override def isOne = numerator.integ == denominator.integ
  override def reduce: AbstractRational =
    if denominator.isOne then numerator.reduce else this // should use GCD

final case class Frac(numerator: Integ, denominator: Integ) extends AbstractRational:
  require(denominator.integ != 0, "denominator must be non-zero")

object Frac:
  def apply(n: BigInt, d: BigInt) = new Frac(Integ(n), Integ(d))
```

```

trait AbstractInteger extends AbstractRational:
  def integ: BigInt
  override def numerator = this
  override def denominator = Number.One
  override def isZero = integ == 0
  override def isOne = integ == 1
  override def decimal: BigDecimal = BigDecimal(integ)
  override def reduce: AbstractInteger =
    if isZero then Number.Zero else if isOne then Number.One else this

final case class Integ(integ: BigInt) extends AbstractInteger

trait AbstractNatural extends AbstractInteger

final case class Nat(integ: BigInt) extends AbstractNatural:
  require(integ >= 0, "natural numbers must be non-negative")

extension (i: Int)    def j = Number.Im(i)
extension (d: Double) def j = Number.Im(d)
```

10.3 Team Lab: snake0

Goals

- ☐ Be able to use inheritance.
- ☐ Be able to override members in a supertype.
- ☐ Be able to explain the concept of dynamic binding.
- ☐ Be able to use abstract classes and create a class hierarchy.

Preparations

- ☐ Do exercise `Excercise09` in chapter 10.2, especially task 4.
- ☐ Read the documentation for `introprog.BlockGame`.
- ☐ Download the provided code via the course GitHub repository.
- ☐ Read through the entire lab and prepare yourself for the first group meeting.
- ☐ Discuss in your collaboration group how you will divide the code between you into several different parts, which you can work on separately. Such a part can be a class, a trait, an object, a package, or a function.
- ☐ Each part should have a **main responsible** individual.
- ☐ The division of work should be roughly evenly distributed among the group members.
- ☐ The person who is the main responsible for a certain part presents that part.
- ☐ You shall develop a common group checklist for code review. Each group member shall review at least one other group member's code according to the checklist.
- ☐ The group collaboration takes place over **two weeks** divided into two partial presentations. At the first presentation, the work plan and ongoing development shall be presented. At the second occasion, the completed solutions shall be presented by the respective main responsible individual.
- ☐ At the first presentation, you shall explain to the supervisor how you have divided the code and who is primarily responsible for what and what your responsibility includes, as well as how you practically work to synchronize your development.
- ☐ The group laboratory work is an extra large assignment and group work requires lead time so that you have enough time to coordinate among yourselves. You therefore need to plan to work on some part of the group lab virtually every day during the available weeks, and be ready to contribute to discussions.
- ☐ Meet in your collaboration group and discuss your working method based on the following questions:
  - Which requirements are you going to implement?
  - How will you work with shared code components?
  - How will you share the code components you develop individually? It is educational to try manual merge of changes initially.
    - If you do not use git: Consider when and how you will merge your changes. Use, e.g., a USB drive or a shared storage location on the network that only your group has access to. Communicate when it is time to sync via chat or email.
    - If you use git: How will you use branches? How should your `.gitignore` look?

10.3.1 Background

The game *Snake*[6] became very popular in Sweden already in the 1980s, often played on the legendary ABC80 computer. The game exists in several variants, both for a single player and as a duel between two players. Each player controls a worm with a head and tail that constantly moves forward. The goal is to avoid running into a worm tail and to collect points by e.g. eating an apple.

Figure 10.2 shows a start dialog where you can select the number of players, as well as examples of a game with one and two players. In the one-player variant, the worm's head approaches the apple and may manage to eat it if the player steers correctly. In the two-player variant, the green worm wins because the blue worm accidentally ran into the green worm's tail.

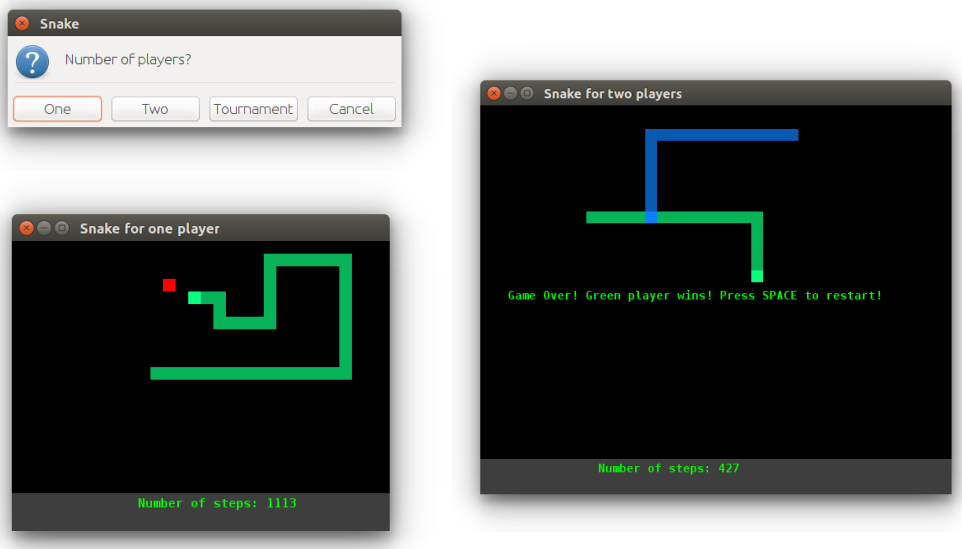


Figure 10.2: The game Snake for one player with apple and for two players without apple.

10.3.2 Mandatory functional requirements

The following functional requirements must be met by your program if you are six people in the group. If you are fewer, the mandatory requirements are those shown in Table 10.1.

- ☐ **`Player`**. There must be players that correspond to human users and who have a name and four keys they can play with. Each player has their own snake which they can control with their keys.
- ☐ **`Snake`**. There must be snakes. A snake consists of a number of blocks, where the foremost block is called the head and the rest of the blocks are called the tail. The head has a lighter color than the body. The tail's length grows during the game. A snake moves in a certain direction and each player can change the direction of their snake with their keys, in one of four directions `North`, `South`, `East` or `West`.

[6] Also called "the worm". https://en.wikipedia.org/wiki/Snake_(video_game)

- ☐ **Apple**. There must be (at least one) apple. An apple consists of a red block and is in a random position. An apple can be eaten by a snake if the snake's head hits the apple. Each time an apple is eaten by a snake, the apple teleports to a new position and can be eaten again.
- ☐ **Banana**. There must be (at least one) banana. A banana consists of three vertical yellow blocks and is in a random position. A banana is eaten by a snake if the snake's head hits the banana. Each time a banana is eaten by a snake it teleports to a new random position and can be eaten again.
- ☐ **Monster**. There must be (at least one) monster. A monster consists of five pink blocks in a cross formation. A monster spawns at a random position and moves in a direction determined at the monster's birth. A monster will be eaten and die if the snake's head touches any monster block.
- ☐ **OneplayerGame**. It must be possible to play alone. In the single-player variant there is one snake and at least one apple (and possibly bananas and monsters as well). Each time the user's snake manages to eat a fruit, the user gets points. When the snake has eaten a certain number of apples, or if the snake has been eaten by a monster, the game ends and the points are shown. A snake's tail should grow at regular time intervals or when it eats fruit.
- ☐ **TwoplayerGame**. It must be possible to play two-player. In the two-player variant there are two snakes. There are also apples, bananas, and monsters. If a snake eats a banana, its tail grows. When the snake has eaten a certain number of apples, or if the snake has been eaten by a monster, the game ends and the points are shown. A snake's tail should grow at regular time intervals or when it eats fruit.
- ☐ **Settings**. Settings for the game must be configurable through a text file which is loaded at the start of the game. Settings should be a context parameter.

Table 10.1: Requirements that must be implemented at minimum depending on group size. If you have special reasons, you can after approval from the course coordinator do the lab individually.

Requirement / Number of people	1	2	3	4	5	6
`Player`	√	√	√	√	√	√
`OnePlayerGame`	√				√	√
`TwoPlayerGame`		√	√	√	√	√
`Snake`	√	√	√	√	√	√
`Apple`	√		√	√	√	√
`Banana`				√		√
`Monster`			√			√
`Settings`	√	√	√	√	√	√

10.3.3 Mandatory design requirements

- ☐ Snake game must be able to start with the main program below. The main program may be changed as needed in accordance with the minimum requirements regarding group size in Table 10.1, as well as optional extra requirements in section 10.3.4, and your own ideas.

```
package snake

def createOnePlayerGame(): Unit =
```

```
  Settings.default.windowTitle = "Snake: One Player"
  OnePlayerGame().play()

def createTwoPlayerGame(): Unit =
  Settings.default.windowTitle = "Snake: Two Player"
  TwoPlayerGame().play()

@main
def run: Unit =
  val buttons =
    Seq("One", "Two", "Competition", "Tournament", "Cancel")
  val selected =
    introprog.Dialog.select("Number of players?", buttons, "Snake")
  selected match
    case "One"         => createOnePlayerGame()
    case "Two"         => createTwoPlayerGame()
    case "Competition" => println(s"TODO: $selected")
    case "Tournament"  => println(s"TODO: $selected")
    case  _            => println("Goodbye!")
```

☐ The game must be built on `introprog.BlockGame` according to the type hierarchy in fig. 10.3.

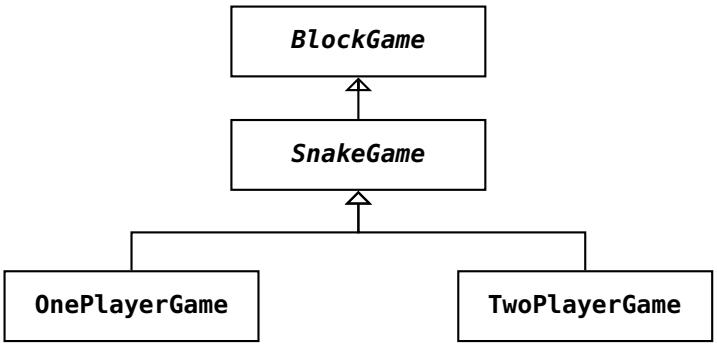


Figure 10.3: Inheritance hierarchy with the class `introprog.BlockGame` as base type.

☐ Snakes, monsters, and fruits must derive from the base type `Entity` according to the type hierarchy in 10.4.

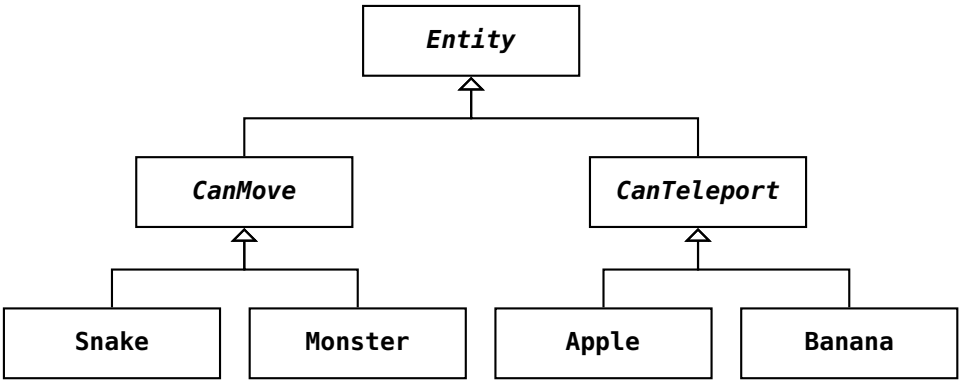


Figure 10.4: Inheritance hierarchy with the class `Entity` as base type.

☐ `Entity` represents a creature in a game and should look as follows:

```
package snake

trait Entity:
  def draw(): Unit

  def erase(): Unit

  def update(): Unit

  def reset(): Unit

  infix def isOccupyingBlockAt(p: Pos): Boolean
```

The methods `draw` and `erase` are called when drawing and erasing respectively. The method `reset` restores the original state. The method `update` is called once in each round in the game loop. The predicate `isOccupyingBlockAt` returns true if the position `p` is among the blocks that the creature occupies on the screen.

☐ `CanMove` represents an entity that can move at a certain speed, as follows:

```
package snake

trait CanMove extends Entity:
  def move(): Unit

  var movesPerSecond: Double = 20.0

  final def millisBetweenMoves: Int =
    (1000 / movesPerSecond).round.toInt max 1

  private var _timestampLastMove: Long = System.currentTimeMillis

  final def timestampLastMove = _timestampLastMove

  override final def update(): Unit =
```

```
    // move if time has passed millisBetweenMoves
    if System.currentTimeMillis >
         _timestampLastMove + millisBetweenMoves
    then
      _timestampLastMove = System.currentTimeMillis
      move()
```

☐ `CanTeleport` represents an entity that exists in a certain location but after a certain number of updates teleports unpredictably to a new position:

```
package snake

trait CanTeleport extends Entity:
  private var _pos = teleport()

  def pos: Pos = _pos

  protected var nbrOfStepsSinceLastTeleport = 0

  def teleportAfterSteps: Int

  def teleport(): Pos

  def update(): Unit =
    nbrOfStepsSinceLastTeleport += 1
    if nbrOfStepsSinceLastTeleport > teleportAfterSteps
    then reset()

  def reset(): Unit =
    nbrOfStepsSinceLastTeleport = 0
    _pos = teleport()
```

☐ There must be an enumeration `State` in the singleton object `SnakeGame` representing the game's overall state as follows:

```
package snake

object SnakeGame:
  enum State:
    case Starting, Playing, GameOver, Quitting
  export State.* // makes all states visible in SnakeGame
```

☐ In each round in the game loop, the following logic shall be executed. This code is preferably placed in `gameLoopAction`, see further `SnakeGame` in section 10.3.5.

```
    if state == Playing && !isPaused then
      _iterationsSinceStart += 1
      entities.foreach(_.erase())
      entities.foreach(_.update())
      entities.foreach(_.draw())
```

```
      onIteration()
      if isGameOver then enterGameOverState()
```

☐ There must be a singleton object `Colors` where all colors used in the game are collected.

☐ The file `pairs.scala` should, according to the lab preparations in exercise 4 on page 96, contain `Pair[T]`, `Dim`, `Pos`, `Dir`, `North`, `South`, `East`, `West`. See workspace here:
https://github.com/lunduniversity/introprog/tree/master/workspace/

☐ The class `Player` should look as follows:

```
package snake

class Player(
  var name: String,
  var keyMap: Player.KeyMap,
  val snake: Snake,
  var points: Int = 0,    // TODO: count points when e.g. eating apple
):
  def handleKey(key: String): Unit =
    ??? //if key is included in keyMap then update snake.dir

object Player:
  enum KeyMap(left: String, right: String, up: String, down: String):
    val dir = Map(left -> West, right -> East, up -> North, down -> South)
    case Letters extends KeyMap("a", "d", "w", "s")
    case Arrows extends KeyMap("Left", "Right", "Up", "Down")
```

10.3.4 Optional requirements – each person must choose at least one

Each person in the group must implement *at least one* (preferably several) of the requirements below. Implementing several of these requirements will make the game significantly more fun.

- ☐ **`Points`**. Introduce a points system where the score depends on e.g. the length of the tail, number of steps, number of turns, number of apples eaten, etc.
- ☐ **`Highscore`**. The game should show a list of players who have achieved the highest scores.
- ☐ **`Apple`**. If not already included among mandatory requirements according to 10.1.
- ☐ **`Monster`**. If not already included among mandatory requirements according to 10.1.
- ☐ **`Banana`**. If not already included among mandatory requirements according to 10.1.
- ☐ **`SelfTailCrash`**. If a player runs into their own snake's tail, the game is lost. (If this requirement is not implemented, then the player *may* pass through their own tail without consequence.)

- ☐ **`BoundaryCrash`**. If a player moves off the game board, the game is lost. (If this requirement is not implemented, the snake should continue on the other side of the board when it reaches the edge.)
- ☐ **`EnterPlayerName`**. Players can enter their name, e.g. via a dialog or through arguments to `main`. The name is used in the message field during scoring and in messages about who won.
- ☐ **`OnePlayerGame`**. You can choose to implement `OnePlayerGame` if it is not already part of the mandatory requirements.
- ☐ **`TwoPlayerComp extends Competition`**. Two players should be able to compete in a best-of-n series of `TwoPlayerGame.play` matches, where the one who wins most matches is the overall winner.
- ☐ **`SinglePlayerComp extends Competition`**. Several players should be able to compete in single-player Snake, where the player with the highest score over n `OnePlayerGame` games is the overall winner.
- ☐ **`Tournament extends Competition`**. Many players should be able to play a tournament.[7] Player names are read from a text file. Optional variants:
  - ☐ **`KnockOut extends Tournament`**. It should be possible to play a knockout tournament, which ends with final after semi-final, etc., depending on the number of players.
  - ☐ **`RoundRobin extends Tournament`**. It should be possible to play a round-robin tournament where all possible player pairs meet in random order.

10.3.5 Tips and suggestions

This section presents sketches for some of the classes needed in accordance with the design requirements. It is allowed to change, remove and add, as long as the mandatory design requirements are met. The code can be found here:
https://github.com/lunduniversity/introprog/tree/master/workspace/
Here is a sketch of the `Snake` class:

```
package snake

import java.awt.Color

class Snake (
  val initPos: Pos,
  val initDir: Dir,
  val headColor: Color,
  val tailColor: Color,
 )(using ctx: SnakeGame, settings: Settings) extends CanMove:
  var dir: Dir = initDir
  val initBody: List[Pos] = List(initPos + initDir, initPos)
  val body: scala.collection.mutable.Buffer[Pos] = initBody.toBuffer

  val initLength: Int = settings.snake.initLength
  val growEvery: Int = settings.snake.growEvery
  val startGrowingAfter: Int = settings.snake.startGrowingAfter

  private var _nbrOfSteps = 0
  def nbrOfSteps: Int = _nbrOfSteps

  private var _nbrOfApples = 0
```

[7] https://en.wikipedia.org/wiki/Tournament

```
  def nbrOfApples: Int = _nbrOfApples

  def reset(): Unit = ???  // restore start state, give correct tail length

  def grow(): Unit = ??? // grow in the right direction with extra tail position

  def shrink(): Unit = ??? // shrink the tail if the body length is greater than 2

  def isOccupyingBlockAt(p: Pos): Boolean = ??? // check if p is in the body

  def isHeadCollision(other: Snake): Boolean = ??? // check if the heads collide

  def isTailCollision(other: Snake): Boolean = ??? // my head in someone else's tail

  private var _isEatenByMonster: Boolean = false
  def isEatenByMonster: Boolean = _isEatenByMonster
  def eatenByMonster(): Unit = ???

  def move(): Unit = ???
    // grow and shrink according to rules
    // actions if eats fruit or get eaten by monster

  override def toString = // bra vid println-debugging
    body.map(p => (p.x, p.y)).mkString(">:)", "~", s" going $dir")

  def draw(): Unit = ???

  def erase(): Unit = ???
```

Here is a sketch of the abstract class `SnakeGame` with the abstract methods `isGameOver` and `play` which are overridden in the subsequent subclasses `OnePlayerGame` and `TwoPlayerGame`:

```
package snake

object SnakeGame:
  enum State:
    case Starting, Playing, GameOver, Quitting
  export State.*

abstract class SnakeGame(settings: Settings) extends introprog.BlockGame(
  title                 = settings.windowTitle,
  dim                   = settings.windowSize,
  blockSize             = settings.blockSize,
  background            = settings.background,
  framesPerSecond       = settings.framesPerSecond,
  messageAreaHeight     = settings.messageAreaHeight,
  messageAreaBackground = settings.messageAreaBackground
):
  // examples of different visibility (discuss choice of visibility based on use)
  var entities: Vector[Entity] = Vector.empty
  protected var players: Vector[Player] = Vector.empty
  private var isPaused = false

  import SnakeGame.*

  protected var state: State = Starting
  private var _iterationsSinceStart = 0
```

```
def iterationsSinceStart = _iterationsSinceStart

def enterStartingState(): Unit = ??? //erase, announce "press space to start"

def enterPlayingState(): Unit = ??? //erase, for each entity: reset & draw

def enterGameOverState(): Unit = ??? // inform "game over"

def enterQuittingState(): Unit =
  println("Goodbye!")
  pixelWindow.hide()
  state = Quitting

def randomFreePos(): Pos =
  ??? // pull random-pos until free space, used by fruit, monster

override def onKeyDown(key: String): Unit =
  println(s"""key "$key" pressed""")
  state match
    case Starting => if key == " " then enterPlayingState()

    case Playing =>
      if key == "Esc" then
        println(s"Toggle pause: isPaused == $isPaused")
        isPaused = !isPaused
      else
        players.foreach(_.handleKey(key))

    case GameOver =>
      if key == " " then enterPlayingState()
      else if key == "Esc" then enterQuittingState()

    case _ =>

override def onClose(): Unit =
  println("Window Closed!")
  enterQuittingState()

/** Implement this with logic for when to end the game */
def isGameOver: Boolean

/** Override this if you want to add game-logic in gameLoopAction
 *  Call super.onIteration() if you want to keep the step counter.
 */
def onIteration(): Unit =
  clearMessageArea()
  drawTextInMessageArea(s"Number of steps: $iterationsSinceStart", 10, 2)

override def gameLoopAction(): Unit =
  if state == Playing && !isPaused then
    _iterationsSinceStart += 1
    entities.foreach(_.erase())
    entities.foreach(_.update())
    entities.foreach(_.draw())
    onIteration()
    if isGameOver then enterGameOverState()

final def start(ps: Player*)(es: Entity*): Unit =
  players = ps.toVector
```

```
    entities = es.toVector
    isPaused = false
    pixelWindow.show()  // enables restart even if the window has been closed
    enterStartingState()
    gameLoop(stopWhen = state == Quitting)

  /** Implement this with a call to start with specific players and entities. */
  def play(playerNames: String*): Unit
```

If the group considers using git and GitHub:

- Discuss in the group whether everyone has sufficient knowledge to work with git and GitHub, as well as pros and cons of doing so.
- If not all are comfortable with git and GitHub, consider whether you want to do manual version control with copying new files via USB drive, ssh or uploading/downloading via cloud storage. After experiencing manual version control concretely, you gain a deeper understanding of the need for tool support for version control, and it is more motivating to learn git.
- Discuss workflows. How will you use GitHub issues, git branches, etc.? Or should everyone push to the main branch? Will you use GitHub pull requests, reviews, etc.?
- Check that you have a `.gitignore` before pushing, so that e.g. binary files do not end up in your repo, which can cause tricky cleanup work and unnecessary merge conflicts. An example of a suitable `.gitignore` is here: https://github.com/lunduniversity/introprog/blob/master/workspace/w10_snake/.gitignore
- **Be careful to make your GitHub repo private!** It is not permitted to share lab solutions on the internet – doing so may result in disciplinary action for assisting cheating, and you may be suspended from your studies.

Chapter 11

Variance and Context Parameters

Concepts included in this week's studies:

- ☐ upper and lower type bound
- ☐ variance
- ☐ contravariance
- ☐ covariance
- ☐ type wildcard
- ☐ context bound
- ☐ type constructor
- ☐ self type
- ☐ type wildcard
- ☐ given value (given)
- ☐ context parameter (using)
- ☐ ad hoc polymorphism
- ☐ type class
- ☐ api
- ☐ code readability
- ☐ reviews

11.1 Theory

11.1.1 Type Parameter, Generic Structure, Type Constructor

- With the help of **type parameters** you can create **generic structures**.
- Type parameters are written inside square brackets, for example: `[T, U]`
- A generic structure works for *arbitrary* types, **unknown** at **declaration**.
- The compiler ensures **correct usage** already at *compile time*.
- At the usage site in the code (at invocation or instantiation), the type parameters are **bound** to "real" types according to the type system's rules.

```
case class Pair[A, B](a: A, b: B):  // A and B are unbound (free) within []
  def swap: Pair[B, A] = Pair(b, a) // A and B bound since swap lacks []
```

- If you don't write out the types, the compiler tries to **infer** them:

```
scala> val p = Pair("hello", 42)
val p: Pair[String, Int] = Pair(hello,42)

scala> p.swap
val res0: Pair[Int, String] = Pair(42,hello)
```

- The class `Pair` is called a **type constructor** since the "final" type `Pair[String, Int]` is "constructed" at usage.

Exercise: Modify the class `Pair` above so that both elements in the pair have the same type.

11.1.2 Different Ways to Restrict Generic Types

In Scala, there are several ways to restrict which types you want to allow – the compiler helps you enforce this.

- **Upper bound** with `A <: B`
- **Lower bound** with `A >: B`
- **Self type** restricts how you can mix in a trait.
  Specify any name followed by a type annotation and a rocket in the class body:

```
trait MyTrait:
  self: ThisType =>

  // Code here can assume that this instance is of type ThisType
  // the name, here self, is arbitrary, but 'self' is the most common choice
```

  The compiler checks that mixin is done with `ThisType`.
- There is also something called **context bound** where `[A: B]` makes the type constructor `B` into a context parameter `B[A]`
  – more about this later in the section on contextual abstractions.

11.1.3 Upper and Lower Type Bounds

Using the type operators `<:` and `>:` you can restrict which types can be bound to a type parameter in a generic structure.

- Mnemonic for type bounds: **colon at the end**.

Assume that `T` is an *unbound* type parameter, while `U` and `L` are **bound**.

- with `T <: U`, `U` becomes an upper bound for `T`
  `U` is the "highest" possible type that `T` can be (like "less or equal")
- with `T >: L`, `L` becomes a lower bound for `T`
  `L` is the "lowest" possible type that `T` can be (like "greater or equal")
- For all types `A`, `A >: Nothing` and `A <: Any` hold

Example:

```
trait Vegetable { def weight: Int }

def f[T <: Vegetable](x: T): Int = x.weight
```

The compiler uses the upper type bound to conclude that the method `weight` is available via the generic parameter `x`.

11.1.4 Example of Upper and Lower Type Bounds

```
class Animal
class Cat extends Animal
class Dog extends Animal
class RobotDog extends Dog

def testUpperBound[T <: Dog](x: T) = println(x)
def testLowerBound[T >: Dog](x: T) = println(x)
```

```
scala> testUpperBound[Cat](Cat())
-- Error:
1 |testUpperBound[Cat](Cat())
  |          ^
  |          Type argument Cat does not conform to upper bound Dog

scala> testLowerBound[RobotDog](RobotDog())
-- Error:
1 |testLowerBound[RobotDog](RobotDog())
  |          ^
  |          Type argument RobotDog does not conform to lower bound Dog
```

11.1.5 What is Variance?

Is a cat carrier also an animal carrier??

Allowing **variance** makes generic structures more **flexible**.

11.1.6 Why is Variance Needed?

```
trait Animal
case class Cat() extends Animal
case class Dog() extends Animal

case class Cage[A](a: A)
```

```
scala> val animalCage: Cage[Animal] = Cage[Cat](Cat())
```

This does not work! The Cage above is **invariant** (inflexible in its type parameter).

```
-- Error:
1 |val animalCage: Cage[Animal] = Cage[Cat](Cat())
  |                   ^^^^^^^^^^^^^^^^^
  |                   Found:    Cage[Cat]
  |                   Required: Cage[Animal]
```

Why does this work then??

```
scala> val animals: Vector[Animal] = Vector[Cat](Cat())
val animals: Vector[Animal] = Vector(Cat())
```

`Vector` is declared as **covariant** and therefore more flexible!

11.1.7 Covariance

- For a **covariant** type constructor `F` it holds that:
  if `T <: U` **then** `F[T] <: F[U]` (subtype-flexible "in the same direction")

- In Scala, you get a covariant type constructor with a + before the type parameter:

```
trait Animal
case class Cat() extends Animal
case class Dog() extends Animal

case class Cage[+A](a: A)  // covariant thanks to + before A
```

- This now works :)

```
scala> val animalCage: Cage[Animal] = Cage[Cat](Cat())
val animalCage: Cage[Animal] = Cage(Cat())
```

- `Cage[Cat]` is now a **subtype** of `Cage[Animal]`.
- **Immutable** collections are often covariant, e.g. Vector, Option, List.
- Generic enumerations **need** to be covariant so that the different cases can have a flexible type parameter. `Empty` is a `Toilet[Nothing]` here:

```
enum Toilet[+T]:
  case Occupied(x: T)
  case Empty
```

11.1.8 Contravariance

Is a cat veterinarian also an animal veterinarian?

Sometimes we want the variance the other way around: A veterinarian who can only treat cats should not be allowed to treat any animal.

11.1.9 Contravariance

- For a **contravariant** type constructor `F` it holds that:
  if `T <: U` **then** `F[U] <: F[T]` (subtype-flexible "in the opposite direction")

- You create a contravariant type constructor with - before the type parameter:

```
trait Animal
case class Cat() extends Animal
case class Dog() extends Animal

class Veterinarian[-A]:    // contravariant with -
  def treat(x: A) = println(s"$this has treated $x")
```

- This now works "backwards" (but not the other way):

```
scala> val catVet: Veterinarian[Cat] = Veterinarian[Animal]()
val catVet: Veterinarian[Cat] = Veterinarian@77b6d94c

scala> val catVet: Veterinarian[Animal] = Veterinarian[Cat]()
-- Error:
1 |val catVet: Veterinarian[Animal] = Veterinarian[Cat]()
  |                                    ^^^^^^^^^^^^^^^^^
  |                                    Found:    Veterinarian[Cat]
  |                                    Required: Veterinarian[Animal]
```

11.1.10 Variance Problems – Thanks Compiler!

- A type constructor **cannot** be covariant if the type parameter appears as a method parameter (so-called *contravariant position*):

```
trait Animal
case class Cat() extends Animal
case class Dog() extends Animal

case class Cage[+A](a: A):
  def changeTo(x: A): Cage[A] = Cage(x)     // error!
```

```
-- Error:
6 |  def changeTo(x: A): Cage[A] = Cage(x)
  |          ^^^^
  |  covariant type A occurs in contravariant position in type A of parameter x
```

- Here the type must be allowed to vary "upwards" using a **lower bound**:

```
case class Cage[+A](a: A):
  def changeTo[B >: A](x: B): Cage[B] = Cage(x)
```

```
scala> Cage[Cat](Cat()).changeTo(Dog())
val res1: Cage[Animal] = Cage(Dog())
```

11.1.11 When to Use Which Kind of Variance?

- Immutable generic classes that have methods which are **producers** of new immutable generic types are often best declared **covariant**, e.g. `class Vector[+T]`, `trait Option[+T]`.
- A type constructor over `T` which has methods that **consume** `T` may be **contravariant**, e.g. `class Veterinarian[-T]`.
  It can also be **covariant** if consuming methods **widen** the input parameter type to `B` where `B >: T`.
- A type constructor with multiple parameters can be **both** contravariant **and** covariant, e.g. `trait Function1[-A, +B]`
  Arguments of type `A` are *consumed* and return values of type `B` are *produced*. [1].

11.1.12 Invariance with Mutability

- **Mutable types** can only be **invariant**, otherwise it would be **chaos**!
- Suppose there is a `Cage` that can be mutated in place via the method `changeTo` *and at the same time* were covariant (warning for careful coders):

```
notscala> val catCage: Cage[Cat] = Cage(Cat())

notscala> val animalCage: Cage[Animal] = catCage  // a covariant reference to the same

notscala> animalCage.changeTo(Dog())           // mutate in place

notscala> val cat: Cat = catCage.release      // CHAOS! type-safe dogcat monster :(
```

11.1.13 Wildcard Types: Warning about Unbounded Types

Invariant classes have inflexible type parameters, restricting usage.

```
scala> class Box[A](val value: A)

scala> val b: Box[Any] = Box[Int](42)
-- Error:
1 |val b: Box[Any] = Box[Int](42)
  |                   ^^^^^^^^^^^^
  |                   Found:    Box[Int]
  |                   Required: Box[Any]
```

Questionable solution: A **wildcard type** is written with a question mark and gives a completely unknown type which is **not checked by the compiler**.

```
scala> var whatever: Box[?] = Box[Int](42)
var whatever: Box[?] = Box@70d7a49b

scala> whatever = Box("hello")
whatever: Box[?] = Box@43120a77
```

[1] The function type `A => B` is expanded by the compiler to `Function1[A, B]`

Use wildcards only if you *really* need them.

11.1.14 More about Variance for the Curious

- https://docs.scala-lang.org/scala3/book/types-variance.html
- https://en.wikipedia.org/wiki/Covariance_and_contravariance_(computer_science)
- https://www.artima.com/pins1ed/type-parameterization.html

11.1.15 Self Type + Anonymous Class to "Inject" Dependencies

- You can get an explicit, arbitrary name, e.g. `self`, for "myself", i.e. this instance, by writing `self =>` in the body of a trait or class.
- A type restriction on the own instance is called a **self type**.
- Useful for mixin: you can restrict what this trait may mix in with.

```
trait Vegetable { def weight: Int }

trait CanBePeeled:
  self: Vegetable =>
  def weightAfterPeeling = 0.99 * weight
```

- Note that *even* if `CanBePeeled` *does not* `extend`, it can *still* access `weight` because the self type requires `CanBePeeled` to be a `Vegetable`.
- Now you can combine **anonymous class** and **mixing in** at **runtime** with **`with`**:

```
scala> val g = new Vegetable with CanBePeeled { val weight = 100 }
val g: Vegetable & CanBePeeled = anon1@70a91d72

scala> g.weightAfterPeeling
val res0: Double = 99.0
```

- This is an elegant way to **inject dependencies**. `https://en.wikipedia.org/wiki/Dependency_injection`

11.1.16 What is the Advantage of Self Types Instead of Inheritance?

Inheritance does **not** allow mutual (cyclic) dependencies...

```
scala> trait Tulip extends Rose; trait Rose extends Tulip
-- [E110] Syntax Error: ----------------------------------------------------------
1 |trait Tulip extends Rose; trait Rose extends Tulip
  |                     ^^^
  |                     Cyclic inheritance: trait Tulip extends itself
  |
```

...whereas self types *can* be **mutually dependent**:

```
trait Tulip:
  self: Rose =>

trait Rose:
  self: Tulip =>
```

```
scala> val tulipAndRose = new Tulip with Rose
val tulipAndRose: Tulip & Rose = anon1@1c63b39a
```

https://en.wikipedia.org/wiki/Tulip_rose

11.1.17 Type Unions with the Or-Operator

Type unions are created with the type operator | between types:

```
scala> var x: Int | String = 42
var x: Int | String = 42

scala> x = "hello"
x: Int | String = hello

scala> type IntOrErr = Int | String
// defined alias type IntOrErr = Int | String

scala> def div(nom: Int, denom: Int): IntOrErr =
          if denom != 0 then nom / denom else "div. by zero"
```

Advantage compared to a class: rudimentary simplicity.
Disadvantages: you cannot declare parameters, members and everything else a class can; to some extent, this can be compensated with **extension** and **match**.
Read more here: https://dotty.epfl.ch/docs/reference/new-types/union-types.html

---

11.1.18 Intersection Type with And-Operator

When mixing in traits, the supertype becomes an intersection type, indicated with the type operator & in the example below:

```
trait Vegetable(val weight: Int)
trait HasTaste(val taste: String)

object Cucumber extends Vegetable(42), HasTaste("watery")
object Tomato extends Vegetable(43), HasTaste("tart")
```

```
scala> Vector(Cucumber, Tomato)
val res0: Vector[Vegetable & HasTaste] =
  Vector(Cucumber@560742f7, Tomato@3cc82e23)
```

It is possible to create your own intersection types with **with**:

```
scala> class X { val x = 42 }
scala> trait Y { val y = "hello" }
scala> val a: X & Y = new X         // -- Error: Found: X Required: X & Y
scala> val b: X & Y = new X with Y  // Works! An anonymous class of type X & Y
```

11.1.19 Transparent Types

You can make your trait **transparent** for type inference of the supertype during mixin with the keyword **transparent**:

```
trait Vegetable(val weight: Int)
transparent trait HasTaste(val taste: String)

object Cucumber extends Vegetable(42), HasTaste("watery")
object Tomato extends Vegetable(43), HasTaste("tart")
```

```
scala> Vector(Cucumber, Tomato)
val res0: Vector[Vegetable] =
  Vector(Cucumber@560742f7, Tomato@3cc82e23)
```

What type would be shown without **transparent**?
`Vector[Vegetable & HasTaste]`

11.1.20 What is a Good API?

- An API ((eng. *Application Programming Interface*)) is an interface for application programming.
- By "interface" we mean a boundary in the code between what is visible outside and what is "under the hood".
- Many quality aspects are desirable for an API:
  - Easy to use externally even if internally it is complex.
  - What does "easy to use" mean?
    - Easy to learn
    - Easy to remember
    - Easy to understand
    - Pleasant experience (subjective)
  - Solves a (general) problem well. What does "well" mean?
    - Fast (high performance)
    - Efficient (effective use of resources, e.g. memory)
    - ...
- Interesting talk by Joshua Bloch (Google) on good API design:
  - https://research.google.com/pubs/archive/32713.pdf
  - https://youtu.be/aAb7hSCtvGw

11.1.21 API Design with Scala

- Combine paradigms OO + FP to choose the best fitting solution.
- Advanced abstraction mechanisms that can be challenging for the API designer but at the same time become simple for the API user.
- Makes it possible to create an API that works in several runtime environments:
  - On desktop and in back-end with JVM and Graal VM
  - In frontend browser with Scala JS
  - Directly to platform-specific machine code with Scala Native
- Advanced features we have not covered in this course but might help API designers to make easy-to-use APIs:
  - Context functions `?=>`
  - Opaque types `opaque type`
  - Type class derivation `derives`
  - Metaprogramming `inline`
  - Type lambda and match types `=>>`
  - ... (research ongoing)

11.1.22 Context is Crucial When You Code!

- **The context**, i.e. the setting, controls which names are visible where.
- **Object Orientation** (OO) creates context with:
  - **Namespaces**
  - **State**
  - **Subtype polymorphism**
- **Functional Programming** (FP) creates context with:
  - **Parameters**
  - **Function values**
  - **Parametric polymorphism**
- **The combination** of OO and FP is **extra** powerful and flexible!
- But there are **challenges**:
  - OO: sometimes difficult to reason about changed state and dynamic binding
  - FP: can be many parameters that must be repeated in nested calls
- Our salvation: more **cool stuff** in Scala:
  - **Context parameters**: allow **given** (implicit) values to be automatically summoned by the compiler.
  - **Ad hoc polymorphism**: enables different implementations depending on **static** type without requiring inheritance or dynamic binding.

11.1.23 Review: Default Arguments

- We have previously seen that the compiler, with **default arguments**, can **fill in** values we need not write at function calls:

```
scala> def f(x: Int, y: Int = 42) = x + y
def f(x: Int, y: Int): Int

scala> f(1, 2)        // explicit y value
val res0: Int = 3

scala> f(1)          // we can also omit y
val res1: Int = 43   // default argument used
```

11.1.24 Review: Curried Parameter Lists

- We have previously seen curried parameter lists, which allow stepwise application (so-called Curry functions):

```
scala> def f(x: Int)(y: Int = 42) = x + y
def f(x: Int)(y: Int): Int

scala> val g = f(1)  // a new function without default argument
val g: Int => Int = Lambda1352/0x08406d0840@dbc7e0a

scala> f(1)()   // y value filled in by compiler
val res4: Int = 43
```

- Can we take this one step further and **decouple** declaration of the function **from** the given value, and provide that elsewhere based on type?
- YES! In Scala 3 this is done with **given** values and **context parameters**, created with keywords **given** and **using**
  (in Scala 2 the old keyword **implicit** was used)

11.1.25 Given Values + Context Parameter

Example of usage of **given** and **using**:

```
case class Default(value: Int)

object Default:
  given d: Default = Default(0)  // Use this if no other given value is available locally
```

```
scala> def f(x: Int)(using d: Default) = x + d.value

scala> f(1)(using Default(2))    // explicit value always used first if present
val res0: Int = 3

scala> f(1)  // compiler summons a given value for Default from companion object
val res1: Int = 1

scala> given d: Default = Default(42)  // let d be the given value in this context
lazy val given_Default: Default
```

```
scala> f(1)       // compiler summons the given value in this local context
val res2: Int = 43
```

You can omit the name of the value, since it is often unnecessary:
`given Default = Default(42)` or even shorter: `given Default(42)`

11.1.26 Wouldn't a Global Variable Work as Well?

Warning for careful coders!

- An attempt to create a default value with a global `var` variable:

```
scala> var globalVar = Default(42)
var ctx: Int = 42

scala> def f(x: Int, d: Default = globalVar) = x + d.value

scala> f(1)
val res3: Int = 43

scala> globalVar = Default(0) // THE CHANGE IS GLOBAL THROUGHOUT YOUR PROGRAM!!

scala> f(1)
val res4: Int = 1
```

- Here the "context" is a reference to a mutable value decided at **runtime** in the global namespace. The function `f` is not genuine!
- This solution **cannot** provide **different** context-dependent default values; all function calls use **one** globally mutable value. **Bug risk**!

Given values with `given` are instead resolved at **compile time** from a special **implicit namespace** (eng. *implicit scope*) according to special rules.

11.1.27 Importing a Context Parameter

```
object OneNamespace:
  given oneGivenString: String = "a given value in OneNamespace"
  def summon(using s: String) = s     // context parameters are marked with using
```

It is the *local* context at **usage** time that controls what can be summoned (not the namespace where the context parameter was declared):

```
scala> OneNamespace.summon
-- [E172] Type Error: ------------------------------------------------------------
1 |OneNamespace.summon
  |                    ^
  | No given instance of type String was found for parameter s of method summon
  | in object OneNamespace. The following import might fix the problem:
  | import OneNamespace.oneGivenString
```

You can import given values with a special syntax where the type is specified instead of the name:

```
scala> import OneNamespace.given String    // special import syntax based on type

scala> OneNamespace.summon
val res0: String = a given value in OneNamespace
```

11.1.28 Summon Value with `summon`

In the standard Scala 3 library there is a **generic** variant of the method `summon` in the previous example, defined like this:

```
def summon[T](using x: T) = x
```

The function `summon` can be used to test which value the compiler will summon in a certain context:

```
object AnotherNamespace:
  given String = "taken for granted"    // name is optional, type is enough
```

```
scala> summon[String]
-- Error:
1 |summon[String]
  |              ^
  | no implicit argument of type String was found for parameter x of
  | method summon in object Predef. The following import might fix the problem:
  | import AnotherNamespace.given_String

scala> import AnotherNamespace.given    // imports all given values in MyContext

scala> summon[String]
val res0: String = taken for granted
```

11.1.29 Order of Priority When Summoning Given Values

- There can be several **different** given values of the **same** type if they are in different namespaces.
- It must **not** be **ambiguous** which value should be summoned.
- Therefore, there are special priority rules:
  1. **Explicit** arguments for context parameters marked with `using`
  2. `given` and `import given` ... in the current namespace (eng. *current scope*)
  3. `given` values in *companion objects* for the type in use.
  4. ... (more rules in special cases that we do not cover here)
- *Special rule:* If several given values can be summoned for types which are part of a common **inheritance hierarchy** then the compiler chooses the given value with the **most general** type.

Summoning given values is also called in English:
to summon, or *term inference*, or *implicit resolution*.

11.1.30 Ad hoc Polymorphism

- With so-called **Ad hoc polymorphism** you can have *different* implementations of a function for *different* types *without* using inheritance and dynamic binding.
- This was invented in the ML language and developed further in Haskell.
- In Haskell Ad hoc polymorphism is created with a so-called "**type class**".
- This is done in Scala by combining type parameters and context parameters.
- A "type class" in Scala is a stateless **trait** with at least one type parameter and at least one abstract method.

```
trait Parser[T]: // a type class for parsing and converting strings to arbitrary types
  def fromString(value: String): Option[T]

object Parser:            // implementations for certain types can be offered as given value
  given Parser[Int] with        // the keyword 'with' is needed when implementing abstract m
    def fromString(value: String): Option[Int] = value.toIntOption
```

- The specific implementation is summoned at call site with context parameter:

```
scala> def useParser[T](s: String)(using p: Parser[T]) = p.fromString(s)

scala> useParser[Int]("12")        // finds given value in companion object
val res0: Option[Int] = Some(12)
```

11.1.31 How to Make the Type Class `Parser` Work for More Types?

```
scala> useParser[java.awt.Color]("Color(120,10,0)")
-- Error:
1 |useParser[java.awt.Color]("Color(120,10,0)")
  |                                         ^
  | no implicit argument of type Parser[java.awt.Color] was found
  | for parameter p of method useParser
```

```
given Parser[java.awt.Color] with
  def fromString(value: String): Option[java.awt.Color] =
    if !value.startsWith("Color(") then None else
      val trimmed = value.trim.stripPrefix("Color(").stripSuffix(")")
      trimmed.split(",").map(_.toIntOption) match
        case Array(Some(r),Some(g),Some(b)) => Some(java.awt.Color(r, g, b))
        case _ => None
```

```
scala> useParser[java.awt.Color]("Color(120,10,0)")
val res1: Option[java.awt.Color] = Some(java.awt.Color[r=120,g=10,b=0])
```

This provides **flexibility**: I can provide given values for my **own** types!

11.1.32 Context Parameter Names Can Be Omitted

- The name of the given value is often not needed—it's the *type* that matters.
- Therefore, it is allowed to omit the parameter name at **using**.
- The given value can instead be summoned with `summon` if needed:

```
def useParser[T](s: String)(using Parser[T]) =
  summon[Parser[T]].fromString(s)
```

11.1.33 Context Bound

This form of **using** parameter with a type constructor...

```
def useParser[T](s: String)(using Parser[T])
```

... is so common in Scala that there is a shorter syntax called **context bound**:

```
def useParser[T: Parser](s: String)
```

If `F[A]` is a type constructor and you write `[T: F]`, then `F` becomes a so-called **context bound**. The compiler expands this automatically to the context parameter `(using F[T])`

11.1.34 Even More Convenient Type Class with Extension Method

It is possible to combine context bounds with extension methods:

```
extension [T: Parser](s: String)
  def parseOrElse(default: T): T =
    summon[Parser[T]].fromString(s).getOrElse(default)
```

Now we have convenient dot notation:

```
scala> "12".parseOrElse(default = 42)
val res2: Int = 12

scala> "cucumber".parseOrElse(default = 42)
val res3: Int = 42

scala> "Color(100,100,0)".parseOrElse(default = java.awt.Color.black)
val res4: java.awt.Color = java.awt.Color[r=100,g=100,b=0]

scala> "bad color".parseOrElse(default = java.awt.Color.black)
val res5: java.awt.Color = java.awt.Color[r=0,g=0,b=0]
```

11.1.35 Sorting Collections with Given Ordering

```
scala> case class Cucumber(name: String, weight: Int)

scala> val xs = Vector(Cucumber("a", 100), Cucumber("b", 50), Cucumber("c", 100))
val xs: Vector[Cucumber] = Vector(Cucumber(a,100), Cucumber(b,50), Cucumber(c,100))

scala> xs.sorted
-- Error:
1 |xs.sorted
  | ^
  | No given instance of type Ordering[Cucumber] was found
```

This can be fixed by providing a given ordering for `Cucumber`:

```
given Ordering[Cucumber] with
  def compare(x: Cucumber, y: Cucumber): Int =
    if (x == y) then 0
    else if x.weight < y.weight then -1
    else 1
```

```
scala> xs.sorted   // now it works :)
val res0: Vector[Cucumber] = Vector(Cucumber(b,50), Cucumber(a,100), Cucumber(c
```

11.1.36 Sorting Collections with Even Easier Given Ordering

- It is common to want to define custom ordering relations.
- Therefore there is a convenient helper method in the companion object of the type class `Ordering` called `fromLessThan`:

  ```
  given Ordering[Cucumber] =
    Ordering.fromLessThan((g1, g2) => g1.weight < g2.weight)
  ```

- This takes as input a function which takes two instances to be ordered and returns **`true`** if the first is considered **less** (i.e., comes **before**) according to any definition.
- `fromLessThan` returns an `Ordering` you can mark as given.
- Try this on the week's deepening exercises.
- Read more about type classes in Scala here: https://docs.scala-lang.org/scala3/reference/contextual/type-classes.html

---

11.1.37 Suggested Use of Context Parameters in the Snake Lab

Suppose the class `Snake` has two context parameters:

```
class Snake (
  val initPos: Pos,
```

```
  val initDir: Dir,
  val headColor: Color,
  val tailColor: Color,
  )(using ctx: SnakeGame, settings: Settings) extends CanMove
```

- Where to offer default values for context parameters?
  - A good place to place `given default: Settings = ???` is in the companion object to the class `Settings`.
  - You can in the body of the class `SnakeGame` have a `given SnakeGame = this` Then the compiler will use the current instance of `SnakeGame` as context parameter when the game creates instances of `Snake`.

Note this text in the lab, in section *Tips and suggestions*: ”It is allowed to change, remove and add as long as the required design requirements are met.”

11.2 Exercise Excercise11

Goals

- ☐ Be able to explain what a context parameter is.
- ☐ Be able to explain the benefits of context parameters compared to global variables and default arguments when solving the configuration problem.
- ☐ Be able to use simple contextual abstractions with **given** and **using**.

Preparations

- ☐ Study the concepts in chapter 11

11.2.1 Basic Tasks; preparation for the lab

Task 1. *Context parameter.* Declare the following functions that take an integer as a context parameter. Also create a **given** declaration that provides the given integer value zero:

```
scala> def f(using i: Int) = i + 1

scala> def g(x: Int)(using y: Int) = x + y
```

a) Call the functions `f` and `g` with an explicit given argument that differs from the given integer value using **using** in the call. What happens if you omit **using**?

b) Call the functions `f` and `g` without specifying a **using** argument. Explain what happens.

c) Is it possible to mix regular parameters and context parameters in the same parameter list? If not, what happens?

Task 2. *Multiple given values in local context.* Different values depending on context.

```
case class Delta(value: Int)
object Delta:
  given default: Delta = Delta(1)

def inc(x: Int)(using dx: Delta) = x + dx.value

object Context1:
  val a = inc(1)

object Context2:
  given Delta = Delta(42)
  val a = inc(1)
```

a) What is the value of `Context1.a`?

b) What is the value of `Context2.a`?

c) Explain what happens.

Task 3. *Solution to the configuration problem using given values.* Suppose we want to be able to configure the behavior of a function to make it more flexible. Below are three conceptually different ways to do this for a function `greet` that prints a greeting: 1) a globally accessible variable, 2) default argument, and 3) contextual abstraction with **given** and **using**.

```
object GlobalVar:
  case class GreetConfig(greeting: String, receiver: String)
  object GreetConfig:
    val default = GreetConfig(greeting = "Hello", receiver = "World")
    var config = default

  def greetMsg =
    s"${GreetConfig.config.greeting} ${GreetConfig.config.receiver}!"

object DefaultArgs:
  case class GreetConfig(greeting: String, receiver: String)
  object GreetConfig:
    val default = GreetConfig(greeting = "Hello", receiver = "World")

  def greetMsg(config: GreetConfig = GreetConfig.default) =
    s"${config.greeting} ${config.receiver}!"

object GivenVal:
  case class GreetConfig(greeting: String, receiver: String)
  object GreetConfig:
    given default: GreetConfig = GreetConfig("Hello", "World")

  def greetMsg(using g: GreetConfig) = s"${g.greeting} ${g.receiver}"
```

a) Write code to test the different variants above. Show especially how you can use the default configuration and then provide a configuration that differs from `default`.

b) What are the pros and cons of the different variants above? Discuss especially which solution(s) allow multiple local configurations without affecting each other.

c) Explain what happens when calling `summon[GivenVal.GreetConfig]`.

d) What happens if you try to summon a given value for a type that has none?

e) Must the given value be unique?

11.2.2 Extra Tasks; additional practice

Task 4. *Context parameter and given value.* Try the following in REPL.

```
1 scala> def add(x: Int)(using y: Int) = x + y
2 scala> add(1)(using 2)
3 scala> add(1)
4 scala> given someName = 42
5 scala> add(1)
```

a) What is the error message on line 3 above?

b) Why does it work on line 5 without error?

c) Define and test a corresponding function `sub` that can subtract a given value.

11.2.3 Advanced Tasks; challenges

Task 5. *Variance and type bounds.* The code below models pets with the following meaning: Pets can be healthy or sick and are normally born healthy. There can be many cats and dogs, which are all different kinds of pets.

```
trait Pet(var isHealthy: Boolean = true)
class Cat extends Pet()
class Dog extends Pet()
```

a) Modify the code below so that the subsequent REPL statement *does not* give a compilation error?

```
case class Box[A](x: A)
```

```
scala> val b: Box[Any] = Box[Cat](Cat())
```

b) Try below in REPL and explain what happens.

```
scala> val v: Vector[Pet] = Vector[Cat](Cat())

scala> val s: Set[Pet] = Set[Cat](Cat())

scala> :settings -explain

scala> val s: Set[Pet] = Set[Cat](Cat())
```

Hint: In Scala's standard library, `Set[T]` inherits the function type `T => Boolean` which is declared contravariant in its input parameter.

c) There should be veterinarians who can treat pets and make them healthy. Why does the following code not work? Is it a compilation error or runtime error?

```
class Vet[-A]:
  def treat(x: A): Unit = x.isHealthy = true
```

d) Add a type bound to the veterinarian's type parameter to fix the error.

e) Write code to show 1) that the compiler allows cat veterinarians to treat cats but 2) prevents cat veterinarians from treating arbitrary pets, and that 3) a veterinarian who has competence to treat arbitrary pets can treat both cats and dogs. Explain why the compiler allows/prevents these.

Task 6. *Type classes and context parameters.* In Scala there are possibilities for advanced functional programming with so-called **type classes** (i.e. *ad hoc polymorphism*). A type class defines general behaviors that work for arbitrary existing types without needing to change their implementation. We explore here how contextual abstractions can be used to create type classes in Scala, illustrated using given orderings for sorting.

By combining concepts of given values, generic classes, and context parameters, one obtains ad hoc polymorphism, exemplified with the type class `CanCompare` below, which we can get to work for existing types *without* needing to change them. Especially, we

do not have the possibility to add methods to existing types in the standard library since that is not our own code.

a) What happens below? Which lines give error messages? Why?

```
scala> trait CanCompare[T]:
         def compare(a: T, b: T): Int

scala> def sort[T](a: T, b: T)(using cc: CanCompare[T]): (T, T) =
         if cc.compare(a, b) > 0 then (b, a) else (a, b)

scala> sort(42, 41)

scala> given intComparator: CanCompare[Int] with
         override def compare(a: Int, b: Int): Int = a - b

scala> sort(42, 41)

scala> sort(42.0, 41.0)
```

b) Define a given value so that `sort` works for values of type `Double`.

c) Define a given value so that `sort` works for values of type `String`. *Tip:* You can use existing comparison operators on strings, but keep in mind that `compare` still must return an integer even for string comparisons.

Task 7. *Usage of given ordering.* We are now going to create a function `isSorted` that is generally useful by making given orderings available for different types. The function `def isSorted(xs: Vector[Int]): Boolean = ???` works only for collections of type `Vector[Int]`.
If instead we use `def isSorted(xs: Seq[Int]): Boolean = ???` it works for various collections of integers, including `Vector` and `List`.

a) Test the following function in REPL with integer sequences of different types.

```
def isSorted(xs: Seq[Int]): Boolean = xs == xs.sorted
```

b) There is a problem with trying to make `isSorted` generic below. What is the error message? What is missing according to the message?

```
scala> def isSorted[T](xs: Seq[T]): Boolean = xs == xs.sorted
```

c) We want `isSorted` to work for arbitrary types `T` that have an ordering definition. This can be done with the function below where the special type parameter syntax `[T:Ordering]` means that `isSorted` is defined for all collections where `T` has a given ordering `Ordering[T]`. In particular, this applies to all primitive types `Int`, `Double`, `String`, etc., which all have specific implementations of the type class `Ordering`.

```
def isSorted[T:Ordering](xs: Seq[T]): Boolean = xs == xs.sorted
```

Test the method above in REPL as shown below.

```
scala> isSorted(Vector(1,2,3))
scala> isSorted(List(1,2,3,1))
scala> isSorted(Vector("A","B","C"))
scala> isSorted(List("A","B","C","A"))
scala> case class Person(firstName: String, familyName: String)
```

```
scala> val persons = Vector(Person("Kim", "Finkodare"), Person("Voldemort","Ful
scala> isSorted(persons)
```

What error message does the last line give? Why?

d) *Implicit orderings.* A type parameter of the form `[T:Ordering]` is called a context bound and causes the compiler to automatically expand the function header for `isSorted` with a context parameter. Instead of using `[T:Ordering]` we can manually add a context parameter corresponding to the context bound. This also gives us access to a name, here `ord`, for the implicit ordering and can use that name in the function body and call methods that are members of the type class `Ordering`. (The name of the context parameter can also be omitted, but then we must go via the built-in function `summon[T]` to ask the compiler to search for the given instance for the type provided at the call.)

```
def isSorted[T](xs: Seq[T])(using ord: Ordering[T]): Boolean =
  xs.zip(xs.tail).forall(x => ord.lteq(x._1, x._2))
```

Objects of type `Ordering` have comparison methods such as `lteq` (short for *less than or equal*) and `gt` (short for *greater than*).
There are given orderings for all primitive types in the standard library, e.g. `Ordering[Int]`, `Ordering[String]`, etc. Test that the compiler finds the ordering for collections of some primitive types. Also verify below that it still causes problems for user-defined classes, e.g. `Person` (which we will fix in task 8).

```
scala> isSorted(Vector(1,2,3))
scala> isSorted(Array(1,2,3,1))
scala> isSorted(Vector("A","B","C"))
scala> isSorted(List("A","B","C","A"))
scala> class Person(firstName: String, familyName: String)
scala> val persons = Vector(Person("Kim", "Finkodare"), Person("Robin","Fulhack
scala> isSorted(persons)
```

e) *Import implicit ordering operators from an `Ordering`.* If you import members of an `Ordering` object, you get implicit conversions that make comparison operators work. Test the variant of `isSorted` below on different sequence types and verify that <=, >, etc., now work as expected.

```
def isSorted[T](xs: Seq[T])(using ord: Ordering[T]): Boolean =
  import ord.*
  xs.zip(xs.tail).forall(x => x._1 <= x._2)
```

Task 8. *Create your own implicit ordering with `Ordering`.*

a) One way to create a custom, specialized ordering for your own classes is to map your objects to types that already have an implicit ordering. Using the method `by` in the object `scala.math.Ordering`, you can create orderings by attaching a function `T => S` where `T` is the type of the objects you want to order and `S` is some other type, e.g. `String` or `Int`, where an ordering already exists.

```
scala> case class Team(name: String, rank: Int)
scala> val xs =
         Vector(Team("fnatic", 1499), Team("nip", 1473), Team("lumi", 1601))
scala> xs.sorted  // What is the error message? Why does it error?
scala> val teamNameOrdering: Ordering[Team] = Ordering.by(t => t.name)
```

```
scala> xs.sorted(using teamNameOrdering)   //explicit ordering
scala> given Ordering[Team] = Ordering.by(t => t.rank)
scala> xs.sorted   // Why does it work now?
```

b) To sort in reverse order one can use `Ordering.fromLessThan` which takes a function `(T, T) => Boolean` which should return **`true`** if the first parameter should come before, otherwise **`false`**. To sort by `rank` in reverse order you can do:

```
scala> val highestRankFirst: Ordering[Team] =
         Ordering.fromLessThan((t1, t2) => t1.rank > t2.rank)
scala> xs.sorted(using highestRankFirst)
```

c) If you have a case class with multiple fields and want a predefined implicit sorting order as well as provide an alternative sorting order, you can place a default ordering definition in the companion object; this is one of the places where the compiler searches last for implicit values before giving up.

```
case class Team(name: String, rank: Int)
object Team:
  given highestRankFirst: Ordering[Team] =
    Ordering.fromLessThan((t1, t2) => t1.rank > t2.rank)
  val nameOrdering: Ordering[Team] = Ordering.by(t => t.name)
```

```
scala> val xs =
         Vector(Team("fnatic", 1499), Team("nip", 1473), Team("lumi", 1601))
scala> xs.sorted
scala> xs.sorted(Team.nameOrdering)
```

d) It is also possible with the companion object above to get comparison operators working with your case class by importing members of the appropriate ordering object. Verify that this works as below:

```
scala> Team("fnatic",1499) < Team("gurka", 2)  // What error? Why?
scala> import Team.highestRankFirst.given
scala> Team("fnatic",1499) < Team("gurka", 2)  // No error? Why?
```

Task 9. *Custom ordering by inheriting from `Ordered`* If there is only *one* well-defined, specific order that you want to apply to your case class instances, you can make it ordered by mixing in the trait `Ordered` and implementing the abstract method `compare`. (This illustrates use of subtype polymorphism (i.e. inheritance) instead of ad hoc polymorphism with type classes.)

Background: A trait used this way is called an **interface**, and if you *implement* an interface you fulfill a "contract," which in this case means implementing what is required of ordered objects, namely a concrete `compare` method. You will learn more about interfaces in upcoming courses.

a) Implement the case class `Team` so that it is a subtype of `Ordered` as sketched below. Higher ranked teams should come before lower ranked teams. The method `compare` should return an integer that is negative if `this` comes before `that`, zero if they are equal in ordering, otherwise positive.

```
case class Team(name: String, rank: Int) extends Ordered[Team]:
  override def compare(that: Team): Int = ???
```

Tip: You can call the `compare` method on all primitive types, e.g. `Int`, since they implement the interface `Ordered`. Negating the expression reverses the order.

```
scala> -(2.compare(1))
```

b) Test that your case class now fulfills the requirements to be ordered.

```
scala> Team("fnatic",1499) < Team("gurka", 2)
```

c) Discuss with an instructor or peers the differences and similarities between interfaces and type classes, based on this and the previous exercises.

11.3 Lab: `snake1`

Goals

- ☐ See goals in last week's exercise. This lab extends over two weeks. Last week's presentation concerned the work plan and ongoing development, as well as your individual responsibility. This week, the completed solutions shall be presented by the respective main responsible individual.

Preparations

- ☐ Do exercise `Excercise11` in section 11.2

11.3.1 Group Lab Presentation

1. Explain the overall structure of the code.
2. Describe which parts you have contributed to in the code.
3. Give a brief explanation of concepts you have practiced.
4. Account for what you have learned about the challenges of system development in a group.
5. Account for the feedback you have received from reviews and how you have worked to improve readability during your stepwise extensions of your code.
6. Account for how and what feedback you have given when you have reviewed someone else's code.

Chapter 12

Advanced, Project

Concepts included in this week's studies:

- ☐ choose any advanced topic
- ☐ start project

12.1 Project assignment

12.1.1 About your final project assignment

Read carefully Study Guide Part 1, chapter -1 section ”Project assignment”!
Some important points:

- Goal: create a large program with many cooperating classes/modules.
- In study guide Part 2, chapter 13, there are several proposals to choose from.
- Before the presentation you must create automatically generated documentation based on relevant documentation comments for at least half of your public methods, according to instructions in Appendix E.
- Presentation takes place in a coach review:
  - Explain how your code works.
  - Describe the development of your program.
  - Go through the generated documentation of your code.

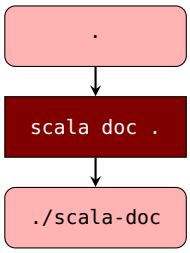
12.1.2 Project assignments

If you don’t know which to choose, feel free to pick `bank` which trains the basics.

- `bank`
  - well-known domain: create a bank with transaction history
  - immutable data together with state changes
- `music`
  - create a simple composition tool that plays music
  - getting into a domain model
- `photo`
  - a simple variant of photoshop
  - insight into basic matrix mathematics
- self-defined project
  - Moderately difficult: not too simple a task, but don’t take on more than you can handle!
  - Discuss with a supervisor and document your own task and its scope and relation to the learning goals of the course so that other supervisors and the course coordinator can understand why the project fits in the course.
  - You must get OK from a supervisor before you start a self-defined project.

12.1.3 Creating documentation

Scala CLI can take the documentation comments in the source code and create a website with documentation.



Current directory with `.scala` files

`index.html` and lots of other files that give a whole website!

Open `scala-doc/index.html` in a web browser.

12.1.4 Documentation comments

To make code understandable for humans it is good to document what it does. There are **three different kinds of comments**:

```
// Single-line comments start with double slashes
//       but only apply until the end of the line

/* Multi-line comments start with
   slash-asterisk
   and end with asterisk-slash.  */

/** Documentation comments are placed before
 *   e.g. a function and explain what it does
 *   and what any parameters are used for.
 *   Start with slash-asterisk-asterisk.
 *   Each new comment line starts with an asterisk.
 *   End with asterisk-slash.
 */
```

Comments do not affect how the machine executes the code, but help people to use the code and tools to display help. See Appendix E.

12.2 Exercise `Excercise12`

Goals

- ☐ This week's exercise contains optional deepening.
- ☐ Searching and sorting:
  - Be able to use built-in search methods.
  - Understand when binary search is appropriate and possible.
  - Be able to implement binary search.
  - Be able to implement selection sort, both to a new collection and in place.
- ☐ Threads and concurrent execution:
  - Know what a thread is and be able to explain the concept of concurrent execution.
  - Know what the methods `run` and `start` do in the class `Thread`.
  - Be able to create and start a thread with an overridden `run` method.
  - Be able to create a simple program where two threads compete to update a variable and explain why the behavior can become unpredictable.
  - Be able to use a `Future` to start multiple parallel computations.
  - Be able to register a callback on a `Future` with the method `onComplete`.

12.2.1 Tasks on searching and sorting

Task 1. *Timing measurement.* In the upcoming tasks, you will find the function `timed` below useful.

```
def timed[T](code: => T): (T, Long) =
  val now = System.nanoTime
  val result = code
  val elapsed = System.nanoTime - now
  println("\ntime: " + (elapsed / 1e6) + " ms")
  (result, elapsed)
```

a) Paste `timed` in the REPL and test that it works by measuring how long the expression below takes to execute.

```
scala> val (v, t1) = timed{ (1 to 1000000).toVector.reverse }
scala> val (s, t2) = timed{ v.toSet }
scala> timed{ v.find(_ == 1) }
scala> timed{ s.find(_ == 1) }
scala> timed{ s.contains(1) }
```

b) Try to explain the differences in execution time between the different ways to search for the number 1 in the collection. Approximately how many times would you need to use `contains` on the integer set `s` for it to be worth creating `s` instead of linear searching in `v` with `find` in the example above?

Task 2. *Searching with built-in search methods.*

a) *Linear search forward with `indexOfSlice`.* Study the documentation for Scala's collection method `indexOfSlice`[1] and write 8 different expressions in the REPL that, using both a string and a vector of integers, test 4 different cases: (1) present at the beginning, (2) present somewhere in the middle, (3) present at the end, and (4) not present.

b) *Linear search backward with `lastIndexOfSlice`.* Study the documentation for Scala's collection method `lastIndexOfSlice`[2] and write 8 different expressions in the REPL that, using both a string and a vector of integers, test 4 different cases: (1) present at the beginning, (2) present somewhere in the middle, (3) present at the end, and (4) not present.

c) *Searching with built-in binary search.* If a collection is sorted, this can be exploited to perform faster search. In **binary search**[3] one begins by looking in the middle and checking which half to continue searching in; then one divides this half in half again to check which quarter to continue in, and so forth.
In the object `scala.collection.Searching`[4] there is a method `search` which, if imported, offers binary search for all sorted sequence collections. If the collection is sorted it returns an object of case class `Found` containing the index where the element was first found; alternatively if the searched-for element does not exist, it returns an object of case class `InsertionPoint` containing the index where the element would have been placed if it existed. Note that if the collection is not sorted, the result is "undefined", i.e., something is returned but it is *not* reliable; so one must first sort the collection or be completely certain that it is sorted.
Investigate how `search` works by explaining what happens below. Which is fastest, `lin` or `bin` below? Use `timed` from task 1.

```
scala> val udda = (1 to 1000000 by 2).toVector
scala> import scala.collection.Searching._
scala> odd.search(odd.last)
scala> odd.search(odd.last + 1)
scala> odd.reverse.search(odd(0))
scala> def lin(x: Int, xs: Seq[Int]) = xs.indexOf(x)
scala> def bin(x: Int, xs: Seq[Int]) = xs.search(x) match
         case Found(i) => i
         case InsertionPoint(i) => -i
scala> timed{ lin(odd.last, odd) }
scala> timed{ bin(odd.last, odd) }
```

d) If a collection contains n elements, how many comparisons are needed in the worst case during binary search? *Tip:* Read about the algorithm on Wikipedia[3].

Task 3. *Search among LTH's courses with linear search.*
TODO!!! Update this exercise so it benefits from the new API on kurser.lth.se, see also `https://github.com/lunduniversity/lth-courses/issues/3`

a) Via this URL, you can download a tab-separated list of all courses offered at LTH this academic year: `https://fileadmin.cs.lth.se/pgk/lthkurser201819.txt`
What data is in the file? You can investigate with e.g.:

[1] docs.scala-lang.org/overviews/collections/seqs.html
[2] docs.scala-lang.org/overviews/collections/seqs.html
[3] `https://en.wikipedia.org/wiki/Binary_search_algorithm`
[4] http://www.scala-lang.org/api/current/scala/collection/Searching$.html

```
scala> import scala.io.Source.fromURL
scala> val url = "https://fileadmin.cs.lth.se/pgk/lthkurser201819.txt"
scala> val data = fromURL(url,"UTF-8").getLines.mkString("\n")
```

b) Paste the object `courses` on page 148 in the REPL.[5] What does the code do? How many courses does `courses.lth` contain?

```
object courses:
  def download(): Vector[Course] =
    val url = "https://fileadmin.cs.lth.se/pgk/lthkurser201819.txt"
    val lines = scala.io.Source.fromURL(url, "UTF-8").getLines.toVector
    lines.drop(1).map(s => Course.fromLine(s, '\t'))

  lazy val lth: Vector[Course] = download()

  case class Course(
    code:    String,
    nameSv:  String,
    nameEn:  String,
    credits: Double,
    level:   String
  )

  object Course:
    def fromLine(line: String, separator: Char): Course =
      import scala.util.Try
      val xs = line.split(separator).toSeq
      def str(i: Int): String = Try(xs(i)).getOrElse("")
      def num(i: Int): Double = Try(xs(i).toDouble).getOrElse(0.0)
      Course(str(0), str(1), str(2), num(3), str(4))
```

Figure 12.1: Code to download data about all courses at LTH.

c) *Linear search with find.* Student Oddput Clementina wants to attend the first available Computer Science course at G2 level. Help Oddput search for the first such course using linear search with the collection method `find`. CS course codes start with EDA or ETS[6]. *Tip:* Define the predicate `def isCS(s: String): Boolean` which preferably uses the string method `startsWith`.

d) *Implement linear search.* As practice, implement your own linear search function with signature:

```
def linearSearch[T](xs: Seq[T])(p: T => Boolean): Int = ???
```

The function takes a sequence collection `xs` and a predicate `p` which is a function that takes an element and returns a boolean. The type `Seq` is a supertype to all sequence collections, so using it as the parameter type for `xs` makes the function work for

[5] You can download the code from:
github.com/lunduniversity/introprog/tree/master/compendium/examples/lth-courses/courses.scala

[6] This is a simplified view of LTH's course code system. Some courses from the EIT department will also sneak in, but we ignore that here.

`Vector`, `Array`, `List`, etc. By the type parameter `T`, the function is generic and works for any type. The function `p` should return **`true`** if the parameter is the sought element. `linearSearch` should return the index of the first element in `xs` for which `p` holds. If no such element exists, `-1` is returned. First write pseudocode for the function by hand. Use **`while`**.

e) Implement a function **`def`** `rndCode: String` that generates random course codes consisting of 4 letters from A to Z followed by 2 digits from 0 to 9. *Tip:* Use REPL combined with an editor to create and test helper functions gradually solving appropriate subproblems.

f) Use `rndCode` from the previous subtask to fill a vector called `xs` with half a million random course codes. For each random code in `xs`, search with your function `linearSearch` for the index of the code in the vector `courses.lth` from subtask b. Measure the total time for the 500,000 linear searches with the function `timed` from task 1. How many of the random course codes were found among the real course codes at LTH?

g) How can you implement `linearSearch` with the built-in collection method `indexWhere`?

Task 4. *Search among LTH's courses with binary search.* The search algorithm BINSEARCH can be formulated with the pseudocode below:

```
Input  : An ascending sorted sequence xs of length n of integers and a
         searched integer key
Output : An integer i ≥ 0 giving the position of x if found, or a negative integer
         i where −i corresponds to the place where x should be inserted in
         sorted order if x is not in the collection.
set interval (low, high) to (0, n − 1)
found ← false
mid ← −1
while low ≤ high and not found do
    mid ← position midway between low and high
    if xs(mid) == key then
        found ← true
    else
        if xs(mid) < key then
            low ← mid + 1
        else
            high ← mid − 1
        end
    end
end
if found then
    mid
else
    −(low + 1)
end
```

a) Try the algorithm above on paper with a sorted sequence of fewer than 10 integers. Try with even and odd number of elements, a sequence with one integer, and an empty sequence. Try both present and absent elements.

b) Implement binary search in a function with signature

```
def binarySearch(xs: Seq[String], key: String): Int = ???
```

and test in REPL for different cases. What happens if the sequence is not sorted?

c) Use `binarySearch` below to search for LTH courses. Use `rndCode`, `timed`, and `courses` from previous tasks.

```
def binarySearch(xs: Seq[String], key: String): Int = ???

val lthCodesSorted = courses.lth.map(_.code).sorted
val xs = Vector.fill(500000)(rndCode)
val (_, elapsedBin) =
  timed{xs.map(x => binarySearch(lthCodesSorted, x))}
val (_, elapsedLin) =
  timed{xs.map(x => linearSearch(lthCodesSorted)(_ == x))}
println(elapsedLin / elapsedBin)
```

d) How much faster was binary search compared to linear search?[7]

Task 5. *Insertion sort.* Implement sorting of an integer sequence to a sorted sequence with **insertion sort** in a function with this signature:

```
def insertionSort(xs: Seq[Int]): Seq[Int] = ???
```

Idea: Create a new empty sequence which will be our sorted result. For each element in the unsorted sequence: insert it in the correct place in the new sorted sequence.

a) *Pseudocode:* Run the pseudocode below by hand on the sequence 5 1 4 3 2 1. Draw the memory state after each loop iteration. Here we internally use a mutable `ArrayBuffer` which is fast at insertions and finish with `toVector` so that we return an immutable sequence.

1 $result \leftarrow$ a new, empty ArrayBuffer
2 **foreach** *element* e **in** xs **do**
3 | $pos \leftarrow$ find the right position in $result$
4 | insert e at position pos in $result$
5 **end**
6 $result$.toVector

b) Implement `insertionSort`. Use a `while` loop to implement line 3 in the pseudocode. Look up the documentation for `insert` on `ArrayBuffer`. Test `insert` on `ArrayBuffer` in REPL and verify it can be used to insert at the end in the "unused" position just after the last index. What happens if you do `insert` at position `size + 2`? Paste your implementation of `insertionSort` in REPL and test it works:

```
scala> insertionSort(Vector())
res0: Seq[Int] = Vector()

scala> insertionSort(Vector(42))
res1: Seq[Int] = Vector(42)

scala> insertionSort(Vector(1,2,3))
res2: Seq[Int] = Vector(1, 2, 3)

scala> insertionSort(Vector(5,1,4,3,2,1))
```

[7] In a run on an i7-4970K at 4.0GHz, `elapsedLin` was about 3000 ms and `elapsedBin` about 60 ms. Binary search was about 50 times faster in this case.

```
11 res3: Seq[Int] = Vector(1, 1, 2, 3, 4, 5)
```

Task 6. *In-place sorting.* Implement in-place sorting in an `Array[String]` with selection sort.
Idea: For all indices i: find $minIndex$ of the smallest string from position i to the end and swap elements at position i and $minIndex$. See animation here: sv.wikipedia.org/wiki/Urvalssorte
Implement according to the sketch below. *Tip:* You may find a modified version of the solution to task 13 in chapter 2 useful.

```
def selectionSortInPlace(xs: Array[String]): Unit =
  def indexOfMin(startFrom: Int): Int = ???
  def swapIndex(i1: Int, i2: Int): Unit = ???
  for i <- 0 to xs.size - 1 do swapIndex(i, indexOfMin(i))
```

Task 7. *Check if a sequence is sorted.* A simple and readable way to check if a sequence is sorted is shown below.

```
scala> def isSorted(xs: Vector[Int]): Boolean = xs == xs.sorted
```

a) If `xs` has 10^6 elements, how many comparisons are done in the worst case using `isSorted` above? The method `sorted` uses the Timsort algorithm[8]. Look up the number of comparisons in the worst case on Wikipedia. ✎
This solution is relatively slow for large collections. One does not need to sort first to check if sorted (unless you intend to sort anyway). It suffices to check that elements are in ascending order.

b) Implement a more efficient variant of `isSorted` that uses a **`while`** loop checking elements are in ascending order. Your algorithm should stop searching as soon as unsorted elements are found.

c) How many comparisons are done in the worst case with the method in subtask b if you have n elements? ✎

d) One can check if a sequence is sorted by the clever trick of first zipping the sequence with its tail and then checking that all pairs satisfy the sorting condition, i.e. `xs.zip(xs.tail).forall(???)` where `???` is replaced by a suitable predicate. What is the type of the 2-tuple `xs.zip(xs.tail)` if `xs` is of type `Vector[Int]`? Implement `isSorted` with this trick.

Task 8. *In-place insertion sort.* Implement and test in-place sorting in an integer array with the [9] algorithm.
Implement and test the function below in Scala with the following signature:

```
def insertionSort(xs: Array[Int]): Unit
```

Place the method in an appropriate object, and create a main program to test. Compile and run from the terminal. Start by writing the sorting algorithm in pseudocode.

[8] stackoverflow.com/questions/14146990/what-algorithm-is-used-by-the-scala-library-method-vector-sorted

[9] https://en.wikipedia.org/wiki/Insertion_sort

Task 9. *Sorting to new sequence with selection sort.* Implement and test sorting to a new sequence with selection sort [10] in Scala, according to the sketch below. You may find the solution to task 13 in chapter 2 useful.

```
def selectionSort(xs: Seq[String]): Seq[String] =
  def indexOfMin(xs: Seq[String]): Int = ???
  val unsorted = xs.toBuffer
  val result = scala.collection.mutable.ArrayBuffer.empty[String]
  /*
  while unsorted is not empty
    minPos = indexOfMin(unsorted)
    elem   = unsorted.remove(minPos)
    result.append(elem)
  */
  result.toVector
end selectionSort
```

12.2.2 Tasks on threads and concurrent execution

Task 10. *Threads.* The class `java.lang.Thread` is used to create **threads** with concurrent execution. This way, different parts of code can run simultaneously.
The class `Thread` defines an empty `run` method. To make the thread do something meaningful, one overrides `run` with the desired code.
A thread is started with the method `start`, which automatically calls the `run` method and runs the code in `run` concurrently with other threads. If `run` is called directly, it is *not* concurrent execution.

a) Create a thread that does something that takes a little time and run it with `run` and `start`, respectively.

```
1 def zzz = { print("zzzzzz"); Thread.sleep(5000); println(" AWAKE!")}
2 zzz
3 val t2 = new Thread{ override def run = zzz }
4 t2.run; println("Good morning!")
5 t2.start; println("Good morning!")
6 t2.start
```

b) What happens if you call `start` more than once on the same thread?

c) Create two threads with overridden `run` methods and start them simultaneously as below. In which order are the greetings printed after line 3 and line 4? Explain.

```
1 val g = new Thread{ override def run = for i <- 1 to 100 do print("Cucumber ")
2 val t = new Thread{ override def run = for i <- 1 to 100 do print("Tomato ") }
3 g.run; t.run
4 g.start; t.start
```

d) Use `Thread.sleep` as below. Is the behavior fully predictable (deterministic)? Explain. You can interrupt REPL with CTRL+C or SHIFT+CTRL+C, depending on terminal settings.[11].

[10] https://en.wikipedia.org/wiki/Selection_sort
[11] stackoverflow.com/questions/6248884/can-i-stop-the-execution-of-an-infinite-loop-in-scala-repl

```
def sometimes(block: => Unit) = new Thread {
  override def run = while(true) { block; Thread.sleep(600) }
}.start
sometimes(print("zzz ")); sometimes(print("snark ")); sometimes(println("hi!"))
```

Task 11. *Concurrent variable update.* Write the classes `Bank` and `Kund` in an editor and then paste them in the REPL.

```
class Bank:
  private var _saldo = 0;
  def saldo: Int = _saldo
  def deposit(): Unit = _saldo += 1
  def withdraw(): Unit = _saldo -= 1
end Bank

class Customer(bank: Bank):
  def spendSave(): Unit =
    bank.withdraw()
    Thread.sleep(1)
    bank.deposit()
  end spendSave
end Customer
```

a) Use the function `sometimes` from the previous task and run the lines below in REPL. The concurrent variable update here produces nonsense results that lead to very upset bank customers and owners. Explain what happens.

```
val bank = new Bank
println(bank.saldo)
bank.deposit()
println(bank.saldo)
bank.withdraw()
println(bank.saldo)

val teddybear = new Customer(bank)
val ing = new Customer(bank)

teddybear.spendSave()
ing.spendSave()
println(bank.saldo)

def often(block: => Unit) = new Thread { // every millisecond
  override def run = while true do { block;  Thread.sleep(1)}
}.start

often(teddybear.spendSave()); often(ing.spendSave())

sometimes(println(bank.saldo))
```

Task 12. *Thread-safe `AtomicInteger`.* The JVM supports updates that cannot be interrupted by other threads during memory writing. An operation that cannot be

interrupted is called **atomic**. Study the documentation for `AtomicInteger` [12] and try the code below. Explain what happens.
Use the functions `often` and `sometimes` from earlier tasks.

```
class SafeBank:
  import java.util.concurrent.atomic.AtomicInteger
  private var _saldo = new AtomicInteger
  def saldo: Int = _saldo.get
  def deposit(): Unit = _saldo.incrementAndGet()
  def withdraw(): Unit = _saldo.decrementAndGet()
end SafeBank

class SafeCustomer(bank: SafeBank):
  def spendSave =
    bank.withdraw()
    Thread.sleep(1)
    bank.deposit()
  end spendSave
end SafeCustomer
```

```
val sb = new SafeBank
val grandma = new SafeCustomer(sb)
val wolf = new SafeCustomer(sb)

often(grandma.spendSave); often(wolf.spendSave)

sometimes(println(sb.saldo))
```

Task 13. *Concurrent execution with `scala.concurrent.Future`.* Managing threads can be complicated. Using `scala.concurrent.Future` allows simpler concurrent execution.

Background: With a `Future`, concurrent execution is created "under the hood" using a framework called Akka[13], written in Scala and Java. Akka provides automatic multithreading with thread pools and enables advanced parallel programming at a high abstraction level, so one does not have to create instances of the class `Thread` oneself. Instead one can simply wrap code inside `Future{ "runs concurrently" }` after importing what is needed.

a) To create concurrent execution with `Future`, you must first import as below; this creates an execution context with thread pools ready for use. Restart REPL and observe the error after line 1. Then import as shown. What is the type of `f`?

```
1 scala> concurrent.Future { Thread.sleep(1000); println("One second later!") }
2 scala> import scala.concurrent._
3 scala> import ExecutionContext.Implicits.global
4 scala> val f = Future { Thread.sleep(1000); println("One second later!") }
```

b) Create a procedure `printLater` as below that prints its argument after a random time. Explain what happens below.

```
1 scala> def printLater(a: Any): Unit =
```

[12] docs.oracle.com/javase/8/docs/api/java/util/concurrent/atomic/AtomicInteger.html
[13] http://akka.io/

```
        Future { Thread.sleep((math.random() * 10000).toInt); print(a + " ") }
scala> (1 to 42).foreach(i => printLater(i)); println("all launched!")
```

c) Create as below a `Future` that counts how many digits a very large number has. With `onComplete` one can specify what to do when the heavy computation finishes; this is called registering a callback. What is the return type of `big`? How many digits does the large number have? What is the type of `r`? Adjust the argument to `big` if you do not want to wait too long.

```
scala> BigInt(10).pow(100)
scala> BigInt(10).pow(100).toString.size
scala> def big(n: Int) = Future { BigInt(n).pow(n).toString.size }
scala> big(1234567).onComplete{r => println(r + " digits") }
```

d) The big advantage with `Future` is that you can continue running while waiting, so `Future` calls are **non-blocking**. Sometimes you want to block execution while waiting for the result. You can then use `scala.concurrent.Await` and its method `result` as below. Use `big` from the previous task and do a blocking wait for a result below. What happens? What if you wait too little?

```
scala> import scala.concurrent.duration._
scala> Await.result(big(1234567), 20.seconds)
```

Task 14. *Use `Future` to do multiple things concurrently.* In this task you will download web pages in parallel using `Future`, so that one download can finish while another is delayed.

a) The code for a minimal webpage is below. You can view it here: http://fileadmin.cs.lth.se/pgk/mini.html or write the code below in a file ending with `.html` and open it in your browser.

```
<!DOCTYPE html>
<html>
<body>
HELLO WORLD!
</body>
</html>
```

b) To simulate slow servers, you can download a page via the site
`http://deelay.me/`
Download the above page with 2 seconds delay:
http://deelay.me/2000/http://fileadmin.cs.lth.se/pgk/mini.html

c) You can download webpages with `scala.io.Source`. What happens below? Try to estimate, given how `delay` is computed, how long you have to wait on average, best case, and worst case before you can see the first webpage in vector `loads` below?

```
scala> def load(url: String) = scala.io.Source.fromURL(url).getLines.toVector
scala> def slowload(url: String) =
         val delay = (math.random() * 1000 + 2000).toInt
         val delaySite = s"http://deelay.me/$delay/"
         load(delaySite+url)
       end slowload
scala> load("http://fileadmin.cs.lth.se/pgk/mini.html")
```

```
scala> def seg = slowload("http://fileadmin.cs.lth.se/pgk/mini.html")
scala> val loads = Vector.fill(10)(seg)
scala> loads(0)
```

d) Before we can start a `Future`, as shown in task 13, we must import the underlying execution environment ready for parallelization without creating threads ourselves. What happens below?

```
scala> import scala.concurrent._
scala> import ExecutionContext.Implicits.global
scala> val f = Future(seg)
scala> f   // check if done or try again later
scala> f
```

e) Load input without blocking (non-blocking input). Explain what happens below.

```
scala> val nonBlocking = Future(Vector.fill(10)(seg))
scala> nonBlocking   // check later if not done
scala> nonBlocking
```

f) Load input separately in different parallel threads. Explain what happens below. Run line 3 below repeatedly quickly in REPL using up-arrow + Enter.

```
1 scala> val para = Vector.fill(10)(Future(seg))
2 scala> para
3 scala> para.map(_.isCompleted)
4 scala> para.map(_.isCompleted) // watch how they complete one by one
5 scala> para(0)
```

g) Register a callback with `onComplete`. Explain what happens below.

```
scala> val action = Vector.fill(10)(Future(seg))
scala> action(0).onComplete(xs => println(s"ready:$xs"))
scala> // wait until download at index 0 is done
```

h) Register a callback for error handling in case of exceptions with `onFailure`. Explain what happens below.

```
scala> def happy  = { Thread.sleep(3000); println(":)") }
scala> def unhappy = { Thread.sleep(3000); 42 / 0; happy }
scala> Future(happy ).onFailure{ case e => println(s":( $e") }
scala> Future(unhappy).onFailure{ case e => println(s":( $e") }
```

Task 15. Calculate large primes in parallel using the functions below. Implement `isPrime` according to the pseudocode from the English Wikipedia page on primality test [14] using the "naive algorithm". Compute 10 random primes with 16 digits each. Perform the computations in parallel using `Future`.

```
def isPrime(n: BigInt): Boolean = ???

def nextPrime(start: BigInt): BigInt =
  var i = start
  while !isPrime(i) do i += 1
```

[14] https://en.wikipedia.org/wiki/Primality_test

```
  i
end nextPrime

def randomBigInt(nDigits: Int): BigInt =
   def rndChar = ('0' + (math.random() * 10).toInt).toChar
   val str = Array.fill(nDigits)(rndChar).mkString
   BigInt(str)
randomBigInt
```

Task 16. *Answer theory questions.* ✎

a) What is a thread?

b) How do you create a thread with class `Thread`?

c) How do you start a thread?

d) What problems can arise if multiple threads update the same resource?

e) What does it mean that code is *thread-safe*?

f) Name some advantages of using Future compared to using threads directly.

Task 17. *Classes with atomic updates.* Read about and test classes AtomicBoolean, AtomicDouble, and AtomicReference for atomic updates in package `java.util.concurrent.atomic`.
Use some of these together with `scala.concurrent.Future`.

Task 18. *Create your own multithreaded web server.*

a) Type in [15] the code below in an editor and save it as `webserver.scala`, compile and run with `scala run webserver.scala`, then describe what happens when you browse to:
http://localhost:8089/abbasillen

```
package webserver

import java.net.{ServerSocket, Socket}
import java.io.OutputStream
import java.util.Scanner
import scala.util.Try

object html:
  def page(body: String): String = //minimal web page
    s"""<!DOCTYPE html>
       |<html>
       |<head><meta charset="UTF-8"><title>Min Sörver</title></head>
       |<body>
       |$body
       |</body>
       |</html>
       """.stripMargin

  def header(length: Int): String = //standardized header of reply
    s"HTTP/1.0 200 OK\nContent-length: $length\n" +
     "Content-type: text/html\n\n"
```

[15] Or download here: github.com/lunduniversity/introprog/blob/master/compendium/examples/simple-web-server/webserver.scala

```
22
23
24 object start:
25   def handleRequest(cmd: String, uri: String, socket: Socket): Unit =
26     val os = socket.getOutputStream
27     val response = html.page("Backwards: " + uri.reverse)
28     os.write(html.header(response.size).getBytes("UTF-8"))
29     os.write(response.getBytes("UTF-8"))
30     os.close
31     socket.close
32   end handleRequest
33
34   def serverLoop(server: ServerSocket): Unit =
35     println(s"http://localhost:${server.getLocalPort}/hej")
36     while true do
37       Try {
38         var socket = server.accept  // blocks thread until connect
39         val scan = new Scanner(socket.getInputStream, "UTF-8")
40         val (cmd, uri) = (scan.next, scan.next)
41         println(s"Request: $cmd $uri")
42         handleRequest(cmd, uri, socket)
43       }.recover{ case e: Throwable => s"Connection failed: $e" }
44
45   def main(args: Array[String]) =
46     val port = Try{ args(0).toInt }.getOrElse(8089)
47     serverLoop(new ServerSocket(port))
```

b) Now create a web server that does something more interesting. It should respond with the 13th Fibonacci number[16] if browsed to `http://localhost:8089/fib/13`. Save your previous webserver as `fibserver.scala` and add the following code to calculate Fibonacci numbers in a deliberately slow exponential-time manner so the server really works, then later implement `compute.fib` with linear time complexity to avoid unnecessary CPU waste.

```
// add below in webserver.scala from
//    https://github.com/lunduniversity/introprog/blob/master/compendium/examples/simple-web

object compute:
  def fib(n: BigInt): BigInt =
    if n < 0 then 0 else
    if n == 1 || n == 2 then 1
    else fib(n - 1) + fib(n -2)
  end fib
end compute

def fibResponse(num: String) =
  num.toIntOption match
    case Some(n) => html.page(s"fib($n) == " + compute.fib(n))
    case None    => html.page(s"ERROR: enter an integer, not $num")

def errorResponse(uri:String) = html.page(s"Error: $uri </br> use /fib/integer")


// change handleRequest in start in webserver.scala to
  def handleRequest(cmd: String, uri: String, socket: Socket): Unit =
```

[16] https://sv.wikipedia.org/wiki/Fibonaccital

```
    val os = socket.getOutputStream
    val afterSlash = uri.toString.drop(1) // skip initial slash
    println(s"afterSlash:$afterSlash")
    val response: String =
      if afterSlash.startsWith("fib/") then fibResponse(afterSlash.stripPrefix("fib/"))
      else errorResponse(uri)
    os.write(html.header(response.size).getBytes("UTF-8"))
    os.write(response.getBytes("UTF-8"))
    os.close
    socket.close
  end handleRequest
```

Run in terminal with `scala run webserver.scala` and describe what happens in your browser when you navigate to the server.

c) Open several large Fibonacci numbers simultaneously in multiple browser tabs. How does it show that the server is single-threaded?

d) Make your server multithreaded using the new server loop below.

```
import scala.concurrent._
import ExecutionContext.Implicits.global

  def serverLoop(server: ServerSocket): Unit = {
    println(s"http://localhost:${server.getLocalPort}/hej")
    while (true) {
      Try {
        var socket = server.accept  // blocks thread until connect
        val scan = new Scanner(socket.getInputStream, "UTF-8")
        val (cmd, uri) = (scan.next, scan.next)
        println(s"Request: $cmd $uri")
        Future { handleRequest(cmd, uri, socket) }.onFailure {
          case e => println(s"Request failed: $e")
        }
      }.recover{ case e: Throwable => s"Connection failed: $e" }
    }
  }
```

e) Open several large Fibonacci numbers simultaneously in different tabs. How does it show the server is multithreaded?

f) It is unnecessary to compute the same Fibonacci number multiple times. With a cache in the form of a mutable `Map` you can store already computed values. This does not work with a regular `scala.collection.mutable.Map` in our multithreaded web server because it is not **thread-safe**. With thread-safe mutable data structures the same concurrency problems as in task 11 occur.
Instead, use `java.util.concurrent.ConcurrentHashMap`. Look up the documentation for `ConcurrentHashMap` and try to understand the code below. How do the methods `containsKey`, `put`, and `get` work?

```
object compute {
  import java.util.concurrent.ConcurrentHashMap
  val memcache = new ConcurrentHashMap[BigInt, BigInt]

  def fib(n: BigInt): BigInt =
    if (memcache.containsKey(n)) {
      println("CACHE HIT!!! no need to compute: " + n)
```

```
      memcache.get(n)
    } else {
      println("cache miss :( must compute fib:  " + n)
      val f = fastFib(n)
      memcache.put(n, f)
      f
    }

  private def fastFib(n: BigInt): BigInt = {
    if (n < 0) 0 else
    if (n == 1 || n == 2) 1
    else fib(n - 1) + fib(n -2)
  }
}
```

g) Use the above `fib` object in a new version of your server. Save it as `fibserver-memcached.scala`. Examine how fast it computes large Fibonacci numbers with the new variant. How large numbers can you compute? Can the server continue after stack overflow? Explain why.

h) Since we can now get very large Fibonacci numbers, it may be useful to insert line breaks on the webpage. The html tag `</br>` produces a line break.

```
  def insertBreak(s: String, n: Int = 80): String = {
    if (s.size < n) s
    else s.take(n) + "</br>" + insertBreak(s.drop(n),n)
  }
```

Use this recursive function to insert line break tags every n characters in long strings. Test how it looks on the webpage with very large numbers.

i) We will now use the larger heap memory instead of stack, avoiding stack size limitations. Rewrite `fastFib` to use a **`while`** loop instead of recursive calls. This is tricky, but if stuck you may peek at the solutions in the Appendix for inspiration. How large numbers can your server handle now? What happens when memory runs out? How can you protect the server from hanging?

12.3 Project Assignment: bank

12.3.1 Focus

☐ Be able to implement a complete program based on a given specification
☐ Be able to combine different parts from different modules
☐ Understand how Java classes can be used in Scala
☐ Understand and assess when immutable/mutable as well as var/val should be used in larger contexts
☐ Be able to use companion objects
☐ Be able to read from and write to files
☐ Be able to search in different data structures in various ways

12.3.2 Background

In this project, you are to write a program that keeps track of bank accounts and customers in a bank. Besides tracking the bank's current state, the program should also keep a history of all state changes. The history should be detailed enough that the current state can be reconstructed by replaying all changes stored in the history. The program should be entirely text-based, i.e., interaction with the program is done via the terminal where a menu is displayed and input is received via the keyboard. You should write most of the program yourself, without any pre-written code. The program must follow the specifications given in the task, as well as the object-oriented principles you have learned in the course.

12.3.3 Requirements

The requirements for the bank application are listed below. To pass this assignment, all requirements must be met:

- The program must have the following menu options:
  - 1. Find accounts for a certain account holder with a given ID.
  - 2. Search for customers by (part of) name.
  - 3. Deposit money into an account.
  - 4. Withdraw money from an account.
  - 5. Transfer money between two different accounts.
  - 6. Create a new account.
  - 7. Remove an existing account.
  - 8. Print all bank accounts, sorted alphabetically by account holder.
  - 9. Print the history of all changes to the bank's state.
  - 10. Restore the bank to the state it had on a given date. For simplicity, you must permanently discard all history created after the date to which the bank is restored.
  - 11. Exit.
- When any of the following occur, the program must note this in the history:
  - Money is deposited into an account.

  - Money is withdrawn from an account.
  - Money is transferred between two accounts.
  - An account is created.
  - An account is removed.

- The history must be saved both in memory and in a file.
- When the program starts, it must read the history file to reconstruct the bank's previous state.
- The format of the history file must be the same as in this example file: https://github.com/lunduniversity/introprog/blob/master/workspace/w13_bank_proj/bank_log.txt
- Everything related to the user interface (such as prints to the terminal and input reading from the terminal) must take place in `BankApplication` or helper classes to `BankApplication`, not in any other classes specified in the assignment.
- All methods and attributes must have appropriate visibility, so that internal, mutable parts are not unnecessarily exposed.
- The choice of val/var and immutable/mutable must be appropriate.
- The program must work as in the attached examples of program execution.
- Reasonable error handling must be present. That is, it is desirable that the program does not crash when the user inputs invalid data, but instead informs the user that the input is invalid. However, you can assume that the history file is always in the correct format.
- The program design must follow the specifications given below.
- It is sufficient that the bank can handle integers, but this must be done with the `BigInt` class to allow large amounts. If your bank handles decimals, this must be done with `BigDecimal` to avoid rounding errors.
- The `BankAccount` class must generate a unique account number for each account. These must be restored if the bank's state is restored to an earlier date, i.e., if a restoration removes an account, its account number should become available again.
- The only way to change the state of a `Bank` must be (besides calling `returnToState`) by calling `doEvent` with a `BankEvent` that describes the state change. At first glance, this may seem a bit cumbersome, but when implementing the change history, it will be very helpful that there exists a `BankEvent` representing each change.
- For the report, you should generate automatic documentation based on documentation comments according to the instructions in Appendix E. You should write relevant documentation comments for at least half of your public methods. It is often useful to write the documentation comments *before* the implementation of the method body.

12.3.4 Design

Below follows a description of members that the different classes in the bank application must contain. These starter classes exist in the course workspace, together with the pre-written classes `HistoryEntry` and `Date` as well as `BankEvent` with associated subtypes: https://github.com/lunduniversity/introprog/tree/master/workspace/w13_bank_proj

```
package bank

/**
 * A customer of a bank with provided name and id.
 */
case class Customer(name: String, id: Long):
  override def toString(): String = ???
```

```
package bank

/**
 * A bank account for hold by a specific customer.
 * The account is given a unique account number and initially
 * has a balance of 0 kr.
 */
class BankAccount(val holder: Customer):
  val accountNumber: Int = ???

  /**
   * Deposits the provided amount in this account.
   */
  def deposit(amount: BigInt): Unit = ???

  /**
   * Returns the balance of this account.
   */
  def balance: BigInt = ???

  /**
   * Withdraws the provided amount from this account,
   * if there is enough money in the account. Returns true
   * if the transaction was successful, otherwise false.
   */
  def withdraw(amount: BigInt): Boolean = ???

  override def toString(): String = ???
```

```
package bank

import time.Date

/**
 * A bank with no accounts and no history.
 */
```

```
class Bank():
 /**
   * Returns a list of every bank account in the bank.
   * The returned list is sorted in alphabetical order based
   * on customer name.
   */
  def allAccounts(): Vector[BankAccount] = ???

  /**
   * Returns the account holding the provided account number.
   */
  def findByNumber(accountNbr: Int): Option[BankAccount] = ???

  /**
   * Returns a list of every account belonging to
   * the customer with the provided id.
   */
  def findAccountsForHolder(id: Long): Vector[BankAccount] = ???

  /**
   * Returns a list of all customers whose names match
   * the provided name pattern.
   */
  def findByName(namePattern: String): Vector[Customer] = ???

  /**
   * Executes an event in the bank.
   * Returns a string describing whether the
   * event was successful or failed.
   */
  def doEvent(event: BankEvent): String = ???

  /**
   * Returns a log of all changes to the bank's state.
   */
  def history(): Vector[HistoryEntry] = ???

  /**
   * Resets the bank to the state it had at the provided date.
   * Returns a string describing whether the event was
   * successful or failed.
   */
  def returnToState(returnDate: Date): String = ???
```

12.3.5 Tips

- Use a `match` expression to handle the different subtypes of `BankEvent` when implementing `doEvent`.

```
event match {
  case Deposit(account, amount) => ???
  case Withdraw(account, amount) => ???
```

```
  case Transfer(accFrom, accTo, amount) => ???
  case NewAccount(id, name) => ???
  case DeleteAccount(account) => ???
}
```

- To write to a file easily, you can for example use static methods in the `Files` class available in `java.nio.file`. To avoid portability issues, you can use a fixed `Charset`, e.g., `UTF_8`, which is available in `java.nio.charset.StandardCharsets.UTF_8`.
- To read from a file you can use `introprog.IO`. Pay particular attention to the method `appendString` and how newline characters are handled in `appendLines` https://github.com/lunduniversity/introprog-scalalib/blob/master/src/main/scala/introprog/IO.scala
- Be thorough in that your tests cover more cases than the provided examples (see 12.3.9), as these may be needed for more extensive testing and debugging and may be requested during demonstrations.

12.3.6 Mandatory tasks

Task 1. Implement the class `Customer`. Test that it works in the REPL.

Task 2. Implement the class `BankAccount`. Test that it works in the REPL.

Task 3. Create a main program in the singleton object `BankApplication`. Make sure the main program prints the menu correctly and can read input from the keyboard corresponding to the menu choices. Let menu selections give a message informing the user that they are not yet implemented.

Task 4. Implement the class `Bank`.

a) Implement menu choice 6. When the user chooses to create a new account, `BankApplication` should create a `NewAccount` object which it then uses as an argument in a call to `doEvent` in `Bank`. It is in `doEvent` (or a private function called from `doEvent`) where the account is actually created.

b) Implement menu choice 8. Verify that both menu choices 6 and 8 work correctly.

c) Implement menu choice 9. Every time `doEvent` executes without error, its `BankEvent` argument should be added to the history along with the current date.

d) Implement all other menu options, except menu option 10. Test the new menu options carefully as you implement them, especially so that the change history works correctly. Make extensions you consider necessary.

Task 5. Implement backup of the history.

a) When something is added to the history, it must also be immediately written to a history file. The bank must not have to be closed for the output to be recorded in the file; if this were the case, information could be lost if the bank crashes. Use the `toLogString` method in `HistoryEntry` to get output in the correct format.

b) When the program starts, it must read all events from the history file and replay them one by one. This way the bank's state can be restored, even though only the change history is saved and not the state itself. Use the `fromLogString` method in `HistoryEntry` when reading strings from the file.

Task 6. Implement menu choice 10 by first resetting the bank's state and then replaying everything in the history that happened before the given date. The rest of the history should be permanently removed, both in memory and in the history file.

12.3.7 Optional extra tasks

First complete the mandatory parts of the project. Then, if you wish, you can extend your program as follows.

Task 7. Implement a new menu option that prints all account history for a given person. The history should include the type of event with associated parameters, the balance at the time of the event, as well as the date of the event. (This can be useful when testing your program.)

Task 8. Write one or more of the classes `Customer` and `BankAccount` in Java instead and use these in your Scala code.

12.3.8 Example of history file

An example history file is included in the workspace directory for this project assignment. Reading and printing must be done using its format. Each line represents an event, and the format for a line is: **Year Month Day Hour Minute BankEventType Argument**. The different kinds of `BankEvent` are represented by the following letters: D for `Deposit`, W for `Withdraw`, T for `Transfer`, N for `NewAccount`, and E for `DeleteAccount`.

12.3.9 Example of program execution

Below are possible examples of running the program. Data entered by the user is marked in bold. Your program does not have to look exactly the same, but the overall structure as well as the execution result must be the same. When the first example starts, the bank is assumed to have no accounts.
The list of options, which is italicized in the first example, is not printed in later examples to save paper. Your program should always print the list of options before the user is to enter a choice.

- -
1. Find accounts for a given customer
2. Search for customers by (part of) name
3. Deposit money
4. Withdraw money
5. Transfer money between accounts
6. Create new account
7. Delete existing account
8. Print all accounts in the bank
9. Print change history
10. Restore the bank to an earlier date
11. Exit
Choice: **6**
Name: **Adam Asson**
Id: **6707071234**
New account created with account number: 1000
10:03 14/5-2016

- -
Choice: **1**
Id: **6707071234**
Account 1000 (Adam Asson, id 6707071234) 0 kr
10:04 14/5-2016

- -
Choice: **6**
Name: **Berit Besson**
Id: **8505255678**
New account created with account number: 1001
10:12 14/5-2016

- -
Choice: **8**
Account 1000 (Adam Asson, id 6707071234) 0 kr
Account 1001 (Berit Besson, id 8505255678) 0 kr
10:13 14/5-2016

- -
Choice: **2**
Name: **adam**
Adam Asson, id 6707071234
10:15 14/5-2016

- -
Choice: **6**
Name: **Berit Besson**
Id: **8505255678**
New account created with account number: 1002
13:56 14/5-2016

- -
Choice: **2**
Name: **erit**
Berit Besson, id 8505255678
14:01 14/5-2016

- -
Choice: **3**
Account number: **1000**
Amount: **5000**
Transaction successful.
14:36 14/5-2016

- -
Choice: **5**
Account number to transfer from: **1000**
Account number to transfer to: **1001**
Amount: **1000**
Transaction successful.
14:37 14/5-2016

- -
Choice: **8**
Account 1000 (Adam Asson, id 6707071234) 4000 kr
Account 1001 (Berit Besson, id 8505255678) 1000 kr
Account 1002 (Berit Besson, id 8505255678) 0 kr
14:52 14/5-2016

- -
Choice: **7**
Enter account to delete: **1002**
Transaction successful.
14:54 14/5-2016

- -
Choice: **8**
Account 1000 (Adam Asson, id 6707071234) 4000 kr
Account 1001 (Berit Besson, id 8505255678) 1000 kr
14:55 14/5-2016

- -
Choice: **9**
10:03 14/5-2016: Created an account belonging to Adam Asson, id 6707071234
10:12 14/5-2016: Created an account belonging to Berit Besson, id 8505255678
13:56 14/5-2016: Created an account belonging to Berit Besson, id 8505255678
14:36 14/5-2016: Deposited 5000 kr in account 1000
14:37 14/5-2016: Transferred 1000 kr from account 1000 to account 1001
14:54 14/5-2016: Deleted account 1002
14:58 14/5-2016

- -
Choice: **10**
Which date do you want to restore the bank to?
Year: **2016**
Month: **5**
Day: **14**
Hour: **10**
Minute: **5**
Bank restored.
15:00 14/5-2016

- -
Choice: **9**
10:03 14/5-2016: Created an account belonging to Adam Asson, id 6707071234
15:00 14/5-2016

- -

Choice: **8**
Account 1000 (Adam Asson, id 6707071234) 0 kr
15:01 14/5-2016

- -

Choice: **3**
Account number: **1001**
Amount: **5000**
Transaction failed. No such account found.
15:06 14/5-2016

- -

Choice: **4**
Account number: **1000**
Amount: **1000**
Transaction failed. Insufficient balance.
15:23 14/5-2016

12.4 Project Assignment: `music`

Preparations

- ☐ Test that the computer you will use for the presentation can play sound with `javax.sound.midi` by running `Main` in the provided code.[17]
- ☐ It is good if you can bring headphones to the computer lab so that you do not disturb others.
- ☐ Download the given code via the course's GitHub location or via the course website under Links.

12.4.1 Background

When writing programs, one often creates models of a certain real *domain*, which can be e.g. the rules of the social insurance office or a physics simulation in a computer game. To be able to create such models, one often needs to acquire *domain knowledge* by carefully understanding what different concepts in the domain mean and how they are related. With this knowledge, you can create code that models the domain, based on carefully chosen simplifications of the complex reality. The ability to create domain models forms an important foundation for the art of developing good software systems, and you will learn more about this in upcoming courses.
In this lab assignment, you will create a program based on a simplified model of the domain *music*. You will receive ready-made code that models how notes are built, and how different string instruments work. With this domain model, you will create your own music program that uses *chords* which consist of several notes played together.

12.4.2 Domain model

Pitch

A **note** (eng. *note*) played on an instrument, e.g. a piano or a guitar, has a **pitch** (eng. *pitch*) that is related to the specific fundamental frequency of the note's sound. In our model of the music domain, we assign different distinct pitches a unique integer. A pitch can then be described by a `case class Pitch(nbr: Int)` where we use `nbr` in the range `(0 to 127)`. The integer `60` corresponds to a certain note, which also has the name `"C5"`, and which lies approximately in the middle of the keyboard on a piano. In (Western) music, one starts from 12 different *pitch classes* (eng. *pitch classes*). These twelve pitch classes are ordered in a sequence of so-called *half steps* and have the following **pitch class names**:

```
val pitchClassNames: Vector[String] =
  Vector("C","C#","D","D#","E","F","F#","G","G#","A","A#","B")
```

After the pitch class named `B`, the pitch class named `C` returns. The symbol `#` represents a raising by a half step. The pitch class `C#` is pronounced *ciss* in Swedish, and *see sharp* in English.[18] Note that there is a half step between `E` and `F`, and between `B` and `C` (therefore there are neither `E#` nor `B#` in the list of pitch class names.[19])

[17] At the time of writing, sound does not work under WSL (Windows Subsystem for Linux). Feel free to search online for possible solutions to sound issues in WSL.

[18] The flat sign ♭ is also used, which is pronounced *flat* in English, for lowering a note by a half step, but for simplicity we disregard this naming convention in our model.

[19] You can read more about why this is the case on e.g. Wikipedia, but you may also just accept that it is simply like this due to historical reasons.

On a piano, the white keys correspond to the pitch class names `C D E F G A B` and the black keys correspond to the pitch class names `C# D# F# G# A#`.
A so-called **pitch class** is a positive integer in the interval `0 until 12` corresponding to the index of the pitch class name in `pitchClassNames`. A pitch `Pitch(nbr)` belongs to the pitch class `nbr \% 12`.
Using integer division by 12, one can find the pitch's so-called **octave**, i.e. `nbr / 12`. A piano normally spans 7 or 8 octaves. A pitch `Pitch(nbr)` can also be named by combining the pitch class name and the pitch's octave, e.g. `"C5"`.
With this domain description, we can create a more detailed model of the concept of pitch using a case class and accompanying companion object:

```
package music

case class Pitch(nbr: Int):  //Tonhöjd
  assert((0 to 127) contains nbr, s"Error: nbr $nbr outside (0 to 127)")
  def pitchClass: Int        = nbr % 12
  def pitchClassName: String = Pitch.pitchClassNames(pitchClass)
  def name: String           = s"$pitchClassName$octave"
  def octave: Int            = nbr / 12
  def +(offset: Int): Pitch  = Pitch(nbr + offset)
  override def toString      = s"Pitch($name)"

object Pitch:
  val defaultOctave = 5  // mittenoktaven på ett pianos tangentbord

  val pitchClassNames: Vector[String] =
    Vector("C", "C#", "D", "D#", "E", "F", "F#", "G", "G#", "A", "A#", "B")

  val pitchClassIndex: Map[String, Int] = pitchClassNames.zipWithIndex.toMap

  def fromString(s: String): Option[Pitch] = scala.util.Try {
    val (pitchClassName, octaveName) = s.partition(c => !c.isDigit)
    val octave = if octaveName.nonEmpty then octaveName.toInt else 5
    Pitch(pitchClassIndex(pitchClassName) + octave  * 12)
  }.toOption

  def apply(s: String): Pitch =
    fromString(s).getOrElse(throw new IllegalArgumentException)
```

The companion object has two factory methods that can create `Pitch` objects from a string representation of a pitch.

- The method `fromString` converts a string to an `Option[Pitch]`.
- The method `apply` throws an exception if it is not possible to convert a string to a `Pitch` object.

✎ **Task 1.** Which two expressions in the `Try` block can cause the exceptions `NumberFormatException` and `NoSuchElementException` respectively? Investigate similar expressions in REPL that cause these exceptions. How can the factory method `fromString` be rewritten to use `toIntOption` instead of `toInt` on the string and `get` instead of `apply` on the key-value table and without using `scala.util.Try`? What are the disadvantages of detouring via `scala.util.Try` instead of methods that directly produce `Option`?

Task 2. Investigate the class `Pitch` in REPL.

```
> sbt
sbt> console
Welcome to Scala 2.13.3 (OpenJDK 64-Bit Server VM, Java 11.0.8).
Type in expressions for evaluation. Or try :help.

scala> import music._
import music._

scala> Pitch("C#5").nbr
res0: Int = 61

scala> Pitch("C") + 1
res1: music.Pitch = Pitch("C#5")
```

a) Give an example of an argument to `Pitch.apply` that causes an exception to be thrown. ✎

b) Give an example of an argument to `Pitch.fromString` that returns `None`. ✎

c) Give an example of an argument to `Pitch.+` that causes an exception to be thrown. ✎

Task 3. Modify the implementation of `fromString` so that instead of `.toOption`, you perform pattern matching with `match` on the `Try` results `Success` and `Failure` in separate case clauses of the form `case Success(x) =>` and `case Failure(e: ???) =>` `???`, returning the appropriate value. Only handle the two expected exception types you identified in task 1. Ensure that all other possible exceptions are thrown through this clause: `case Failure(e) => throw e`

a) Test that your solution works both in normal cases and when given an incorrect pitch name.

b) Investigate what happens if you comment out various case clauses. When does the compiler warn? Why? ✎

c) Are there any advantages or disadvantages to catching only certain exceptions? ✎

Chords

A chord consists of several notes played together. One can play a chord on a string instrument by striking a set of notes simultaneously or a sequence of notes in quick succession. One chooses to call one of the notes in the chord (usually the lowest/first note) the **root** (eng. *root*).
An **interval** is a note's relative pitch distance from the root. Chords have different names depending on which intervals are included in the chord. There are many different chord names, but here we limit ourselves for simplicity to four different chord types: [20]

- major chord, denoted e.g. `"C"`,
- minor chord, denoted e.g. `"Cm"`,
- seventh chord, denoted e.g. `"C7"`,
- major seventh chord, denoted e.g. `"Cmaj7"`.

[20] If you want to learn more about chord names, read here: https://en.wikipedia.org/wiki/Chord_(music)

In the case class `Chord` below there is a method `name` that defines which intervals are included in the different chord types above, except major seventh chords. The complicated modulo-12 calculation before the matching ensures that intervals in different octaves are treated equally, even for negative intervals.

```
package music

case class Chord(ps: Vector[Pitch]):
  assert(ps.nonEmpty, "Chord pitch sequence is empty")

  val pitchClasses: Vector[Int]   = ps.map(_.pitchClass).toVector

  def apply(i: Int): Pitch = ps(i)

  def intervals(root: Pitch = ps(0)): Vector[Int] = ps.map(_.nbr - root.nbr)

  def relativePitchClasses(root: Pitch = ps(0)): Vector[Int] =
    intervals(root).map(i => (i%12 + 12) % 12).distinct.sorted

  def name(root: Pitch = ps(0)): String = relativePitchClasses(root) match
    case Vector(0, 4, 7)     => root.pitchClassName
    case Vector(0, 3, 7)     => root.pitchClassName + "m"
    case Vector(0, 4, 7, 10) => root.pitchClassName + "7"
    case _ => root.pitchClassName + intervals(root).mkString("[",",","]")

  override def toString = ps.map(_.name).mkString("Chord(",",",")")

object Chord:
  def apply(xs: String*): Chord = Chord(xs.map(Pitch.apply).toVector)

  def random(pitchNumbers: Seq[Int] = (60 to 72), n: Int = 3): Chord =
    val shuffled = scala.util.Random.shuffle(pitchNumbers).toVector
    Chord(shuffled.take(n).map(Pitch.apply))
```

Task 4.

a) Major seventh chords have the same intervals as seventh chords, except that the fourth interval should be `11` half steps from the root instead of `10`. Add a case clause in `Chord.name` so that major seventh chords get a name ending with the suffix `"maj7"`.

b) Test your code and check that the chord `Chord("D4","F#4","A4","C#5")` gets the name `"Dmaj7"`

```
scala> Chord("D4","F#4","A4","C#5").name()
res2: String = "Dmaj7"
```

c) Which four notes does a `Cmaj7` chord with root `"C5"` consist of?

String instruments

A string instrument, e.g. a piano or a guitar, is characterized by the ability to play chords because several strings can be vibrated so that many notes are played together. In our model, we capture this feature with a trait `StringInstrument` that has a method `toChordOpt` which returns some chord if at least one string is played.

Guitar and ukulele are examples of string instruments with a fretboard. One plays a fretted instrument by pressing strings against the fretboard with one hand while plucking the strings with the other. Different instances of these instruments may differ in the number of strings and how these strings are tuned. A normal guitar has 6 strings, while a normal ukulele only has 4 strings. These features are modeled in the code below.

Each string has a tuning peg by which one can change the string's tension, the string's so-called **tuning** (eng. *tuning*). If you pluck all open strings on a guitar with standard tuning, the tones E3, A3, D4, G4, B4, E5 are played, counted from the thickest to the thinnest string.

```
package music

trait StringInstrument { def toChordOpt: Option[Chord] }

case class Piano(isKeyDown: Set[Int]) extends StringInstrument:
  def toChordOpt: Option[Chord] =
    if isKeyDown.nonEmpty then
      Some(Chord(isKeyDown.toVector.sorted.map(Pitch.apply)))
    else None

trait FrettedInstrument extends StringInstrument:
  def nbrOfStrings: Int
  def tuning: Vector[Pitch]
  def grip:   Vector[Int]
  def toChordOpt: Option[Chord] =
    val notes =
      for i <- grip.indices if grip(i) >= 0
      yield tuning(i) + grip(i)
    if notes.nonEmpty then Some(Chord(notes.toVector)) else None

case class Guitar(pos: (Int,Int,Int,Int,Int,Int)) extends FrettedInstrument:
  val grip = Vector(pos._1, pos._2, pos._3, pos._4, pos._5, pos._6)
  val nbrOfStrings = 6
  val tuning =
    "E3 A3 D4 G4 B4 E5".split(' ').map(Pitch.apply).toVector

case class Ukulele(pos: (Int,Int,Int,Int)) extends FrettedInstrument:
  val grip = Vector(pos._1, pos._2, pos._3, pos._4)
  val nbrOfStrings = 4
  val tuning =
    "A5 D5 F#5 B5".split(' ').map(Pitch.apply).toVector
```

If you press the fretboard at different positions, you get different notes, depending on which position you press. The positions on the fretboard are counted from one and upwards. Position `0` corresponds to a **open string**, meaning that you pluck a string without pressing down on the fretboard over that string. A negative position, e.g. `-1`, indicates that a string is not played at all; many guitar chords are played by only plucking a subset of the strings. An example of a guitar chord is shown in figure 12.2.

Task 5. Study the model of string instruments above and use REPL to answer these questions: ✎

Figure 12.2: A C major chord on a guitar corresponding to `Guitar(3,3,2,0,1,0)`.

a) What is the name of this piano chord if we choose the lowest note in the chord as the root: `Piano(Set(60, 64, 67, 70))`

b) What are the names of the notes included in the chord `Guitar(3,3,2,0,1,0)`?

c) What is the name of this chord with root A: `Ukulele(0,2,1,2)`

Electronic instruments

An electronic instrument synthesizes sound using analog and/or digital electronics, and is therefore called a **synthesizer**, often abbreviated *synth* (eng. *synth*).
Most modern PC operating systems include software-implemented synths that follow the so-called MIDI standard. The Java package `javax.sound.midi` contains classes that can make such a MIDI synth play music.
The MIDI standard is based on a model of a piano keyboard where different notes can be "on" or "off" depending on whether a key is pressed or not. These notes' pitches are modeled similarly to our class `Pitch`, where pitch `60` corresponds to the note `"C5"`, etc. A key can be pressed with varying force, represented by an integer value in `Range(0,128)` called `velocity`. A high value produces a strong note, while a low value corresponds to a weak (quiet) note.
A synth following the MIDI standard can play sound via 16 different so-called **channels** (eng. *channel*), numbered `(0 until 16)`, where each channel can be set to play a sound resembling a certain real instrument, such as piano or guitar.
In the course workspace in the package `music` there is a `Synth` module that simplifies the use of the Java package `javax.sound.midi`. The module `Synth` has the method `playBlocking` which can play multiple notes for a certain duration using the synth on your sound card. The execution of your program is blocked until the notes have finished playing, hence the *"blocking"* in the name.
The method `playBlocking` has the following parameters, default arguments, and return type: [21]

[21] If you are curious, you can study the implementation of the `Synth` module here: https://github.com/lunduniversity/introprog/tree/master/workspace/w13_music_proj The

```
def playBlocking(
  noteNumbers: Seq[Int] = Seq(60), // a sequence of pitches
  velocity: Int         = 60,      // how hard the strike in Range(0, 128)
  duration: Long        = 300,     // how long in milliseconds
  spread:   Long        = 50,      // milliseconds between notes
  after:    Long        = 0,       // milliseconds before the first note
  channel:  Int         = 0        // MIDI channel that plays the notes
): Unit
```

Task 6. Call `playBlocking()` in REPL and check if your computer can play the note `"C5"`. Use headphones so you do not disturb your lab mates. Try what happens when giving different arguments to `playBlocking`.

Task 7. Complete the module `ChordPlayer` below so that the method `play` can play a chord. The case class `Strike` represents a chord strike.

```
package music

object ChordPlayer:

  case class Strike(
    velocity: Int         = 50,   // hur hårt anslag i Range(0, 128)
    duration: Long        = 500,  // hur länge i millisekunder
    spread:   Long        = 50,   // millisekunder mellan tonerna
    after:    Long        = 0     // millisekunder innan första tonen
  )

  def play(chord: Chord, strike: Strike = Strike(), channel: Int = 0): Unit =
    strike match
      case Strike(v, d, s, a) => ???
```

Task 8. Implement a singleton object named `Test` with a `main` method that, using your `play` method from the previous task, plays some different chords.

Task 9. Create a terminal app that can play chords. In the course workspace in `w13_music_proj` there is a started terminal app you can build on. It already has a `Main.main` method that starts a loop where the user can give commands (eng. *Command Line Interface, CLI*). The command `?` gives help and the command `:q` quits.

```
*** Welcome to music!
music> ?
?         print help
:q        quit this app
!         play chord TODO
music> !
play chord TODO
music> :q
Goodbye music!
```

code is easier to understand if you also read the API documentation of the package `javax.sound.midi` and learn more about the MIDI standard using e.g. Wikipedia.

As seen above, there is also a started command `!` which is intended to play a chord, but currently only prints a TODO message. Make it so that the user can play chords from different instruments using `!` as shown below:

```
music> ! p 60 64 67
Play Piano(Set(60, 64, 67)) Chord(C5,E5,G5)
music> ! g 0 2 2 0 0 0
Play Guitar((0,2,2,0,0,0)) Chord(E3,B3,E4,G4,B4,E5)
```

Task 10. Create a command that lets users define their own command names that can then be easily run using the defined name. If defined with an existing name, the old definition should be replaced.

```
music> def Em = ! g 0 2 2 0 0 0
defined Em = ! g 0 2 2 0 0 0
music> Em
Play Guitar((0,2,2,0,0,0)) Chord(E3,B3,E4,G4,B4,E5)
```

It should be possible to make nested def-commands, meaning the body of a def contains the name of another def. Try defining a whole song that consists of defined chords.

Task 11. For the report, you should generate automatic documentation based on documentation comments according to the instructions in Appendix E. You should write relevant documentation comments for at least half of your public methods. It is often useful to write the documentation comments *before* the implementation of the method body.

12.4.3 Optional tasks

Task 12. Make it so that definitions are saved between runs.

Task 13. Implement more optional commands. You can e.g.

- create a command that draws a fingering chart for guitar chords or finger positioning on piano with `introprog.PixelWindow`.
- create a command to make drumbeats, e.g. `drum mygrove x x o x` for 2 hi-hat + bass drum + 1 hi-hat and `start mygrove` and `stop mygrove` for playback of a beat in the background with `playConcurrently`. See `Synth.scala` for which instruments produce drum sounds based on their names `music.Synth.instruments.map(_.getName)`, e.g. `Standard Kit` or `Synth Drum`. Use channel 10 to play the drum sounds.

Task 14. Use the open-source project `jline` instead of `scala.io.StdIn.readLine` to automatically get up-arrow history, Ctrl+A Ctrl+K, TAB-completion, etc. See an example of usage of `jline` here: https://github.com/bjornregnell/termut

12.5 Project Assignment: `photo`

12.5.1 Background

This project involves implementing your own image processing application, a very simplified version of *Photoshop* or *Gimp*.
A digital image consists of a grid, a so-called matrix, of pixels, each with a certain color. When many small pixels are placed next to each other in a grid, they blend together visually, and the viewer perceives an image.
Images can be manipulated by applying various so-called *filters* that modify the image's pixels in interesting ways. You shall, based on the given mathematical theory, implement different filters using special matrix operations.
There are different systems for coloring pixels. For example, the CMYK system (cyan, magenta, yellow, black) is used for mixing color to be printed on paper or other materials. On a computer, however, the RGB system is usually used, which has the three primary colors red, green, and blue. The saturation of each primary color is specified by an integer which we henceforth assume lies in the interval [0, 255]. 0 denotes "no color" and 255 denotes "maximum color". Thus, one can represent 256 × 256 × 256 = 16,777,216 different shades of color. One can also represent grayscale images; this is done with colors that have the same value for all three primary colors: (0, 0, 0) is completely black, (255, 255, 255) is completely white.
In this project, we will create matrices of integers to compute interesting properties of an image, for example the intensity for each pixel. To save space when processing large images, we use integer matrices with the type `Short`, which use 16 bits in memory, instead of `Int`, which uses 32 bits in memory.

12.5.2 Preparations

In this project, the following parts of `introprog-scalalib` and `java.awt` are useful:

- `introprog.Image` for image handling.
- `introprog.PixelWindow` and `introprog.Dialog` for user interaction.
- `introprog.IO` for file handling.
- `java.awt.Color` for pixel color handling.

Read the documentation carefully for the classes in introprog here and do your own experiments in REPL so you understand how they can be used: https://fileadmin.cs.lth.se/pgk/api
Read about the class `java.awt.Color` here:
https://docs.oracle.com/en/java/javase/11/docs/api/java.desktop/java/awt/Color.html Download and study carefully the code given for this project here:
https://github.com/lunduniversity/introprog/tree/master/workspace/

12.5.3 Matrix with values of type `Short`

Task 1. **`Matrix`**. In the given code file `Matrix.scala` there are helper functions for creating and updating matrices with values of type `Short`, to save memory with large images.
Complete the missing implementations and test thoroughly in REPL so that everything works as it should before you proceed. *Tip:* You will find `Array.tabulate` useful.

```
scala> import photo.*

scala> val m = Matrix(3,3)(1,2,3,4,5,6,7,8,9) // a 3x3 matrix with Short values
val m: photo.Matrix = Array(Array(1, 4, 7), Array(2, 5, 8), Array(3, 6, 9))

scala> m(0,1)
val res0: Short = 4

scala> m(1,0) = 42

scala> m
val res1: photo.Matrix = Array(Array(1, 4, 7), Array(42, 5, 8), Array(3, 6, 9))

scala> m.row(0)
val res2: Array[Short] = Array(1, 42, 3)
```

12.5.4 User Interface

When the app starts, a window is displayed according to fig. 12.3, implemented by the given code in `Main.scala`. With the help of the given `Button.scala`, a column with clickable buttons is created. Study the code in `Main.scala` and `Button.scala` carefully so that you understand what happens. So far, no ImageEditor window opens, but that is part of the next task.

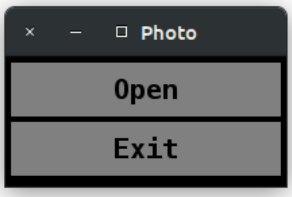


Figure 12.3: The Photo application's start window.

Task 2. **`ImageEditor`**. The following requirements shall be implemented:

- You shall create a code file `ImageEditor.scala` containing a class of the same name that implements an image editing window with a column of buttons on the left and an image loaded from a file on the right, as shown in fig. 12.4.
- When pressing the Exit button, a warning question "Ok to Exit without save?" shall be displayed using `introprog.Dialog.isOK` and the user shall be able to cancel the exit. If exit is chosen anyway, this shall be done with `System.exit(0)` so that all possibly active windows and associated threads are terminated immediately.
- When pressing the Open button, a file shall be selected using `introprog.Dialog.file`. If there is a subdirectory named `images` in the current directory, the file chooser shall start there; otherwise in the current directory.
- After OK in file opening, an image shall open in an image editing window as shown in fig. 12.4 with the buttons Save, Undo, Close, plus a button for each

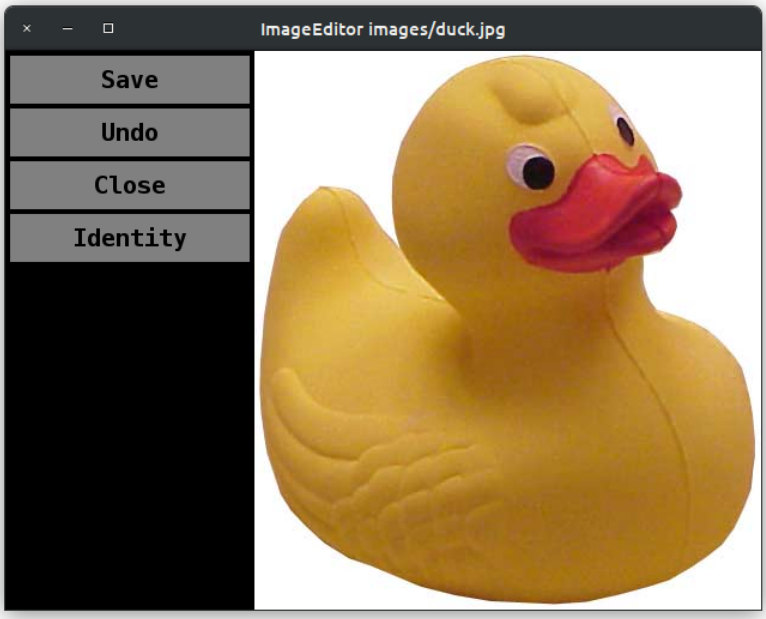


Figure 12.4: The image editing window before any filters (beyond the identity filter) are implemented. Each implemented filter should have a corresponding button.

filter, to the left of the image. The window's height and width shall be adjusted so that the entire image and all buttons fit.

- The main window and all image editing windows shall work in parallel. You shall achieve this by running the event loop in the `ImageEditor` class as an argument to the method `runInParallell` as below:

```
def runInParallell(block: => Unit) =
  new Thread{ override def run(): Unit = block }.start

def startEventLoop(): Unit = runInParallell:
  // initialization and event loop here
```

- It shall be possible to do Undo in multiple steps and restore all images before applying filters in turn. *Tip:* introduce an attribute `var history: Vector[Image]` initially containing the original image.

- The window title shall contain the name `ImageEditor` and the last two parts of the opened path, according to the example in fig. 12.4, since a full path such as `/home/userxyz/workspace/photo/images/duck.jpg` may not fit in the window's title bar.

- When saving, a filename query shall be asked with `introprog.Dialog.file` and checks made whether the file exists or not; if existing, a question with `introprog.Dialog.isOK` shall ask whether to overwrite or not.

- All implemented filters shall have a button that applies the filter and saves the result in history so that the filter can be undone with Undo. If the filter has arguments, an informative dialog shall open where the user can enter the

arguments via `introprog.Dialog.input`. The filter buttons shall be sorted alphabetically by filter name, see fig. 12.5.

12.5.5 Filters

You shall build upon the given code in `Filter.scala` shown below. You shall implement and test a number of different filters that modify images in interesting ways using different matrix algorithms.

```
package photo

import introprog.Image

trait Filter:
  def name: String

  def argDescriptions: Seq[String] = Seq()

  def nbrOfArgs = argDescriptions.length

  def apply(im: Image, args: Double*): Image

object Filter:
  val byIndex: Vector[Filter] = Vector(Identity)

  val byName: Map[String, Filter] = byIndex.map(f => f.name -> f).toMap

  def intensity(im: Image): Matrix = ???

  def convolve(p: Matrix, x: Int, y: Int, kernel: Matrix, weight: Int): Short = ???

  object Identity extends Filter:
    val name = "Identity"
    def apply(im: Image, args: Double*): Image =
      val result = Image.ofDim(im.width, im.height)
      result.updated((x, y) => im(x, y))
```

`Filter.scala` contains a base type for all filters with several members that all filters shall implement, according to the requirements below. There is one filter already implemented, `Identity`, which can be used for testing purposes; this filter simply copies all pixels to a new image and therefore has no editing effect.

- The method `apply` shall return a new image where the filter has been applied, without modifying the input image.
- A filter can have zero or more arguments of type `Double` that can influence what happens when the filter is applied. Each such argument shall have a short, instructive description in the sequence `argDescription`, in order.
- The method `intensity` shall calculate a so-called *intensity matrix* and is needed in implementing grayscale, Gauss, and Sobel filters. How an intensity matrix is calculated is described below.
- The method `convolve` shall perform a so-called convolution (averaging in matrices) and is needed in the implementation of the Gauss and Sobel filters. How convolution is performed is described below.

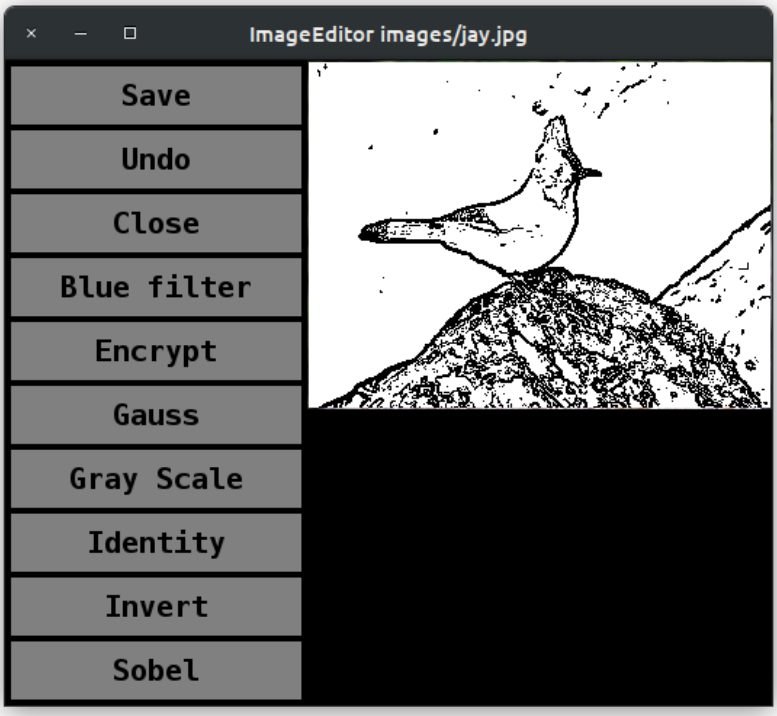


Figure 12.5: Image editing window with all filters implemented. A contour-enhancing so-called Sobel filter is applied with threshold value 150.

- All implemented filters shall be in the sequence `byIndex`, which is used in the table `byName`. These are needed to create all filter buttons and apply the respective filter.

You shall implement and test all filters in the tasks below, one at a time. The tasks are ordered in ascending difficulty.

Task 3. Blue filter. Create a filter in a singleton object named `Blue` which when applied gives a blue version of the image, where each pixel only contains the blue component. *Tip:* You have the method `getBlue` in the class `java.awt.Color`.

Task 4. Negative. Create a filter `Invert` that inverts an image, i.e. creates a "negative" copy of the image. Bright colors become dark and dark colors become bright. Consider what is meant by an inverted or negative copy. *Tip:* The new RGB values shall also be integers in the range 0 – 255. The new RGB values are *not* computed by any division expression over the old values (then the new values would be decimals and not integers in 0 – 255).

Task 5. Grayscale filter. Create a filter `GrayScale` that converts the image to grayscale. First implement the `intensity` method in `trait Filter` by averaging all three RGB components. Then use the intensity for each pixel to determine the grayscale level. If the intensity in a pixel, for example, is 105, then the new grayscale pixel should be a `Color` object with RGB values (105, 105, 105).

Task 6. Encryption. Create a filter `XOrCrypto` that encrypts the image with the xor operator `^`. This operator does a binary xor between the bits of an integer. For

example, 8 ^ 127 yields 119. If you do xor again with 127, i.e., 119 ^ 127, you get back 8. Each pixel is encrypted by applying xor between the original red, green, and blue values and random integers generated from a new instance of `scala.util.Random`. Three new random numbers are drawn for each pixel's RGB component from the same Random instance. Let the user give an argument that you use as the random seed when creating the `Random` instance. In this way, the image can be restored by applying the encryption filter again, with the same argument, on the now encrypted image.
If the filter's `argDescriptions` sequence is non-empty, `ImageEditor` shall prompt for each argument in turn and show each description in the dialog. The user's input is converted to a decimal number of type `Double` before the arguments are used in the `apply` method. Decide for yourself how to handle default values and error handling if the user enters a string that cannot be converted to `Double`. *Tip:* `toDoubleOption` and `getOrElse` will be helpful.

```
object XOrCrypto extends Filter:
  val name = "Encrypt"
  override val argDescriptions = Seq("Encryption key")

  def apply(im: Image, args: Double*): Image = ???
```

```
scala> Seq(8, 127, 8 ^ 127).map(_.toBinaryString)
val res0: Seq[String] = List(1000, 1111111, 1110111)

scala> 8 ^ 127
val res1: Int = 119

scala> 119 ^ 127
val res2: Int = 8
```

Task 7. Gauss filter. A Gauss filter makes the image a bit less sharp. Gauss filtering is an example of so-called *convolution filtering*. Convolution is a kind of localized averaging. New pixels are created by combining each pixel with its surrounding pixels according to a special matrix algorithm.
To achieve this, one uses a so-called *convolution kernel* K which is a small square integer matrix. Place K over each element in the intensity matrix and multiply each element of K with the corresponding element in the intensity matrix. Sum the products and divide the sum by the sum of the elements in K to obtain the new intensity value at that point (a kind of average). The division is done so that the new intensity falls within the correct range (0 – 255). Example:

$$intensity = \begin{pmatrix} 5 & 4 & 2 & 8 & \dots \\ 4 & 3 & 4 & 9 & \dots \\ 9 & 8 & 7 & 7 & \dots \\ 8 & 6 & 6 & 5 & \dots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{pmatrix} \qquad K = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

The sum of the elements in K is $1+1+4+1+1=8$. To calculate the new intensity value at point `(1, 1)` where the current value is 3, calculate:

$$newintensity = \frac{0\cdot 5+1\cdot 4+0\cdot 2+1\cdot 4+4\cdot 3+1\cdot 4+0\cdot 9+1\cdot 8+0\cdot 7}{8} = \frac{32}{8} = 4$$

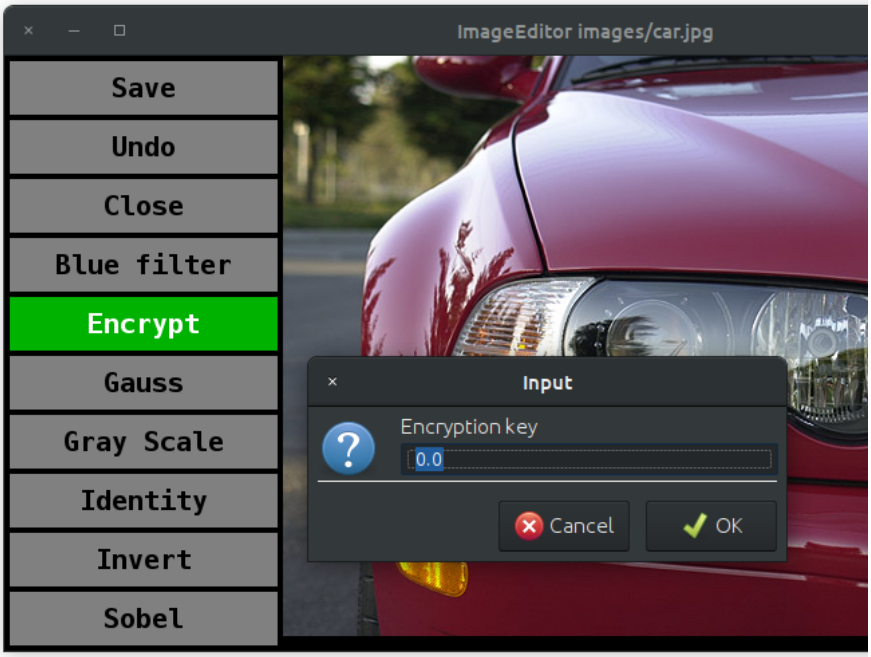


Figure 12.6: Encryption filter before application, during entering key.

Then continue by moving K one step to the right and calculate in the same way a new value for element with index `(1)(2)` (where the current value is 4 and the new value will be 5). Thereafter, do the same for all elements except for the "frame", i.e. the elements at the edges of the matrix.
Implement and test thoroughly first the method
`convolve(p: Matrix, x: Int, y: Int, kernel: Matrix, weight: Int): Short` in `trait Filter` which shall give the normalized sum of products of `kernel` and the points near `(x,y)` in matrix `p`, normalized by `weight`. `Tip: round` and `toShort` methods will be useful.

```
scala> import photo.*

scala> val p = Matrix(4,4)(5,4,2,8,4,3,4,9,9,8,7,7,8,6,6,5)
val p: photo.Matrix = Array(Array(5, 4, 9, 8), Array(4, 3, 8, 6), Array(2, 4, 7

scala> val K = Matrix(3,3)(0,1,0,1,4,1,0,1,0)
val K: photo.Matrix = Array(Array(0, 1, 0), Array(1, 4, 1), Array(0, 1, 0))

scala> Filter.convolve(p, 1, 1, K, K.flatten.sum)
val res0: Short = 4
```

Then create a filter `Gauss` which convolves using `convolve` for each color component separately. Proceed as follows:

1. Create three `short` matrices and store the red, green, and blue components of the pixels in them.

2. Perform convolution of the three components for each element and update `result` with the computed values.

3. The elements at the edges are not processed, but `result` must also have values for these. The simplest approach is to copy these elements unchanged from `im` to `result`. (Alternatively, you may set them to `Color.BLACK`, but then the filtered image might look smaller.)

Use the kernel `K` as above, and let `weight` be the sum of all elements in K.
It might be interesting to try values other than 4 in the center of the convolution matrix. With zero in the center, you get a stronger smoothing effect since the current pixel's value is ignored. Let the user enter an argument for the middle value, between 0 and 50, and describe this in `argDescriptions`. [22]

Task 8. Sobel filter. Sobel filtering is, like Gauss filtering, a type of convolution filtering. However, Sobel filtering gives the opposite effect compared to Gauss filtering, i.e. it enhances edges in an image. Essentially, it takes derivatives in the x and y directions and combines the result.

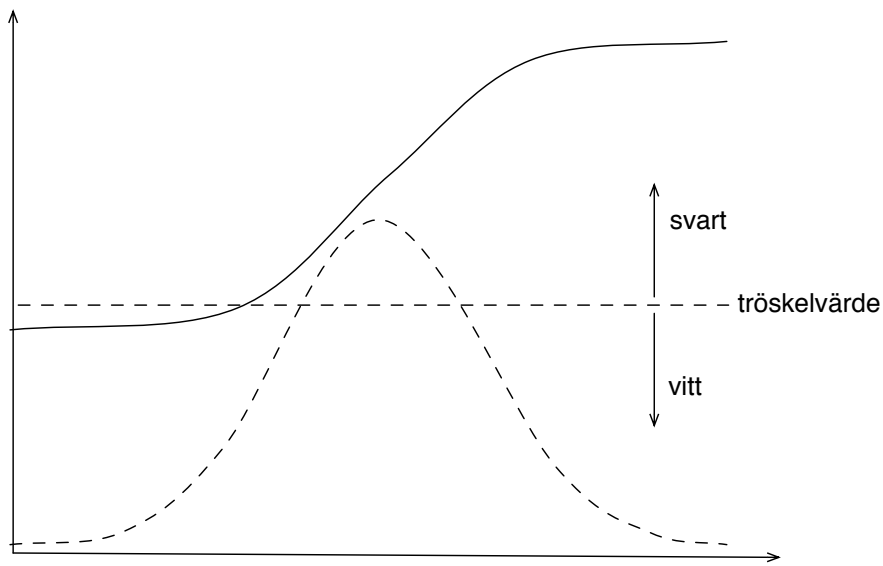


Figure 12.7: A function (solid line) and its derivative (dashed line).

In figure 12.7 a function f (solid line) and its derivative f' (dashed line) are shown. We see that where the function makes a "jump", the derivative has a large value. If the function represents the intensity of pixels along a line in the x or y direction, then those "jumps" correspond to edges in the image. If one then decides that pixels where the derivative value exceeds a certain threshold should be black and other pixels white, one obtains an image with strong edges.
Here the pixel intensity is not a continuous function that one can differentiate by ordinary mathematical rules. But it can be approximated by, for example, the formula:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

[22] It can sometimes be hard to see any difference between the Gauss-filtered image and the original. To get a really blurred image, you have to use a larger matrix as convolution kernel. You are encouraged to try this as an extra task.

As h approaches zero, this becomes the derivative definition. Further theoretical elaborations show that derivation can be expressed in a matrix with convolution as follows:

1. Calculate the intensity matrix with the method `intensity`.

2. For each point of the intensity matrix do two convolutions with these kernels:

$$SobelX = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix} \qquad SobelY = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$

   Use the method `convolve` with weight 1. The coefficients in the matrix $SobelX$ express derivation in the x direction, while $SobelY$ expresses derivation in the y direction. Explaining why the coefficients are sometimes 1, sometimes 2, sometimes positive and sometimes negative requires studying the underlying theory closely, but we will not do that here.

3. If the convolution results at a point are denoted by `sx` and `sy` then an indicator for the presence of an edge is given by `math.abs(sx) + math.abs(sy)`. Absolute values are needed since there are negative coefficients in the convolution matrices.

4. Set the pixel to black if the indicator exceeds the threshold value, otherwise white. Let the threshold be an argument that the user can specify.

Create a filter `Sobel` that implements edge enhancement with the above algorithm. See example in fig. 12.5. The user shall specify the threshold value with argument description `"Threshold (0.0 - 255.0)"`.

Task 9. For the report, you should generate automatic documentation based on documentation comments according to the instructions in Appendix E. You should write relevant documentation comments for at least half of your public methods. It is often useful to write the documentation comments *before* the implementation of the method body.

12.5.6 Optional extra tasks

Task 10. Keyboard shortcut. Make it possible to apply filters via key presses. Extend the `trait Filter` so that all filters can have a shortcut. Write on the button what the shortcut is so that the user can discover it.

Task 11. Contrast filter. If one applies contrast filtering on a color image, the image will be converted to grayscale. (One can of course enhance contrast in a color image and get a color image as a result. Then one treats the three color channels separately.) Many images suffer from too low contrast. This is because the image does not use the full available range 0–255 for intensity. You get an image with better contrast if you "stretch" the interval as follows (linear interpolation):

```
val newIntensity = 255 * (intensity - 45) / (225 - 45)
```

As can be seen, a pixel with intensity 45 will get new intensity 0 and a pixel with intensity 225 will get new intensity 255. The intermediate pixels spread evenly over the interval `[0, 255]`. For pixels with intensity less than 45, set the new intensity to 0; for pixels with intensity greater than 225, set the new intensity to 255. We call the interval where most of the pixels are `[lowCut, highCut]`. Points with intensity less than `lowCut` are set to 0, points with intensity greater than `highCut` set to 255. For other points, interpolate with the formula above (replace 45 with `lowCut`, 225 with `highCut`).
Now remains the finding of suitable values for `lowCut` and `highCut`. This is not something that can be done simply, since the values depend on the intensity distribution of the pixels in the image. One therefore begins by computing the image's intensity histogram, i.e. how many pixels have intensity 0, how many have 1, ..., up to 255.
In most image processing programs, one can then look at the histogram and interactively decide values of `lowCut` and `highCut`. We won't do that here. Instead, we decide on a percentage `cutOff` that the user can enter as argument, which calculates `lowCut` such that `cutOff` percent of pixels have intensity less than `lowCut`, and `highCut` such that `cutOff` percent of pixels have intensity greater than `highCut`.

Example: Suppose an image contains 100,000 pixels and `cutOff` is 1.5. Compute the intensity histogram by registering each intensity in an integer array
```
val histogram = Array.ofDim[Int](256)
```
and compute `lowCut` so that
`histogram(0) + ... + histogram(lowCut) = 0.015 * 100000`
as close as possible, it probably won't be exactly equal. Compute `highCut` similarly.

Summary of the algorithm:

1. Compute the intensity matrix.
2. Compute the intensity histogram of the image.
3. User argument is used as `cutOff`.
4. Compute `lowCut` and `highCut` as above.
5. Compute the new intensity for each pixel according to the interpolation formula and store the new pixels in `result`.

Create a filter `Contrast` that implements the algorithm. In the *images* directory, the image *moon.jpg* might be suitable for testing, since it has low contrast. Note: if `cutOff` is 0, then this filtering produces the same result as `GrayScale`. This can be seen when studying the interpolation formula.

Task 12. Own filter. Create your own custom filter. For example, you can create a filter that takes five arguments, where the first two values represent an intensity interval and the last three values represent red, green, and blue components of a pixel which shall replace pixels whose intensity lies outside the given interval.

Task 13. Own interactive tools. Create optional interactive editing tools with mouse and keyboard input. Start with a selection tool that allows a rectangular portion of the image to be selected using the mouse. Make it possible to apply filters to the selected portion of the image. You can also make arguments for e.g. the Gaussian filter settable with a slider that you draw below the button and that can be adjusted with the mouse or arrow keys.

Chapter 13

Revision

Concepts included in this week's studies:

- ☐ practice exercises
- ☐ report project
- ☐ prepare for oral exam

13.1 Tips

13.1.1 Summary FAQ

Unclear

1. named calls and value calls w03
2. constructor w05
3. pattern matching with `match` ... `case` w06
4. enumerations w07
5. `flatten`, `flatMap` w07
6. Difference between `map` and `Map`? w09
7. visibility, import/export, private/protected w10

Curious about

1. safe code and error handling

13.1.2 More FAQ

Unclear

1. When is it good/bad to use anonymous functions? w03
2. Classes and companion objects: what fits best where? w05
3. How to do error handling with `Option` and `Try`? w06
4. The difference between statement & expression, e.g. `if`, `for`? w01

Curious about

1. Multithreaded programming
2. Window management in intro programming under the hood
3. Generic type bounds `<:` `>:`

13.1.3 Review: What is an algorithm?

An algorithm is a step-by-step description of how to solve a problem.
Examples: SWAP, MIN, Registration, Searching, Sorting
The different steps in the problem-solving process (not necessarily in this order):

- Break down the problem into simpler subproblems and combine.
- Is there already a ready-made solution to the (sub)problem?
- Formulate the (sub)**problem** and clearly specify input and output:
  example MIN: input: sequence of integers; output: the smallest number
- Come up with a **solution idea**: (can be very tricky and difficult)
  example MIN: iterate over the numbers and keep track of the "smallest so far"
- Formulate a **stepwise description** that solves the problem:
  example: pseudo-code with a sequence of instructions

- Implement an **executable solution** in "real" code:
  example: a Scala method in a class or in a singleton object
- Does the algorithm have acceptable time and memory requirements?

Creativity is often required in the steps above – even in **recognizing** the problem!
Simple example: You encounter the problem MAX and see the similarity with MIN.
Exercise: Discuss how you solve this problem in relation to the steps above:

Counting the occurrences of different unique words in a text string.

13.1.4 Review: Rules of thumb/tips when choosing abstraction

Extension method, singleton object, case class, class, trait, or enum?

- If you want to add a method to an existing type without need for new attributes etc., use `extension`.
- Use `object` if you need to collect methods (and variables) in a module that exists only in one instance. You get local namespace and dot notation for free.
- Need to model **immutable data**, use a `case class` or `enum`.
- If you want enumerated values that you want to iterate over and match on in a sealed structure, with values in their own namespace, use `enum`.
- With `case class` and `enum` you also get :wq:content equality and lots of other goodies for free!
- Need **mutable state** use a regular `class`. The normal is that the mutable state (the attributes that are mutable) are `private` or `protected` and that all updates and accesses to the state happen indirectly via methods (getters/setters/...).
- Need an abstract base type use a `trait`, especially if you want mixin support. If you want to prevent mixin or facilitate use from Java, use `abstract class`.

13.1.5 Review: Tips on choice of collection

It is often easier with immutable collections of immutable elements and create new collections when changing. But for some algorithms it can be simpler or more efficient if you mutate in place in mutable collections.

- Need to handle values in sequence?
  - If you can manage without changing the contents after construction:
    `val`-reference to `Vector`
  - If you need to change contents but **not** the number of elements:
    `val`-reference to `Array`
  - If you need to change contents **and** number of elements:
    `var`-reference to `Vector` and e.g. the method `patch`, or
    `val`-reference to `ArrayBuffer` and e.g. the method `insert`
- Need to handle values `x` that should be unique?
  - Immutable: `Set`
  - Mutable: `val`-reference to `scala.collection.mutable.Set`

- Need to look up values `x:Int` by key e.g. String?
  - Immutable: `Map[String, Int]`
  - Mutable: **`val`**-reference to `scala.collection.mutable.Map[String, Int]`

13.2 Exercise Excercise13

Task 1. *Complete your project.*

Task 2. *Take a final exam.*

Task 3. *Prepare your project presentation.*

Task 4. *Create documentation for your project.* Read more in Appendix E about how to create documentation.

Task 5. *Review exercises and laboratory work.*

Chapter 14

ORAL EXAM

At the scheduled time you are invited to take a oral examination (CGI) for supervisors. You must have passed all laboratory exercises to be eligible to skip the oral exam. The purpose of the exam is to verify that you have an approved understanding of the concepts included in the course. You are recommended to prepare thoroughly for the exam, for example by reviewing fundamental concepts for each course module and revising basic exercises and laboratory work.
The exam will be a random sampling from the course content. You will be given some randomly selected questions where you will be asked to explain some of the concepts covered in the course. You will also receive assignments to write code similar to the course exercises and explain how the code works. You can practice typical questions here: `https://fileadmin.cs.lth.se/pgk/muntabot/`

Part III

Appendix

Appendix A

Kojo

A.1 What is Kojo?

Kojo[1] is an integrated development environment for Scala that is specially adapted for programming education in elementary school. Kojo is used at LTH's Science Center Vattenhallen for training elementary school teachers in programming and during school visits and other visitor activities, in which teachers and students at LTH work as supervisors.

Kojo is open source and the development community is led by Lalit Pant from India. Kojo also includes easily accessible libraries that lower the threshold to program moving graphics and simple games.

During the course's first lab, we use the graphics library in Kojo to illustrate basic concepts, such as sequence, alternative, repetition, and abstraction.

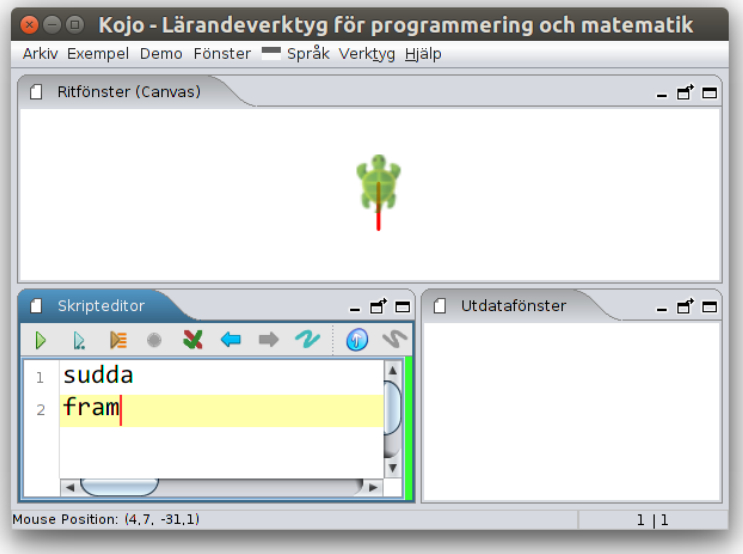


Figure A.1: The beginner-friendly development environment Kojo for Scala in Swedish.

[1] en.wikipedia.org/wiki/Kojo_(programming_language)

A.2 Using the graphics library in Kojo

Kojo is based on the proven educational idea of turtle graphics[2], where you write programs that control a turtle with a pen under its belly. When the turtle moves, a line of any color you choose is drawn on the screen. Depending on how you decide the turtle should move and the color you set for the pen, you can create various interesting pictures while simultaneously learning the fundamentals of programming.
During the course's first lab, you should use the graphics library in Kojo together with the VS `code` editor and `scala repl` in the terminal (see appendices B and C). Download the file `kojo.scala` from `https://fileadmin.cs.lth.se/kojo.scala` and save it in a new directory using your browser, or via these commands (note the capital letter `O`, not a zero, in the option `-sLO`):

```
> mkdir w01-kojo
> cd w01-kojo
> curl -sLO https://fileadmin.cs.lth.se/kojo.scala
```

Now you can start Scala REPL and draw with Kojo like this:

```
> scala repl .
Welcome to Scala 3.1.2 (17.0.2, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.

scala> fram; höger; fram; vänster
```

You can start VS `code` in the current directory like this:

```
> code .
```

Write the program below in VS `code` and save it in the same directory as the previously downloaded file, under a new filename of your choice, e.g. `rita.scala`:

```
@main def rita = { fram; höger; fram; vänster }
```

Run your standalone program with:

```
> scala run .
```

You should now get a window called Kojo Canvas with a turtle that has drawn two lines. When you close the window, the program ends. Try more turtle functions according to table A.1.
Instead of downloading the file `kojo.scala` you can place its contents appropriately in your program as shown below. Note that the line starting with `//> using dep` should be a single long line without line breaks.

```
//> using scala 3
//> using dep "net.kogics:kojo-lib:0.3.1,url=https://github.com/litan/kojo-lib/
    releases/download/v0.3.1/kojo-lib-assembly-0.3.1.jar"

export net.kogics.kojo.Swedish.*, padda.*, CanvasAPI.*, TurtleAPI.*
export builtins.activateCanvas
export java.awt.Color

def selectColor(
  message: String = "Select a color",
```

[2] `https://en.wikipedia.org/wiki/Turtle_graphics`

```
  default: java.awt.Color = java.awt.Color.red
): java.awt.Color =
  import javax.swing.JColorChooser
  Option(JColorChooser.showDialog(null, message, default)).getOrElse(default)
```

The Scala code for the Swedish turtle API is available here:
github.com/litan/kojo-lib/blob/main/src/main/scala/net/kogics/kojo/i18n/Swedish.scala

A.3 Kojo Desktop

Kojo is available as a standalone desktop application, called Kojo Desktop. Kojo Desktop includes its own editor with syntax highlighting for Scala, but currently only works for Scala 2. One of the most visible differences between Scala 2 and Scala 3 is that curly braces around multi-line functions are mandatory in Scala 2, whereas Scala 3 makes curly braces optional. So if you use Kojo Desktop, you need to remember to surround sequences of lines that belong together with { and }.
Kojo Desktop is preinstalled on LTH's computers and is started with the terminal command `kojo` or via the application menu. For instructions on how to install Kojo Desktop on your own computer, see here: lth.se/programmera/installera
When you start Kojo for the first time, choose "Swedish" in the language menu and restart Kojo. After that, the graphics functions work in Swedish according to table A.1 on page 201. After restarting Kojo set to Swedish, the program looks approximately like in figure A.1 on page 199.
There are a number of useful keyboard shortcuts that you find in the menus in Kojo Desktop. Especially check Ctrl+Alt+Space, which gives autocompletion based on what you have started typing.

A.4 Kojo in the Browser

A limited version of Kojo is available for programming directly in your web browser here: `http://kojo.lu.se/`
When you press the play button, your code is compiled on a server to Javascript via ScalaJS and then the Javascript code runs in your browser. Kojo on the web is also currently limited to Scala 2 and requires that you surround sequences of lines that belong together with { and }.

A.5 More about Kojo

In this document there is a simple introduction to Kojo:
"Introduction to Kojo" `http://www.kogics.net/kojo-ebooks#intro`
In table A.1, continued on the following pages, there is a selection of commands in Kojo in Swedish and English.

Table A.1: A selection of functions in Kojo. See also lth.se/programmera

Swedish / English	*What happens?*
`clear` `clear()`	The drawing window is cleared
`forward` `forward()`	The turtle moves forward 25 steps.

`forward(100)` `forward(100)`	The turtle moves forward 100 steps.
`right` `right(90)`	The turtle turns 90 degrees to the right.
`right(45)` `right(45)`	The turtle turns 45 degrees to the right.
`left` `left(90)`	The turtle turns 90 degrees to the left.
`left(45)` `left(45)`	The turtle turns 45 degrees to the left.
`hop` `hop()`	The turtle hops 25 steps without drawing.
`hop(100)` `hop(100)`	The turtle hops 100 steps without drawing.
`jumpTo(100, 200)` `jumpTo(100, 200)`	The turtle jumps to position (100, 200) without drawing.
`moveTo(100, 200)` `moveTo(100, 200)`	The turtle turns and moves to position (100, 200).
`home` `home()`	The turtle returns to the starting position (0, 0).
`east` `setHeading(0)`	The turtle turns so that the nose points to the right.
`west` `setHeading(180)`	The turtle turns so that the nose points to the left.
`north` `setHeading(90)`	The turtle turns so that the nose points upwards.
`south` `setHeading(-90)`	The turtle turns so that the nose points downwards.
`towards(100,200)` `towards(100, 200)`	The turtle turns so that the nose points toward the position (100, 200)
`setHeading(90)` `setHeading(90)`	The turtle turns its nose to angle 90 degrees.
`heading` `heading`	Returns the angle value the turtle's nose is pointing.
`setSlow(5000)` `setAnimationDelay(5000)`	Makes the turtle draw very slowly.
`println("hej")`	Prints the text `hej`.
`setFontSize(100)` `setPenFontSize(100)`	The turtle writes with very large text next time you print.
`arc(100, 90)` `arc(100, 90)`	The turtle draws an arc with radius 100 and angle 90.
`circle(100)` `circle(radius)`	The turtle draws a circle with radius 100.
`visible` `visible()`	The turtle becomes visible.
`invisible` `invisible()`	The turtle becomes invisible.

`position.x` `position.x`	Returns the turtle's x position
`position.y` `position.y`	Returns the turtle's y position
`penDown` `penDown()`	Puts down the turtle's pen so that it draws when it moves.
`penUp` `penUp()`	Lifts up the turtle's pen so that it does NOT draw when it moves.
`penIsDown`	Checks if the pen is down or not.
`setPenColor(pink)` `setPenColor(pink)`	Sets the pen's color to pink.
`setFillColor(purple)` `setFillColor(purple)`	Sets the fill color to purple.
`setFillColor(noColor)` `setFillColor(noColor)`	Makes the turtle not fill anything while drawing.
`setPenThickness(20)` `setPenThickness(20)`	Sets the pen thickness to 20.
`saveStyle` `saveStyle()`	Saves the pen's color, thickness, and fill color.
`restoreStyle` `restoreStyle()`	Loads the previously saved color, thickness, and fill color.
`savePosHe` `savePosHe()`	Saves the pen's position and direction
`restorePosHe` `restorePosHe()`	Loads the previously saved direction and position
`beamsOn` `beamsOn()`	Turns on the sight.
`beamsOff` `beamsOff()`	Turns off the sight.
`setBackground(black)` `setBackground(black)`	The background color becomes black.
`setBackgroundV(green, yellow)` `setBackgroundV(green, yellow)`	Background with a gradient from green to yellow.
`repeat(4){forward; right}` `repeat(4){forward; right}`	The turtle moves forward and turns right 4 times.
`round(3.99, 2)`	Rounds 3.99 to two decimals, i.e. 4.0
`random(100)`	Gives a random number between 0 and 99.
`randomWithDecimals(100)`	Gives a random number between 0 and 99.99999999
`systemTime`	Gives the current system clock in seconds.

Appendix B

Terminal Window

B.1 What is a terminal window?

In a terminal window, you can write commands that run programs and manage files. When programming, you often use terminal commands to compile and execute your programs.

Terminal in Linux

In Ubuntu, the easiest way is to press **Ctrl+Alt+T** or search for ”terminal” in the app menu. This opens a window with a blinking cursor indicating it is ready to receive your text commands. An example of a command is `ls` which prints a list of files in the current directory, as shown in fig. B.1.
What appears in a terminal window is managed by a **command shell** (eng. *command shell*), which is ready to receive commands after a prompt that ends with a $ sign. When you type a command and press Enter, the command shell calls a command interpreter that interprets and executes your commands. If a command cannot be interpreted, an error message is displayed.

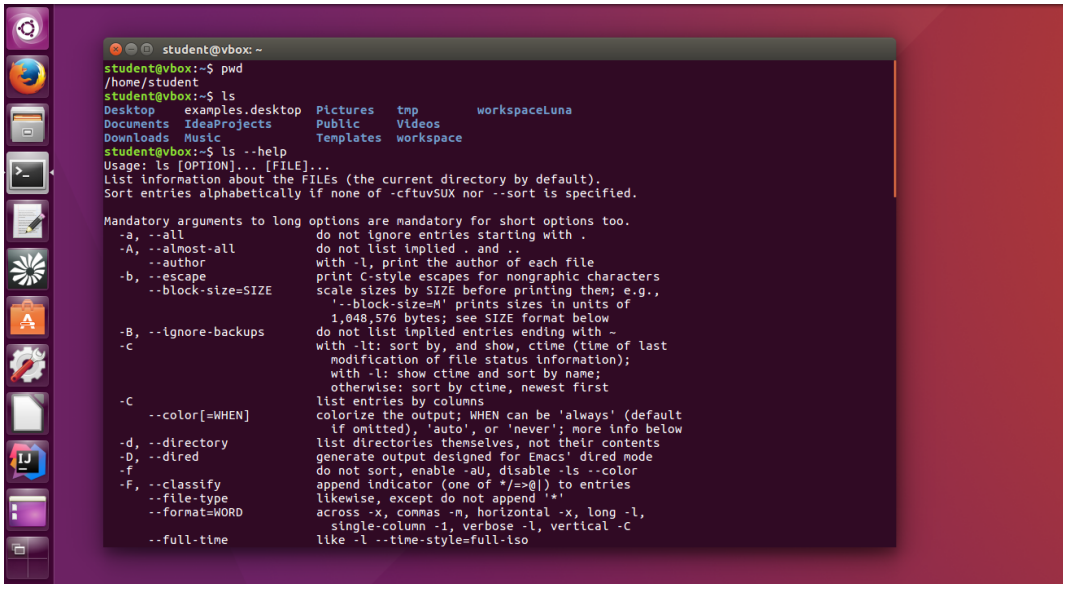


Figure B.1: Terminal window in Ubuntu is opened with Ctrl+Alt+T.

There are many useful keyboard shortcuts, of which the most important are shown in table B.1. It is good if you learn these shortcuts by heart so that your work in the terminal becomes fast and smooth.

up/down arrow	scroll through command history
Tab	”auto-complete”, fill in the rest based on what you have typed so far
Tab Tab	pressing Tab twice lists multiple options, if any exist
Ctrl+A	”ahead”, move cursor to the beginning of the line
Ctrl+E	”end”, move cursor to the end of the line
Ctrl+K	”kill”, remove characters from cursor to end of line
Ctrl+U	”undo”, remove characters from cursor to beginning of line
Ctrl+Y	”yank”, paste the most recently cut text
Ctrl+Z	”zleep”, stop ongoing process, then type `bg` to run in background
Ctrl+L	clear terminal window
Ctrl+D	exit the command shell

Table B.1: Important keyboard shortcuts in Linux terminal window.

Ctrl+C normally causes an interrupt of the running process, and instead *paste* is bound to Shift+Ctrl+C, but if you want Ctrl+C to be ”Copy” as usual for copying selected text and do interrupts with Shift+Ctrl+C, you can change this in the terminal window menu ”Edit → Keyboard Shortcuts”, or similar.

PowerShell, Cmd and Linux in Microsoft Windows

There are several ways to run terminal commands in Windows:

- **Powershell**. In Microsoft Windows, the command interpreter *Powershell* exists with a special command syntax. It is not Linux-based but there are aliases defined for some common Linux commands, including `ls`, `cd` and `pwd`. You start Powershell for example by pressing the Windows key and typing `powershell`. You can also, while browsing files, click the filename bar at the top in the file browser and type `powershell` and press Enter; this will start Powershell in the current directory.

- **Cmd**. Windows also has the original, old command prompt *Cmd* with completely different commands. For example, in Cmd you write the command `dir` instead of `ls` to list files.

- **WSL**. In both Windows 10 and 11, you can also run the Ubuntu terminal using Windows Linux Subsystem (WSL), which is recommended, especially if you cannot do so-called dual boot[1].
  - See here for how you can install WSL on Windows (WSL2 is recommended over WSL1 if your machine supports it): `https://docs.microsoft.com/en-us/windows/wsl/install`
  - There is also a handy extension for VS Code called Remote-WSL that allows you to edit files in Windows that are in WSL, see more here: `https://code.visualstudio.com/docs/remote/wsl-tutorial`

[1] Read more about dual boot here and ask someone for help who has done it before: https://www.linuxtechi.com/dual-boot-ubuntu-22-04-and-windows-11/

- **Windows Terminal**. The new Microsoft app *Windows Terminal* is recommended regardless of whether you use Powershell, Cmd, or WSL. Read more here about how to install Windows Terminal:
  https://docs.microsoft.com/en-us/windows/terminal/

Terminal in Apple macOS/OS X

Apple OS X and macOS are Unix-based operating systems. Most common terminal commands that work in Linux also work under Apple OS X and macOS. You start a terminal window in Apple's operating system by clicking the magnifying glass at the top right, typing `terminal`, and pressing Enter.

B.2 What is a path?

When you type a command in the terminal, or run any program on your computer, the operating system needs to identify where the machine code of the program file is located before the program can be executed.
The location of files is done using a **path**, which indicates a position in the file system. The file system is often viewed as an upside-down tree and is therefore also called the "file tree". The "topmost" position is called the "root" and denoted by a simple slash `/`. Directories inside directories form branches in the tree. A path points out routes through the tree needed to reach the "leaves", which are the actual files.
You can see where a program is located in Linux using the command `which` as below.[2]
The list of directories in the path is separated by slashes.

```
$ which java
/usr/lib/jvm/oracle_jdk8/bin/java
$ which ls
/bin/ls
```

A path can be **absolute** or **relative**. An absolute path starts from the root and shows the entire route from root to destination, e.g. `/usr/bin/firefox`, while a relative path starts from the current directory (where you "are") and *does not* begin with a slash.
All operating systems keep track of a number of different paths to be able to find special files in the file tree. These paths are stored in so-called **environment variables**. There is a *special* environment variable simply called **PATH**, in which all paths to programs exist that should be available for your user identity directly for execution by their filenames, *without* needing to specify absolute paths.
You can see what is in your PATH in Linux using the command `echo \$PATH` while in Windows Powershell you write `\$env:Path` where only the first letter is uppercase. In Linux, the directories in the path are separated by colons, while Windows uses semicolons.
Sometimes you may need to update your PATH for programs you have installed and want to be generally accessible. This is done in different ways on different operating systems, for Linux see e.g. here: stackoverflow.com/questions/14637979/how-to-permanently-set-path-on-linux
When specifying paths, some characters have special meanings:

[2] Type `gcm ls` in Windows Powershell for equivalent of `which ls`
Or type `New-Alias which get-command` to have access to the command `which` in Powershell.
stackoverflow.com/questions/63805/equivalent-of-nix-which-command-in-powershell

~	”tilde”, your home directory
/	”slash”, slash indicates root of the file tree if at the beginning of the path, but acts as directory separator inside the path
.	a dot indicates the current directory, where you ”are”
..	two dots indicate one step ”up” in the file tree
"	surround a path with quotation marks at the beginning and end if it contains anything other than English letters, e.g. spaces
\	*backslash+space* is used to denote spaces in paths that are *not* surrounded by quotation marks

B.3 Some important terminal commands

In table B.2 there is a list of some important terminal commands that are good to learn by heart.
An introduction to LTH's computers with examples of how to use common Linux commands can be found in this document `http://www.ddg.lth.se/perf/unix/` which is used in the introductory week for beginners in the Computer Engineering program at LTH.
On the site `http://ss64.com/` there is a more comprehensive list of useful terminal commands and corresponding explanations for Linux (Bash), Windows (Powershell, Cmd), and Apple OS X (Bash).

`ls`	list files in current directory (where you ”are”)
`ls` *p*	list files in directory *p*
`ls -A`	list all files in current directory, including hidden ones
`man ls`	manual for command `ls`; also try `man` for other commands!
`cd` *p*	”change directory”, change current directory to *p*
`pwd`	”print working directory”, print the path of the current directory
`cp` *p1 p2*	”copy”, copy file at path *p1* to a new file named *p2*
`mv` *p1 p2*	”move”, rename file *p1* to *p2*
`rm` *p*	”remove”, delete file *p*
`rm -r` *p*	”remove recursive”, delete directory *p* with all contents; be careful!
`mkdir` *p*	”make dir”, create a directory *p*
`cat` *p1 p2*	”concatenate”, print the entire contents of one or more files *p1 p2 etc.*
`less` *p*	print the contents of file *p*, one screen at a time
`wget` *url*	download *url*, e.g. `wget https://fileadmin.cs.lth.se/pgk/workspace.zip -o ws.zip`
`unzip` *p*	unpack *p*, e.g. `unzip ws.zip`

Table B.2: Some important terminal commands in Linux. With *p*, *p1*, *p2*, etc. absolute or relative paths (eng. *path*) are meant, see section B.2.

Appendix C

Edit, compile and execute

C.1 What is an editor?

An editor is used to edit program code. There are many different editors to choose from. Experienced developers often spend a lot of energy learning to use their favorite editor's keyboard shortcuts and special features, as this greatly affects how quickly code editing can be done.
A good editor has **syntax highlighting** for the language you are using, so that different parts of the code are shown in different colors. This makes it much easier to read and navigate the code.
Below are listed some important functions that are used many times daily when coding:

- **Navigate**. There are several ways to move the cursor and scroll through the code. All editors offer search capabilities, and most editors also have more advanced search functions where code patterns can be identified and multiple search hits are highlighted across multiple code files.

- **Select**. Selecting code can be done in many ways: with arrow keys + Shift, with various special menu options, with mouse + double-click or triple-click, etc. Some editors also have the ability to have multiple cursors so that several lines can be edited simultaneously.

- **Copy**. By Copy-Paste you avoid typing the same thing many times. The keyboard shortcuts Ctrl+C for Copy and Ctrl+V for Paste become second nature after a while. However, you should be aware that it is easy to make mistakes when copying a large section that is then slightly changed; many Copy-Paste bugs come from not being careful enough and often it is better to write from scratch instead of copying so that you have time to think while typing.

- **Cut**. By Ctrl+X for Cut and Ctrl+V for Paste, you can easily move code. Writing code is a stepwise process where many changes are made along the way to improve and further develop the code. Moving code to create a better structure is very common.

- **Formatting**. With indentations, line breaks, and nested blocks at multiple levels, code gains structure. Many editors can help with this and have special keyboard shortcuts to increase or decrease indentation level.

- **Parenthesis matching**. Different kinds of parentheses, `( { [ ) } ]` , need to be matched for the code to work; otherwise, the compiler often gets completely lost and strange error messages can point to the wrong place in the code. A good code editor can help you highlight which pairs of parentheses belong together so that you avoid spending too much time looking for a missing or misplaced parenthesis.

C.1.1 Choose editor

Table C.1 shows a list of some popular editors. It is a great advantage if your favorite editor is available on multiple platforms so that you can benefit from your acquired skills when you need to switch between different operating systems.
In this course, Visual Studio **`code`** is recommended because it is open, free, available for Linux, Windows and Mac, and has good support for Scala and Java. But if you are already used to any of the other editors in table C.1, they also work well.
An Integrated Development Environment (eng. *integrated development environment, IDE*), see appendix H, offers many advanced features that help you code effectively once you have learned the techniques. VS `code` now has several IDE-like features, and the boundary between a pure editor and an IDE, such as IntelliJ and Eclipse, is no longer as clear as before.

Table C.1: Some popular editors. VS Code is recommended in the course.

Editor	*Description*
VS Code	Open, free and gratis. Available for Linux, Windows, & Mac. Pre-installed on LTH's Linux computers and started with the command `code`. The open-source project was started by Microsoft and has an active community with many developers and many useful extensions. Search for the extension `scalameta.metals` and install it to get syntax highlighting and many other IDE features for Scala. https://code.visualstudio.com/ https://scalameta.org/metals/docs/editors/vscode/#installation
Gedit	Open, free and gratis. Easy to learn but not very advanced. Pre-installed on LTH's Linux computers and started with the command `gedit`. https://wiki.gnome.org/Apps/Gedit
Nano	Open, free and gratis. A simple editor for small tasks in the terminal. Pre-installed on most Linux systems on planet Earth. Started with the command `nano`. https://www.nano-editor.org/
Notepad++	Open, free and gratis. Developed especially for Windows but also available for Linux. https://notepad-plus-plus.org/ https://snapcraft.io/notepad-plus-plus

Vim	Open, free and gratis. High learning curve. Available for Linux, Windows, & Mac. Pre-installed on LTH's Linux computers and started with the command `vim`. With Scala Metals (see link below) you get IDE-like features. You exit vim by pressing Escape and then typing :q and pressing Enter. http://www.vim.org/ https://scalameta.org/metals/docs/editors/vim.html
Emacs	Open, free and gratis. High learning curve. Available for Linux, Windows, & Mac. Pre-installed on LTH's Linux computers and started with the command `emacs`. With Scala Metals (see link below) you get IDE-like features. http://www.gnu.org/software/emacs/ https://scalameta.org/metals/docs/editors/emacs.html
Sublime Text	Closed source. Free to try, but the program periodically suggests buying a license. Available for Linux, Windows, & Mac. With Scala Metals (see link below) you get IDE features. http://www.sublimetext.com/3 https://scalameta.org/metals/docs/editors/sublime.html

C.2 What is a compiler?

A **compiler** (eng. *compiler*) is a program that reads source code and translates it into executable machine code, as shown in figure C.1. The source code being compiled consists of text that follows the rules of a programming language, for example Scala or Java.

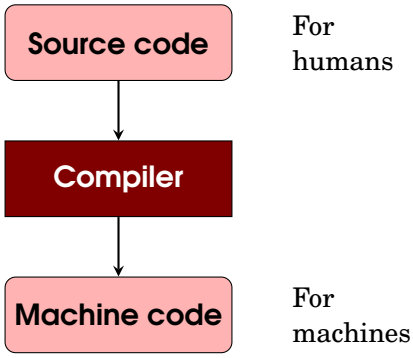


Figure C.1: A compiler translates from source code to machine code.

Some compilers generate code that can be run directly by a processor, while others generate an intermediate format that is interpreted during execution. The latter is the case with Java and Scala, which allows the program to be compiled once for all platforms and then run on all those processors for which there is a so-called Java Virtual Machine (JVM). The code produced by a compiler for the JVM is called **bytecode**.

If the compilation does not succeed, the compiler outputs an error message and no machine code is generated. It is not easy to build a compiler that gives good error messages in all cases, but the error message often provides good clues to the cause of the error once you have learned to understand the programming language-specific vocabulary that the compiler uses.

Even if the program compiles without error messages and generates executable machine code, it is common that the program does not work as intended. Sometimes it is very difficult to figure out what the problem is, and you may need to do extensive investigations of what happens during execution by, for example, printing the values of various variables or changing the code in other ways and seeing what happens. This process is called debugging, and is an essential part of all system development. Read more about debugging in Appendix D.
Exhaustive testing of a larger program, running the program through *all* possible execution paths, is practically impossible to carry out within a reasonable time, as the number of combinations grows very quickly with the size of the program. Therefore, the compiler is a very important tool. With the analysis and checks performed by the compiler, many bugs that would otherwise be very difficult to find can be avoided and fixed in the compilation phase, already *before* you execute the program.

C.3 Java JDK

Scala, Java, and several other languages use the Java platform as execution environment. If you not only want to run programs developed by others but also develop your own programs that work in this environment, you need to install the Java Development Kit (JDK). This development package contains several parts, including:

- The compiler `javac` compiles Java programs to bytecode stored in class files with the filename extension `.class`.
- The execution environment Java Runtime Environment (JRE) with the command `java` which starts the Java Virtual Machine (JVM) that can load and execute bytecode stored in class files.
- The program `jar` which packages many related class files into one jar file that can easily be distributed via the network and then run with the `java` command on any machine with a JRE.
- The program `javap` which reads class files and prints out what they contain in a human-readable format (such a program is called a disassembler).
- The JDK also includes a very large collection of ready-made program libraries with support for network communication, file management, graphics, encryption, and a lot more needed when building modern systems.

You can read more about Java and its history here:
https://en.wikipedia.org/wiki/Java_(programming_language)

C.3.1 Check if you have JDK installed

Open a terminal window (see appendix B) and type (note the final `c` in `javac`):

```
javac -version
```

You should see something like the following printed, where `x` and `y` are numbers:
`javac 21.x.y`
If the output says that `javac` is missing, install JDK as described below.
We use JDK 21 in the course. It is also possible to use the older versions JDK 8 and JDK 11, but JDK 9 or 10 do not work with all the tools we use, and later versions than 21 can also cause problems. Read more under ”Tools” on the course website.

C.3.2 Install JDK

There are several JDK distributions to choose from, of which OpenJDK and Oracle JDK are two examples. We use OpenJDK in the course, which can be installed via https://adoptium.net/temurin/releases/?version=21.
If you install all Scala tools using Coursier according to instructions on the course website under "Tools", https://lunduniversity.github.io/pgk/#verktyg then JDK will be installed automatically (if you don't already have JDK).

C.4 Scala

Scala uses the Java Virtual Machine (JVM) as execution environment, but can also be run in the browser using the ScalaJS compiler which compiles from Scala to JavaScript. In this course we use Scala on the JVM. A Scala installation includes, among other things, the compiler `scalac` and also an interactive command shell called Scala REPL (see below C.4.2) where you can test your Scala code line by line and see what happens immediately.
The official Scala website is here: http://www.scala-lang.org/
You can find more about Scala's history and other background information here: en.wikipedia.org/wiki/Scala_(programming_language)

C.4.1 Install Scala

Scala is pre-installed on LTH's computers. On the course website under "Tools" you find detailed instructions on how to install Scala on your own computer: https://lunduniversity.github.io/pgk/#verktyg

C.4.2 Scala Read-Evaluate-Print-Loop (REPL)

For many languages, e.g., Scala and Python, there is an interactive program intended for the terminal that makes it possible to execute single lines of program and immediately see the effect. Such a program is called a *Read-Evaluate-Print-Loop* (REPL), because it reads and interprets one line at a time. The result of the evaluation of your code is printed in the terminal, and then the command shell is ready for the next code line.
The exercises in the course largely rely on you using the Scala REPL to investigate principles and concepts included in the course through your own code experiments. Even when you develop larger programs with an editor and an IDE in lab sessions, it is good to have Scala REPL on hand. Then you can paste parts of the program you are developing into Scala REPL and gradually develop subprograms that eventually work as you want.
In Scala REPL you see type information for variables and methods, which is very helpful when trying to figure out what a line of code means. By practicing your ability to take advantage of Scala REPL, your productivity will increase.
You start Scala REPL with the command `scala` and type Scala code after the prompt `scala>` and compilation + execution happens when you press Enter.

```
> scala
Welcome to Scala 3.1.2 (17.0.2, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.

scala> 41 + 1
```

```
val res0: Int = 42
```

Each evaluated value is saved in a new variable, here `res0`.
If you write an incomplete line, editing continues on the next line. You can navigate between lines with the up-arrow and down-arrow keys. When you complete with a line that makes your code complete, all lines are compiled and executed. You can prematurely cancel multi-line editing by typing a semicolon `;` and then pressing Enter. If you want to continue editing with a new line and prevent premature evaluation, press Esc+Enter. The Escape key is located in the top-left of the keyboard. See example below:

```
scala> def fleraRader = 42  // Esc+Enter gives new line
     |   + "ny rad".length  // continuation line, finish with Enter
```

Depending on which operating system you run, other key combinations may also work to start a new line in REPL; try for example Linux: Left Alt+Enter, Windows: Left Alt + Shift + Enter, VS Code Terminal in Windows Left Alt + Enter, VS Code Terminal in MacOS: Option + Enter.
Many keyboard shortcuts that work in the terminal shell also work in Scala REPL. Feel free to go through the list in table B.1 on page 206, and test what happens in Scala REPL. Training your dexterity with these shortcuts will make your work in Scala REPL much faster.
With the command `:help` you get a list of special commands for Scala REPL:

```
The REPL has several commands available:

:help                    print this summary
:load <path>             interpret lines in a file
:quit                    exit the interpreter
:type <expression>       evaluate the type of the given expression
:doc <expression>        print the documentation for the given expression
:imports                 show import history
:reset [options]         reset the repl to its initial state, forgetting all session en
:settings <options>      update compiler options, if possible
```

You can also start Scala REPL with the help of the command `scala repl .` with a space and a dot at the end. The dot makes all `.scala`-files in the current directory compiled by Scala CLI and made available for use in REPL.

C.4.3 Compile and run with Scala Command Line Interface

Since 2022, there is a new handy command line interface for compiling, executing and packaging Scala programs called *Scala CLI*. If you install the Scala tools according to instructions on the course website under "Tools", https://lunduniversity.github.io/pgk/#verktyg then Scala CLI is included.
Here are some useful commands:

- The first time you run a newly installed Scala CLI installation, run this command so you get access to handy completions with the TAB key:
  `scala install completions`
- With this command you can prepare VS Code to cooperate with Scala CLI (note space and trailing dot):
  `scala setup-ide .`
  Run the above command before you start VS Code the first time with `code .` in

the current directory, or exit VS Code and run the above command and start VS Code again with `code .` in the current directory.

- Scala CLI can start REPL in the current directory with your Scala and Java programs automatically compiled and made available in REPL with the command below. With the option `-S` you specify which Scala version you want to run:
  `scala repl . -S 3.7.2`

- Instead of specifying the Scala version with the option `-S` on the command line, you can inside your program, on the first line, write this "magic" comment:
  `//> using scala 3.7.2`
  Then Scala CLI will automatically select (and download if needed) the desired version of the Scala compiler (note the > after `//`):

- Compile all Scala and Java programs in the current directory and see possible error messages. Using `--watch` (can be shortened to `-w`) will automatically compile all files as soon as changes are saved in VS Code (shortcut Ctrl+S):
  `scala compile . --watch`

- Run Scala and Java programs in the current directory starting the top-level `def` marked `@main` (if there are several, you are asked which `@main def` you want to run).:
  `scala run .`

- Create an executable file:
  `scala package .`

- Create a copy of your project with directory structure and files adapted for the build tool `sbt` (see Appendix F):
  `scala export . --sbt --output ../nameofnewprojdir`
  Change the directory name `nameofnewprojdir` to any new directory name that does not exist. Note the double dots that place the new directory at the same level as your current project, and *not* in your current directory (to avoid file duplicates that cause compilation errors).

- If you write `scala help` you will see what else you can do.

Appendix D

Fixing bugs

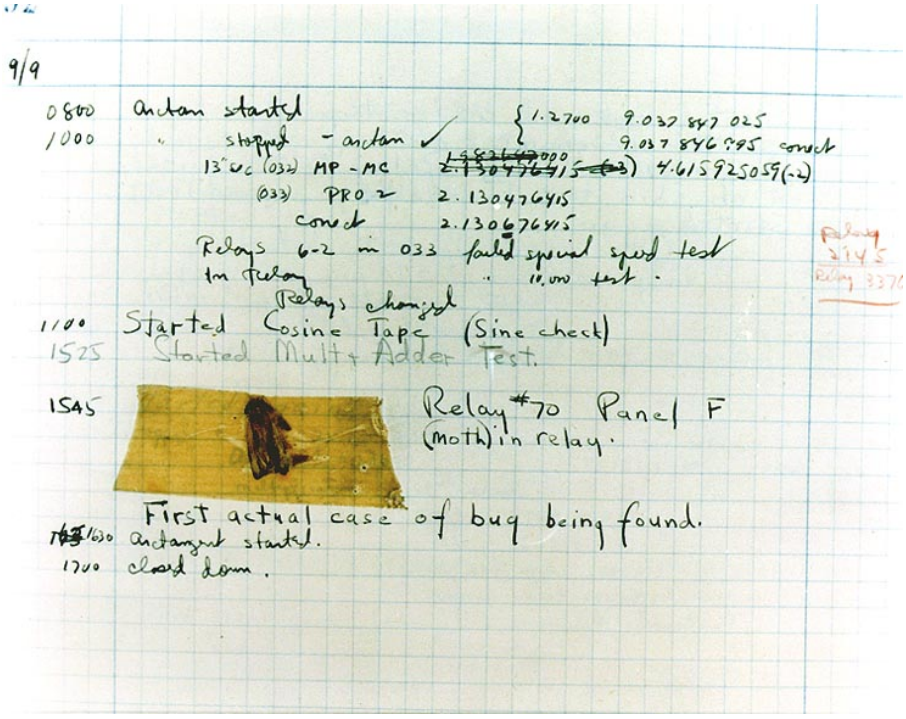


Figure D.1: The first documented bug was found on September 9, 1947 in a Mark II Aiken Relay Calculator by Grace Hopper.[1]

D.1 What is a bug?

A bug, also called a glitch (eng. *bug*), is an error that can cause a program not to behave as intended, and can result in unwanted output, the program crashing, or even sudden death.[2]

The origin of the word's usage in programming context is somewhat unclear, but can be traced back to the English word *bug* which means insect or small creature. It is

[1] commons.wikimedia.org/w/index.php?curid=165211 Courtesy of the Naval Surface Warfare Center, Dahlgren, VA., 1988. - U.S. Naval Historical Center Online Library Photograph NH 96566-KN, Public Domain.

[2] www.theguardian.com/technology/2016/jul/01/tesla-driver-killed-autopilot-self-driving-car-harry-potter

often told that during debugging of a program running on an early computer built with electromechanical relays, a dead moth was discovered crushed between the armature and the coil in a relay, which caused the program not to execute correctly.

D.1.1 Different types of errors

When you want to learn more about errors in software-based systems and how they can be fixed, it is important to clearly distinguish between **mistakes** (eng. *mistake*), **fault** (eng. *fault*), and **failure** (eng. *failure*). By ”mistakes” we mean errors made by humans (developers, system administrators, operators, users, etc.) while creating and using a software system.
Errors can occur in different parts of the process:

- **Requirements errors** occur while figuring out what the system should do and involve mistaking the users' needs and wishes.
- **Design errors** arise when the system's structure is poorly designed.
- **Implementation errors** are made during programming and occur when writing incorrect lines of code.
- **Test errors** happen during system testing when the test code is incorrect and therefore gives false alarms of ”errors”, even though the behavior is actually correct.
- **Operator errors** occur when the system is handed over to those who will install and run the system in production, and system operations (eng. *operations, ”ops”*) are handled in a way that leads to problematic consequences.
- **User errors** happen when users supply incorrect input, possibly in violation of guidelines on how the system should be used, which the system cannot handle correctly, leading to more or less serious faulty behaviors.

Errors can thus be made in different parts of the development process, which can, either immediately or sometime in the future, cause faults. But it is not certain that a fault will ever manifest. Perhaps the incorrect lines of code that *could* cause a fault are never executed. Or no user will ever want to use the system as stipulated by (unnecessary) requirements. Errors only become apparent when faults *manifest* at execution.
Faults can also be categorized based on *how* they are detected during the development process. Common categories are faults discovered during review, compilation errors, and runtime errors, which are discussed below:

- Faults discovered during **review**. An effective way to find errors is for people to carefully read through their own and others' code, looking for possible problems and flaws. It's common to be ”blind” to your own code. Therefore, someone else's independent review with ”fresh eyes” can be very fruitful. During code review, one can also assess how readable and modifiable the code is or whether it has other important qualities relevant for future development. Reviews often find simple programming mistakes such as incorrect conditions and loop counters not incremented correctly, etc.
- **Compilation errors** occur during compilation and are detected thanks to checks performed by the compiler.

With compilation errors, one often also gets information from the compiler about *where* in the code the error is and *why* it is wrong, facilitating the search for fault causes and correction of mistakes. But sometimes compiler error messages are misleading and point to completely wrong places in the code, so it's important not to blindly trust what the compiler says. Also, compiler error messages are often expressed in terms of the language's syntactic and semantic rules, and it takes time to learn to interpret compiler messages. Creating compilers that provide good error messages is a difficult problem studied within the computer science subfield *compiler technology*, which you can learn more about in advanced courses.

Different programming languages offer varying degrees of compile-time checking. A compiler for a language with an advanced type system, such as Scala, offers relatively extensive opportunities to identify errors already during compilation, whereas with languages with weaker type systems, such as JavaScript, one must rely on performance-hampering checks generated internally at machine code level or inserted manually in source code for safety.

- **Runtime errors**, also called execution errors (eng. *runtime error*), occur while the program runs. Specific input under specific execution conditions (a certain processor, certain memory size, certain network capacity, etc.) may be required for a runtime error to manifest. When a runtime error emerges, different things can happen:
  - **Execution produces unwanted results.** It is not guaranteed that a runtime error interrupts execution; often the error "merely" results in incorrect output or otherwise low quality. To detect this before deployment, it is common practice to write carefully thought-out **test cases** and analyze **test results** in detail by comparing output with expected results or accepted quality levels.
  - **Execution hangs** (eng. *hang*). Sometimes a fault manifests as nothing happening during execution, which can be due to e.g.:
    * an **infinite loop** that never finishes,
    * it being **very slow** because processing of input takes unreasonably long,
    * the program **waiting for input** that never arrives,
    * different concurrent parts of the program waiting on each other so that a **deadlock** (eng. *deadlock*) occurs.

    When execution hangs and you no longer want to wait for something to happen, you simply forcibly abort execution with an appropriate command provided by your execution environment.[3] In worst cases, you may have to cut the power.
  - **Execution crashes** (eng. *crash*). Sometimes execution stops suddenly and is aborted with a runtime error message. This can be caused by e.g.:
    * **Memory exhaustion**, either the parameter memory for function calls (eng. *stack memory*) is used up or the memory for objects allocated during program execution (eng. *heap memory*) is full,
    * the mistake of trying to reference a **null reference** that does not refer to any object and has the value `null`, resulting in a *null pointer exception*,

[3] `kill -9 <pid>`, Ctrl+C, Ctrl+Shift+C, Ctrl+Z, or others depending on environment.

  * a so-called **exception** has been ”thrown” (eng. *throw exception*) intentionally by the programmer to cause a crash if the exception is not ”caught” (eng. *catch*) and handled by surrounding code.

When the system crashes you get a list of the current chain of function calls in a **stack trace** (eng. *stack trace*). You can also request a dump of the entire memory contents at the crash (eng. *memory dump*), but these can be hard to interpret.

When the system produces unwanted results, hangs or crashes, you try to reproduce the runtime error in a rerun and, with the help of instrumentation or a debugger, attempt to figure out what happens just *before* the error occurs; see section D.5.
In the course *Software Testing* (eng. *Software Testing*), you learn more about systematic methods for testing systems so errors can be prevented, identified, and fixed.

Bug or feature?

When a (possible) error is discovered, it is appropriate to first ask some basic questions:

- Is this really an ”error” or is it actually intended behavior? It is not always obvious whether it is a bug or a deliberately created system property/feature (eng. *feature*).

- Could it be that the test case has flawed test code while the code under test actually works perfectly? Such problems can be especially hard to solve because one often looks for the cause in the wrong place.

- If the bug concerns a quality attribute of the system, one might ask: Where exactly is the line between ”error” and acceptable? Is this good enough considering the cost of improving quality? Quality requirements concern properties of a program that can be expressed on a sliding scale, where something can be more or less *good* or *bad* from different perspectives. Such requirements often lead to important but difficult trade-off decisions during design and implementation. Also, test results can be hard to assess and there can be differing opinions on whether an observed error is a bug.

  Here are some examples of quality requirements:

  - **Performance requirements** (eng. *performance requirements*) refer to how fast and efficiently the program should work under various conditions.
  - **Capacity requirements** (eng. *capacity requirements*) relate to how much data the system should be able to handle under various conditions.
  - **Usability requirements**[4] (eng. *usability requirements*) concern how easy the system should be to use for a given user category.

In the course *Requirements Engineering* (eng. *Software Requirements Engineering*) you learn more about identifying, specifying, and tracking quality requirements.

[4] sv.wikipedia.org/wiki/Usability

Issue tracking tools

It is common practice in industrial system development to use an issue tracking tool (eng. *issue tracker*) so collaborating developers get support in keeping track of all encountered errors and problems (eng. *issues*). Many popular code hosting sites on the web, such as GitLab, GitHub and BitBucket (see section G.3), offer issue tracking functions. These can include:

- management and compilation of all different issue states, so that one can see which issues are in *Open* or *Closed* states,
- assignment of issues to specific people responsible for fixing the problem,
- grading issues by severity levels,
- notification generation to involved people when an issue is commented on or changes state.

D.2 Preventing errors

Although it is almost inevitable for bugs to slip into code as it becomes more and more complex, it is still important to put great effort into trying to avoid this. It is often much better invested time to work on bug prevention measures while you write code, than to chase bugs that could have been prevented with general carefulness and stricter discipline in coding. Below are summarized some measures that can help reduce the amount of errors.

- **Write comprehensible code**. The basis for avoiding bugs is to strive to write code that is easy to read. This is a constant struggle; code complexity grows with each addition and occasional refactoring is needed to maintain a good structure that facilitates comprehensibility and makes extensions easier.
- **Choose good names**. An important piece in creating understandable code is to come up with good names. This can be surprisingly difficult and may require a lot of discussion and thought. If you realize a name is poorly chosen, it is probably worth the work to refactor and introduce a better name, especially if others have not yet become too accustomed to the concept.
- **Check parameters and variables**. Often you know constraints that must hold for variable values. For example, you often know a particular integer function parameter must not be negative. You can ensure this by inserting checks that the condition holds. For parameter checks, in Scala you commonly invoke `require`, for example: `require(x >= 0, "x must be positive")`. There is also a method `assert` that works similarly[5]; while `require` is used for validating parameters, `assert` is used to verify general conditions that should hold, e.g. `assert(x + y > n, "overflow")`. The advantage of inserting condition checks is that violations are discovered immediately, making debugging easier.
- **Use type checking**. With *type annotations* the compiler helps verify your assumptions about the types of various values. In Scala you can place a colon and a type almost anywhere in an expression to request the compiler check

[5] stackoverflow.com/questions/26140757/what-to-choose-between-require-and-assert-in-scala

the type. For example, `(xs + f(42)) : Set[Int]` enforces that the expression `xs + f(42)` really yields a set of integers. Although you seldom need explicit types in Scala thanks to type inference, specifying expected types in appropriate places, especially in complicated expressions, long chains of method calls, or when method return types are not obvious, contributes to readability and safer code. Also, the compiler may sometimes avoid confusion in particularly difficult type inference situations if you assist it with explicit type annotations.

- **Handle missing values**. It is very common to need to handle situations where a value is missing, cannot be computed, or is not available for other reasons. One can keep track of missing values by representing these with special values, e.g. `-1` or `null`. But this strategy very easily leads to bugs, as it is easy to forget to check these special values elsewhere. Such special values also give no compiler assistance to detect missed handling. Instead, if you use `Option` (see chapter 10) to handle potentially missing values, you get compile-time assistance and avoid runtime errors and difficult debugging. Additionally, it becomes very clear to anyone reading your code, including yourself, that a value may be absent.

- **Handle exceptions**. When exceptions occur, e.g. a file cannot be read or division by zero happens, execution is aborted and the program's users can no longer use it, which in worst cases can have disastrous consequences. Therefore, you want to handle exceptional situations so the program becomes robust and doesn't crash. One effective way is to wrap the exception in a `Try` type value, see chapter 10. Like `Option` for missing values, `Try` makes it clear in code when a result can represent a success (`Success`) or failure (`Failure`) with an encapsulated prevented crash.

- **Review code**. It is common practice in industrial software development to conduct code reviews, where a group of people carefully study someone else's code, provide comments, and identify potential problems. Often there is a checklist used during reading, including points such as readability, naming, parameter checks, handling of missing values and exceptions, etc. Many organizations have agreed coding standards with guidelines for style and appearance to be followed unless special reasons exist. Compliance with style guidelines can be checked via reviews. There are also tool support for such checks. An example of coding guidelines for Scala is available on the official documentation website[6].

- **Test code**. It is common practice in industrial software development to conduct tests on multiple levels. Unit tests (eng. *unit tests*) of individual code parts are often combined with **feature tests** (eng. *feature tests*) to see that inputs in intended use situations yield expected results, and **system tests** (eng. *system tests*) to ensure all parts work together under realistic circumstances.

- **Learn from user experiences**. When code is deployed to production, there is the opportunity to learn through feedback from users. How the system is used and how users experience it is very important information when deciding how best to continue developing the code. From users you can learn about unknown bugs and get brilliant ideas for valuable new features. A software development organization's innovation capability largely depends on its ability to continuously deliver code with more and more features users like without too many annoying or catastrophic bugs.

[6] https://docs.scala-lang.org/style/

D.3 What is debugging?

When a failure manifestation has been identified, e.g. through testing or users reporting problems, the search begins for the underlying fault cause, so we understand *why* the error occurred and then can *fix* the mistake. This process is called **debugging** (eng. *debugging*).

D.3.1 How is the fault found?

The first step in the debugging process is finding the underlying fault cause. This can be very difficult, especially if the system is large and complicated.
If you stare blind at the code without finding the fault cause, it may be because you have a wrong hypothesis about what the code actually does. You are convinced that a certain thing happens, but *actually* it is *not* what you *think* is happening that *really* happens. For example, you may assume a counter is incremented inside a loop, but in reality the increment is missing. If you uncritically accept your incorrect assumption, you risk looking in the wrong place in the code.
The following actions are often appropriate when hunting bugs:

- **Reproduce the bug with a minimal test case**. When you have discovered a failure manifestation it is important to reproduce the fault, so that you can debug the code that runs exactly *before* the bug appears. Best is if you can create a **minimal test case** where precisely the minimal input and individual conditions under which the bug occurs are documented. The description of the minimal test case is the first piece in the detective work that follows during debugging.

- **Formulate and verify hypotheses about the bug**. A fundamental principle during debugging is to explicitly formulate hypotheses you have about what happens in the system while the bug occurs and then *verify* that these really hold, through different investigations of the executing system. You should clearly describe how you think the code works and then with various forms of instrumentation, e.g. printing values of variables to the terminal, check whether this is indeed the case. This can be done by instrumentation as below.

- **Instrumentation with prints, ”print-debugging”**.

  To verify your hypothesis about what leads to the bug, you need to check what is happening. You can do this by placing `println` outputs at well-chosen locations in the code where values of interesting variables are printed. It may require some ingenuity to find exactly the right print statements; if you print everything happening in all loops you get drowned in information, but if you print too little you might overlook a refuted hypothesis and get no help solving the bug puzzle.

  You can also use a debugger (eng. *debugger*), which usually is included in an integrated development environment, to instrument your code. See more in section D.5 on how to use debuggers in Eclipse and IntelliJ IDEA.

D.4 Fixing errors

Often the hardest part is to *find* the bug, while the actual bug fix turns out trivial. For example, once you have found the missing increment of your loop variable, it is obvious what you need to do.

But sometimes it is really tricky to fix the fault. Below are some situations that can arise, making the bug fix especially difficult.

- Maybe the algorithm itself is fundamentally flawed and a completely new algorithm needs to be constructed. Creating new algorithms from scratch can be very difficult in some cases. In advanced courses you will learn more about the noble art of algorithm design.
- Maybe the algorithm works for normal cases, but unusual exceptional cases are not handled correctly. Handling all possible cases properly can be very tricky. Unfortunately, it is often exceptional cases combined with bugs that open security holes for malicious hackers to exploit to crash the system or infect it with viruses.
- Maybe the problem itself is very difficult to solve correctly. The algorithm can be really complicated with many conditions, loops, and nested data structures. If something goes wrong in such an algorithm it can take a long time to get changes to work and ensure that all conditions, loops, and nested data structures fit together again after the fix.
- While fixing a bug you can accidentally introduce new bugs. The risk is especially great if the code is complex. Sometimes one even avoids fixing errors if the system still works somewhat acceptably in other respects and the risk of creating new bugs is too high. Then the system needs to be restructured to become easier to modify.
- Maybe execution time grows exponentially with data size. Then it can be practically impossible to write a program that always finishes within reasonable time. You have to devise clever shortcuts to suboptimal solutions that are still acceptable, which sometimes requires very advanced programming techniques.

There is no one-size-fits-all quick fix when confronting hard errors. Becoming a productive system developer who successfully addresses all sorts of bugs largely involves combining broad general knowledge in computer science with lifelong learning, where each bug you find and fix gives you new knowledge and experience for the future. Even if your bug is irritating, try to see it as a new chance to learn!

D.5 Using a debugger

A professional integrated development environment often comes with an advanced debugger that can help you follow execution and see what happens while code runs. Normally these features are included in a debugger:

- **Setting breakpoints**. With the debugger you can set so-called *breakpoints* at specific places in the code. This is often done by clicking on a code line margin where a breakpoint symbol is placed. When execution reaches the breakpoint, execution is paused and you can step forward or inspect the values of variables at the breakpoint.
- **Stepped execution**. Once you hit a breakpoint, you can step through the code line by line with the debugger and see what happens. If you reach a function call, you can choose to step into the implementing code or simply run the function

and step to the next line after the function call. It can take many reruns from a given breakpoint before you find which functions are irrelevant for the bug and can be skipped, or which functions constitute the puzzle solution and you want to step into and investigate further. Stepping deeply into function call chains can cause you to get lost and have to restart. (You cannot step backwards...)

- **Inspecting variables**. While stepping through the code you can inspect variable values. The debugger usually presents simple values such as integers and strings, but also data structures like vectors and lists can be inspected by browsing arrays and object references. A program can have many variables and deep structures of objects referencing other objects. It is often tricky detective work to figure out how variable values relate to the cause of the bug you are searching for. You often need to alternate between reading code, stepping through, setting new breakpoints, and inspecting variables to understand what is happening.

In Kojo Desktop (see appendix A) there are user-friendly debug functions. For example, you can follow the steps during execution with the bright yellow play button ”Run and trace the program” (shortcut: Alt+Enter). This opens a new window showing execution steps. You can click a step and get information about parameters at function calls, etc.

Appendix E

Documentation

Documentation helps others to use your code, but also facilitates for yourself when you later need to recall how it works and how to use and build upon your code. Modern system development is often based on open source and ready-made APIs (eng. *application programming interface*), where the quality of the documentation is crucial for how easy it is to get started using the code.
Below are examples of different types of documentation[1]:

- **Requirements documentation** describes the overall goal of the software, as well as functional requirements and quality requirements fulfilled by the system.
- **Design documentation** describes the architecture, how the code is organized in modules, and the internal system structure, e.g., in the form of classes, objects, and their relationships.
- **End-user documentation** can for example be manuals for using the system and installation instructions.
- **Technical documentation** can for example be API documentation that describes which functions are included in a program library. Such documentation is often generated with the help of a **documentation tool** (see section E.1). Other types of technical documentation are instructions on how to build the code, possibly with associated build tool configuration files; often the build procedure is described step-by-step in a text file named `README`. (Read more about build tools in appendix F.)

It is a major challenge to keep documentation updated as the code evolves. Even if you get help generating a navigable site from a documentation tool, the *content* of the manually authored documentation comments must be consistent with the current version of the code. If the code is updated, you must be careful to update the documentation comments as well, otherwise great confusion arises.
This problem is so serious that one should carefully consider how to formulate documentation comments in a future-proof way, and how extensive they should be relative to the future effort required to keep them updated. The more extensive the comments, the more work it takes to keep them updated.
In practice, it is difficult to achieve an optimal balance between good and many comments that *help* the user, and on the other hand poorly maintained and outdated comments that *hinder* users.

[1] en.wikipedia.org/wiki/Software_documentation

E.1 What does a documentation tool do?

A documentation tool generates technical documentation of the code based on special **documentation comments** written in the code immediately before declarations of what is to be documented. These documentation comments are written in a special syntax that the documentation tool can interpret.

The output from a documentation tool typically consists of a website with appropriate formatting and navigation links, see figure E.1.

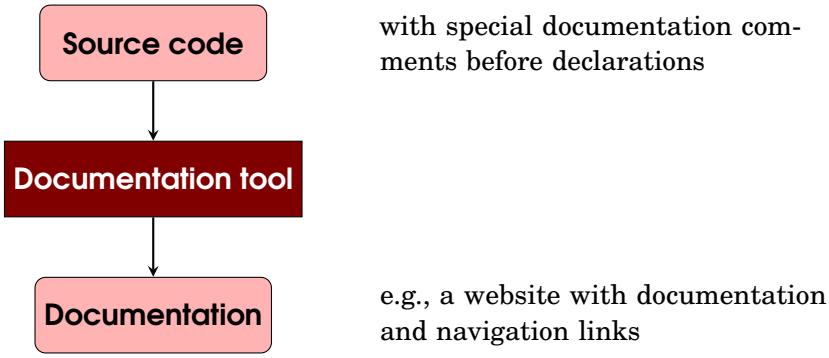


Figure E.1: A documentation tool reads the code and documentation comments and generates documentation, e.g., in the form of a website.

E.2 scaladoc

The Scala installation includes the documentation tool `scaladoc`, which generates a website with appropriate layout and special features to search, filter, and navigate the documentation.

The documentation of large libraries can become extensive and it requires practice in using documentation sites to get the most out of them. The following sections first describe how you use documentation generated with `scaladoc`. Then it shows how you yourself can generate documentation for your own code.

E.2.1 Using documentation from scaladoc

The documentation of Scala's standard library is generated with `scaladoc` and navigating this provides good practice in how to use advanced API documentation. You can find the documentation for Scala's standard library here:
`http://scala-lang.org/api/current`

When you browse there you are met by the documentation for the *root package*, which gives an overview of different packages in the standard library. In the search box at the top left you can type the beginning of the name of classes, traits, or objects you are looking for, as shown in figure E.2.

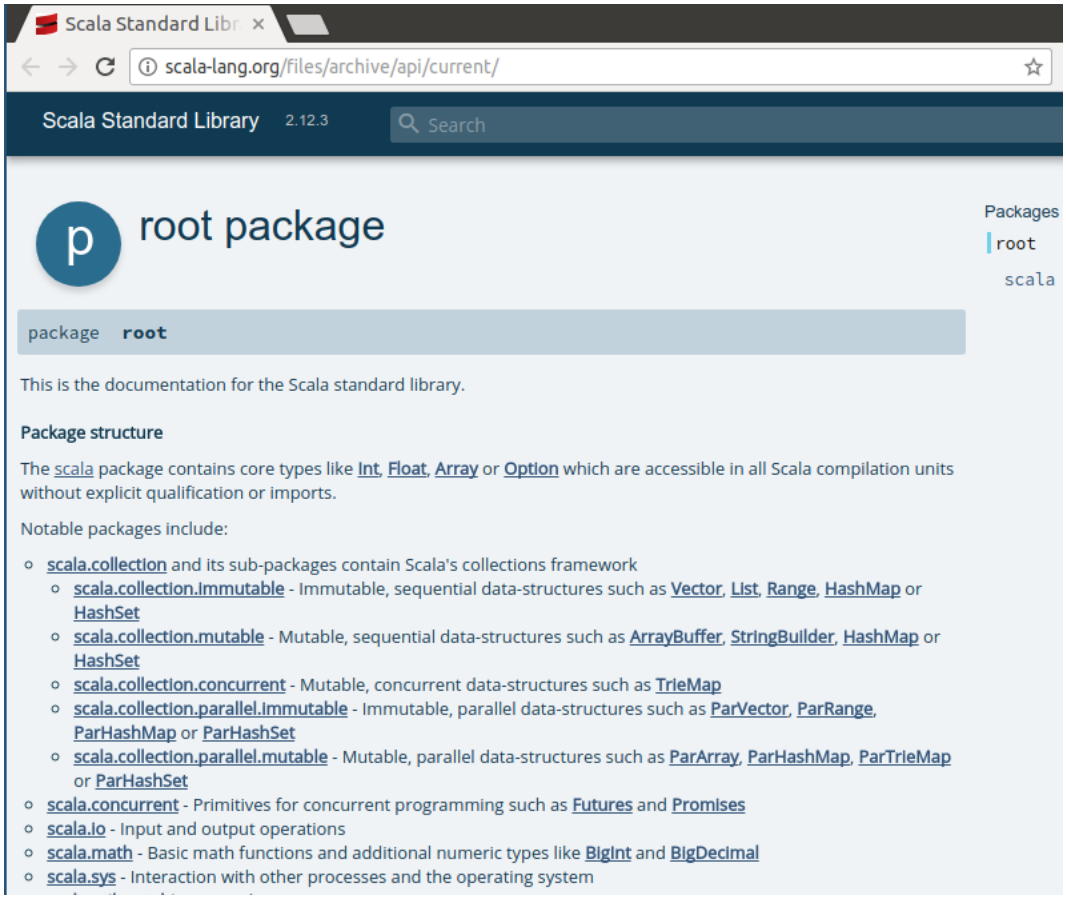


Figure E.2: Front page with the documentation of the Scala standard library.

By typing text in the search box you get a filtered list of everything with a name starting with what you search for. In figure E.3 a search for `vec` is shown.

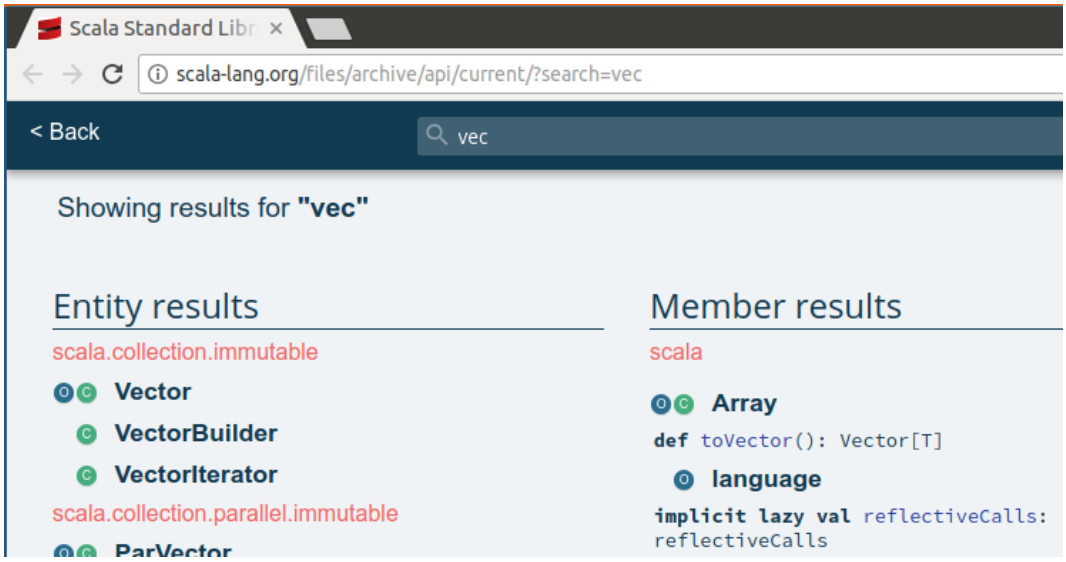


Figure E.3: Search for content starting with `vec`.

If you click on `Vector` you will see the documentation in figure E.4

Scala Standard Libr ×

scala-lang.org/files/archive/api/current/scala/collection/immutable/Vector.html

Scala Standard Library 2.12.3 Search

scala.collection.immutable

Vector

Companion object Vector

```
final class Vector[+A] extends AbstractSeq[A] with IndexedSeq[A] with GenericTraversableTemplate[A, Vector] with IndexedSeqLike[A, Vector[A]] with VectorPointer[A] with Serializable with CustomParallelizable[A, ParVector[A]]
```

Vector is a general-purpose, immutable data structure. It provides random access and updates in effectively constant time, as well as very fast append and prepend. Because vectors strike a good balance between fast random selections and fast random functional updates, they are currently the default implementation of immutable indexed sequences. It is backed by a little endian bit-mapped vector trie with a branching factor of 32. Locality is very good, but not contiguous, which is good for very large sequences.

Note: Despite being an immutable collection, the implementation uses mutable state internally during construction. These state changes are invisible in single-threaded code but can lead to race conditions in some multi-threaded scenarios. The state of a new collection instance may not have been "published" (in the sense of the Java Memory Model specification), so that an unsynchronized non-volatile read from another thread may observe the object in an invalid state (see scala/bug#7838 for details). Note that such a read is not guaranteed to *ever* see the written object at all, and should therefore not be used, regardless of this issue. The easiest workaround is to exchange values between threads through a volatile var.

A the element type

Self Type Vector[A]
Annotations @SerialVersionUID()
Source Vector.scala
See also "Scala's Collection Library overview" section on **Vectors** for more information.

Linear Supertypes

dis

Value Members

```
def distinct: Vector[A]
```
Builds a new sequence from this sequence without any duplicate elements.

```
def foreach(f: (A) → Unit): Unit
```
[use case] Applies a function **f** to all elements of this vector.

```
def groupBy[K](f: (A) → K): Map[K, Vector[A]]
```

Figure E.4: A part of the documentation of the class `Vector`

By typing text in the lower, dark gray search box you can filter the list of class members. If you click on the link `object Vector`, or on the round, green icon with a large C, you will see the companion object's members. You can display a filtering box with more options by clicking the expand arrow in the dark gray search box as shown in figure E.5.

Figure E.5: Expanded search box with additional filtering options.

E.2.2 Writing documentation comments

The tool `scaladoc` reads comments that start with `/**` and end with `*/` and associates them to the following declaration. Note the double asterisks. All lines following `/**` should, according to the convention for Scala documentation comments, start with an asterisk `*` indented with several spaces so that it aligns under the *second* asterisk in the opening comment, as below:

```
/** First comes a summary on a single line.
  *
  * Then possibly a more detailed description,
  * which can span multiple lines.
  */
```

The documentation comment ends with `*/` right under the asterisk column.

- With `@constructor` at the beginning of a line a special comment about constructors is given.
- With `@param` at the beginning of a line a special comment regarding parameters is given.
- With `@return` at the beginning of a line a special comment about what is returned by a method call is given.
- Links are written within double square brackets, as in the example below.

```
/** A person who uses our application.
 *
 *  @constructor create a new person with a name and age.
 *  @param name the person's name
 *  @param age the person's age in years
 */
class Person(name: String, age: Int)

/** Factory for [[mypackage.Person]] instances. */
object Person:
  /** Creates a person with a given name and age.
   *
   *  @param name their name
   *  @param age the age of the person to create
   */
  def apply(name: String, age: Int) = ???

  /** Creates a person with a given name and birthdate
   *
   *  @param name their name
   *  @param birthDate the person's birthdate
   *  @return a new Person instance with the age determined by the
   *          birthdate and current date.
   */
  def apply(name: String, birthDate: java.util.Date) = ???
```

Read more here on how to write documentation comments:
https://docs.scala-lang.org/scala3/guides/scaladoc/
Read more about documentation here:
https://docs.scala-lang.org/scala3/scaladoc.html
https://scala-cli.virtuslab.org/docs/commands/doc

E.2.3 Generating documentation

You generate documentation most easily with the command line tool `scala-cli`. In the terminal write
`scala doc . -o api`
When `scala-cli` finishes generating the documentation, it reports which directory the documentation is in. For the links within the documentation to work, either you run a local web server in the directory or you use a program to convert the links to local ones.

Using a local web server

If you have Python 3 installed on your computer, you have an included web server. You start this in the terminal with `python -m http.server` when you are in the documentation directory. To open the documentation, then visit http://localhost:8000 in your browser.

Using a program to convert links

If you do not have Python installed, you can run a Scala program that replaces all links with local ones, enabling you to open them directly. This can be downloaded from https://github.com/dixine55/Scaladoc-Local-Version-Patcher/blob/main/scaladocPatch.scala.
Place the program in the documentation folder and run it with `scala run .` when you are in the documentation folder. You then open the documentation via the file `index.html` in your browser.

More to read about generating documentation is here:

Appendix F

Build Tools

F.1 What Does a Build Tool Do?

A **build tool** (eng. *build tool*) is used to

- download,
- compile,
- run tests,
- package, and
- distribute

software. A large development project can contain many hundreds of source files, and during development, you want to continuously run tests to check that everything still works; also the code that has not been changed but may still be affected by the change. A build tool is used to *automate* this process.
An important concept in build contexts is **dependency** (eng. *dependency*). If code X needs other code Y to function, code X is said to have a dependency on code Y.
In configuration files, which are written in a format the build tool can read, the dependencies between different code parts are specified. The build tool analyzes these dependencies and, based on file modification timestamps for source files, determines which subset of the source files needs to be **recompiled** after a change. This significantly speeds up compilation compared to a full recompilation from scratch, which for a large project can take many minutes or even hours. After recompiling the changed parts, the build tool can be instructed to run test programs and report test results, as well as upload executable program packages to, e.g., a web server.
A common type of dependency is precompiled program libraries used by the system under development, which in practice often means that a path to the compiled code for the library needs to be made available. In JVM contexts, this means the path to all necessary jar files must be on the path list called **classpath**.
Many build tools can perform so-called **dependency resolution** (eng. *dependency resolution*), meaning that the network of dependencies is analyzed and the correct set of program packages is made available during the build. This can also mean that program packages available via the internet are automatically downloaded before the build, e.g., via repositories for open source.
Even if you only have a small code project with a few source files, it is still convenient to use a build tool. You can have the build tool active in the background and, as soon as you save a code change, trigger recompilation and report any compilation errors.

It is wise to recompile often, preferably at every small change, and fix any errors *before* making new changes, since it is much easier to figure out a single problem after a small change than to fix a bunch of difficult follow-up errors caused by a sequence of extensive changes where the mistake was made sometime far earlier.

An integrated development environment, such as VS Code or IntelliJ IDEA, rebuilds the code continuously and can often be configured to communicate with several different build tools. For example, with VS Code you can choose whether you want Scala CLI or `sbt` to be used to build your project.

There are many different build tools. Some generally well-known build tools are listed below so you can recognize which build tool is used in open-source projects you encounter, e.g., on GitHub.

- `Scala CLI`. The tool Scala CLI (Command Line Interface) is open source developed by VirtusLab[1] to compile and run Scala and Java programs and also contains basic build tool functions, such as running test cases, packaging jar files, and creating documentation. The command `scala-cli` replaced Scala 3.5 as the official `scala` command in 2024. This is the simplest and recommended way to build systems with Scala code. Basic usage of Scala CLI is described in Appendix C.4.3, while a more detailed description is found below in section F.2.

- `sbt`. Also called *Scala Build Tool*. Used to build Java and Scala programs in coexistence, but also to automate many other tasks. `sbt` is developed in Scala and the configuration files, called `build.sbt`, contain Scala code that controls the build process. `sbt` is advanced and can build systems consisting of many projects (eng. *multi-project build*). `sbt` is by far the most common build tool for Scala, and many open source projects use `sbt`. Read more about `sbt` in section F.3 below.

  After criticism that `sbt` is complicated, several alternative build tools for Scala have been developed, including `bleep`[2] and `mill`[3].

- Apache Maven, `mvn`, is also written in Java and is a successor to `ant`. Maven is used by many Java developers. Configuration files are called `pom.xml` and contain a so-called project object model specified in XML according to special rules.

- `gradle` builds upon ideas from `ant` and `maven` and is written in Java and Groovy. Configuration files are written in Groovy and are named `build.gradle`.

- Apache `ant`. This build tool is developed in Java as an alternative to `make` and is still used in many Java projects, although Maven and Gradle are more common nowadays. Configuration files are called `build.xml` and are written in the standardized language XML according to specific rules.

- `make`. This venerable build tool has been around since the 1970s and is still used to build many systems under Linux, and is popular when developing with the programming languages C and C++. A configuration file for `make` is called `Makefile` and has its own, special syntax.

[1] https://scala-cli.virtuslab.org/
[2] https://bleep.build/docs/
[3] https://mill-build.com/mill/Intro_to_Mill.html

F.2 Scala Command Line Interface `scala-cli`

Development of Scala CLI[4] started in 2022 and became the official build and run tool accompanying the official Scala installation via https://www.scala-lang.org in summer 2024. Scala CLI can also be installed separately from https://scala-cli.virtuslab.org and run with the command `scala-cli`. When Scala 3.5 was released, the command `scala` became equivalent to `scala-cli`.
After a fresh installation of Scala CLI, you can enter the following command once and for all to get access to option completions with the Tab key in the terminal:

```
scala install-completions
```

Before you start writing source code in a new directory in VS Code, you can prepare VS Code to use Scala CLI as the build tool in the current directory with the following command (wait to open VS Code until after you have run the command):

```
mkdir myNewDirectory
cd myNewDirectory
scala setup-ide .
code .
```

F.2.1 Example Usage of Scala CLI

Below are the most important Scala CLI commands to incrementally build your code with many small steps and experimenting with partial solutions.

- With the command `scala compile . -w` in a separate terminal window alongside your editor, you start Scala CLI in so-called *watch mode*. Then all file changes are monitored and recompilation happens immediately when a new change is saved, and you can see any compilation errors right away. Preferably fix one compilation error before building more on your code, as follow-up errors can be hard to solve especially if there are many. Divide large code changes into small steps and try to get the latest change to compile without errors as soon as possible.

- With the command `scala repl .` in another terminal window, you start a REPL with your classes in the current directory automatically available on the classpath (hence the dot after the space after `scala repl`) and you can call your methods, after any **`import`**, when experimenting before the next step.

  This way you can create and test small functions and get them to work before you integrate them into your program and combine them with already created functions. It is often easier to debug and build a larger program if you have many small functions interacting, rather than a few very large functions. And it is usually easier to test run new solution ideas in REPL before you create "finished" code in your program.

- With `scala run .`, compilation and execution of the `main` method in the current directory takes place. You can specify another directory by passing the path as an argument to the command. You can also specify the option `-w` for *watch mode* when running. Then your program will run again on every change. Watch mode

[4] CLI stands for *command line interface*. Read more about Scala CLI here: https://scala-cli.virtuslab.org/docs/overview

during execution is useful if the program produces results without waiting for input, but less convenient if each run requires user input or windows must be closed.

- If there are multiple `main` methods in the current directory, you can specify which one to execute with the option `--main-class`, for example
  `scala run .  --main-class hello`

- Arguments to your main program are specified after double dashes `--` as follows:
  `scala run . -- arg1 arg2 arg3`

F.2.2 Basic Build Functions in Scala CLI

The basic functions are summarized below (also see Appendix C.4.3):

`scala repl`	Start Scala REPL. It also works with just `scala`
`scala repl hello.scala`	Start Scala REPL with compiled code in `hello.scala` on the classpath.
`scala repl .`	Start REPL with source files in the current directory available on the classpath.
`scala compile hello.scala`	Compile code in the file `hello.scala`
`scala compile .`	Compile all source files in the current directory.
`scala run hello.scala`	Compile the code in the file `hello.scala` and start the main program if compilation succeeded.
`scala run .`	Compile and run all code files in the current directory.
`scala run .  --list-main-class`	List all main programs.
`scala run .  -M mypkg.myMain`	Run a specific main program. Shortcut `-M` can also be written as `--main-class`
`scala package .`	Package all compiled code files into an executable file.

F.2.3 Using Options to Control Scala CLI

There are many different options that you can add to control what Scala CLI should do. See example below and explanations in the following table:

```
scala run . -S 3.7.2 -O -unchecked --dep se.lth.cs::introprog::1.4.0 -w
```

Here are explanations of some common options that can be used during both compilation and execution:

`--scala 3.7.2`	Use Scala version 3.7.2. The option `--scala` can be shortened to `-S`
`--watch`	Repeat the command on saved change. The option `--watch` can be shortened to `-w`
`--jar introprog.jar`	Add a jar file to the classpath.
`--dep se.lth.cs::introprog::1.4.0`	Add a dependency to the classpath.
`--scalac-option -unchecked`	Add a compiler option that provides extra warnings for unsafe code. The option `--scalac-option` can be shortened to `-O`

The advantage of explicitly specifying a certain Scala version is that the build process becomes *repeatable* even on another computer that might accidentally have a different Scala version installed. It is possible to "pin" the Scala version to a more precise version, e.g., `3.3.1`. If that version of the compiler is not installed on the computer, Scala CLI will automatically download and use the explicitly requested version during the build.
You can also request the absolute latest compiler version if you want to use the very latest in Scala language development with `3.nightly`. Especially experimental features require that you use the `nightly` version[5].

F.2.4 Generate Documentation with Scala CLI

Scala CLI can also create documentation based on documentation comments (see also Appendix E), as follows. With the option `--output` you can specify the destination directory and with `--force` any old documentation is overwritten.

```
scala doc . --output apidoc --force
```

F.2.5 Packaging an Executable File with Scala CLI

Scala CLI can package your code into an executable file as follows:

```
scala package . --force --standalone --output myapp
```

Here are explanations of some common options used during packaging:

`--output`	Specify name of output file with packaged code. The option `--output` can be shortened to `-o`
`--force`	Overwrite output file if it already exists. The option `--force` can be shortened to `-f`
`--standalone`	Create a standalone, executable jar file with your code and its dependencies.
`--library`	Create a jar file with your code for use by other programs.
`--assembly`	Create a fat jar file with your code and all its dependencies for use by other programs.

Some options, e.g., `--assembly` require so-called power mode. If you have not enabled power mode, you will get a warning telling you how to do it: you can either enable

[5] https://stackoverflow.com/questions/40622878/how-do-i-use-a-nightly-build-of-scala

power mode temporarily by also including the initial option `--power`, or enable it permanently with a setting like this:

```
scala config power true
```

F.2.6 Options as Usage Directives in Magic Comments

Instead of using options in the terminal, you can specify these as so-called usage directives (eng. *using directives*) in "magic" comments starting with `//> using` at the beginning of any code file.
If you have multiple source files in the same directory, it is common to create a special file usually called `project.scala` and gather all usage directives there to control the build. Here is an example of what it might look like:

```
//> using scala 3.7.2
//> using option -unchecked -deprecation
//> using option -Wunused:all -Wvalue-discard -Wsafe-init
//> using dep se.lth.cs::introprog::1.4.0
```

The compiler options suggested above for extra warnings have the following meanings:

`-unchecked`	Extra warnings on several unsafe code cases.
`-deprecation`	Explanation when using deprecated functions.
`-Wunused:all`	Warning on unused declarations.
`-Wvalue-discard`	Warning on discarded value.
`-Wsafe-init`	Warn on risk of uninitialized values.

You can find more information about Scala CLI here: `https://scala-cli.virtuslab.org/`

F.3 Scala Build Tool `sbt`

The build tool `sbt` is written in Scala and is the most used build tool among Scala developers. With `sbt` you can write build configuration files in Scala and also control the build process via an interactive command shell in the terminal window. With incremental compilation and parallel execution of different parts of the build process, it can be significantly accelerated.

F.3.1 Installing sbt

`sbt` is preinstalled on LTH's computers and is started with the command `sbt` in the terminal.
If you want to install `sbt` on your own computer, first make sure you have `java` on your computer with the terminal command `java -version`. If `java` is missing, follow the instructions in section C.3.2 on page 213. Then follow the instructions here to install `sbt`: `http://www.scala-sbt.org/download.html`

- **Linux**. If you browse to the above page from a Linux computer, some terminal commands appear that you use to install `sbt` in the terminal.

- **Windows**. If you browse to the above page from a Windows computer, a link to a `.msi` file is shown. Download and double-click on it. Before you start using sbt in a Windows terminal, it is good to type `chcp 65001` so special characters (e.g., ÅÄÖ) will work properly.

- **macOS**. Follow the instructions under the heading *Manual Installation*.

When you run sbt the first time, additional files will be downloaded and installed, and parts of this process can take a long time. Be patient and do not abort the run, even if nothing special seems to be happening for a while.

F.3.2 Using sbt

`sbt` is designed to handle very large projects, but it is easy to use `sbt` even if you only have a small project with a few source files. With `sbt` installed, just start `sbt` and type `run` like this:

```
> sbt
sbt> run
```

in the terminal in the directory where your source files are located. `sbt` then finds and compiles all the `.scala` files in the directory and, if there is only one object with a main method, `sbt` runs this main method directly, provided the compilation completes without errors. `.java` files are also compiled automatically if they are in the same directory.
If you just type `sbt`, the interactive command shell will start, where you can run commands such as `compile` and `run`. If you type a ~ before the command `run`, as below, `sbt` will be active in the background while you edit, and as soon as you save a change, the changed source files will be recompiled, and the main method will be executed if compilation succeeds.

```
> sbt
[info] Set current project to hello (in build file:/home/bjornr/hello/)
> ~run
[info] Running hello
Hello, World!
[success] Total time: 0 s, completed Aug 9, 2016 9:50:16 PM
1. Waiting for source changes... (press enter to interrupt)
[info] Compiling 1 Scala source to /home/bjornr/hello/target/scala-2.10/classes
[info] Running hello
Hello again, World!
[success] Total time: 1 s, completed Aug 9, 2016 9:50:45 PM
2. Waiting for source changes... (press enter to interrupt)
```

In the above run, `sbt` performs a recompilation after a change to the print string was saved.

```
// in file hello.scala

@main def run = println("Hello again, World!") // add 'again'; Ctrl+S
```

Directory Structure

If you have code in subdirectories, `sbt` assumes that you follow a certain, specific directory structure. This directory structure is also used by other build tools, such as Maven, and also works in many IDEs like Eclipse and IntelliJ.

It also becomes less messy and easier for everyone to find things in the project's directories if your source files are placed according to a commonly accepted structure. Therefore, preferably place your source files in subdirectories as shown in figure F.1.

```
src/
  main/
    resources/
       <files to include in main jar here>
    scala/
       <main Scala sources>
    java/
       <main Java sources>
  test/
    resources
       <files to include in test jar here>
    scala/
       <test Scala sources>
    java/
       <test Java sources>
```

Figure F.1: Directory structure in an `sbt`-project. Only directories that have contents need to exist.

According to this structure, place your `.scala` files in the subdirectory `src/main/scala/` and your `.java` files in the subdirectory `src/main/java/`. If you place code in the directories `src/test/scala/` or `src/test/java/` the code will be run when you type the `sbt` command `test`. If you place files in the subdirectory `src/main/resources/`, these files will be packaged into the jar file created when you run the `sbt` command `package`.

If you use, e.g., `package x.y.z;` in your Java code, the subdirectory structure must also match, and the code file must be located in `src/main/java/x/y/z/`.

In Scala it is actually not necessary that the code is located in the same directory as the compiled `.class` files, but it can be good practice to follow the package structure also for Scala source code; especially if you later want to run your code with Eclipse, which requires this consistency between package and source code directories, not only for Java but also for Scala.

Configure Your Builds in the File `build.sbt`

If you want to make settings and also help others to be able to reproduce your builds, create a configuration file named `build.sbt` and place it in the project's root directory. Figure F.2 shows a build configuration file that specifies which version of the Scala compiler you are using, so others can build your code under the same conditions as you.

```
scalaVersion := "3.7.2"
```

Figure F.2: Example of configuration file for `sbt`. The file should be named `build.sbt` and be placed in the project's root directory.

Here is an example of a more extensive `build.sbt`:

```
scalaVersion   := "3.7.2"
scalacOptions  := Seq("-unchecked", "-deprecation") // more info during compilation

fork           := true   // run in a separate JVM, good if sound and graphics are used
connectInput   := true   // connect input to the correct JVM during fork
outputStrategy := Some(StdoutOutput)  // connect output to the correct JVM during fork

ThisBuild / useSuperShell := false // disables noisy progress info
```

You can read more about all features of `sbt` and how to create more advanced build configuration files here: http://www.scala-sbt.org

Pin sbt Version in `project/build.properties`

If you create a directory `project` (if it does not already exist) you can pin the version of sbt in a file named `build.properties` by letting the file contain the following (note the dot and lack of quotes):

```
sbt.version=1.8.2
```

This way you avoid inconsistencies between an old `build.sbt` and future updates of sbt; the above setting guarantees that your build will always use this version of sbt, and others can build your code under the same conditions as you.

Adding the Course Library `introprog` as a Dependency

With the following text in `build.sbt`, you get automatic downloading and access to the course Scala library `introprog` with, among others, the class `PixelWindow` for graphical windows:

```
scalaVersion := "3.7.2"
libraryDependencies += "se.lth.cs" %% "introprog" % "1.4.0"
```

Change the version number if needed to the latest version. Note the double percent signs before the library name, which are used for Scala libraries that are cross-published for different Scala versions, e.g., 3, 2.12, and 2.13, which ensures that the correct library version for the correct compiler version is downloaded.
You can read more about `introprog` here:

- Code: https://github.com/lunduniversity/introprog-scalalib
- Documentation: https://fileadmin.cs.lth.se/pgk/api

Adding Other Dependencies

In the file `build.sbt` you can add many dependencies to various code libraries. There is the online repository *Maven Central*, a very extensive code database searchable here http://search.maven.org, with many useful open source projects. You can ask `sbt` to download the precompiled code for any of the projects on *Maven Central* and automatically add the jar file to the `classpath` so the code becomes directly available in your program.
For example, you can add the Java library `jline` which enables terminal input from the keyboard with many useful features, such as command history with the up-arrow, simply by adding this line to your `build.sbt` with the specific version you want (note single percent signs for Java libraries):

```
libraryDependencies += "org.jline" % "jline" % "3.26.3"
```

You can read more about `jline` here: https://jline.github.io/

Appendix G

Version Control and Code Hosting

G.1 What is Version Control?

Version control[1] (eng. *version control* or *revision control*) of software means keeping track of different versions of the code in a development project as the code changes. Version control is a sub-discipline within **configuration management** (eng. *software configuration management*) that encompasses everything in the process of identifying, deciding, implementing, and following up changes.
An important part of version control is to *store* different versions of the code as it develops, so that previous versions can be *recreated* if needed. Good tool support and a well-defined work process for version control, which everyone in the development project follows, enables several developers to *work in parallel* and to merge each other's additions and changes into the common codebase without chaos and confusion.
Good version control is absolutely crucial for the developers' productivity, especially for large projects with many developers working in parallel on a comprehensive codebase with many different internal and external components. But even a small project with a single developer can benefit from a version control tool and a disciplined procedure for naming versions, e.g., to be able to recreate earlier versions of the project's various code files when a change turns out less successful.
There are several different models for how code storage occurs:

- **local**; all developers work in the same local file system where all different versions are stored.
- **centralized**; a repository (abbr. repo), i.e., a database with the code, exists centrally on a server that everyone works with using a version control client.
- **distributed**; every developer has their own local repo and each developer initiates sharing of changes between different repos individually.

G.2 The Git Version Control Tool

There are many different version control tools[2] that use different models for code storage; local, centralized, distributed, or combinations thereof. Recently, the tool **Git**[3] has gained a strong position, especially in the open source world. Git was originally

[1] en.wikipedia.org/wiki/Version_control
[2] https://en.wikipedia.org/wiki/List_of_version_control_software
[3] https://en.wikipedia.org/wiki/Git_(software)

developed by Linus Torvalds to version control the Linux kernel, but has grown into a comprehensive open source project with wide distribution and many users and contributors.
Git is designed for **distributed** version control where everyone can work quickly and smoothly in their own local repo without having to wait for a client to synchronize the code with a central repo on a server over the network. Changes are shared between repos on request by individual developers.
Each new version of the code is stored as a delimited set of changes from the previous version, a so-called **commit**[4], and managed internally by Git in a local database in the directory `.git` located at the root of your project directory. Via various terminal commands, or through a client with a graphical user interface, your code can be transferred to and from the local code database, or shared with other repos over the network.
There is a well-written book called *"Pro Git"* that explains Git in depth and is freely available here: `https://git-scm.com/book/en/v2`. Read chapters 1 and 2 to get a solid foundation.
These terms are good to know by heart before you start working with Git:

- **repo** (*noun*: a repository) A code database with change history.
- **commit** (*noun*: a submission, *verb*: to submit). A delimited set of new changes is submitted to the local repo. The repo's change history consists of the sequence of all submissions.
- **push** (*noun*: a delivery, *verb*: to deliver, to push). One or more commits are pushed up to another repo.
- **pull** (*noun*: a fetch, *verb*: to fetch, to pull). One or more commits are pulled down from another repo.
- **merge** (*noun*: a merge, *verb*: to merge). One or more commits are combined into a new commit.
- **merge conflict** (*noun*: a merge conflict) Problems during merging; changes cannot be easily merged in an unambiguous way.
- **pull request** (abbr. PR, *noun*: a pull request, *verb*: to request a pull). Developer A asks another developer B to fetch one or more commits from A's repo and merge them with B's repo.

G.2.1 Installing git

Git is pre-installed on LTH's Linux computers. You can check if Git is already on your machine by typing `git help` in the terminal.
There are good instructions on how to install Git on your own machine here: `https://git-scm.com/book/en/v2/Getting-Started-Installing-Git`
VS Code has special support for git, and from within VS Code you can e.g. add, commit, push, and pull via the editor's graphical interface. Read more here: `https://code.visualstudio.com/docs/editor/versioncontrol`
There are also many other graphical user interfaces for git, e.g. GitHub Desktop (Windows/Mac) or GitKraken (Linux/Windows/Mac). See more examples here: `https://git-scm.com/downloads/guis`

[4] In Swedish, e.g., "inlämning" (submission) can be used, but the loan word commit is already established.

G.2.2 Customizing Git

Before you start using git, configure your name and email with the terminal commands below, replacing `Förnamn Efternamn` with your name and `mejladr@plats.se` with your email address. Your name and email will be stored in every commit you make so that it is possible to see who made a given change.

```
> git config --global user.name "Förnamn Efternamn"
> git config --global user.email mejladr@plats.se
```

Read more about how to make other settings here, e.g., how to specify which editor git launches when you write commit messages:
https://git-scm.com/book/en/v2/Getting-Started-First-Time-Git-Setup

G.2.3 Using git

Below are some common terminal commands in Git.

- Create a repo in a directory:

```
> cd myproject
> git init
```

- See which files have changed and not yet committed:

```
> git status
> git status -s
```

- See changes that have been made in files not yet committed:

```
> git diff
```

- See the commits in the change history:

```
> git log
> git log --oneline -5
```

- Add files to be included in the next commit and then commit; give the commit a good description explaining what it includes:

```
> git add *.scala
> git commit -m 'initial project version'
```

- Undo all staged additions for a commit (the changes remain and can be added again if you want):

```
> git reset
```

- You can discard the most recent, not yet committed, changes to the file `filename` by doing "undo" with the `git checkout` command on the file as below. Only do this if you are absolutely sure you want to undo your latest changes.
  WARNING! Your latest changes in the file will be lost forever; cannot be undone!

```
> git checkout filename
```

- You want to prevent version control of certain files, e.g., binary files like `.class` files and other generated files. You do this by creating a file named `.gitignore` and entering file extensions according to the syntax below, where `**/` means all directories and subdirectories and `*` can be any filename start. The symbol # precedes a comment line.

```
# this is my .gitignore

# Java / Scala
**/*.class

# Sbt
**/target
```

G.3 Code Hosting Sites on the Internet

Many developers use code hosting sites on the internet ("in the cloud") (eng. *code hosting*) to facilitate collaboration on code and to share open source. There are many different code hosting sites that can be used for free under certain conditions or for payment with associated extra services.

> **NOTE!** You *must not* store your solutions to the course's lab assignments in a public repo. If you want to use a code hosting site, you must ensure that your solutions remain in a private repo without others being able to access it.

Below are some common sites for open and closed code hosting, all Git-based:

- **GitHub**, `https://github.com`, is one of the most popular code hosting sites for open source, but has also become a popular place for job-seeking developers to showcase their code work samples for future employers. GitHub is free to use for you as a private individual. Many companies pay GitHub to store their closed code with additional services to test, build, and deploy code, etc. The code governing the GitHub code hosting site itself is closed source, unlike GitLab. GitHub was bought in 2018 by Microsoft for 65 billion SEK.

- **GitLab**, `https://gitlab.com`, offers free code hosting for open source, but it is also free for individuals and community projects to have private repos. Companies can pay for private code hosting with extra services to test, build, and deploy code, etc. GitLab itself is an open source project and the code powering the code hosting site is open and free. This means you can download the code and start your own code hosting site. LTH has a GitLab-based code hosting site here: `https://git.cs.lth.se`

- **BitBucket**, `https://bitbucket.org`, is a popular code hosting site for both open and closed source and is operated by the Australian company Atlassian. It is free for individuals and small teams to have both public and private repos, but only if there are few contributors. Costs arise if the number of contributors exceeds a certain level. University employees and students can get more favorable conditions after application. Atlassian offers a full suite of tools to manage bugs and collaborate over the internet. BitBucket supports, besides Git, also other version control tools.

Using code hosting sites

If you have not already done so, it is good to register your username, preferably `fornamnefternamn` as one word without Swedish characters and in lowercase, on one or all of the above sites, partly to reserve your name and partly to start collaborating with developers worldwide. It is good to choose *one* username for *all* code hosting sites on the internet, especially if you work with open source where your name will be associated with all the code contributions you make during your professional life.
If you do not know which site to choose, start with `https://github.com`. If you want the code hosting site itself to be run by open source, try `https://gitlab.com`.
With a Git-based code hosting site, you have the opportunity to sync your local repo against a server on the internet using `git` commands in the terminal or via a Git client with graphical interface, see section G.2.1.
Before you start using a code hosting site, it is useful to understand the concepts below.

- **clone** (*noun*: a clone is a copy of a (remote) repo, *verb*: to clone, to create a copy). By cloning a repo located on a hosting site, you can build, examine, and further develop the code locally on your computer. If you have rights to submit code to the central original, you can push your commits directly via terminal command or Git client.

- **fork** (*noun*: a branching of an entire repo, *verb*: to fork a repo). By forking a repo, you create a copy, usually also hosted on a code hosting site, which you can develop separately from the original. It then becomes possible for you to submit changes and push them even if you do not have rights to deliver ("push") to the original. Make a change request (Pull Request, PR) if you want to contribute your changes, so the owner of the original can choose to merge your changes with the original. Many hosting sites, such as GitHub, have a special button that you click to easily create a fork of a repo under your user.

- **upstream** (*preposition*: upstream, *noun*: upstream repo) An upstream repo constitutes the original of a forked repo.
  - Here is how to link a fork upstream:
    https://help.github.com/articles/configuring-a-remote-for-a-fork/
  - Here is how to sync a fork upstream:
    https://help.github.com/articles/syncing-a-fork/

If you want to contribute to an open source project, start by forking the repo on the hosting site and then clone the repo to your local computer. Thereafter you can commit changes and push to your fork and finally make a pull request from your fork to upstream. Read about how a contribution can proceed in section 0.3.
Here are some useful commands:

- Create a local copy of a remote repo; here is how you clone the course repo from GitHub:

```
$ git clone --depth 1 https://github.com/lunduniversity/introprog
```

- Pull new commits from a remote repo:

```
$ git pull
```

- Push new local commits to a remote repo:

```
$ git push
```

Appendix H

Integrated Development Environment

H.1 What is an integrated development environment?

An integrated development environment (eng. *integrated development environment, IDE*) gathers several tools for creating, running, and testing programs, including an advanced **editor** and specialized debugging tools. There are several development environments to choose from, which can be used for both Scala and Java.

An IDE provides support for **code completion** (eng. *code completion*), where available methods are shown in a list and the rest of a name can be filled in automatically after you have typed the first letters of the name. An IDE can help you with formatting and also generate skeleton code based on **code templates** (eng. *code templates*). With **error highlighting** (eng. *error highlighting*) you get underlining of certain errors directly in the code, and sometimes you can also get suggestions for actions to correct simple errors. Features for **debugging** (eng. *debugging*) help you troubleshoot while running your code. With features for **refactoring** (eng. *refactoring*) you get help from the editor in collaboration with the compiler to make extensive structural changes across many code files simultaneously, e.g. renaming with regard to the language's visibility rules.

All these advanced features can significantly increase productivity, but at the same time they take time to learn and an IDE can require a lot of computing power and some waiting compared to a regular, standalone editor. Initially, all functionality can feel overwhelming and it can be hard to find your way in all menus and settings. That is why many prefer a standalone, quickly started code editor over a full-fledged, heavy IDE, especially if it concerns a smaller program. On the other hand, an IDE with code completion can be a great help when learning a new API and experimenting with an unfamiliar codebase. Over time, common editors, such as VS Code, have gained more and more features that earlier only existed in a full-fledged IDE[1], and the practical difference between a "regular" editor and an IDE is becoming increasingly smaller.

In the course, we use several development environments. In the first lab, we use Kojo (see appendix A), which is an IDE specially adapted for beginners. Then we use an editor e.g. VS Code, preferably in combination with the build tool `sbt`. You can in the second half of the course choose to switch from VS Code to using another IDE, but it is perfectly fine to continue with VS Code which nowadays has a good debugger for Scala via the Metals extension. If you want to use an IDE instead of VS Code, IntelliJ IDEA

[1] See e.g. LSP: https://en.wikipedia.org/wiki/Language_Server_Protocol

with the Scala plugin is recommended. If you have already learned Eclipse in depth and really want to continue with this IDE, then choose ScalaIDE – however, this IDE has not kept up with the technical development and remains on Scala 2.12.

H.2 Visual Studio Code with the Scala Metals Extension

Visual Studio Code[2], abbreviated VS Code or simply `code`, is a free development environment that is mostly open source[3]. The project was started and is led by Microsoft and has an active community with many developers and many useful extensions.
VS Code is often called "just" an editor, but over the years it has developed into a full-fledged IDE with, among other things, a built-in debugger and support for many different languages via an extensive library of extensions.

- Read more about how to use VS Code here:
  `https://code.visualstudio.com/docs`
- Read more about how to use Scala in VS Code here:
  `https://scalameta.org/metals/docs/editors/vscode`

There are many useful keyboard shortcuts that make you faster and faster when coding, as you learn new shortcuts. A good tip is to learn at least one new shortcut per day and after a while you will know quite a few. Here is a summary of the most important keyboard shortcuts for VS Code on Linux:
`https://code.visualstudio.com/shortcuts/keyboard-shortcuts-linux.pdf`
Replace `linux` with `windows` or `macos` in the above address for the corresponding platform.

H.2.1 Installing VS Code and Metals

VS Code is pre-installed on LTH's computers, but you need to install the Scala extension **Metals** yourself the first time you run VS Code on LTH's computers. Read about installing Metals here:
`https://marketplace.visualstudio.com/items?itemName=scalameta.metals`
Read more about how to install VS Code on your own computer here:
`https://code.visualstudio.com`
More information about tool installation is available on the course website:
`https://lunduniversity.github.io/pgk/#verktyg`

H.2.2 Running Programs in VS Code

There are different ways to start the main program in a VS Code project:

1. Use `scala run .` in a separate terminal window. Read more about `scala-cli` in Appendix C.4.3.
2. Start `sbt` in a separate terminal window and run the command `run` from inside `sbt`. This requires you to have a valid `build.sbt`, see Appendix F.

[2] en.wikipedia.org/wiki/Visual_Studio_Code
[3] The VS Codium variant `https://vscodium.com/` is completely free from closed source and telemetry.

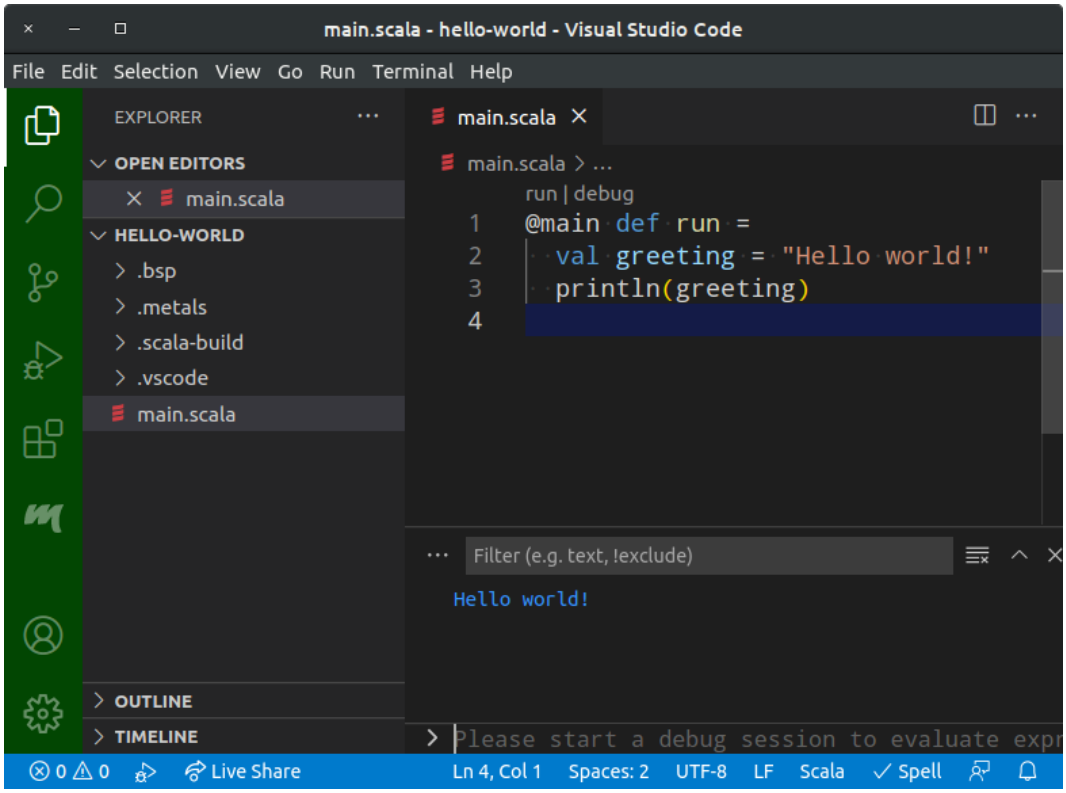


Figure H.1: Run programs by clicking on run above the main program.

3. Run programs from inside VS Code. This requires that you have opened the directory containing your code via the File menu's "Open Folder", or that you start VS Code with `code .` from the top of your project directory (you can see this is done if the lower message bar is blue instead of purple). Before you start VS Code for the first time, you need to run `scala setup-ide .` (see Appendix C.4.3), or create a valid `build.sbt` file (see Appendix F) that you import in VS Code when prompted in the lower right corner.

4. Combine Scala CLI or `sbt` with VS Code. Run this command in a separate terminal window: `scala compile . -w` where `-w` means *watch* and makes it watch for changes. If instead you use `sbt` run `sbt ~compile` in a separate terminal window (note the tilde character which makes it watch for changes). With change watching enabled, compilation errors will appear every time you save a change in VS Code with Ctrl+S. When all compilation errors are fixed and you are ready to test run, click run.

You will see that VS Code is ready to run your program when (after a while) an extra line appears above your main program with the text run | debug and then you can click run to execute your program. Output from the run appears in a tab below the code. Note that it may take some time for VS Code to prepare everything needed to run your program. Keep an eye on whether VS Code is busy with these preparations in the blue message bar at the bottom right. When everything is ready after starting VS Code it says "Index complete!" next to a rocket icon in the message bar.
If something is not working and you do not get run | debug above your `@main` function, even though you have started VS Code as described above, try clicking on one of "Restart build server" or "Import build" (the latter takes longer but starts completely

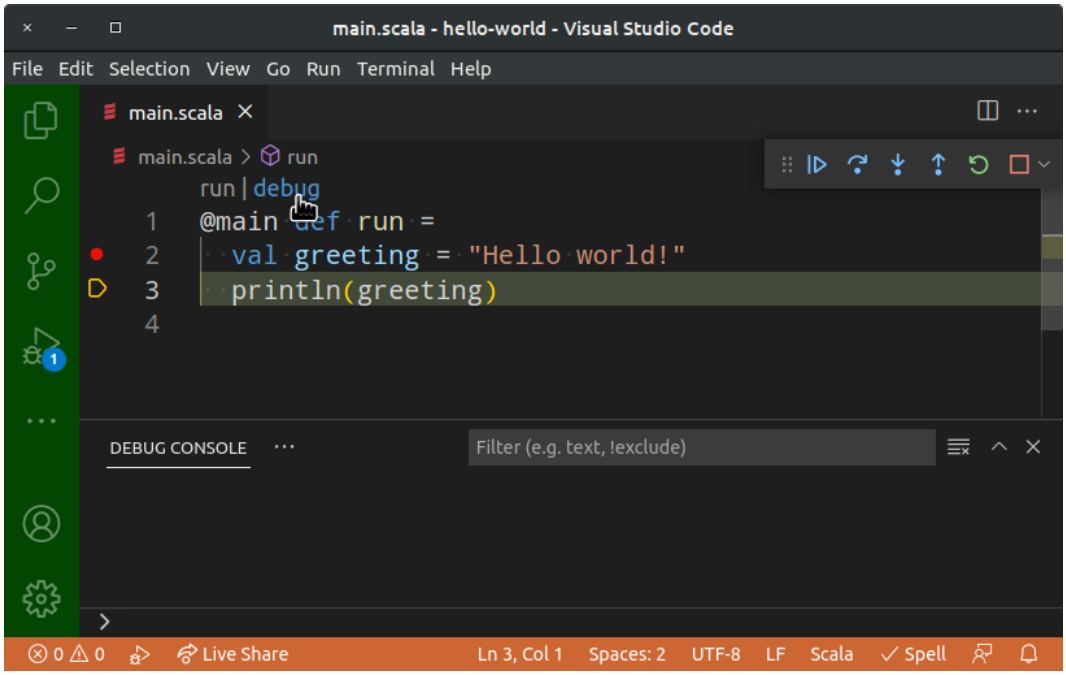


Figure H.2: The debugger in VS Code. The bottom bar is orange when the debugger is running.

fresh) under the Metals tab (the icon with the stylized M in the green toolbar) and wait until it says "Index Complete!" in the blue message bar; then run | debug should appear above your `@main` function.

H.2.3 Using the Debugger in VS Code

Before you start using the debugger, first read about general error handling in Appendix D.

You can enable the debugger in VS Code for your Scala programs by clicking "debug" above your `main` method, provided you have the Metals extension installed in VS Code. You also need to run `scala setup-ide .` once (see Appendix C.4.3), or have a valid `build.sbt` file (see Appendix F) that you import in VS Code when prompted in the lower right corner.

Figure H.2 on page 252 shows how it can look when the debugger in VS Code is activated. When the debugger is running, the bottom message bar becomes orange (instead of blue). To the left of the line number column you can click to enable and disable breakpoints. Enabled breakpoints are shown as a red dot in the left margin. The hollow yellow arrow in the margin points to the line that will be executed next. Note the panel with various buttons at the top edge of the editor window. With these buttons you can control execution as follows (try to learn the shortcuts to become faster):

- **Continue**. The blue play button continues running until the next breakpoint or until the program is finished if no breakpoint is hit. Shortcut "Continue": F5.

- **Step Over**. The blue curved forward arrow executes one line at a time *without* stepping into functions. Shortcut "Step Over": F10.

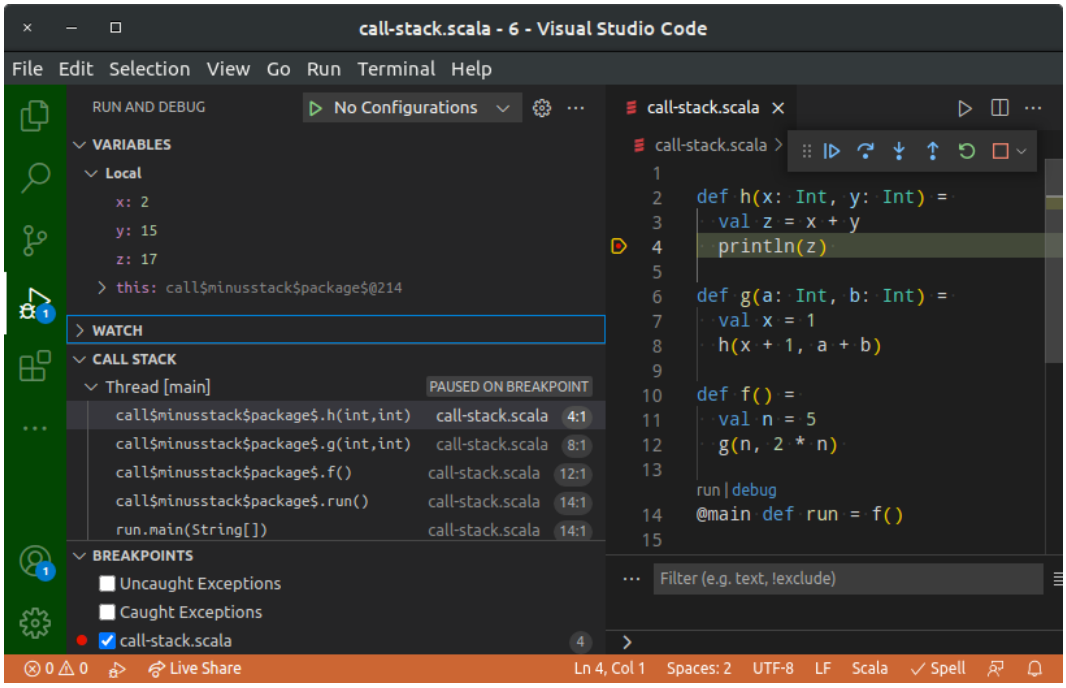


Figure H.3: Call stack and variables in VS Code.

- **Step Into**. The blue downward arrow runs one line at a time and steps into functions if the line contains a function call. Shortcut "Step Into": F11.
- **Step Out**. The blue upward arrow finishes running the current function. Shortcut "Step Out": Shift+F11.
- **Restart**. The green restart icon reruns your program. Shortcut "Restart": Ctrl+Shift+F5.
- **Stop**. The red stop button terminates this debugging session. Remember to stop before starting a new debugging session; otherwise it can easily get confusing with many concurrently running sessions. Shortcut "Stop": Shift+F5.

Figure H.3 on page 253 shows how VS Code presents the call stack and the value of variables visible at the current execution point. You can open this by clicking the icon with a bug and a play button in the vertical green toolbar on the far left. A blue ring shows a one if you have started a debugging session. If there is a two or more then you have multiple sessions running and it may be wise to close all but one, to avoid confusion about which session is currently active.

H.3 JetBrains IntelliJ IDEA with Scala Plugin

IntelliJ IDEA[4] is a professional IDE that supports many different programming languages. IntelliJ is written in Java and developed by the Czech company JetBrains. IntelliJ IDEA is available in two versions: a free community edition with an open source license (eng. *Community edition*), and a paid edition with closed source and support services.

[4] en.wikipedia.org/wiki/IntelliJ_IDEA

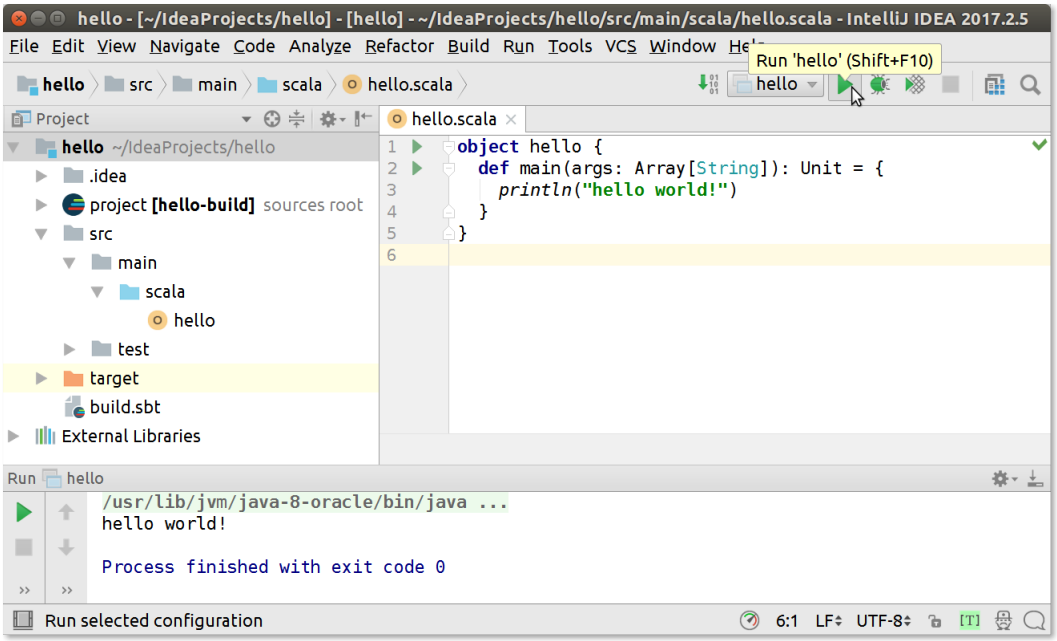


Figure H.4: The integrated development environment IntelliJ IDEA.

IntelliJ IDEA is a comprehensive and advanced programming environment with many features and settings. There is also an extensive collection of plugins and add-ons that facilitate development of, for example, mobile apps, web applications, databases, and much more.

For IntelliJ IDEA there is a plugin that supports Scala along with its standard libraries and the build tool `sbt`, among others. The Scala plugin can be included by selecting Scala in one of the dialogs shown during the first run, according to the instructions below.

This section provides links to installation as well as tips on how to get started with using IntelliJ IDEA with Scala. It is fairly quick to learn the basics, but it requires some effort to learn the more advanced features. There are extensive online resources that will help you further.

Google announced in 2013 that the company would switch from Eclipse to IntelliJ as the officially supported development environment for Android, and in 2014 the development environment Android Studio[5] was launched, which builds on IntelliJ.

H.3.1 Installing IntelliJ IDEA

IntelliJ with Scala plugin is preinstalled on LTH's computers and is started with the command `idea` in a terminal window.

You can install IntelliJ on your own computer by following the instructions for your operating system (Windows/macOS/Linux) here:

`https://www.jetbrains.com/help/idea/run-for-the-first-time.html`

You need the Scala plugin which you can select during the IntelliJ installation, but it is also possible to install the Scala plugin afterwards, see more here:

`https://www.jetbrains.com/help/idea/discover-intellij-idea-for-scala.html`

[5] en.wikipedia.org/wiki/Android_Studio

The provided text consists only of LaTeX commands to input content from external files:
“‘latex

Appendix J

Introduction to Java

J.1 Theory

J.1.1 Exercise `scalajava` and lab `javatext`

Optional tasks that give you a **flying start** in the subsequent course:

- Exercise `scalajava`:
  - Translate from Java to Scala and from Scala to Java
  - Investigate autoboxing (eng. *autoboxing*)
  - Use `import scala.jdk.CollectionConverters.*`
- Laboratory `javatext`:
  - Make a text game for the terminal mainly in Java but some parts in Scala, according to requirements, tips, and inspiration in the lab instructions.
- Note! Scala CLI and sbt can mix `.scala` and `.java` in the same project.

```
> scala run .
```

The above command will also compile `.java` files.
Note Java's filename rules: class == filename, package == directory
Good place to put `.java` files: `src/main/java/`
Good place to put `.scala` files: `src/main/scala/`

J.1.2 Many new features in recent Java versions

Java has in recent years evolved rapidly and borrowed much from e.g. Scala.

Version	Year	Some important news
5–8	2004–2014	generics, lambda, Optional
9–11	2017–2018	jshell, var type inference, Unicode, parallel GC
12–17	2019–2021	switch expressions, multi-line strings, sealed classes, records
18–21	2022–2023	pattern matching, virtual threads, sequenced collections
22–25	2024–2025	simple main methods, unnamed variables (underscore), compact object headers

8, 11, 17, 21, 25 are LTS = *Long Term Support*, maintained extra long
Read more here: https://en.wikipedia.org/wiki/Java_version_history

---

J.1.3 Try Java in `jshell`

- Java has a counterpart to Scala's REPL: the command `jshell`

```
> jshell
|  Welcome to JShell -- Version 11.0.11
|  For an introduction type: /help intro
```

```
jshell> /help intro
|
|                                    intro
|                                    =====
|
|  The jshell tool allows you to execute Java code, getting immediate results.
|  You can enter a Java definition (variable, method, class, etc), like:  int x = 8
|  or a Java expression, like:  x + x
|  or a Java statement or import.
|  These little chunks of Java code are called 'snippets'.
|
|  There are also the jshell tool commands that allow you to understand and
|  control what you are doing, like:  /list
|
|  For a list of commands: /help

jshell>
```

J.1.4 Basic similarities and differences Java–Scala

Some similarities:

- Compiles to bytecode that runs on JVM on many different platforms.
- Static typing: gives fast machine code, the compiler can support code changes (so-called refactoring) and finds many bugs already at compile time.

Similar but some difference:

Java

- **Object orientation**, but not "pure" (eng. *pure*) because not all values are objects
- Primitive types are not objects; represented efficiently, usually **without boxing**
- Some support for **functional programming**

Scala

- **Pure object-oriented** because all values are objects, including functions
- `AnyVal` instances are real objects but represented efficiently, usually **without boxing**
- Extensive support for **functional programming**

J.1.5 Main program in Scala and Java

Scala

```scala
object Main {
  def main(args: Array[String]): Unit = {
    println("Hello!")
  }
}
```

Java

```java
public class JMain {
    public static void main(String[] args){
        System.out.println("Hello!");
    }
}
```

Scala 3

```
@main def Hello(): Unit =
  println("Hello!")
```

Java 25

```
void main() {
    System.out.println("Hello!");
}
```

J.1.6 Looping through arguments in a Java main program

```
> code HelloJavaArgs.java
```

```
public class HelloJavaArgs {
    public static void main(String[] args) {
        int i = 0;
        while (i < args.length) {
            System.out.println(args[i]);
            i = i + 1;
        }
    }
}
```

Compile and run:

```
> javac HelloJavaArgs.java
> java HelloJavaArgs hej gurka tomat
hej
gurka
tomat
```

J.1.7 HIGHSCORE implemented in Java

```
import java.util.Scanner;

public class HighScore {
    public static void main(String[] args){
        Scanner scan = new Scanner(System.in);
        System.out.println("How many points did you get?");
        int points =  scan.nextInt();
        System.out.println("What was the highscore before the last game?");
        int highscore = scan.nextInt();
        if (points > highscore) {
            System.out.println("CONGRATS!");
        } else {
            System.out.println("Try again!");
        }
    }
```

```
}
```

J.1.8 Some things present in Java but not in Scala

- + Faster compilation
- + More mature tooling support
- Variable declaration without initialization
- Mutable parameters
- C-like prefix and postfix increment and decrement: `i++ ++i i-- --i`
- C-like **`for`** statement
- Semicolons required after all statements
- **`return`** required in all methods that have return value
- Keyword **`void`**
- Parentheses after all methods
- Special syntax for indexing arrays `[]` not like other collections
- Checked exceptions and **`throws`**
- ...

J.1.9 Some things present in Scala but not yet in Java

- **`case`** classes
- Local functions
- Methods as operators
- Infix operator notation
- Default arguments
- Named arguments
- One-time initialization: **`val`**
- Lazy initialization: **`lazy val`**
- Uniform access for **`def`**, **`val`**, **`var`**
- Custom setters with **`def`** `name_=`
- Name calls, delayed evaluation
- Matching, patterns, and guards
- Class parameters, primary constructor
- Singleton objects: **`object`**
- Companion objects
- Mixins: **`trait`**
- **`for-yield`** expressions
- Blocks are expressions; no need for **`return`**
- Empty value () of type `Unit`
- `Option`, `Some`, `None` (Java has `Optional` which provides some but not all...)
- `Try`, `Success`, `Failure`
- Collections in Scala standard library, especially the **immutable** collections `Vector`, `Map`, `Set`, `List`, etc.
- Content equality with == for immutable structures, incl. `<` `<=` `>` `>=` on strings
- **Uniform** use of collections **including Array** (except content equality for Array)

- Contextual abstractions `given using`
- More precise visibility `private[mypack]`
- Renaming on `import`
- Flexible file structure and naming
- Flexible nesting of classes, objects, traits
- Type alias and abstract types with `type`
- Extension methods `extension`
- ...

J.1.10 Limitations of functional programming in Java

Of all these functional programming concepts in Scala...

- **overloading**
- **anonymous functions**
- **pattern matching**
- top-level functions etc.
- omitting empty parameter lists (uniform access)
- default arguments
- named arguments
- local functions
- functions as real values
- braces on single parameter
- multiple parameter lists
- custom control structures
- name calls (delayed evaluation)
- curried functions
- captured variable scope in function objects (closures)
- ad hoc polymorphism (type classes)
- contextual abstractions

...in Java only: **overloading**, **anonymous functions** (lambdas), **pattern matching** (the latter two with strong restrictions)

Read more about Java here:
https://en.wikipedia.org/wiki/Java_version_history https://en.wikipedia.org/wiki/Anonymous_function#Java_Limitations

---

J.1.11 Basic types in Scala and primitive types in Java

Basic type in **Scala**	Number of bits	Range minimum/maximum value	**primitive type** in **Java**
`Byte`	8	$-2^7 \ldots 2^7 - 1$	`byte`
`Short`	16	$-2^{15} \ldots 2^{15} - 1$	`short`
`Char`	16	$0 \ldots 2^{16} - 1$	`char`
`Int`	32	$-2^{31} \ldots 2^{31} - 1$	`int`
`Long`	64	$-2^{63} \ldots 2^{63} - 1$	`long`
`Float`	32	$\pm\, 3.4028235 \cdot 10^{38}$	`float`
`Double`	64	$\pm\, 1.7976931348623157 \cdot 10^{308}$	`double`

J.1.12 Java's switch statement

Most C-like languages (but not Scala) have a **`switch`** statement which you can use instead of (some) nested if-else statements:

```
import java.util.Scanner;

public class Switch {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        System.out.println("Write a vegetalbe: ");
        String g = scan.next();
        switch (g) {
        case "cucumber":
            System.out.println("good!");
            break;
        case "tomato":
            System.out.println("good!");
            break;
        case "broccoli":
            System.out.println("very good...");
            break;
        default:
            System.out.println("less good...");
            break;
        }
    }
}
```

`switch` from Java 21 has limited pattern matching https://docs.oracle.com/en/java/javase/21/language/pattern-matching.html

---

J.1.13 Java switch statement without break

Missing **break** statement ”falls through” to subsequent case:

```
import java.util.Scanner;

public class SwitchNoBreak {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        System.out.println("Write a vegetable:");
        String g = scan.next();
        switch (g) {
        case "cucumber":
        case "tomato":
            System.out.println("good!");
            break;
        case "broccoli":
            System.out.println("very good...");
            break;
        default:
            System.out.println("less good...");
            break;
        }
    }
}
```

A forgotten **break** can cause a hard-to-find bug...

J.1.14 Java switch statement with forgotten break

```
import java.util.Scanner;

public class SwitchForgotBreak {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        System.out.println("Write a vegetable: ");
        String g = scan.next();
        switch (g) {
        case "cucumber":
            System.out.println("gott!");
        case "tomato":
            System.out.println("very good!");
            break;
        case "broccoli":
            System.out.println("delicious...");
            break;
        default:
            System.out.println("less good...");
            break;
        }
    }
```

```
}
```

```
> java SwitchForgotBreak
Write vegetable:
gurka
good!
very good!
```

J.1.15 Syntax for variable declaration in Scala and Java

Example of variable declarations in

Scala

```
var i1: Int = 0
var i2 = 0
var i3 = (i2: Int) + 0
var p1: Point = new Point(0, 0)
var p2 = new Point(0, 0)
var (x, y) = (0, 0)
val a = 0
final val Constant = 42
```

- i2 inferred type; not possible in Java 8,9 but present with `var` in Java 10
- i3 type anywhere in expression; not possible in Java
- (x, y) pattern in init; not possible in Java
- `val` gives "single-time init"; no exact equivalent in Java but `final` can often be used instead

Java

```
int i1 = 0;
var i2 = 0; // from Java 10
int i4;
Point p1 = new Point(0, 0);
var p2 = new Point(0, 0);
final int CONSTANT = 42;
```

- i4 no explicit init; not possible in Scala

J.1.16 For-loop in Scala and Java

Scala

```
val s = "Abbasillen"

// Loop over indices forwards:

for i <- 0 until s.length do
  println(s(i))


// Loop over indices backwards:

for i <- s.length-1 to 0 by -1 do
  println(s(i))
```

Java

```
String s = "Abbasillen";

// Loop over indices forwards:

for (int i = 0; i < s.length(); i++) {
    System.out.println(s.charAt(i));
}

// Loop over indices backwards:

for (int i = s.length()-1; i >= 0; i--) {
    System.out.println(s.charAt(i));
}
```

In Scala `s.indices` is preferred!

J.1.17 For-loop in Scala with indices

Scala

```scala
val s = "Abbasillen"

// Loop over indices forwards:

for i <- s.indices do
  println(s(i))


// Loop over indices backwards:

for i <- s.indices.reverse do
  println(s(i))
```

Java

```java
String s = "Abbasillen";

// Loop over indices forwards:

for (int i = 0; i < s.length(); i++) {
    System.out.println(s.charAt(i));
}

// Loop over indices backwards:

for (int i = s.length()-1; i >= 0; i--) {
    System.out.println(s.charAt(i));
}
```

J.1.18 For-loop and arrays in Java

A for-loop in Java has the following structure:

```java
for (initialization; condition; increment) {
    stmt1;
    stmt2;
    ...
}
```

A primitive integer array is declared in Java like this:

```java
int[] xs = new int[42];  // holds 42 integers, initialized with 0s
int[] ys = {10, 42, -1}; // initialize with 3 integers
```

Example of for-loop: fill an array with ones

```java
for (int i = 0; i < xs.length; i++){
  xs[i] = 1;    // index with [i]
}
```

J.1.19 Implementation of SEQ-COPY in Java with `for` loop

```java
public class SeqCopyForJava {

    public static int[] arrayCopy(int[] xs){
        int[] result = new int[xs.length];
        for (int i = 0; i < xs.length; i++){
            result[i] = xs[i];
        }
        return result;
    }

    public static String test(){
        int[] xs = {1, 2, 3, 4, 42};
        int[] ys = arrayCopy(xs);
        for (int i = 0; i < xs.length; i++){
            if (xs[i] != ys[i]) {
                return "FAILED!";
            }
        }
        return "OK!";
    }

    public static void main(String[] args) {
        System.out.println(test());
    }
}
```

Some syntax and semantics for Java:

- A Java class with only static members corresponds to a singleton object in Scala.
- The type comes **before** the name.
- You **must** write `return`.
- You **must** have semicolon after every statement.
- Method name **must** be followed by parentheses; if no parameters use `()`
- An array in Java is not a normal object but has an "attribute" `length` giving the number of elements.
- **Exercise**: rewrite with Java's `while` loop instead.

J.1.20 Element by element with special for-each in Java

Scala

```scala
val s = "Abbasillen"

// Loop over all characters:

for ch <- s do println(ch)
```

Java

```java
String s = "Abbasillen";

// Loop over all characters:

for (char ch: s.toCharArray()) {
    System.out.println(ch);
}
```

`s.foreach(println)` does not work in Java but from Java 8 there is method `chars` giving an `IntStream` and then one can:
`str.chars().forEachOrdered(i -> System.out.println((char) i));`

J.1.21 Typical design of Java class

Typical "anatomy" of a Java class:

```java
public class ClassName {
    attributes, usually private
    constructors, usually public
    methods: public getters, and for mutable objects also setters
    methods: private abstractions for internal use
    methods: public abstractions intended to be used by client code
}
```

J.1.22 Static members in Java

- You can **not** declare explicit singleton objects in Java and there is no keyword `object`.
- Instead you can declare **static members** in a class with the Java keyword `static`.
- Example how to do this inside the class `JPerson`:

```
public static final int ADULT_AGE = 18;
```

- The effect is the same as a singleton object in Scala:
  - All static members in a Java class are automatically allocated and end up in a singular "class instance" existing independently of dynamic instances.
  - The static members are accessed with dot notation through the class name, **without new**:

```
System.out.println(JPerson.ADULT_AGE);
```

J.1.23 Example: immutable class in Scala and Java

Scala:

```
class Person(val name: String, val age: Int):
  def isAdult = age >= Person.AdultAge

object Person:
  val AdultAge = 18
```

Java:

```
public class JPerson {
    private String name;
    private int age;
    public static final int ADULT_AGE = 18;

    public JPerson(String name, int age) {
      this.name = name;
      this.age = age;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public boolean isAdult() {
```

```
        return age >= ADULT_AGE;
    }
}
```

Learn this pattern for a typical Java class by heart so you quickly get things in place!

Exercise:
Make `Person` + `JPerson` **mutable** so that name and age can be updated and the following requirements are met:

- name to be given on construction,
- age initialized to 0 at construction,
- age must never be negative.

J.1.24 Example: Scala class Complex

```
class Complex(val re: Double, val im: Double):
  def r  = math.hypot(re, im)
  def fi = math.atan2(im, re)
  def +(other: Complex) = new Complex(re + other.re, im + other.im)
  override def toString = s"$re + $im${Complex.imSymbol}"

object Complex:
  var imSymbol = 'i'
```

Scala: https://github.com/lunduniversity/introprog/blob/master/compendium/examples/complex7.scala
Java: https://github.com/lunduniversity/introprog/blob/master/compendium/examples/JComplex.scala

---

J.1.25 Example: Corresponding Java class JComplex

```
public class JComplex { // you cannot declare class parameters in Java
    private double re;  // initialized in the constructor below
    private double im;  // initialized in the constructor below
    public static char imSymbol = 'i'; // public, mutable attribute (unusual in Java)

    public JComplex(double real, double imag){ // constructor, runs on new
        re = real;
        im = imag;
    }

// a "getter" that returns the attribute value, prevents modification of re
    public double getRe(){
        return re;
    }
```

```java

// not customary formatting in Java, so methods are at least 3 lines
    public double getIm(){ return im; }

    public double getR(){
        return Math.hypot(re, im);       // Math with a capital M in Java
    }

    public double getFi(){
        return Math.atan2(re, im);
    }

// Java method names may not contain operator symbols such as +, hence the name add
    public JComplex add(JComplex other){
        return new JComplex(re + other.getRe(), im + other.getIm());
    }

    @Override public String toString(){
        return re + " + " + im + imSymbol;
    }
}
```

J.1.26 Example: Using JComplex in Scala code

```
$ javac JComplex.java
$ scala
Welcome to Scala 2.12.9 (OpenJDK 64-Bit Server VM, Java 1.8.0_222).
Type in expressions for evaluation. Or try :help.

scala> val jc1 = JComplex(3, 4)
jc1: JComplex = 3.0 + 4.0i

scala> val polarForm = (jc1.getR, jc1.getFi)
polarForm: (Double, Double) = (5.0,0.6435011087932844)

scala> val jc2 = JComplex(1, 2)
jc2: JComplex = 1.0 + 2.0i

scala> jc1 add jc2
res0: JComplex = 4.0 + 6.0i
```

J.1.27 Example: Using JComplex in Java code

```java
public class JComplexTest {
    public static void main(String[] args){
        JComplex jc1 = new JComplex(3,4);
```

```
        String polar = "(" + jc1.getR() + ", " + jc1.getFi() + ")";
        System.out.println("Polar form: " + polar);
        JComplex jc2 = new JComplex(1,2);
        System.out.println(jc1.add(jc2));
    }
}
```

- In Java you must write **new**.
- In Java you must write **empty parentheses pairs** after method name when **calling parameterless methods**.
- **Tuples do not exist** in Java, so it is not easy to create pairs of values as in Scala; above we convert polar form to a string for printing.
- **Operator notation for methods does not exist** in Java, so in Java one must use dot notation and write: `jc1.add(jc2)`

J.1.28 Example: Mutable class in Scala and Java

Scala:

```
class MutablePerson(var name: String):
  private var _age = 0

  def age: Int = _age

  def age_=(a: Int): Unit =
    if (a >= 0) _age = a else _age = 0
      // or better throw exception?

  def isAdult: Boolean =
    age >= MutablePerson.AdultAge

object MutablePerson:
  val AdultAge = 18
```

Java:

```
public class JMutablePerson {
    private String name;
    private int age = 0;
    public static final int ADULT_AGE = 18;

    public JMutablePerson(String name) {
      this.name = name;
    }

    public String getName() {
        return name;
```

```
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        if (age >= 0) {
          this.age = age;
        } else {
          this.age = 0;
        }
    }

    public boolean isAdult() {
        return age >= ADULT_AGE;
    }
}
```

J.1.29 Scala's "case class goodness" not in Java

An immutable data type is implemented in **Scala** preferably as a **case** class:

```
case class Person(name: String, age: Int):
  def isAdult = age >= Person.AdultAge

object Person:
  val AdultAge = 18
```

In Java there are nowadays records similar to case classes:

```
public record Person (String name, int age) {}
```

Constructor, getters, and equals + hashcode are generated.
An immutable class in **Java** with **equivalent** functionality requires custom implementation of these methods:

- a getter for each attribute
- `equals`
- `hashCode` (explained in advanced course)
- `apply`
  (but you probably call it `create` or similar; the name must be written)
- `toString`
- `copy`
  (but since there are no named parameters and default arguments, this is clumsy)
- `unapply`
  (but since there is no pattern matching, this is probably left out)

J.1.30 Review: The primitive type Array in JVM

- Primitive arrays (`Array` in Scala, `[]` in Java) have **advantages**:[1]
  - It is the fastest indexable data structure in JVM: reading and updating an element at a given position is very efficient if you know the index.
  - Works equally well with both primitive values and object references
- ... but also **disadvantages**:
  - You must decide the number of elements to allocate when you do **new**.
  - You can allocate a bit extra if you need more space later, but then you have to keep track of how many slots you're using and where the next free slot is.
  - It is complicated to insert and delete elements.
  - If you want more slots, you must allocate a completely new, larger array and copy all existing elements over.

J.1.31 Syntax for Array in Scala and Java

[1] stackoverflow.com/questions/2843928/benefits-of-arrays

Scala

```scala
var xs = Array(42, 43, 44)




val n = xs.length

var strings = new Array[String](42)
// or      Array.ofDim[String](42)
// or      Array.fill(42)(null: String)

strings(0) = "first"

strings(1) = "second"
```

Java

```java
int[] xs = new int[]{42, 43, 44};

// same as above but shorter:
int[] xs2 = {42, 43, 44};

int n = xs.length;  // NOT length()

String[] strings = new String[42];



strings[0] = "first";

strings[1] = "second";
```

J.1.32 Example: Polygon with primitive array in Java

```java
public class Polygon {
    private Point[] vertices; // array of corner points
    private int n;            // number of corners

    /** Creates a polygon */
    public Polygon() {
        vertices = new Point[1];
        n = 0;
    }

    ...
```

J.1.33 Polygon with primitive array in Java: insert last and create more space if needed

Implement:

```java
private void extend()                // double size
public void addVertex(int x, int y)  // add vertex
```

```java
    private void extend(){
        Point[] oldVertices = vertices;
        vertices = new Point[2 * vertices.length]; // create double space
        for (int i = 0; i < oldVertices.length; i++) {  // copy
            vertices[i] = oldVertices[i];
        }
    }

```

```
    public void addVertex(int x, int y) {
        if (n == vertices.length) extend();
        vertices[n] = new Point(x, y);
        n++;
    }
```

J.1.34 Polygon with primitive array in Java: insert in the middle at given position

Implement:

```
/** Insert vertex at position pos */
public void insertVertex(int pos, int x, int y)
```

```
    public void insertVertex(int pos, int x, int y) {
        if (n == vertices.length) extend();   // extend if needed
        for (int i = n; i > pos; i--) {       // move elements from back
            vertices[i] = vertices[i - 1];
        }
        vertices[pos] = new Point(x, y);
        n++;
    }
```

J.1.35 Scan files and strings with `java.util.Scanner`

- In Scala you can read from file like this (see quickref page 3 at bottom):

```
val names = scala.io.Source.fromFile("src/names.txt").getLines.toVector
```

- The class `java.util.Scanner` can also read from files (see Java Quickref page 4):

```
def readFromFile(fileName: String): Vector[String] = {
  val file = new java.io.File(fileName)
  val scan = new java.util.Scanner(file)
  val buffer = scala.collection.mutable.ArrayBuffer.empty[String]
  while (scan.hasNext) {
    buffer += scan.next
  }
  scan.close
  buffer.toVector
}
```

- With `new java.util.Scanner(System.in)` you can also scan the keyboard.
- With `new java.util.Scanner("hej 42")` you can scan a string too.
- Scan `Int` and `Double` with methods `nextInt` and `nextDouble`.
  See doc: docs.oracle.com/javase/8/docs/api/java/util/Scanner.html

---

J.1.36 Example: Scanner

```
scala> val scan = new java.util.Scanner("hej 42 42.0   42 slut")

scala> scan.hasNext
res0: Boolean = true

scala> scan.hasNextInt
res1: Boolean = false

scala> scan.next
res2: String = hej

scala> scan.hasNextInt
res3: Boolean = true

scala> scan.nextInt
res4: Int = 42

scala> while (scan.hasNext) println(scan.next)
42.0
42
slut
```

J.1.37 Using Java collections in Scala with `CollectionConverters`

With help of **`import`** `scala.jdk.CollectionConverters.*`
you get smooth **interoperability** with Java and its standard library,
especially methods **`asJava`** and **`asScala`**:

```
scala> import scala.jdk.CollectionConverters.*

scala> Vector(1,2,3).asJava
res0: java.util.List[Int] = [1, 2, 3]

scala> val xs = new java.util.ArrayList[String]()
xs: java.util.ArrayList[String] = []

scala> xs.add("hej")
res1: Boolean = true

scala> xs.asScala
res2: scala.collection.mutable.Buffer[String] = Buffer(hej)
```

Read more here:
https://docs.scala-lang.org/overviews/collections-2.13/conversions-between-java-and-scala-collections.html

---

J.1.38 Generic collections in Java

- Starting from Java version 5 (2004) generics were introduced enabling classes to offer generic handling of different kinds of objects.

- Generic classes in Java use syntax `ClassName<Type>`, e.g. `ArrayList<Point>`
- For deeper study: docs.oracle.com/javase/tutorial/extra/generics/intro.html, more about this in advanced course.

J.1.39 About ArrayList in Java

`java.util.ArrayList` is similar to `scala.collection.mutable.ArrayBuffer` which both have these advantages:

- Stores its elements internally in fast indexed primitive arrays.
- Works for all kinds of objects.
- Expands the collection's size automatically when needed.

However, there are some disadvantages with `ArrayList` in Java
(which do not apply to `ArrayBuffer` in Scala):

- Does **not** work directly with primitive types `int`, `double`, `char`, ...
  (but there are ways around this, thanks to so-called wrapper classes and auto-boxing; more on this soon)
- The name `ArrayList` is not very fortunate, since "list" normally means linked rather than array-like structures.

J.1.40 Polygon with ArrayList in Java

Class `Polygon`, now with attribute of type `ArrayList<Point>`:

```
public class Polygon {
    private ArrayList<Point> vertices; // list of corner points

    /** Creates a polygon */
    public Polygon() {
        vertices = new ArrayList<Point>();
    }

    ...
```

No attribute `n` is needed since we do not have to keep track of allocated slots ourselves: allocation, insertion, and increasing number of slots is managed automatically by `ArrayList` when needed.

J.1.41 Some important operations on ArrayList<E>

https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/ArrayList.html

```
/** Returns element at position index */
public E get(int index);

/** Adds object e at end and returns true */
public boolean add(E e);

/** Inserts object at position index; subsequent elements shifted */
public void add(int index, E e);

/** Removes element at position index and returns it */
public E remove(int index);

/** Returns number of elements in list */
public int size();
```

Learn what exists about ArrayList in Java quick reference
Advanced: check implementation of ArrayList here:
https://hg.openjdk.org/jdk8/jdk8/jdk/file/tip/src/share/classes/java/util/ArrayList.java#l106

J.1.42 Exercise ArrayList: new and add

Write Java code that creates a list with elements of type `Point` and adds three points to the list with coordinates:
(50, 50), (50,10) and (30, 40).

Solution:

```
ArrayList<Point> vertices = new ArrayList<Point>();
vertices.add(new Point(50, 50));
vertices.add(new Point(50, 10));
vertices.add(new Point(30, 40));
```

J.1.43 For-each statement in Java:

- Suppose we want to iterate over all elements in a list.

```
ArrayList<String> words = new ArrayList<String>();
```

- There are two types of **for** loops in Java that can do this:
  - Regular **for** loop:

```
for (int i = 0; i < words.size(); i++) {
    System.out.println(i + ": " + words.get(i));
}
```

- So-called **for-each loop** with syntax:
  `for (ElementType element: collection) { ... }`
  Example:

```
for (String s: words) {
    System.out.println(s);
}
```

But we do not get an index variable then...

J.1.44 Polygon with ArrayList: methods become simpler

```
public void addVertex(int x, int y) {
    vertices.add(new Point(x, y));
}

public void move(int dx, int dy) {
    for (Point p: vertices){
        p.move(dx, dy);
    }
}

public void insertVertex(int pos, int x, int y) {
    vertices.add(pos, new Point(x, y));
}

public void removeVertex(int pos) {
    vertices.remove(pos);
}
```

See complete solution here: compendium/examples/scalajava/list/Polygon.java

---

J.1.45 Polygon with ArrayList: iterate over all vertices in draw with indexing

```
public void draw(SimpleWindow w) {
    if (vertices.size() == 0) {
        return;
    }
    Point start = vertices.get(0);
    w.moveTo(start.getX(), start.getY());
    for (int i = 1; i < vertices.size(); i++) {
        w.lineTo(vertices.get(i).getX(),
                 vertices.get(i).getY());
    }
    w.lineTo(start.getX(), start.getY());
```

```
    }
```

Exercise: rewrite with for-each loop.

J.1.46 Polygon with ArrayList: iterate over all vertices in draw with for-each

```
    public void draw(SimpleWindow w) {
        if (vertices.size() == 0) {
            return;
        }
        Point start = vertices.get(0);
        w.moveTo(start.getX(), start.getY());
        for (Point p: vertices){
            w.lineTo(p.getX(), p.getY());
        }
        w.lineTo(start.getX(), start.getY());
    }
```

See complete solution here: compendium/examples/scalajava/list/Polygon.java

---

J.1.47 Exercise ArrayList: implement the method hasVertex

Write code that implements this method in the class `Polygon`:

```
/** Checks if the polygon has any vertex with coordinates x, y. */
public boolean hasVertex(int x, int y) {
    ???
}
```

J.1.48 Solution ArrayList: implement the method hasVertex

```
    public boolean hasVertex(int x, int y) {
        for (Point p: vertices) {
            if (p.getX() == x && p.getY() == y) {
                return true;
            }
        }
        return false;
    }
```

J.1.49 For-each loop with array

For-each loop also works with primitive array:

```
String[] stringArray = {"hej", "på", "dej"};
for (String s: stringArray) {
    System.out.println(s);
}
```

J.1.50 Generic classes (e.g. ArrayList) with primitive types

This is unfortunately **NOT** possible in Java:
~~`ArrayList<int> list = new ArrayList<int>();`~~

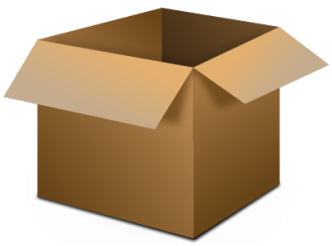
- How to do if you want integer elements (or other primitive values) in a generic collection?
- Java's solution to this problem consists of two parts:
  - Classes that wrap primitive types, (eng. *wrapper classes*)
  - Special rules for implicit conversions, so-called "autoboxing" (eng. *Boxing / Unboxing conversions*)

Often works fine, but there are pitfalls.
(If you are curious about all intricate details, see Java tutorial and Java specification.)

J.1.51 Wrapper class `Integer`

A sketch of the class `Integer`
(in package `java.lang` and thus imported implicitly):

```
public class Integer {
    private int value;

    public static final MIN_VALUE = -2147483648;
    public static final MAX_VALUE = 2147483647;

    public Integer(int value) { // deprecated; will be private in future
        this.value = value;
    }

    public static valueOf(int value) {
        return new Integer(value)
    }

    public int intValue() {
        return value;
    }
    ...
}
```



Javadoc

for class `Integer` is here:

http://docs.oracle.com/javase/8/docs/api/java/lang/Integer.html

---

J.1.52 Wrapper classes in `java.lang`

Primitive type	Wrapped type
boolean	Boolean
byte	Byte
short	Short
char	Character
int	Integer
long	Long
float	Float
double	Double

J.1.53 Exercise: primitive versus wrapped types

- Declare a variable named `gurka` of primitive integer type and initialize it to 42.
- Declare a reference variable named `tomat` of wrapped (”wrapped”) integer type and initialize it to 43.
- Draw how it looks in memory.

J.1.54 Example: List with integers without autoboxing

```
import java.util.ArrayList;
import java.util.Scanner;

public class TestIntegerList {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<Integer>();
        Scanner scan = new Scanner(System.in);
        System.out.println("Write integers with a space between them. End with <CTRL+D>");
        while (scan.hasNextInt()) {
            int nbr = scan.nextInt();
            Integer obj = Integer.valueOf(nbr);
            list.add(obj);
        }
        System.out.println("Your integers in reverse order:");
        for (int i = list.size() - 1; i >= 0; i--) {
            Integer obj = list.get(i);
            int nbr = obj.intValue();
            System.out.println(nbr);
        }
    }
}
```

Code is here: compendium/examples/scalajava/TestIntegerList.java

---

J.1.55 Special rules for wrapper classes

- If an `int` value appears where an `Integer` object is needed, the compiler **automatically** inserts code that creates an `Integer` object wrapping the value.
- If an `Integer` object appears where an `int` value is needed, the compiler **automatically** inserts code calling method `intValue()`.

Same applies between all primitive types and their wrapper classes:

```
 boolean  ⇔  Boolean
    byte  ⇔  Byte
   short  ⇔  Short
    char  ⇔  Character
     int  ⇔  Integer
    long  ⇔  Long
   float  ⇔  Float
  double  ⇔  Double
```

J.1.56 Example: List with integers and autoboxing

```java
import java.util.ArrayList;
import java.util.Scanner;

public class TestIntegerListAutoboxing {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<Integer>();
        Scanner scan = new Scanner(System.in);
        System.out.println("Write integers with a space between them. End with <CTRL+D>");
        while (scan.hasNextInt()) {
            int nbr = scan.nextInt();
            list.add(nbr);   // motsvarar: list.add(Integer.valueOf(nbr));
        }
        System.out.println("Your integers in reverse order:");
        for (int i = list.size() - 1; i >= 0; i--) {
            int nbr = list.get(i);  // motsvarar: int nbr = list.get(i).intValue();
            System.out.println(nbr);
        }
    }
}
```

Code is here: scalajava/generics/TestIntegerListAutoboxing.java

---

J.1.57 Pitfalls with autoboxing

- Comparisons with == and `!=`
  compendium/examples/scalajava/generics/TestPitfall1.java

- The compiler does not notice swapped parameter order, e.g. `add(pos, item)` in wrong order: ~~`add(item, pos)`~~
  compendium/examples/scalajava/generics/TestPitfall2.java

J.1.58 Reference equality or structural equality in Scala and Java

There are two **principally different** kinds of **equality**:

- **Reference equality**: two references are equal if they refer to the **same instance** in memory.
- **Structural equality**, aka content equality: two references are equal if they refer to objects with the **same content**.
- In Scala there are several methods to test equality:
  - the method `eq` tests **reference equality** and `r1.eq(r2)` gives **`true`** if `r1` and `r2` refer to the **same** instance.
  - the method `ne` tests reference **inequality** and `r1.ne(r2)` gives **`true`** if `r1` and `r2` refer to **different** instances.
  - the method == which calls the method `equals` which by default tests reference equality but can be **overridden** if one **wants to decide** if it should be reference or structural equality.
- Scala's **standard library** and **basic types** `Int`, `String` etc. test **structural equality** via the method ==
- In **Java** it is **different**: the symbol == is not a method in Java but **special syntax** which always tests **reference equality** for instance comparison, while method `equals` can be overridden with arbitrary equality test.

J.1.59 Pitfall with collections: contains method requires equals implementation

Suppose we want to implement `hasVertex()` in class `Polygon` by using the method `contains` on a list. How to do then?

```
public boolean hasVertex(int x, int y) {
    return vertices.contains(new Point(x, y)); // NOT WORKING if ...
    // ... Point does not have an equals checking content equality
}
```

We need to implement the method `equals(Object obj)` in the class `Point` that checks content equality and replaces the `equals` in `Object` which checks reference equality, because method `contains` in class `ArrayList` calls `equals` when searching the list for equal objects.
See example here: compendium/examples/scalajava/generics/TestPitfall3.java

Usually one must also override `hashCode`, more on this in advanced course.

J.1.60 Complete recipe for `equals`

For the curious before the advanced course after Christmas:

Read about pitfalls implementing equals in **Java** here:
www.artima.com/lejava/articles/equality.html

Read recipe for implementing equals in **Scala** here:
www.artima.com/pins1ed/object-equality.html#28.4

J.1.61 Conditional expressions in Java

You can use conditional expressions in Java, but with syntax from C:

Scala

```scala
var r = math.random()
var answer = if r > 0.5 then 42 else 0
```

Java

```java
double r = Math.random();
int answer = (r > 0.5) ? 42 : 0;
```

J.1.62 Type test and cast

Scala

```scala
var x = "hej"

var isString = x.isInstanceOf[String]

var y = 42

var z = y.asInstanceOf[Double]
```

Java

```java
String x = "hej";

boolean isString = x instanceof String;

int y = 42;

double z = (double) y;
```

This is better done in Scala with `match` and type patterns!

J.1.63 Rules for overriding in Java

http://docs.oracle.com/javase/tutorial/java/IandI/override.html

---

J.1.64 Catching exceptions in Scala and Java

Typical difference between Scala and Java:
constructs that are **expressions** in Scala are often **statements** in Java.

Scala

```scala
val a = try 2 / 0 catch
  case e: ArithmeticException => 0


val b = try 4 / 2 catch
  case e: ArithmeticException => 0
```

Java

```java
int a;
try {
    a = 2 / 0;
} catch (ArithmeticException e) {
    a = 0;
}

int b;
```

```
try {                                          b = 0;
    b = 4 / 2;                             }
} catch (ArithmeticException e) {
```

J.1.65 Interface `List` in Java

- In Java there are no **`trait`**s and mixins.
- Instead there are **`interface`**s similar to **`trait`** but more limited what members can exist.
- You can only do **`extends`** on exactly one other class, but you can do **`implements`** for several **`interface`**s.
- Example:

  ```
  public class ArrayList<E> extends AbstractList<E>
      implements List<E>, RandomAccess, Cloneable, java.io.Serializable
  ```

- Implementing an interface means fulfilling a contract that promises certain specific methods are available.
- Interface `List` is fulfilled by one of its implementations `ArrayList`

  similarly in Scala where interface `Seq` is fulfilled by `Vector` etc.
  `List<String> xs = new ArrayList<String>();`
- Similar example from Hangman exercise:
  `Set<Character> found = new HashSet<Character>();`
- A Scala trait with only abstract members is compiled to a Java interface in JVM bytecode.
- More about interfaces in Java in the advanced course.

J.1.66 Cannot create generic Array in Java

- In Java you **cannot** create primitive arrays of generic type parameter: ~~`T[] xs = new T[42]`~~
- Instead you must create array of the most general reference type:
  `Object[] xs = new Object[42]`
  and then type test and cast at runtime; see e.g. implementation of `ArrayList` at line 119: http://developer.classpath.org/doc/java/util/ArrayList-source.html

- This can actually be done in Scala with help of `reflect.ClassTag` like this:

```
scala> def fill[T](n: Int, x: T): Array[T] = Array.fill(n)(x)
-- Error:
1 |def fill[T](n: Int, x: T): Array[T] = Array.fill(n)(x)
  |                                                     ^
  |  No ClassTag available for T

scala> def fill[T: reflect.ClassTag](n: Int, x: T): Array[T] = Array.fill(n)(x)

scala> fill(42, "hej")
res2: Array[String] = Array(hej, hej, hej, hej, hej, hej, hej, hej, hej, hej, hej
```

J.1.67 Compare strings in Java

- In Java you can **not** compare strings with operators <, <=, >, and >=
- Also operators == and != do *not* mean content equality but **reference equality** :(
- For content equality you must use methods `equals` and `compareTo`.
- `s1.compareTo(s2)` returns an int which is:
  - `0` if s1 and s2 have same content
  - **negative** if s1 < s2 lexicographically, i.e. s1 should sort **before**
  - **positive** if s1 > s2 lexicographically, i.e. s1 should sort **after**
- Check the following:

```
scala> new String("hej") eq new String("hej") // corresponds to == in Jav
scala> "hej".equals("hej")                    // same as == in Scala
scala> "hej".compareTo("hej")
scala> "hej".compareTo("HEJ")         // all uppercase are 'before' lower
scala> "HEJ".compareTo("hej")
```

- Sometimes `"hej" == "hej"` surprisingly gives **`true`** in Java. This is due to a memory optimization called string interning which often happens, but not always. This causes a hard-to-find bug!

docs.oracle.com/javase/8/docs/api/java/lang/String.html#compareTo-java.lang.String-

J.1.68 Compare strings in Java: example

What does this Java program print?

```java
public class StringEqTest {
    public static void main(String[] args){
        boolean eqTest1 =
          (new String("hej")) == (new String("hej"));
        boolean eqTest2 =
          (new String("hej")).equals(new String("hej"));
        int eqTest3 =
          (new String("hej")).compareTo(new String("hej"));
        System.out.println(eqTest1);
        System.out.println(eqTest2);
        System.out.println(eqTest3);
    }
}
```

```
$ javac StringEqTest.java
$ java StringEqTest
false
true
0
```

J.2 Exercise `java`

Goals

- ☐ Be able to explain and describe key differences between Scala and Java.
- ☐ Be able to translate simple algorithms, classes, and singleton objects from Scala to Java and vice versa.
- ☐ Know what a case class contains in terms of a Java class.
- ☐ Be able to use the Java types `List`, `ArrayList`, `Set`, `HashSet` and translate them to their Scala counterparts using `CollectionConverters`.
- ☐ Be able to explain how autoboxing works in Java, and describe advantages and pitfalls.

Preparations

- ☐ Study theory at the beginning of this Appendix.

J.2.1 Basic Tasks; preparation for the lab

Task 1. *Translate methods from Java to Scala.* In this task, you should translate a Java class used as a module[2] containing only static methods and no mutable state that can be changed externally. (In the next task you will then translate classes with mutable state.)
We start by translating from Java to Scala line by line, and you should keep as much as possible of the syntax and semantics so that the Scala code becomes as Java-like as possible. In a subsequent subtask, you will then reshape the translation so that the Scala code becomes more idiomatic[3].

a) Study the class `Hangman` below. You shall translate it from Java to Scala according to the guidelines and tips that follow after the code. Read all the guidelines and tips before you start.

```java
import java.net.URL;
import java.util.ArrayList;
import java.util.Set;
import java.util.HashSet;
import java.util.Scanner;

public class Hangman {
    private static String[] hangman = new String[]{
        " ======  ",
        " |/   |  ",
        " |    O  ",
        " |   -|- ",
        " |   / \\ ",
        " |       ",
        " |       ",
        " ==========================   RIP  :("};

    private static String renderHangman(int n){
```

[2] en.wikipedia.org/wiki/Modular_programming
[3] sv.wikipedia.org/wiki/Idiom_%28programmering%29

```
19          StringBuilder result = new StringBuilder();
20          for (int i = 0; i < n; i++){
21              result.append(hangman[i]);
22              if (i < n - 1) {
23                  result.append("\n");
24              }
25          }
26          return result.toString();
27      }
28
29      private static String hideSecret(String secret,
30                                       Set<Character> found){
31          String result = "";
32          for (int i = 0; i < secret.length(); i++) {
33              if (found.contains(secret.charAt(i))) {
34                  result += secret.charAt(i);
35              } else {
36                  result += '_';
37              }
38          }
39          return result;
40      }
41
42      private static boolean foundAll(String secret,
43                                      Set<Character> found){
44          boolean foundMissing = false;
45          int i = 0;
46          while (i < secret.length() && !foundMissing) {
47              foundMissing = !found.contains(secret.charAt(i));
48              i++;
49          }
50          return !foundMissing;
51      }
52
53      private static char makeGuess(){
54          Scanner scan = new Scanner(System.in);
55          String guess = "";
56          do {
57             System.out.println("Gissa ett tecken: ");
58             guess = scan.next();
59          } while (guess.length() != 1);
60          return Character.toLowerCase(guess.charAt(0));
61      }
62
63      public static String download(String address, String coding){
64          String result = "lackalänga";
65          try {
66              URL url = new URL(address);
67              ArrayList<String> words = new ArrayList<String>();
68              Scanner scan = new Scanner(url.openStream(), coding);
```

```
            while (scan.hasNext()) {
                words.add(scan.next());
            }
            int rnd = (int) (Math.random() * words.size());
            result = words.get(rnd);
        } catch (Exception e) {
            System.out.println("Error: " + e);
        }
        return result;
    }

    public static void play(String secret){
        Set<Character> found = new HashSet<Character>();
        int bad = 0;
        boolean won = false;
        while (bad < hangman.length && !won){
            System.out.print("\nFelgissningar: " + bad + "\t");
            System.out.println(hideSecret(secret, found));
            char guess = makeGuess();
            if (secret.indexOf(guess) >= 0) {
                found.add(guess);
            } else {
              bad++;
              System.out.println(renderHangman(bad));
            }
            won = foundAll(secret, found);
        }
        if (won) {
            System.out.println("BRA! :)");
        } else {
            System.out.println("Hängd! :(");
        }
        System.out.println("Rätt svar: " + secret);
        System.out.println("Antal felgissningar: " + bad);
    }

    public static void main(String[] args){
        if (args.length == 0) {
            String runeberg =
                "http://runeberg.org/words/ord.ortsnamn.posten";
            play(download(runeberg, "ISO-8859-1"));
        } else {
            int rnd = (int) (Math.random() * args.length);
            play(args[rnd]);
        }
    }
}
```

Guidelines and tips for the translation:

1. Write the Scala code with a text editor in a file called `hangman1.scala` and

compile.

2. Translate in this first subtask line by line so that the appearance (syntax) of the original Java code is kept as much as possible, with as few changes as possible. You shall thus keep these Scala unusual features, even if they are not idiomatic Scala:
   (a) long indentation,
   (b) unnecessary semicolons,
   (c) unnecessary `()`,
   (d) unnecessary `{}`,
   (e) unnecessary `System.out`, and
   (f) unnecessary **`return`**.
3. Also try in this subtask to make the meaning (semantics) as much as possible correspond to that in Java, e.g. by using **`var`** everywhere, even where one normally uses **`val`** in Scala.
4. A Java class with only static members corresponds to a singleton object in Scala, i.e. an **`object`** declaration containing "normal" members.
5. To clarify that you use Java's `Set` and `HashSet` in your Scala code, use the following import statements in `hangman1.scala`, which renames your imported names and prevents clashes with Scala's built-in `Set`. This form of import is not possible in Java.

```
import java.util.{Set => JSet};
import java.util.{HashSet => JHashSet};
```

6. Java's `i++` does not work in Scala; you need to write `i += 1` or less commonly `i = i + 1`.
7. Type parameters in Java are written within <> while Scala's syntax for type parameters uses `[]`.
8. Unlike Java, Scala method declarations have an equals sign = after the return type, before the body.
9. You can download the Java code for the `Hangman` class below from the course repo[4]. In the same directory, solutions to the Scala translations also reside, but do *not* look at these until you have finished your translations and gotten them to compile and run without errors! The idea is to practice reading compiler error messages and correcting them in repeated compile-test-fix cycles.

b) Create a new file `hangman2.scala` which initially contains a copy of your direct translation from Java in the previous subtask. Reshape the code so that it becomes more like how one normally writes Scala, i.e., more Scala-idiomatic. Try to simplify and shorten as much as you can without sacrificing readability.

Tips and guidelines:

1. Name the Scala object `hangman`. When using a Scala object as a module (i.e., a collection of functions in a common, bounded namespace), it is common to use a lowercase initial letter, similar to the convention for package names. A package is also a kind of module, and with a common naming convention one can later, without changing code that uses the module, switch from a singleton object to a package and vice versa if desired.
2. Make all methods publicly accessible and also make the string vector `hangman` publicly accessible. Declare `hangman` as a **`val`** and construct it with `Vector`.

[4] github.com/lunduniversity/introprog/blob/master/compendium/examples/scalajava/Hangman.java

Because `Vector` is immutable and one cannot inherit from singleton objects, and `hangman` is declared with **`val`**, there are no particular risks with making the constant vector public unless we don't mind other code reading (and possibly becoming dependent on) our hangman text.

3. In the method `renderHangman`, use `take` and `mkString`.
4. In the method `hideSecret`, use `map` instead of a **`for`** loop.
5. It is possible to replace the method `foundAll` with the concise expression `(secret forall found)` where `secret` is a string and `found` is a set of characters (try in REPL how this works). Therefore, skip that method altogether and use the shorter expression directly.
6. In the method `makeGuess`, instead of `Scanner`, use `scala.io.StdIn.readLine`.
7. If you want to practice using recursion instead of imperative loops: Make the method `makeGuess` recursive instead of using **`do-while`**.
8. In the method `download`, instead of `java.net.URL` and `java.util.ArrayList`, use `scala.io.Source.fromURL(address, coding).getLines.toVector` and make a local import of `scala.io.Source.fromURL` at the top of the block where it is used. Local **`import`** statements are not possible in Java.
9. Let the method `download` return an `Option[String]` which returns `None` if the download fails.
10. In the method `download`, instead of **`try-catch`**, use `scala.util.Try` and its convenient method `toOption`.
11. If you want to practice using recursion instead of imperative loops: Use, instead of the **`while`** loop in the method `play`, a local recursive function with this signature:

```
def loop(found: Set[Char], bad: Int): (Int, Boolean)
```

The function `loop` returns a 2-tuple with the number of incorrect guesses and **`true`** if all letters were found or **`false`** if the hangman was completed.

Task 2. *Translate between classes in Scala and classes in Java.* The class `Point` below is a model of a point that can be stored on demand in a list. The list is private to the companion object and can be printed with a method `showSaved`. In the code, an `ArrayBuffer` is used, but in the future, one might want to change from `ArrayBuffer` to another sequence collection implementation, e.g. `ListBuffer`, which fulfills the properties of the supertype `Buffer` but has different performance characteristics for different operations. Therefore, the attribute `saved` in the companion object is declared with the more general type.

```
class Point(val x: Int, val y: Int, save: Boolean = false):
  import Point.*

  if save then saved.prepend(this)

  def this() = this(0, 0)

  def distanceTo(that: Point) = distanceBetween(this, that)

  override def toString = s"Point($x, $y)"

object Point:
  import scala.collection.mutable.{ArrayBuffer, Buffer}
```

```
14
15    private val saved: Buffer[Point] = ArrayBuffer.empty
16
17    def distanceBetween(p1: Point, p2: Point) =
18      math.hypot(p1.x - p2.x, p1.y - p2.y)
19
20    def showSaved: Unit =
21      println(saved.mkString("Saved: ", ", ", "\n"))
```

a) Translate the class `Point` above from Scala to Java. In the next subtask, we will compile both the Scala program above and your corresponding Java program in the terminal and test in REPL that the classes have corresponding functionality.

Tips and guidelines:

1. To avoid name conflicts in upcoming tests, call the Java type `JPoint`.
2. Instead of Scala's `ArrayBuffer` and `Buffer`, use Java's `ArrayList` and `List`, both in the package `java.util`.
3. Investigate the documentation for `java.util.List` to find a counterpart of `prepend` for adding to the beginning of the list.
4. Instead of the default argument in Scala's primary constructor, use an additional Java constructor.
5. There are no singleton objects or companion objects in Java; instead, static class members can be used. Place the companions' members equivalents *inside* the Java class and make them **`static`**.
6. Code in the class body in the Scala class, such as the if-statement at line 4, is placed in an appropriate constructor in the Java class.
7. Prints with `print` and `println` need to be preceded by `System.out` in Java.
8. There is no **`override`** keyword in Java, but there is an annotation that gives the same compiler help. It is written with an at symbol and capital letter, like this: `@Override` before the method declaration.
9. In Java, the convention is to start getter methods with the word `get`, e.g. `getX()`.
10. There is no counterpart to `mkString` for `List`, so you need to iterate over the list and get element references for printing with a **`for`** loop. Note that after the last element, a newline should be printed and no comma after the last element.
11. In Java, a new **`import`** declaration is needed if you want to import an additional type from the same package. You can also use an asterisk $*$ (corresponding to _ in Scala) to import everything in a package, but then you get all kinds of names and may not want that.
12. Methods in Java end with `()` if they lack parameters.
13. All statements in Java end with easily forgotten semicolon. (After having written a lot of Java code and switching to Scala code, it is difficult to get used to not writing semicolons...)

b) Start REPL in the same directory as you compiled the code files. Test that the classes `Point` and `JPoint` behave the same according to below. Also write test code in REPL to read the attribute values that have getters and check that everything works as it should.

```
> scalac Point.scala
> javac JPoint.java
> scala
scala> val (p, jp) = (new Point, new JPoint)
scala> p.distanceTo(new Point(3, 4))
```

```
scala> Point.showSaved
scala> jp.distanceTo(new JPoint(3, 4))
scala> JPoint.showSaved
scala> for (i <- 1 to 10) { new Point(i, i, true) }
scala> Point.showSaved
scala> for (i <- 1 to 10) { new JPoint(i, i, true) }
scala> JPoint.showSaved
```

c) Translate the Java class `JPerson` below into a Scala **`case class`** `Person` with corresponding functionality.

```java
public class JPerson {
    private final String name;
    private final int age;

    public JPerson(final String name, final int age){
        this.name = name;
        this.age = age;
    }

    public JPerson(final String name){
        this(name, 0);
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public boolean canEqual(Object other) {
        return (other instanceof JPerson);
    }

    @Override public boolean equals(Object other){
        boolean result = false;
        if (other instanceof JPerson) {
            JPerson that = (JPerson) other;
            result = that.canEqual(this) &&
              this.getName() == that.getName() &&
              this.getAge()  == that.getAge();
        }
        return result;
    }

    @Override public int hashCode() {
      return name.hashCode() * 41 + age;
    }

    @Override public String toString() {
```

```
        return "JPerson(" + name + ", " + age + ")";
    }
}
```

✎ d) Investigate in REPL which functionality in the Scala case class `Person` is *not* implemented in the Java class `JPerson` above. Write down the names of some of the case class's extra methods and their signatures by pressing the TAB key on a `Person` instance and on the companion object `Person`. Try some of the extra methods in REPL and explain what they do.

```
scala> val p = Person("Björn", 49)
scala> p.      // press TAB once
scala> Person. // press TAB once
scala> p.copy  // press TAB once
scala> p.copy()
scala> p.copy(age = p.age + 1)
scala> Person.unapply(p)
```

Task 3. *Immutable Java class.* Translate the Scala class below to the Java class `JPoint3D`. All attributes should be private (why?). Translate the default argument to an alternative constructor. Call getters e.g. `getX()`. Compile with `javac` and test in REPL.

```
class Point3D(val x: Int, val y: Int, val z: Int = 0)
```

Task 4. *Mutable Java class.*
Translate the Scala class below to the Java class `JMutablePoint3D`. All attributes should be private (why?). Translate the default argument to an alternative constructor. Call setters e.g. `setX`. Compile with `javac` and test in REPL.

```
class MutablePoint3D(var x: Int, var y: Int, var z: Int = 0)
```

Task 5. *Comparing strings in Java.* In Java, one **cannot** compare strings with the operators <, <=, >, and >=. Furthermore, the operators == and != do not give content (in)equality but reference (in)equality. Instead, one must use the methods `equals` and `compareTo`, which also work in Scala because strings in Scala and Java have the same type, namely `java.lang.String`.

a) What results do the following expressions have?

```
scala> "hej".getClass.getTypeName
scala> "hej".equals("hej")
scala> "hej".compareTo("hej")
```

b) Study the documentation for `compareTo` in `java.lang.String`[5] and write at least 3 different expressions in Scala REPL that test how the method works in different cases.

c) Study the documentation for `compareToIgnoreCase` [6] and write at least 3 different string expressions in Scala REPL that test how the method works in different cases.

[5] docs.oracle.com/javase/8/docs/api/java/lang/String.html#compareTo-java.lang.String-
[6] docs.oracle.com/javase/8/docs/api/java/lang/String.html#compareToIgnoreCase-java.lang.String-

d) What does the following Java program print?

```
public class StringEqTest {
    public static void main(String[] args){
        boolean eqTest1 =
          (new String("hej")) == (new String("hej"));
        boolean eqTest2 =
          (new String("hej")).equals(new String("hej"));
        int eqTest3 =
          (new String("hej")).compareTo(new String("hej"));
        System.out.println(eqTest1);
        System.out.println(eqTest2);
        System.out.println(eqTest3);
    }
}
```

Task 6. *Linear search in Java.* This task builds upon task 13 in chapter 8. You shall create a variant of linear search which involves finding the first yatzy row in a matrix where each row contains the results of 5 dice rolls.

a) You shall add methods `isYatzy` and `findFirstYatzyRow` in class `ArrayMatrix` in task 13 in chapter 8 as outlined below. We start with `isYatzy` in this subtask (next subtask is about `findFirstYatzyRow`). NOTE! There is a bug in `isYatzy` – fix the bug and test that it works.

```
    public static boolean isYatzy(int[] dice){ /* has one bug! */
        int col = 1;
        boolean allSimilar = true;
        while (col < dice.length && allSimilar) {
          allSimilar = dice[0] == dice[col];
        }
        return allSimilar;
    }

    /** Finds first yatzy row in m; returns -1 if not found */
    public static int findFirstYatzyRow(int[][] m){
        int row = 0;
        int result = -1;
        while (???) {
             /* linear search  */
        }
        return result;
    }
```

b) Implement `findFirstYatzyRow`. First create pseudo-code for the linear search algorithm before writing the Java implementation. Test your program by adding the following lines in the main program. The method `fillRnd` is included in task 13 in chapter 8.

```
        int[][] yss = new int[2500][5];
        fillRnd(yss, 6);
        int i = findFirstYatzyRow(yss);
```

```
        System.out.println("First Yatzy Index: " + i);
```

Task 7. *Comparison support in Java.* Java has counterparts to Scala's `Ordering` and `Ordered`, called `java.util.Comparator` and `java.lang.Comparable`. In fact, Scala's `Ordering` is a subtype of Java's `Comparator`, while Scala's `Ordered` is a subtype of Java's `Comparable`.

- Java's `Comparator` and Scala's `Ordering` are used to create standalone orderings that can compare *two different* objects. These can be made implicitly available in Scala. In Java's collections library, instances of `Comparator` are passed as explicit arguments.
- Java's `Comparable` and Scala's `Ordered` are used as a supertype for classes that want to compare "themselves" to other objects and have a natural ordering.

✎ a) Look up the documentation for `java.util.Comparator`. Which abstract method must be implemented and what does it do?

b) In the package `java.util.Arrays` there is a method `sort` which takes an `Array[T]` and a `Comparable[T]`. Try to use these in REPL as per the sketch below. Restart the REPL so that any previous implicit orderings for `Team` are gone.

```
scala> import java.util.Comparator
scala> val teamComparator = new Comparator[Team]{
         def compare(o1: Team, o2: Team) = ???
       }
scala> val xs =
         Array(Team("fnatic", 1499), Team("nip", 1473), Team("lumi", 1601))
scala> java.util.Arrays.sort(xs.toArray, teamComparator)
scala> xs
```

c) In Scala, there is a convenient method `Ordering.comparatorToOrdering` which creates an implicitly available ordering if one has a `java.util.Comparator`. Test this as below in REPL, with the declarations from the previous subtask.

```
scala> implicit val teamOrd = Ordering.comparatorToOrdering(teamComparator)
scala> xs.sorted
```

✎ d) Look up the documentation for `java.lang.Comparable`. Which abstract method must be implemented and what does it do?

e) Make the class `Point` implement `Comparable` so that points closer to the origin sort before points further away according to the sketch below. In Scala, types that are `Comparable` are implicitly also `Ordered`, so sorting below works. Verify this in REPL once you've figured out how to implement `compareTo`.

```scala
case class Point(x: Int, y: Int) extends Comparable[Point] {
  def distanceFromOrigin: Double = ???
  def compareTo(that: Point): Int = ???
}
```

```
scala> val xs = Seq(Point(10,10), Point(2,1), Point(5,3), Point(0,0))
scala> xs.sorted
```

Task 8. *`java.util.Arrays.binarySearch`* The class `java.util.Arrays`[7] has a static method `binarySearch` which can be used as follows.

```
scala> val xs = Array(5,1,3,42,-1)
scala> java.util.Arrays.sort(xs)
scala> xs
scala> java.util.Arrays.binarySearch(xs, 42)
scala> java.util.Arrays.binarySearch(xs, 43)
```

Write a Java program that tests `java.util.Arrays.binarySearch`. Use an array of type `int[]` with some integers that are first sorted with `java.util.Arrays.sort`. Print the return values from `java.util.Arrays.binarySearch` in various cases by searching for numbers at the start, middle, end, and numbers not present. *Tip:* You can declare an array, allocate it, and fill it with values in Java like this:
`int[] xs = new int[]{5, 1, 3, 42, -1};`

Task 9. *Auto(un)boxing.* In the JVM, type parameters for generic classes must be of reference type. In Scala, the compiler solves this for us so that we can still have e.g. `Int` as a type argument, whereas in Java you *cannot* directly have the primitive type `int` as a type parameter for e.g. `ArrayList`.
In Java and on the underlying JVM platform, wrapper classes are used for this, e.g. through the wrapper class `Integer` which boxes the primitive type `int`. The Java compiler supports automatically boxing primitive values into such wrapper classes to create reference types and can also automatically unbox them.

a) Study how the Scala compiler lets us work with a `Cell[Int]` even though the underlying JVM runtime type is a wrapper class. You can see the JVM runtime type with the methods `getClass` and `getTypeName` as below.

```
scala> class Cell[T](var value: T){
         val typeName: String = value.getClass.getTypeName
         override def toString = "Cell[" + typeName + "]((" + value + ")"
       }
scala> val c = new Cell[Int](42)
scala> c.value.getClass.getTypeName
```

b) What is the runtime type of `c.value` above? Explain how this can be despite that we declared the type argument `Int`?

c) Study the documentation for `java.lang.Integer`[8] and test in REPL some of the *class methods* (those which are `static` and can be called with dot notation on the class name without `new`) and some *instance methods* (those which are not `static`).

```
scala> Integer.  //press TAB
scala> Integer.
scala> Integer.toBinaryString(42)
scala> Integer.valueOf(42)
scala> val i = new Integer(42)
scala> i.  // press TAB
scala> i.toString
scala> i.compareTo  // press TAB twice
scala> i.compareTo(Integer.valueOf(42))
scala> i.compareTo(42)  // why does this work?
```

[7] docs.oracle.com/javase/8/docs/api/java/util/Arrays.html
[8] docs.oracle.com/javase/8/docs/api/java/lang/Integer.html

✎ d) According to the documentation[9] the instance method `compareTo` in class `Integer` takes an `Integer` as parameter. How can it be that the last line above works with an `Int`?

e) Study the Java program below and describe what will be printed *before* you compile and run it.

```
1  import java.util.ArrayList;
2
3  public class Autoboxing {
4      public static void main(String[] args) {
5          ArrayList<Integer> xs = new ArrayList<Integer>();
6          for (int i = 0; i < 42; i++) {
7              xs.add(new Integer(i));
8          }
9          for (Integer x: xs) {
10             int y = x.intValue() * 10;
11             System.out.print(y + " ");
12         }
13         int pos = xs.size();
14         xs.add(pos, new Integer(0));
15         System.out.println("\n\n[0]: " + xs.get(0).intValue());
16         System.out.println("[" + pos + "]: " + xs.get(pos));
17         if (xs.get(0) == xs.get(pos)) {
18             System.out.println("EQUAL");
19         } else {
20             System.out.println("NOT EQUAL");
21         }
22     }
23 }
```

f) Change the program above so that autoboxing and autounboxing are used in all places where possible. Also exploit that `toString` method on `Integer` gives the same string representation as `int` when printed. Also fix to avoid the *pitfall* in Java of comparing with reference equality instead of using `equals`. Test that everything works as it should after your changes.

✎ g) Suppose you accidentally write `xs.add(0, pos)` on line 14 in your program from the previous task. Explain how autoboxing trips you up in a *pitfall* there.

✎ h) Based on the two previous subtasks: summarize the two mentioned pitfalls with autoboxing in Java in two general points, so that you can memorize them for your future Java programming.

Task 10. *CollectionConverters.* With `import scala.jdk.CollectionConverters._` you get in your Scala programs access to the handy methods `asJava` and `asScala` that translate between corresponding collections in the respective languages' standard libraries. Run the below in REPL and do the following subtasks.

```
scala> val sv = Vector(1,2,3)
scala> val ss = Set('a','b','c')
scala> val sm = Map("gurka" -> 42, "tomat" -> 0)
scala> val ja = new java.util.ArrayList[Int]
```

[9] docs.oracle.com/javase/8/docs/api/java/lang/Integer.html#compareTo-java.lang.Integer-

```
scala> ja.add(42)
scala> val js = new java.util.HashSet[Char]
scala> js.add('a')
scala> import scala.jdk.CollectionConverters._
```

a) To what types are the Scala collections `Vector[Int]`, `Set[Char]` and `Map[String, Int]` converted if you call the method `asJava` on them?

b) To what types are the Java collections `ArrayList[Int]` and `HashSet[Char]` converted if you call the method `asScala` on them? Are they mutable or immutable counterparts?

c) What is the resulting type if you run `toSet` on a collection of type `mutable.Set`?

d) Investigate how you can, after having done `sm.asJava.asScala`, call an additional method to get back an immutable `immutable.Map`.

e) Read more in the documentation about CollectionConverters[10] and try some more conversions.

Task 11. *How does a* **`switch`** *statement in Java (and many other languages) work?* It often happens that one wants to test if a value is one of many alternatives. One can use a sequence of many **`if-else`**, one for each alternative. But there is another way in Java and many other languages: you can use **`switch`** which checks many alternatives in the same statement, see e.g. en.wikipedia.org/wiki/Switch_statement.

a) Write in the code below in an editor. Save with the name `Switch.java` and compile the file with the command `javac Switch.java`. Run it with `java Switch` and give your favorite vegetable as an argument to the program. What happens? Explain how the **`switch`** statement works.

```
1  public class Switch {
2      public static void main(String[] args) {
3          String favorite = "celleri";
4          if (args.length > 0) {
5              favorite = args[0];
6          }
7          System.out.println("Your favorite vegetable: " + favorite);
8          char firstChar = Character.toLowerCase(favorite.charAt(0));
9          System.out.print("I think that ");
10         switch (firstChar) {
11         case 'c':
12             System.out.println("cucumber is good!");
13             break;
14         case 't':
15             System.out.println("tomato is good!");
16             break;
17         case 'b':
18             System.out.println("broccoli is good!");
19             break;
20         default:
21             System.out.println(favorite + " is less good...");
22             break;
23         }
24     }
25 }
```

b) What happens if you remove the **`break`** statement on line 16?

[10] docs.scala-lang.org/overviews/collections-2.13/conversions-between-java-and-scala-collections.html

Task 12. *Catch exceptions in Java with a* ***try-catch*** *statement.* As we saw in the previous task, there is built-in support in the JVM for handling when programs are interrupted unexpectedly, e.g. due to division by zero or unexpected input from the user. Save the code below[11] in a file named `TryCatch.java` and compile with `javac TryCatch.java` in the terminal.

```
1  // TryCatch.java
2
3  public class TryCatch {
4      public static void main(String[] args) {
5          int input;
6          int output;
7          if (args[0].equals("safe")) {
8              try {
9                  input = Integer.parseInt(args[1]);
10                 System.out.println("Skyddad division!");
11                 output = 42 / input;
12             } catch (Exception e) {
13                 System.out.println("Undantag fångat: " + e);
14                 System.out.println("Dividerar ändå med säker default!");
15                 input = 1;
16                 output = 42 / input;
17             }
18         } else {
19             input = Integer.parseInt(args[0]);
20             System.out.println("Oskyddad division!");
21             output = 42 / input;
22         }
23         System.out.println("42 / " + input + " == " + output);
24     }
25 }
```

a) Explain what happens when you run the program with different inputs:

```
> java TryCatch 42
> java TryCatch 0
> java TryCatch safe 42
> java TryCatch safe 0
> java TryCatch
```

b) What happens if you "forget" line 15 and thereby miss initializing input? What is the error message? Is it a runtime error or a compile-time error?

Task 13. *Matrices with arrays in Java.* If you already know how many elements a matrix should have, in Java you often use an array of arrays. An integer matrix (an array of arrays of integers) is written in Java with double square brackets `int[][]` directly after the type. When allocating, use the keyword `new` and specify the number of elements in each dimension within brackets; e.g., `new int[42][21]` gives a matrix with 42 rows and 21 columns, corresponding to Scala's `Array.ofDim[Int](42,21)`[12]. All elements get the default value for the type, here `0` for integers.

[11] https://github.com/lunduniversity/introprog/blob/master/compendium/examples/TryCatch.java

[12] Another way to write this in Scala, where initial values are explicit: `Array.fill(42,21)(0)`

a) Write the program below in an editor and save the code in the file `JavaArrayTest.java` and compile with `javac JavaArrayTest.java` and run in the terminal with `java JavaArrayTest` and examine the output. Explain what happens. Note some differences in how matrices are used in Scala and Java.

```
public class JavaArrayTest {

    public static void showMatrix(int[][] m){
        System.out.println("\n--- showMatrix ---");
        for (int row = 0; row < m.length; row++){
            for (int col = 0; col < m[row].length; col++) {
                System.out.print("[" + row + "]");
                System.out.print("[" + col + "] = ");
                System.out.print(m[row][col] + "; ");
            }
            System.out.println();
        }
    }

    public static void main(String[] args) {
        System.out.println("Hello JavaArrayTest!");
        int[][] xss = new int[10][5];
        showMatrix(xss);
    }
}
```

b) Implement the following method `fillRnd` inside the class `JavaArrayTest`. Write code that fills the matrix `m` with random numbers between `1` and `n`.

```
    public static void fillRnd(int[][] m, int n){
        /* ??? */
    }
```

Tip: this expression creates a random number between 1 and 42 in Java:
`(int) (Math.random() * 42 + 1);`
where type casting `(int)` has the same effect as calling `toInt` in Scala; it converts a double-precision floating-point number to an integer by truncating all decimals.
Change the main program so it calls `fillRnd(xss, 6)`. The program should give output similar to the following:

```
Hello JavaArrayTest!

--- showMatrix ---
[0][0] = 6; [0][1] = 2; [0][2] = 6; [0][3] = 3; [0][4] = 5;
[1][0] = 2; [1][1] = 4; [1][2] = 6; [1][3] = 1; [1][4] = 1;
[2][0] = 5; [2][1] = 4; [2][2] = 4; [2][3] = 1; [2][4] = 5;
[3][0] = 4; [3][1] = 6; [3][2] = 6; [3][3] = 1; [3][4] = 3;
[4][0] = 4; [4][1] = 6; [4][2] = 2; [4][3] = 3; [4][4] = 2;
[5][0] = 2; [5][1] = 4; [5][2] = 5; [5][3] = 5; [5][4] = 3;
[6][0] = 6; [6][1] = 5; [6][2] = 2; [6][3] = 4; [6][4] = 3;
[7][0] = 1; [7][1] = 6; [7][2] = 1; [7][3] = 6; [7][4] = 2;
[8][0] = 1; [8][1] = 1; [8][2] = 5; [8][3] = 3; [8][4] = 2;
[9][0] = 1; [9][1] = 1; [9][2] = 1; [9][3] = 5; [9][4] = 4;
```

Task 14. *Translate from Java to Scala.* Translate the code below from Java to Scala. Write the code in a file named `showInt.scala` and call the Scala object with the `main` method `showInt`. Read the tips that follow after the code before starting.

```java
import java.util.Scanner;

public class JShowInt {
    private static Scanner scan = new Scanner(System.in);

    public static void show(Object obj) {
        System.out.println(obj);
    }

    public static void show(Object obj, String msg) {
        System.out.println(msg + obj);
    }

    public static String repeatChar(char ch, int n) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) {
            sb.append(ch);
        }
        return sb.toString();
    }

    public static String readLine(String prompt) {
        System.out.print(prompt);
        return scan.nextLine();
    }

    public static void showInt(int i) {
        int leading = Integer.numberOfLeadingZeros(i);
        String binaryString =
            repeatChar('0', leading) + Integer.toBinaryString(i);
        show(i,                        "Heltal: ");
        show((char) i,                 "Tecken: ");
        show(binaryString,             "Binärt: ");
        show(Integer.toHexString(i),   "Hex   : ");
        show(Integer.toOctalString(i), "Oktalt: ");
    }

    public static void loop() {
        boolean hasExploded = false;
        while (!hasExploded) {
            try {
                String s = readLine("Heltal annars pang: ");
                showInt(Integer.parseInt(s));
            } catch (Throwable e){
                show(e);
                hasExploded = true;
            }
```

```
        }
        show("PANG!");
    }

    public static void main(String[] args){
        if (args.length == 0) {
            loop();
        } else {
            for (String arg: args) {
                showInt(Integer.parseInt(arg));
                System.out.println();
            }
        }
    }
}
```

Tips:

- A Java class with only static members corresponds to a singleton object in Scala, i.e. an **`object`** declaration. Scala does not have a **`static`** keyword.
- The type `Object` in Java corresponds to Scala's `Any`.
- You can use Scala's default arguments (which don't exist in Java) to define only a single `show` method with an empty string as default `msg` argument.
- In Scala, objects of type `Char` have a method **`def`** `*(n: Int): String` which creates a string with the character repeated `n` times. But you can still implement the method `repeatChar` with `StringBuilder` as below if you want to practice translating a **`for`** loop from Java to Scala.
- Instead of `Scanner.nextLine`, you can use `scala.io.StdIn.readLine` which takes a prompt as parameter, but you can also use `Scanner` in Scala if you want to practice that.
- In Java you *must* use the keyword **`return`** if the method is not **`void`**, whereas in Scala you are allowed to use **`return`** even though it is usually avoided and instead one exploits the fact that expressions in Scala are also expressions.

Compile your Scala code and run in terminal and test that everything works. If you want to also compile the Java code it is in the course repo in file `compendium/examples/scalajava/JShowInt.java`

Task 15. *Content equality and reference equality in Java.* Study and try this pitfall with content equality: TestPitfall3.java

Task 16. *Implement content equality in Java.* Study pitfalls for how to write an `equals` method in Java here: www.artima.com/lejava/articles/equality.html and compare with the full recipe for how to write a well-functioning `equals` and `hashcode` in Scala here: www.artima.com/pins1ed/object-equality.html ✎

a) What differences and similarities are there when overriding equals in Java versus Scala, which should give a working content test for a hierarchy with base types and subtypes?

b) What pitfalls are common to both Java and Scala?

Task 17. *Arrays and* **`for`** *loops in Java.* Download the program below from the course's GitHub repo: `compendium/examples/DiceReg.java`

a) Compile with `javac DiceReg.java` and run with `java DiceReg 10000 42` and explain what happens.

```
// DiceReg.java
import java.util.Random;

public class DiceReg {
    public static void main(String[] args) {
        int[] diceReg = new int[6];
        int n = 100;
        Random rnd = new Random();
        if (args.length > 0) {
            n = Integer.parseInt(args[0]);
        }
        System.out.print("Rolling the dice " + n + " times");
        if (args.length > 1) {
            int seed = Integer.parseInt(args[1]);
            rnd.setSeed(seed);
            System.out.print(" with seed " + seed);
        }
        System.out.println(".");
        for (int i = 0; i < n; i++) {
            int pips = rnd.nextInt(6);
            diceReg[pips]++;
        }
        for (int i = 1; i <= 6; i++) {
            System.out.println("Number of " + i + "'s: " +
                diceReg[i-1]);
        }
    }
}
```

b) Describe the differences between Scala and Java, regarding array and **for** loop syntax. Describe some other differences visible in the program above.

c) Amend the program above so that the loop variable `i` is printed in every round in each **for** loop. Compile again and run.

d) Rewrite the program above by abstracting the main parts into static methods `parseArguments`, `registerPips` and `printReg` according to the skeleton below. Save the program in the file `DiceReg2.java` and compile with `javac DiceReg2.java` in the terminal.

```
// DiceReg2.java
import java.util.Random;

public class DiceReg2 {
    public static int[] diceReg = new int[6];
    private static Random rnd = new Random();

    public static int parseArguments(String[] args) {
        // ???
        return n;
```

```
    }

    public static void registerPips(int n){
        // ???
    }

    public static void printReg() {
        // ???
    }

    public static void main(String[] args) {
        int n = parseArguments(args);
        registerPips(n);
        printReg();
    }
}
```

e) Start Scala REPL in the same directory as the file `DiceReg2.class` is located, and run the following 7 lines in REPL and explain what happens:

```
scala> DiceReg2.main(Array("1000","42"))
scala> DiceReg2.diceReg
scala> DiceReg2.registerPips(1000)
scala> DiceReg2.printReg
scala> DiceReg2.registerPips(1000)
scala> DiceReg2.printReg
scala> DiceReg2.rnd
```

Task 18. *Read in a sequence of numbers with `Scanner` in Java.* Read in the Java part of the quick reference about `java.util.Scanner`. With `new Scanner(System.in)` an object is created that can read numbers from strings that the user types in the terminal window, as shown in the Java program below:

```
// DiceScanBuggy.java
import java.util.Random;
import java.util.Scanner;

public class DiceScanBuggy {
    public static int[] diceReg = new int[6];
    public static Scanner scan = new Scanner(System.in);

    public static void registerPips(){
        System.out.println("Enter pips separated by blanks.");
        System.out.println("End with -1 and <Enter>.");
        boolean isPips = true;
        while (isPips && scan.hasNextInt()) {
            int pips = scan.nextInt();
            if (pips >= 1 && pips <=6 ) {
              diceReg[pips]++;
            } else {
              isPips = false;
            }
```

```
        }
    }

    public static void printReg() {
        for (int i = 0; i < 6; i++) {
            System.out.println("Number of " + i + "'s: " +
                diceReg[i-1]);
        }
    }

    public static void main(String[] args) {
        registerPips();
        printReg();
    }
}
```

Download the program from `compendium/examples/DiceScanBuggy.java` and compile and run with input sequence `1 2 3 4 -1` and note how the registration proceeds.

a) Look up and read the JDK8 documentation of `java.util.Scanner`. What do `hasNextInt()` and `nextInt()` do?

b) The program does not work as intended. You need to correct 3 issues for the program to work correctly. Fix the bugs and save the corrected program as `DiceScan.java`. Compile and test the fixed program.

J.3 Lab: `java`

Goals

- ☐ Understand the difference between primitive types and objects in Java.
- ☐ Be able to explain how autoboxing works in Java.
- ☐ Be able to explain what static methods and attributes in Java mean.
- ☐ Be able to use `ArrayList` and arrays in Java.
- ☐ Be able to use the Java class `Scanner`.
- ☐ Be able to create a for-loop in Java.
- ☐ Be familiar with how to simplify the use of Java and Scala in the same program using `scala.jdk.CollectionConverters`.

Preparations

- ☐ Do the exercises earlier in this Appendix.
- ☐ Study given code here:
  github.com/lunduniversity/introprog/tree/master/workspace/javatext/

J.3.1 Requirements

You are to create a text game for the terminal that is (reasonably) interesting/fun to play and saves scores per game round for different players. To help you, you have the finished file `Main.java` (which can be changed if needed) as well as the two code skeletons `Game.java` and `UserInterface.scala`. Your text game should run in the terminal and fulfill the following requirements and guidelines:

1. When your program runs, it should be possible to start several game rounds one after another without having to close the program.

2. For each game round, the program should remember the player's name[13] with the associated result.

3. Upon request, the program should be able to display a leaderboard with the best scores, both for all players and for a specific player name.

4. The code for the actual game should be written in Java, but Scala should be used to implement the functions in the singleton object `UserInterface`.

5. In the Scala code, you should for practice use the Java class `java.util.Scanner` when reading data from the terminal.

6. The code in the singleton object `UserInterface` should use the conversion method `asScala` after `import scala.jdk.CollectionConverters._` to convert arguments of type `java.util.ArrayList` to `scala.collection.mutable.Buffer`[14].

7. Your game should, in Java code, use at least one of the data structures `ArrayList`, `HashSet`, `HashMap` from the package `java.util`, as well as at least one array. (The given code in `Main.java` does not count toward this requirement.)

8. You should play someone else's half-finished game and, after studying the code, provide feedback on the code's readability.

[13] or players' names if it is a game for two or more people

[14] Note that `asJava` on `Buffer` gives a Java collection of the type `List`.

9. You should let someone else play your half-finished game and show your code and ask for feedback on readability. You should note the feedback you receive.

10. Before the presentation, you should prepare the following:

    (a) a short overview of the game's idea,

    (b) a brief explanation of the code structure and the responsibility of the various Java classes,

    (c) a short report on the feedback you received on your code's readability and how you have worked to improve readability during your stepwise extensions of your code,

    (d) a list of concepts you have practiced while creating your text game.

J.3.2 Optional extra tasks

1. Save results to a file after each game round, and read results from the file either when the program starts or when the user wants to see the score list, so that it is possible to see game results from previous runs of the program. The code you need to add to achieve this can be written in either Java or Scala. Keep in mind that you may also need to make changes in the `Main` class.

2. Measure the playtime for each game round and save the time together with the score result for each player.

J.3.3 Inspiration and tips

1. Start from Hangman in this week's exercise or,

2. Yatzy from earlier exercises, or

3. create a card game inspired by the `shuffle` labs, or

4. be inspired by the list of party games on Wikipedia: sv.wikipedia.org/wiki/Kategori:Sällskapsspel

5. or invent your own text game.

6. Start with a strongly simplified variant that you then build upon.

7. Compile and test after each change, so that you always have a working program.

8. Divide your game code into several methods, and also several classes if appropriate.

9. There is a lot of information online about how to write Java code and use the JDK, e.g., at `https://stackoverflow.com/`

10. Practice using the JDK documentation here: `https://docs.oracle.com/javase/8/docs/api/`

“‘

Part IV

Solutions

Appendix L

Solutions to the exercises

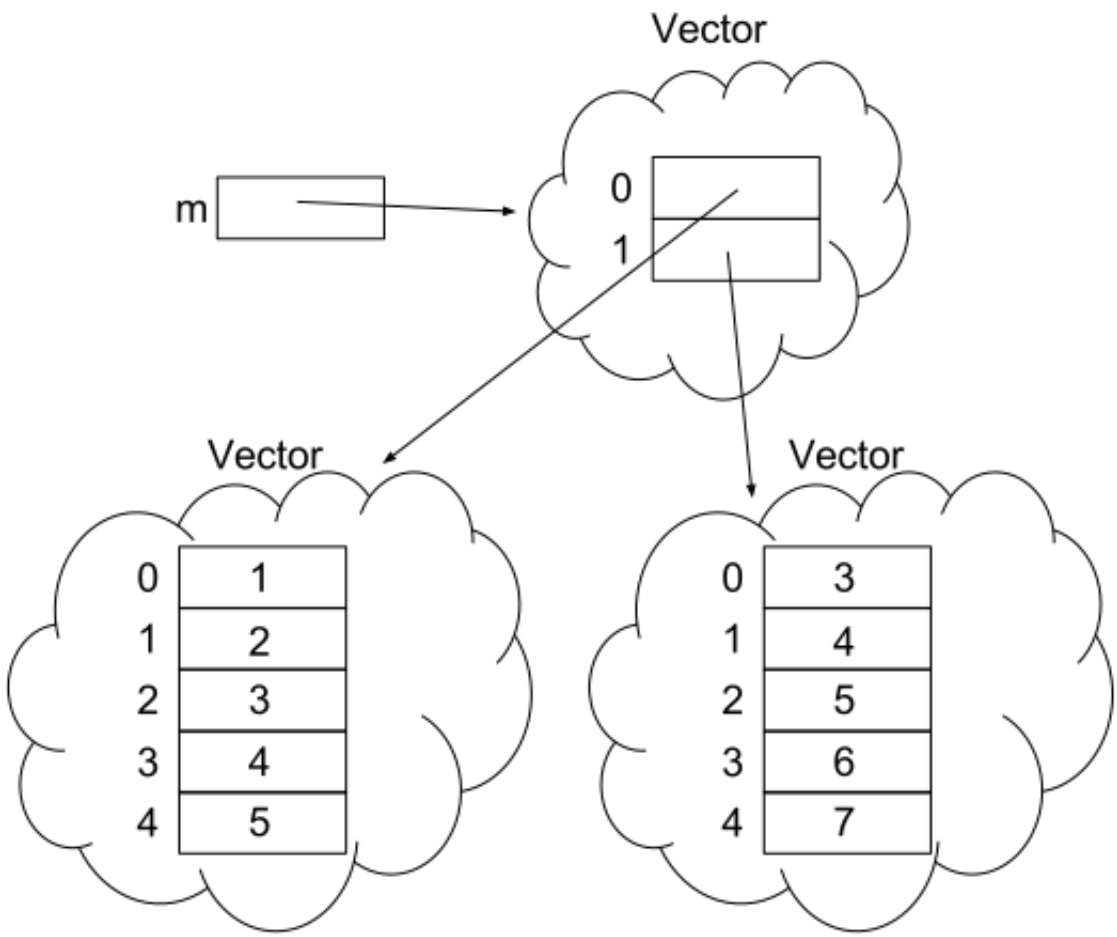
L.8 Solution `Excercise08`

L.8.1 Basic Tasks; preparation for the lab

Solution Task 1. *Match concepts with descriptions.*

matrix	1	$\rightsquigarrow$	A	indexable 2D data structure
row vector	2	$\rightsquigarrow$	F	matrix of dimension $1imesm$ with m horizontal values
column vector	3	$\rightsquigarrow$	G	matrix of dimension $mimes1$ with m vertical values
column	4	$\rightsquigarrow$	C	another word for column
generic	5	$\rightsquigarrow$	B	has abstract type parameter, type is general
type argument	6	$\rightsquigarrow$	D	concrete type, binds to type parameter at compile time
type inference	7	$\rightsquigarrow$	E	compiler deduces type from context

Solution Task 2. *Create matrices using nested collections.*



a)

Type: `Vector[Vector[Int]]`

Value: `Vector(Vector(1, 2, 3, 4, 5), Vector(3, 4, 5, 6, 7))`

Dimensions: 2×5

In mathematics, indexing is conventionally with 1 as the lowest index. In Scala, the lowest index is 0, called 0-based indexing. This is not the case in all programming languages; you can read more at https://en.wikipedia.org/wiki/Array_data_type#Index_origin.

b)

```
scala> val m = Vector((1 to 5).toVector, (3 to 7).toVector)
m: Vector[Vector[Int]] = Vector(Vector(1, 2, 3, 4, 5), Vector(3, 4, 5, 6, 7))

scala> m.apply(0).apply(1)
res4: Int = 2

scala> m(1)
res5: Vector[Int] = Vector(3, 4, 5, 6, 7)

scala> m(1)(4)
res6: Int = 7
```

c)
m2: `Vector[Vector[Int]]`
m3: `Vector[Vector[Int | Double]]`
m4: `Vector[Vector[Int | Double | String]]`
m5: `Vector[Vector[Int]]`

d) m5, 42×2

Solution Task 3. *Create and iterate over matrices.*

a)

```
def throwDie: Int = (math.random() * 6).toInt + 1
```

Or:

```
def throwDie: Int = scala.util.Random.nextInt(6) + 1
```

b) Matrix dimension in mathematical notation: 1000×5, which corresponds to a matrix with 1000 rows and 5 columns.

c)

```
ds1: IndexedSeq[IndexedSeq[Int]]
ds2: IndexedSeq[IndexedSeq[Int]]
ds3: IndexedSeq[Vector[Int]]
ds4: IndexedSeq[Vector[Int]]
ds5: Vector[Vector[Int]]
ds6: Vector[Vector[Int]]
```

Both `IndexedSeq` and `Vector` above are in the package `scala.collection.immutable`.

d)

```
def roll(n: Int) = Vector.fill(n)(throwDie).sorted
```

e)

```
def isYatzy(xs: Vector[Int]): Boolean = xs.forall(_ == xs(0))
```

f)

```
def diceMatrix(m: Int, n: Int): Vector[Vector[Int]] =
  Vector.fill(m)(roll(n))
```

g)

```
def diceMatrixToString(xss: Vector[Vector[Int]]): String =
  xss.map(_.mkString(" ")).mkString("\n")
```

h)

```
def filterYatzy(xss: Vector[Vector[Int]]): Vector[Vector[Int]] =
  xss.filter(isYatzy)
```

i)

```
def yatzyPips(xss: Vector[Vector[Int]]): Vector[Int] =
  filterYatzy(xss).map(_.head)
```

Solution Task 4. *An immutable, generic matrix class for this week's lab* `life`.

a) The type of `m` becomes `Matrix`.

b) The type of `e` becomes `String`.

c) You need to change 3 places from `String` to `Int`.

d) Generic matrix `Matrix[T]` for elements of any type `T`:

```
case class Matrix[T](data: Vector[Vector[T]]):
  def apply(row: Int, col: Int): T = data(row)(col)

object Matrix:
  def fill[T](dim: (Int, Int))(value: T): Matrix[T] =
    Matrix[T](Vector.fill(dim._1, dim._2)(value))
```

e) Thanks to compiler type inference, `bm` has type `Matrix[Boolean]`.

f) The type of `be` becomes `Boolean`.

g) h) i) j) k) are all implemented in the code below:

```
case class Matrix[T](data: Vector[Vector[T]]):
  require(data.forall(row => row.length == data(0).length))

  val dim: (Int, Int) = (data.length, data(0).length)

  def apply(row: Int, col: Int): T = data(row)(col)

  def updated(row: Int, col: Int)(value: T): Matrix[T] =
    Matrix(data.updated(row, data(row).updated(col, value)))

  def foreachIndex(f: (Int, Int) => Unit): Unit =
    for r <- data.indices; c <- data(r).indices do f(r, c)
```

```
  override def toString =
    s"""Matrix of dim $dim:\n${ data.map(_.mkString(" ")).mkString("\n") }"""

object Matrix:
  def fill[T](dim: (Int, Int))(value: T): Matrix[T] =
    Matrix[T](Vector.fill(dim._1, dim._2)(value))
```

L.8.2 Extra Tasks; additional practice

Solution Task 5. *Imperative matrix algorithms.*

a)

```
def isYatzy(xs: Vector[Int]): Boolean =
  var foundDiff = false
  var i = 0
  while (i < xs.size && !foundDiff) do
    foundDiff = xs(i) != xs(0)
    i += 1
  end while
  !foundDiff
```

b) The function goes through each matrix row, and for each it iterates over each element in the row and appends to the `StringBuilder` object. If the element is not the last in its row, a space is appended; otherwise, a newline is appended. Exception is the last row, where no newline is appended. Finally, the `StringBuilder` is converted to a `String` and returned.
If `xss` is empty, `xss.indices` is an empty `Range` and the outer for-loop is skipped, returning an empty string. If all rows are empty, the inner for-loops are skipped, with the same result.
Advantage: `StringBuilder` is faster for appending to large strings (though not noticeable here because the string is small).
Disadvantage: The `StringBuilder` code is often perceived as harder to read.

c)

```
def filterYatzy(xss: Vector[Vector[Int]]): Vector[Vector[Int]] =
  var result: Vector[Vector[Int]] = Vector()
  for i <- xss.indices if isYatzy(xss(i)) do result = result :+ xss(i)
  result
```

d) Each loop iteration yields a vector `xss(i)` if the filter condition is satisfied, resulting in a vector of vectors that are yatzy throws.

Solution Task 6. *String table with column headers.*

a)

```
case class Table(
  data: Vector[Vector[String]],
  headings: Vector[String],
  sep: Char
):

  val dim: (Int, Int) = (data.size, headings.size)

  def apply(r: Int, c: Int): String = data(r)(c)

  def row(r: Int): Vector[String]= data(r)

  def col(c: Int): Vector[String] = data.map(r => r(c))
```

```
  lazy val indexOfHeading: Map[String, Int] = headings.zipWithIndex.toMap

  def col(h: String): Vector[String] = col(indexOfHeading(h))

  def values(h: String): Vector[String] = col(h).distinct.sorted

  override def toString: String =
    val s = sep.toString
    headings.mkString(s) + "\n" +data.map(_.mkString(s)).mkString("\n")

object Table:
  def fromFile(fileName: String, sep: Char = ';'): Table =
    val lines = scala.io.Source.fromFile(fileName).getLines.toVector
    val matrix= lines.map(_.split(sep).toVector)
    new Table(matrix.tail, matrix.head, sep)
```

b)

```
@main
def run(fileName: String, separator: String): Unit =
  require(separator.length == 1, "separator ska vara exakt ett tecken")
  val t = Table.fromFile(fileName, separator.head)
  val counts: Vector[Vector[String]] =
    (0 until t.dim._2)
      .map(i => t.values(t.headings(i))
      .map(x => s"$x: ${t.col(i).count(_ == x)}"))
      .toVector
  for (i <- 0 until t.dim._2) do
    println(s"\nColumn: ${i + 1}, ${t.headings(i)}:")
    for (j <- 0 until counts(i).length) do
      println(counts(i)(j))
```

Solution Task 7. *Create a yatzy game for terminal use.*

–

L.8.3 Advanced Tasks; challenges

Solution Task 8. *Generic functions.*

a)

1. –
2. The string representation of `42` is reversed
3. `"hej"` is reversed — `toString` of a string returns the string itself
4. –
5. The string representation of the `Cucumber` object is reversed
6. The function's type parameter does not match the parameter's type: `42` is not a string
7. Implicit conversion to `Double` happens to match the type parameter, giving a string representation with decimal

b)

1. A function is defined that takes two other functions as arguments, composes them, and applies the composed function to a third argument.
2. A function that increments an integer by 1 is defined.
3. A function that halves a floating point number is defined.
4. `42` is passed to `inc()`, and the result (`43`) is passed to `half()`. Inside `half()`, implicit conversion to `Double` happens since the number is divided by a floating point number (2.0), resulting in `43.0 / 2.0`, that is, `21.5`.
5. The result from `half()` is of type `Double`, but `inc()` expects an `Int`. Since floating point numbers cannot be implicitly converted to integers without loss of information, no implicit conversion occurs, and a compilation error results.

c)

```
def inc(x: Double): Double = x + 1.0
```

Now the compilation error moves to line 4 instead, which can be fixed with the following change:

```
def half(x: Double): Double = x / 2.0
```

Solution Task 9. *Generic classes.*

a) –

b)

```
class Cell[T](var value: T):
  override def toString = "Cell(" + value + ")"
  def concat[U](that: Cell[U]): Cell[String] =
    Cell(s"$value${that.value}")
```

c) Only cells with the same type parameter can now be concatenated. Because `concat()` returns an object of type `Cell[String]`, an odd number of cells of any other type than `String` cannot be concatenated. If the number is even, they can be concatenated pairwise and then concatenate the resulting `Cell[String]` objects, but that is somewhat cumbersome.

Solution Task 10. *Implement more generic methods in `Matrix[T]`.*

```
case class Matrix[T](data: Vector[Vector[T]]):
  require(data.forall(row => row.size == data(0).size))

  val dim: (Int, Int) = (data.length, data(0).length)

  def apply(row: Int, col: Int): T = data(row)(col)

  def updated(row: Int, col: Int)(value: T): Matrix[T] =
    Matrix(data.updated(row, data(row).updated(col, value)))

  def foreach(f: T => Unit): Unit = data.foreach(_.foreach(f))

  def foreachIndex(f: (Int, Int) => Unit): Unit =
    for r <- data.indices; c <- data(r).indices do f(r, c)

  def map[U](f: T => U): Matrix[U] = Matrix(data.map(_.map(f)))

  def mapIndex[U](f: (Int, Int) => U): Matrix[U] =
    var result = Matrix.fill(dim)(f(0,0))
    for
      r <- data.indices
      c <- data(r).indices
    do
      result = result.updated(r, c)(f(r, c))
    end for
    result

  override def toString =
    s"""Matrix of dim $dim:\n${ data.map(_.mkString(" ")).mkString("\n") }"""

object Matrix:
  def fill[T](dim: (Int, Int))(value: T): Matrix[T] =
    Matrix[T](Vector.fill(dim._1, dim._2)(value))
```

L.9 Solution `Excercise09`

L.9.1 Basic Tasks; preparation for the lab

Solution Task 1. *Match concepts with description.*

set	1	$\rightsquigarrow$	H	unordered collection with unique elements
key-value table	2	$\rightsquigarrow$	F	unordered collection of mappings with unique keys
mapping	3	$\rightsquigarrow$	G	`key -> value`
key	4	$\rightsquigarrow$	C	a unique identifier
persistence	5	$\rightsquigarrow$	D	property of remaining after program ends
serialize	6	$\rightsquigarrow$	E	encode object into decodable sequence of symbols
deserialize	7	$\rightsquigarrow$	B	decode symbol sequence and recreate object in memory
linear search	8	$\rightsquigarrow$	A	search sequence until criterion met

Solution Task 2. *What is a set?* A set is a collection that can quickly answer whether a certain element belongs to the collection or not. The elements in a set are unique. Adding an already existing element is ignored. A set is not a sequence, since traversal with e.g. `map` or `foreach` does not (necessarily) happen in the order in which the elements were given when the set was constructed or updated.

Solution Task 3. *Using sets.*

`Set(1, 2) ++ Set(1, 2)`	1	$\rightsquigarrow$	I	`Set(1, 2)`
`(1 to 3).toSet`	2	$\rightsquigarrow$	G	`Set(1) + 2 + 3`
`Vector.fill(3)(1).toSet`	3	$\rightsquigarrow$	F	`Set(1, 2) - 2`
`Set(1, 2, 3) diff Set(1, 2)`	4	$\rightsquigarrow$	B	`Set(3)`
`(1 to 7).toSet.apply(8)`	5	$\rightsquigarrow$	H	**`false`**
`Set(1, 2, 3).sorted`	6	$\rightsquigarrow$	D	`error: ...`
`Set(2,4) subsetOf (1 to 7).toSet`	7	$\rightsquigarrow$	E	**`true`**
`Set(1, -1, 2, -2).map(_.abs).sum`	8	$\rightsquigarrow$	A	`3`
`Set(1, 1, 1, 1, 1, 5).sum`	9	$\rightsquigarrow$	C	`6`

Solution Task 4. *Count unique words using a set.*

a)

```
scala> val hej = "hej hej hemskt mycket hej"
scala> val n = hej.split(' ').toSet.size
n: Int = 3
```

b) The method `distinct` returns a sequence with unique elements and preserved original order.

Solution Task 5. *Creating 2-tuples with the method `->` which can be read as "maps to".*

a) Yes, the factory method returns a completely normal pair:

```
scala> val hereILive = "Skåne" -> "Lund"
val hereILive: (String, String) = (Skåne,Lund)

scala> hereILive._1
val res0: String = Skåne

scala> hereILive._2
val res1: String = Lund
```

b)

```
val capital = Vector(
  "Sverige"  -> "Stockholm",
  "Danmark"  -> "Köpenhamn",
  "Grönland" -> "Nuuk",
  "Skåne"    -> "Lund"
)
```

c)

```
scala> capital(3)._2
val res2: String = Lund
```

Solution Task 6. *Linear search for key in sequence of mappings.*

a)

```
def lookupIndex(xs: Vector[(String, String)])(key: String): Int =
  xs.indexWhere(_._1 == key)
```

b)

```
scala> val i = lookupIndex(huvudstad)("Skåne")
val i: Int = 3

scala> capital(i)._2
val res2: String = Lund
```

Or with functions as reusable partial solutions:

```
scala> val indexOf = lookupIndex(huvudstad) _

scala> def capital(key: String) = huvudstad(indexOf(key))._2

scala> capital("Skåne")
val res3: String = Lund

scala> capital("Sverige")
val res4: String = Stockholm
```

Solution Task 7. *Key-value table.*

```
scala> telnr("Fröken Ur")
```

```
val res0: Long = 464690510

scala> :type telnr
Map[String,Long]

scala> :type telnr.toVector
Vector[(String, Long)]
```

Solution Task 8. *Using key-value table.*

a) No; the key-value pairs are stored in a special order determined by an internal, clever storage principle based on a so-called hash function[1], to achieve fast lookups of values from keys, which normally does not coincide with the order of the sequence from which they were created.

b)

`xs(2) + xs(4)`	1	⇝ A	`8`
`ys(0)`	2	⇝ C	`(10, 11)`
`xs(0)`	3	⇝ I	`NoSuchElementException`
`(xs + (0 -> 1)).apply(0)`	4	⇝ D	`1`
`xs.keySet.apply(2)`	5	⇝ G	`true`
`xs isDefinedAt 0`	6	⇝ H	`false`
`xs.getOrElse(0, 7)`	7	⇝ B	`7`
`xs.maxBy(_._2)`	8	⇝ E	`(16, 17)`
`xs.map(p => p._1 -> -p._2)(8)`	9	⇝ F	`-9`

Solution Task 9. *Registration in mutable key-value table.*

```
class FreqMapBuilder:
  private val register = scala.collection.mutable.Map.empty[String,Int]
  def toMap: Map[String, Int] = register.toMap
  def add(s: String): Unit =
    register.addOne(s -> (register.getOrElse(s, 0) + 1))

object FreqMapBuilder:
  def apply(xs: String*): FreqMapBuilder =
    val result = new FreqMapBuilder
    xs.foreach(result.add)
    result
```

Solution Task 10. *The method `sliding`.*

a)

```
scala> val xs = Vector("five", "cucumbers", "is", "more", "than", "four", "toma
val xs: Vector[String] =
  Vector(five, cucumbers, is, more, thane, four, tomatoes)

```

[1] https://sv.wikipedia.org/wiki/Hashfunktion

```
scala> xs.sliding(2).toVector
val res9: Vector[Vector[String]] =
  Vector(Vector(five, cucumbers), Vector(cucumbers, is), Vector(is, more), Vect

scala> xs.sliding(3).toVector
val res10: Vector[Vector[String]] =
  Vector(Vector(five, cucumbers, is), Vector(cucumbers, is, more), Vector(is, m

scala> xs.sliding(10).toVector
val res11: Vector[Vector[String]] =
  Vector(Vector(five, cucumbers, is, more, than, four, tomatoes))
```

`xs.sliding(n).toVector` creates a sequence that contains sequences of length `n` created by taking each element and its `n - 1` following elements.

b)

```
scala> xs.sliding(2).map(ys => ys(0) -> ys(1)).toMap
val res0: Map[String,String] =
  Map(is -> more,
      than -> four,
      four -> tomatoes,
      cucumbers -> is,
      five -> cucumbers,
      more -> than
  )
```

The table can be used to look up the following word. It works because all words are unique. If there had been multiple identical words with different following words, we would need to create a table with keys that map to a collection registering the following words. This we will do in this week's lab.

Solution Task 11. *Reading text from file and web servers.*

a)

```
val populationOf = data.tail.map(v => v(0) -> v(1).toInt).toMap
val sizeOf       = data.tail.map(v => v(0) -> v(2).toInt).toMap
val capitalOf    = data.tail.map(v => v(0) -> v(3)).toMap
```

```
scala> capitalOf("Sverige")
res2: String = Stockholm

scala> populationOf("Sverige")
res3: Int = 9223766

scala> sizeOf("Sverige")
res4: Int = 449964
```

```
scala> val filename = "europa.txt"
scala> val xs = io.Source.fromFile(filename, "UTF-8").getLines.toVector
scala> val data = xs.map(_.split(';').toVector)
scala> data.map(_.map(_.take(15).padTo(15,' ')).mkString(" ")).foreach(println)
```

L.9.2 Extra Tasks; additional practice

Solution Task 12. –

L.9.3 Advanced Tasks; challenges

Solution Task 13. *Registration with `groupBy`.*

a) The method `groupBy` creates a key-value table where the values in the table are a sequence with the elements grouped in a special way. More precisely:
The result of `xs.groupBy(f: K => V)` for a sequence `xs` of type `Vector[K]` is a table of type `Map[V,Vector[K]]` where each element `e` in `xs` is grouped in the same table value if they are equal according to `f(e)`. Each group gets the table key `f(e)`.
Clever trick: If you let the function `f` be the identity function that maps each element onto itself, i.e. `x => x`, then the values are grouped in the same sequence if they are equal.

```
scala> val xs = Vector(1, 1, 2, 2, 4, 4, 4).groupBy(x => x > 2)
val xs: Map[Boolean,Vector[Int]] =
  Map(false -> Vector(1, 1, 2, 2), true -> Vector(4, 4, 4))

scala> val ys = Vector(1, 1, 2, 2, 4, 4, 4).groupBy(x => x)
val ys: Map[Int,Vector[Int]] =
  Map(2 -> Vector(2, 2), 4 -> Vector(4, 4, 4), 1 -> Vector(1, 1))
```

b)

```
def freq(xs: Vector[Int]): Map[Int, Int] =
  xs.groupBy(x => x).map(p => p._1 -> p._2.size)
```

Explanation: the method `groupBy` creates a table with pairs `k, v` where `v` is a sequence with as many `k` as the number of times `k` occurs in `xs`. By converting all values `p._2` to the size `p._2.size` we get a frequency table.

```
scala> freq(toss(1000))
val res0: Map[Int,Int] =
  Map(5 -> 163, 1 -> 174, 6 -> 161, 2 -> 169, 3 -> 167, 4 -> 166)

scala> freq(toss(1000)).toVector.sortBy(_._1).foreach(println)
(1,183)
(2,167)
(3,169)
(4,179)
(5,154)
(6,148)
```

c)

```
shoppinglist.groupMap(_.department)(_.item)
```

d)

```
shoppinglist.groupMapReduce(_.department)(_.price)(_ + _)
```

Solution Task 14. –

L.10 Solution Excercise10

L.10.1 Basic Tasks; preparation for the lab

Solution Task 1. *Match concepts with descriptions.*

base type	1	⇝	N	most general type in an inheritance hierarchy
super type	2	⇝	O	a type that is more general
subtype	3	⇝	D	a type that is more specific
runtime type	4	⇝	J	may be more specific than static type
dynamic binding	5	⇝	L	runtime type determines which method executes
polymorphism	6	⇝	B	can have many forms, e.g., one of several subtypes
trait	7	⇝	I	abstract, can be mixed in, can have parameters
mix-in	8	⇝	H	add properties with `with` and a trait
overridden member	9	⇝	M	subtype member replaces supertype member
anonymous class	10	⇝	C	class without name, extended with extra implementation
protected member	11	⇝	K	visible only in subtypes
abstract member	12	⇝	F	has no implementation
abstract class	13	⇝	E	can have parameters, cannot be instantiated, cannot be mixed i
sealed type	14	⇝	P	subtyping outside this file is prevented
reference type	15	⇝	A	has supertype `AnyRef`, allocated on heap via reference
value type	16	⇝	G	has supertype `AnyVal`, stored directly on stack

Solution Task 2. *Common base type.*

a) `Vector[Object]`. The type `Object` in the JVM corresponds to the type `AnyRef` which is the base type for all reference types.

b) Error message:

```
scala> vegetables.map(_.weight).sum
-- Error:
1 |vegetables.map(_.weight).sum
  |               ^^^^^^
  |              value weight is not a member of Object - did you mean wait?
-- Error:
1 |vegetables.map(_.weight).sum
  |                          ^
  |ambiguous implicit arguments: both object DoubleIsFractional in object Nume
```

The first error is because the elements of the vector are of type `Object` and the member `weight` is not defined for this type. The second error is a follow-up error caused by the fact that a sequence with elements of type `Object` cannot be summed because the compiler cannot infer that the element type is numeric.

c) The attribute `weight` is initialized during the construction of `Cucumber` respectively `Tomat`. The value is given by the respective class parameter.

d) `Vector[Vegetable]`.

e) Yes. Since the static type of the elements in the sequence is `Vegetable` (the dynamic type can be any subtype of `Vegetable`) and all instances of this type are

guaranteed to have the attribute `weight` of type `Int`, the compiler can at *compile time* conclude that summing is valid and thus can compile the code to executable machine code.

f)

```
scala> new Vegetable
-- Error:
1 |new Vegetable
  |    ^^^^^^^
  |    Vegetable is a trait; it cannot be instantiated
```

g)

```
scala> val anonymousVegetable = new Vegetable { val weight = 42 }
val anonymousVegetable: Vegetable = anon$1@1edde8b6
scala> anonymousVegetable.toString
val res0: String = anon$1@1edde8b6
```

The type is `Vegetable` and here becomes a so-called *anonymous class*, since we did not use a named class with **`extends`**, but only "attached" a class body within braces directly during construction. When creating anonymous classes you must use the keyword **`new`**.
The compiler invents a unique class name, here anon$1, to keep track of the anonymous class during compilation to machine code. The string representation contains a hexadecimal number that is unique for the instance, here `1edde8b6`.

h)

```
scala> new Vegetable { }
-- Error:
1 |new Vegetable { }
  |^
  |object creation impossible, since val weight: Int in trait Vegetable is not defined
```

Solution Task 3. *Polymorphism in inheritance, i.e. subtype polymorphism.*

a)

```
def createAnimal(): Animal =
  if math.random() > 0.5 then Cow() else Pig()
```

b)

```
class Horse extends Animal:
  def makeNoise = println("Neigh")

def createAnimal(): Animal =
   math.random() match
    case r if r < 0.33 => Cow()
    case r if r < 0.67 => Pig()
    case _             => Horse()
```

Solution Task 4. *Various types of integer pairs for the lab `snake0`.*

a)

```
trait Pair[T]:
  def x: T
  def y: T
  def tuple: (T, T) = (x, y)
```

b)

- Does the above code also work without keyword **override**? Why?

  Yes, it works because **override** is not required when implementing inherited *abstract* members in a subtype. Abstract members lack implementation and there is nothing to override.

- When must **override** be used?

  It is required if you want to give an inherited member a *different* implementation in the subtype, if this member already has an implementation in the supertype. Your new implementation overrides (replaces) the inherited member's implementation.

- What are the advantages and disadvantages of using **override** even when not needed?

  Advantage: the compiler helps check that you are actually implementing an inherited member and not e.g. misspelled the member's name.

  Disadvantage: more to type and thus longer to read.

c)

```
case class Dim(x: Int, y: Int) extends Pair[Int]
object Dim:
  def apply(dim: (Int, Int)): Dim = Dim(dim._1, dim._2)
```

d)

```
case class Pos private (x: Int, y: Int, dim: Dim) extends Pair[Int]:
  def +(p: Pair[Int]): Pos = Pos(x + p.x, y + p.y, dim)
  def -(p: Pair[Int]): Pos = Pos(x - p.x, y - p.y, dim)

object Pos:
  def apply(x: Int, y: Int, dim: Dim): Pos =
    import java.lang.Math.floorMod as mod
    new Pos(mod(x, dim.x), mod(y, dim.y), dim) //NOTE: new is necessary here!

  def random(dim: Dim): Pos =
    import scala.util.Random.nextInt as rni
    Pos(rni(dim.x), rni(dim.y), dim)
```

e) If you forget to write **new** explicitly in the companion object's `apply` method, it becomes a recursive call resulting in an infinite loop at runtime. With **new**, it is guaranteed that the private primary constructor for `Pos` is called.
In `Dim.apply` the parameter types differ between the factory method and the primary constructor, and the compiler chooses the primary constructor since it matches the given two separate integers and not a 2-tuple.

f)

```
enum Dir(val x: Int, val y: Int) extends Pair[Int]:
  case North extends Dir( 0, -1)
  case South extends Dir( 0,  1)
  case East  extends Dir( 1,  0)
  case West  extends Dir(-1,  0)
export Dir.*  // makes North etc visible within package snake
```

Solution Task 5. *Supertype with parameter.*

a)

```
val person = new Person("Person1")
val academic = new Academic("Person2", "LTH")
val student = new Student("Person3", "LTH", "D")
val researcher = new Researcher("Person4", "LTH", "PhD Student")
```

b)

```
val vec = Vector(person, academic, student, researcher)
for(i <- vec){ print(i.toString + i.name) }
```

c) Error message when instantiating `abstract class Academic`: `Academic is abstract; it cannot be instantiated` It does *not* work equally well with a `trait` in the special case of `Academic`, since a trait may not forward parameters to a supertype. Error message: `trait Academic may not call constructor of trait Person`

```
trait Person(val name: String)
abstract class Academic(
name: String,
val university: String) extends Person(name)
class Student(
name: String,
university: String,
program: String) extends Academic(name, university)
class Researcher(
name: String,
university: String,
title: String) extends Academic(name, university)
```

d)

```
scala>
|trait Person(val name: String)
|
| abstract class Academic(
| name: String,
| val university: String) extends Person(name)
|
| case class Student(
| name: String,
| university: String,
| program: String) extends Academic(name, university)
```

```
|
| case class Researcher(
| name: String,
| university: String,
| title: String) extends Academic(name, university)
-- Error:
8 | name: String,
| ^
| error overriding value name in trait Person of type String;
| value name of type String needs override modifier
-- Error:
9 | university: String,
| ^
| error overriding value university in class Academic of type String;
| value university of type String needs override modifier
-- Error:
13 | name: String,
| ^
| error overriding value name in trait Person of type String;
| value name of type String needs override modifier
-- Error:
14 | university: String,
| ^
| error overriding value university in class Academic of type String;
| value university of type String needs override modifier
```

```scala
trait Person(val name: String)
abstract class Academic(
  name: String,
  val university: String) extends Person(name)
case class Student(
  override val name: String,
  override val university: String,
  program: String) extends Academic(name, university)
case class Researcher(
  override val name: String,
  override val university: String,
  title: String) extends Academic(name, university)
```

```
scala> val ps = Vector(Student("Kim", "Lund", "CS"), Researcher("Herz", "Lund", "Dr"))
val ps: Vector[Academic] = Vector(Student(Kim,Lund,CS), Researcher(Herz,Lund,Dr))
scala> ps :+ new Person("Abstract") {}
val res0: Vector[Person] =
  Vector(Student(Kim,Lund,CS), Researcher(Herz,Lund,Dr), anon1@1941bbf3)
```

e)

```scala
trait Person:
  val name: String
  val nbr: Long
trait Academic extends Person:
  val university: String
case class Student(
  name: String,
```

```
  nbr: Long,
  university: String,
  program: String) extends Academic
case class Researcher(
  name: String,
  nbr: Long,
  university: String,
  title: String) extends Academic
case class NonAcademic(
  name: String,
  nbr: Long) extends Person
```

L.10.2 Extra Tasks; additional practice

Solution Task 6. *Base type* `Shape` *and subtypes* `Rectangle` *and* `Circle`.

a)

```
val c1 = Circle(Point(1, 1), 42)
val r1 = Rectangle(Point(3, 3), 20, 30)
c1.move(2, 3)
r1.move(3, 2)
```

b)

```
case class Point(x: Double, y: Double):
  def move(dx: Double, dy: Double): Point = Point(x + dx, y + dy)
  def moveTo(x: Double, y: Double): Point = Point(x, y)

trait Shape:
  def pos: Point
  def move(dx: Double, dy: Double): Shape
  def moveTo(x: Double, y: Double): Shape

case class Rectangle(pos: Point, width: Double, height: Double) extends Shape:
  def move(dx: Double, dy: Double): Shape = copy(pos = pos.move(dx, dy))
  def moveTo(x: Double, y: Double): Shape = copy(pos.moveTo(x, y))

case class Circle(pos: Point, radius: Double) extends Shape:
  def move(dx: Double, dy: Double): Shape = copy(pos = pos.move(dx, dy))
  def moveTo(x: Double, y: Double): Shape = copy(pos.moveTo(x, y))
```

c) `def distanceTo(that: Point): Double = math.hypot(that.x - x, that.y - y)`

d) `def distanceTo(that: Shape): Double = pos.distanceTo(that.pos)`.

e)

```
case class Point(x: Double, y: Double):
  def move(dx: Double, dy: Double): Point = Point(x + dx, y + dy)
  def moveTo(x: Double, y: Double): Point = Point(x, y)
  infix def distanceTo(that: Point): Double = math.hypot(that.x - x, that.y - y)

trait Shape:
  def pos: Point
  def move(dx: Double, dy: Double): Shape
```

```
  def moveTo(x: Double, y: Double): Shape
  infix def distanceTo(that: Shape): Double = pos.distanceTo(that.pos)

case class Rectangle(pos: Point, width: Double, height: Double) extends Shape:
  def move(dx: Double, dy: Double): Shape = copy(pos = pos.move(dx, dy))
  def moveTo(x: Double, y: Double): Shape = copy(pos.moveTo(x, y))

case class Circle(pos: Point, radius: Double) extends Shape:
  def move(dx: Double, dy: Double): Shape = copy(pos = pos.move(dx, dy))
  def moveTo(x: Double, y: Double): Shape = copy(pos.moveTo(x, y))
```

L.10.3 Advanced Tasks; challenges

Solution Task 7. *Mixins.*

a) There are many ways, here are some example solutions:

```
val numberOfFlyingDucks: Int =
  flock.count(f => f.isInstanceOf[Duck] && f.isAbleToFly)
```

```
val numberOfFlyingDucks: Int =
  flock.filter(f => f.isInstanceOf[Duck] && f.isAbleToFly).size
```

```
val numberOfFlyingDucks: Int =
  flock.collect{case f: Duck if f.isAbleToFly}.size
```

```
val numberOfFlyingDucks: Int = flock.map(_ match
  case f: Duck if f.isAbleToFly => 1
  case _ => 0
).sum
```

b)

```
val numberOfCrows: Int = flock.filter(f => !f.isAbleToSwim).size * 2
val numberOfDucks: Int = flock.filter(f => f.isAbleToSwim).size * 4
```

Solution Task 8. *Final classes.*

a) Put `final` before `class` in the classes.

b) error: illegal inheritance from final class Crow.

Solution Task 9. *Access rules in inheritance and the keyword* `protected`.

a)

```
2 |  def reveal = myHome
  |                ^^^^^^^^^
  |                Not found: myHome
```

b)

```
scala> class Sub extends Super:
         def cryptic = ourHome * math.Pi
scala> (new Sub).ourHome
-- Error:
1 |(new Sub).ourHome
  |^^^^^^^^^^^^^^^^^^
  |value ourHome in trait Super cannot be accessed as a member of Sub.
  | Access to protected value ourHome not permitted because enclosing object
  | is not a subclass of trait Super where target is defined
```

c) Yes.

Solution Task 10. *Using* ***protected****.*

a) In Bird:

```
protected var soundCount: Int = 0
def makeNoise: Unit = { print(sound * 2); soundCount += 2 }
```

In Duck: `override def makeNoise = { print(sound * 4); soundCount += 4 }`

b) `def numberOfSounds: Int = soundCount`

c) If a class representing a bird that would produce more/fewer sounds than a typical `Bird` needs to change `makeNoise`, that method needs access to `soundCount`, which cannot be **`private`**.

d) The counter variable should not be modifiable from any other part of the program.

Solution Task 11. *Mixin of properties.*

```
trait Bird:
  val sound: String
  def makeNoise: Unit = { print(sound * 2); soundCount += 2 }
  protected var soundCount: Int = 0
  val canSwim: Boolean
  val canFly: Boolean
  val isBig: Boolean
  def numberOfSounds: Int = soundCount

trait CanSwim { val canSwim = true }
trait CannotSwim { val canSwim = false }
trait CanFly { val canFly = true }
trait MaybeCanFly { val canFly = math.random() < 0.8 }
trait MaybeCanSwim { val canSwim = math.random() < 0.2 }
trait IsBig { val isBig = true }
trait IsSmall { val isBig = false }

final class Crow extends Bird, CanFly, CannotSwim, IsBig:
  val sound = "caw"

final class Duck extends Bird, CanSwim, MaybeCanFly, IsBig:
  val sound = "quack"
  override def makeNoise = { print(sound * 4); soundCount += 4 }
```

```
final class Chick extends Bird, CanFly, MaybeCanSwim, IsSmall:
  val sound = "chirp"
  override def makeNoise = { print(sound * 8); soundCount += 8 }
```

In REPL:

```
val birds = Vector.fill(42)(
  if math.random() < 0.33 then Crow()
  else if math.random() < 0.5 then Duck()
  else Chick()
)
fyle.filter(f => f.isInstanceOf[Crow]).size * 2
fyle.filter(f => f.isInstanceOf[Duck]).size * 4
fyle.filter(f => f.isInstanceOf[Chick]).size * 8
```

L.11 Solution `Excercise11`

L.11.1 Basic Tasks; preparation for the lab

Solution Task 1. *Context parameter.*

a)

```
scala> given Int = 0
val res0: Int = 83

scala> f(using 41)
val res1: Int = 42

scala> g(41)(using 42)
val res2: Int = 83
```

If you forget **using** when giving an explicit context argument, it results in a compilation error. The compiler becomes "confused" and thinks you are trying to provide a "regular" argument to a (in this case) nonexistent regular parameter list.

```
scala> f(41)
-- [E050] Type Error: ----------------------------------------------
1 |f(41)
  |^
  |method f does not take more parameters
  |
  | longer explanation available when compiling with `-explain`
1 error found

scala> :setting -explain

scala> f(41)
-- [E050] Type Error: ----------------------------------------------
1 |f(41)
  |^
  |method f does not take more parameters
  |----------------------------------------------------------------
    | Explanation (enabled by `-explain`)
  |- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
  | You have specified more parameter lists than defined in the
    method definition(s).
   ----------------------------------------------------------------

scala> g(41)(42)
-- [E050] Type Error: ----------------------------------------------
1 |g(41)(42)
  |^^^^^
  |method g does not take more parameters
  |----------------------------------------------------------------
    | Explanation (enabled by `-explain`)
  |- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
  | You have specified more parameter lists than defined in the
    method definition(s).
   ----------------------------------------------------------------
```

It is uncommon to specify **using** parameters explicitly; it is more common to let the compiler summon a given value.

b) The given value `0` is bound to the corresponding context parameter, which should be declared in a separate parameter list starting with **`using`**.

```
scala> f
val res3: Int = 1

scala> g(42)
val res4: Int = 42
```

c) No, it results in a compilation error if you try to mix regular and context parameters in the same parameter list:

```
scala> def h(i: Int, using j: Int) = i + j
-- [E040] Syntax Error: ----------------------------------------------
1 |def h(i: Int, using j: Int) = i + j
  |                     ^
  |                     ':' expected, but identifier found
```

It is a deliberate design choice to require separate parameter lists so that there is no confusion about whether a regular parameter or context parameter is meant.

Solution Task 2. *Multiple given values in local context.*

a) 2

b) 43

c) When the compiler tries to summon a given value to automatically use as argument to the **`using`** parameter `dx`, it searches first in the context nearest to the call. If a given value exists in the companion object for the parameter type, the compiler takes this as a last resort if no other given value is found closer to the call.

Solution Task 3. *Solution to the configuration problem using given values.*

Below is a test of the three different solutions given in the task. a)

After each test we discuss associated pros and cons, as requested in the task. b)

```
def testGlobalVar(useDefault: Boolean = true) =
  import GlobalVar.*
  if useDefault then println(greetMsg) else
    GreetConfig.config = GreetConfig("Good morning", "world")
    println(greetMsg)
```

Since `config` here is a mutable variable, a change in one place can affect completely different parts of the program, which sometimes can be an advantage, but often a disadvantage because it can be hard to understand what is happening by reading only one part of the program – a change of `config` can occur anywhere. It is easy to forget to change back to the default value if that is what is expected.

```
scala> testGlobalVar(); testGlobalVar(false); testGlobalVar()
Hello World!
Good morning world!
Good morning world!
```

```
def testDefaultArgs(useDefault: Boolean = true) =
  import DefaultArgs.*
```

```
3   if useDefault then println(greetMsg()) else
4     println(greetMsg(GreetConfig("Godd morning","world")))
```

Here there is no state change and default usage is simple, but it is not easy to deviate from the default that applies in a local context; at *each* individual call you have to explicitly supply all the arguments that should not be default, as shown on line 4 above. Changing the default has only local impact. If all arguments are to follow the default, then you must not forget to call with empty parentheses: `greetMsg()`. (What happens otherwise?)

```
scala> testDefaultArgs(); testDefaultArgs(false); testDefaultArgs()
Hello World!
Good morning world!
Hello World!
```

With context parameters the flexibility is greater; **using** parameters let the user control what applies in different contexts and the call becomes simple regardless of whether the default value or other, locally given values are desired. Changing default has only local impact, but this affects arbitrarily many calls in the local context – arguments that differ can be given once without needing to be repeated at each call. For calls where you want the compiler to summon given values for the context parameter no parentheses should be used, making the calls short and simple.

```
def testGivenVal(using g: GivenVal.GreetConfig) = println(g.greetMsg)
```

```
scala> testGivenVal
Hello World

scala> def localContext =
         import GivenVal.*
         given GreetConfig = GreetConfig("Good morning","world")
         testGivenVal

scala> localContext
Good morning world

scala> testGivenVal
Hello World
```

c) The compiler summons a given value in the local context:

```
scala> summon[GivenVal.GreetConfig]
val res0: GivenVal.GreetConfig = GreetConfig(Hello,World)
```

The compiler follows this priority order when searching for a unique given value:

1. **Explicit** arguments to context parameters marked with **using**
2. **given** and **import given** ... in the current scope
3. **given** values in the **companion objects** for the used type.

If multiple given values can be summoned for types in a common inheritance hierarchy, the compiler chooses the given value of the *most general* type.

d) It results in a compilation error if the compiler does not find a given value for the type in question.

```
scala> summon[Long]
-- Error: ----------------------------------------------------------------
1 |summon[Long]
  |            ^
  |            no given instance of type Long was found for parameter x of
                 method summon in object Predef
1 error found
```

e) Yes! It must *not* be ambiguous which given value should be summoned:

```
scala> def ambiguous =
     |   given a: Int = 42
     |   given b: Int = 43
     |   summon[Int]
     |
-- Error: ----------------------------------------------------------------
4 |  summon[Int]
  |             ^
  |ambiguous given instances: both given instance b and given instance a
  |match type Int of parameter x of method summon in object Predef
1 error found
```

Read more about contextual abstractions here:
https://docs.scala-lang.org/scala3/reference/contextual/

L.11.2 Extra Tasks; additional practice

Solution Task 4. *Context parameter and given value.*

a)

```
scala> add(1)
-- Error: ----------------------------------------------------------------
1 |add(1)
  |      ^
  |    no given instance of type Int was found for parameter y of method add
1 error found
```

b) Now there is a given value that the compiler can automatically fill in at the call site.

c) `def sub(x: Int)(using y: Int) = x - y`

L.11.3 Advanced Tasks; challenges

Solution Task 5. *Variance and type bounds.*

a) Make the box flexible in its type parameter with a + before the type parameter as below.

```
case class Box[+A](x: A)
```

The compiler applies the rules for covariance because the type parameter has a plus sign: `Box[Cat]` is a subtype of `Box[Any]` if `Cat` is a subtype of `Any`, which it is since all types are subtypes of `Any`.

b) The explanation for behavior concerns different variance:

- The collection `Vector` is covariant and thus flexible in its type parameter (as are other immutable sequences in Scala's standard library). The compiler thus considers `Vector[Cat]` as a subtype of `Vector[Pet]` since `Cat` is a subtype of `Pet`. Where a `Vector[Pet]` is expected, `Vector[Cat]` conforms and is accepted.

- A set has an apply method from element type to `Boolean` which checks membership. Hence `Set[T]` inherits `Function1[T, Boolean]` which is declared contravariant in `T`, allowing a set to be used where a function `T => Boolean` is expected. Even though it would be practical if `Set[T]` were covariant like `Vector`, `List`, `Seq` etc., a type parameter cannot be both covariant and contravariant at the same time. Therefore `Set` is invariant and sets are not flexible in their type parameter. Thus `Set[Cat]` is *not* a subtype of `Set[Pet]` even though `Cat` is a subtype of `Pet`, which causes the compilation error in the example. See also https://stackoverflow.com/questions/676615/why-is-scalas-immutable-set-not-covariant-in-its-type

- Using `:settings -explain` the compiler outputs a longer explanation clarifying the proof performed during compilation's type checking.

c) It is a compilation error because the method `isHealthy` does not exist for arbitrary types.

d) Add an upper bound to ensure the method `isHealthy` exists for all types bound to the type parameter `A`:

```
class Vet[-A <: Pet]:
  def treat(x: A): Unit = x.isHealthy = true
```

The compiler guarantees that the type parameter `A` is "less than or equal to" `Pet`.

e) The veterinarian `Vet` is flexible in its type parameter and the minus sign indicates contravariance, meaning `Vet[Pet]` is a subtype of `Vet[Cat]` since `Cat` is a subtype of `Pet`. This can be demonstrated with the example below:

```
scala> val pinkPanther = Cat()
val pinkPanther: Cat = Cat@33e7ece5

scala> val somePet: Pet = Cat()
val somePet: Pet = Cat@57f1cb96

scala> val catVet = Vet[Cat]()
val catVet: Vet[Cat] = Vet@1060e784

scala> pinkPanther.isHealthy = false

scala> catVet.treat(pinkPanther)

scala> pinkPanther.isHealthy
val res2: Boolean = true

scala> somePet.isHealthy = false

scala> catVet.treat(somePet)
-- [E007] Type Mismatch Error: --------------
1 |catVet.treat(somePet)
  |             ^^^^^^^
```

```
  |             Found:    (somePet : Pet)
  |             Required: Cat

scala> val powerVet = Vet[Pet]()
val powerVet: Vet[Pet] = Vet@2eb90ae9

scala> pinkPanther.isHealthy = false

scala> powerVet.treat(pinkPanther)

scala> pinkPanther.isHealthy
val res3: Boolean = true

scala> val pluto = Dog()
val pluto: Dog = Dog@6f27db5d

scala> pluto.isHealthy = false

scala> powerVet.treat(pluto)

scala> pluto.isHealthy
val res4: Boolean = true
```

Solution Task 6. *Type classes and context parameters.*

a)

- First we declare a **trait**, `CanCompare`, with a generic type parameter `T`. It contains an abstract method `compare` that takes two parameters of type `T` and returns an `Int`.
- Then a method `sort` is defined, also with a generic type parameter `T`. This method takes two parameters, `a` and `b` of type `T`, and a **using** parameter `cc` which must be an instance of `CanCompare[T]`. Inside the method, the `compare` method from `CanCompare` is used to determine if `a` and `b` should be swapped.
- When we try to run `sort(42, 41)` we get a compiler error because there is no given instance of `CanCompare[Int]`.
- We fix this by defining **given** `intComparator` of type `CanCompare[Int]` overriding the `compare` method. Now the following call `sort(42, 41)` works and returns `(Int, Int) = (41, 42)`.
- When we try `sort` with `Double` arguments, we get a similar error as before because no `CanCompare` for `Double` exists.

b)

```
scala> given doubleComparator: CanCompare[Double] with
         override def compare(a: Double, b: Double): Int = (a - b).toInt
```

c)

```
scala> given stringComparator: CanCompare[String] with
         override def compare(a: String, b: String): Int = a.compareTo(b)
```

Solution Task 7. *Usage of given ordering.*

a) Example tests:

```
scala> isSorted(Vector(1,2,3))
val res0: Boolean = true

scala> isSorted(Vector(1,2,4,3))
val res1: Boolean = false

scala> isSorted(List(1,2,3))
val res2: Boolean = true

scala> isSorted(List(1,2,4,3))
val res3: Boolean = false
```

b)

```
scala> def isSorted[T](xs: Seq[T]): Boolean = xs == xs.sorted
-- [E172] Type Error: -----------------------------------------------------------
1 |def isSorted[T](xs: Seq[T]): Boolean = xs == xs.sorted
  |                                                     ^
  | No implicit Ordering defined for T..
  | I found:
  |
  |     scala.math.Ordering.ordered[T](/* missing */summon[scala.math.Ordering.
  |
  | But no implicit values were found that match type scala.math.Ordering.AsCon
  |
  | The following import might make progress towards fixing the problem:
  |
  |     import scala.math.Ordered.orderingToOrdered
  |
1 error found
```

The error means the compiler cannot derive an ordering for the generic type `T`.

c)

```
scala> isSorted(persons)
-- [E172] Type Error: -----------------------------------------------------------
1 |isSorted(persons)
  |                 ^
  | No implicit Ordering defined for Person..
  | I found:
  |
  |     scala.math.Ordering.ordered[Person](/* missing */summon[scala.math.Orde
  |
  | But no implicit values were found that match type scala.math.Ordering.AsCon
  |
  | The following import might make progress towards fixing the problem:
  |
  |     import scala.math.Ordered.orderingToOrdered
  |
1 error found
```

The error occurs because the compiler does not know any ordering for the `Person` class.

d) -

e) -

Solution Task 8. *Create your own implicit ordering with `Ordering`.*

a)

```
scala> xs.sorted
-- [E172] Type Error: -----------------------------------------------------
1 |xs.sorted
  |         ^
  | No implicit Ordering defined for Team..
  | I found:
  |
  |     scala.math.Ordering.comparatorToOrdering[Team](/* missing */summon[java
  |
  | But no implicit values were found that match type java.util.Comparator[Tean
  |
  | One of the following imports might make progress towards fixing the problem
  |
  |     import scala.math.Ordering.comparatorToOrdering
  |     import scala.math.Ordering.ordered
  |
1 error found
```

The error appears because the compiler doesn't know any ordering for `Team`.
After assigning `given Ordering[Team] = Ordering.by(t => t.rank)` there exists a given ordering for `Team` in the current context. Therefore the following call `xs.sorted` works as intended.

b) -

c) -

d) -

e)

```
scala> Team("fnatic",1499) < Team("gurka", 2)
-- [E008] Not Found Error: ------------------------------------------------
1 |Team("fnatic",1499) < Team("gurka", 2)
  |^^^^^^^^^^^^^^^^^^^^
  |value < is not a member of Team, but could be made available as an extension
  |
  |One of the following imports might fix the problem:
  |
  |  import scala.math.Ordered.orderingToOrdered
  |  import scala.math.Ordering.Implicits.infixOrderingOps
  |
1 error found
```

As the compiler says, `<` is not a method defined for `Team`. However, after `import Team.highestRankFir` the implicit conversions defined for `highestRankFirst: Ordering[Team]` (because it is an object of type `Ordering`) are imported. Thus, `<` works. See task 7e for more background.

Solution Task 9. *Custom ordering by inheriting from `Ordered`*

a)

```
case class Team(name: String, rank: Int) extends Ordered[Team]:
```

```
  override def compare(that: Team): Int = -rank.compare(that.rank)
```

b)

```
scala> Team("fnatic",1499) < Team("gurka", 2)
val res1: Boolean = true
```

c) Subtype polymorphism, i.e. using inheritance and interfaces builds on each subtype implementing certain members. The compiler can then guarantee that *something* will happen at calls with a subtype. However, it cannot guarantee *what* will happen, which is decided at runtime by dynamic binding.
Ad hoc polymorphism on the other hand means that the called function behaves differently for different types. This uses some form of type classes. It is the type given at the call that determines what happens, not which behavior is defined in the subtype the type "happens" to have. By compile time the compiler can thus determine what will happen based on the type, i.e. static binding. This makes it easier to reason about how the program behaves.

L.12 Solution Excercise12

L.12.1 Tasks on searching and sorting

Solution Task 1. *Timing measurement.*

a) Execute the code and you should find that it takes longer to find the value 1 in our Set s than in the vector v.

b)
A vector has a sequential order that `find` can use, whereas a `Set` is internally organized differently for very fast membership checks. The reason that it takes time for `find` on `Set` is that an iterator must first be created before the set can be traversed from start to end. The method `contains` on `Set` is lightning fast due to the internal structure of objects of type `Set` (which is smartly designed with so-called hash codes, allowing constant time lookup regardless of element position).

Solution Task 2. *Searching with built-in search methods.*

a) Suggested test of `indexOfSlice`:

```
scala> List(1,2,3,35,1,23).indexOfSlice(List(35,1,23))
res73: Int = 3
scala> List(1,2,3,35,1,23).indexOfSlice(List(35,1,3))
res74: Int = -1
```

b) Suggested test of `lastIndexOfSlice`:

```
Vector(1,2,3,4,1,2).lastIndexOfSlice(Vector(1,2))
res2: Int = 4
Vector("apa", "banan", "majs", "banan").lastIndexOfSlice(Vector("banan"))
res3: Int = 3
Vector("apa", "banan", "majs", "banan").lastIndexOfSlice(Vector("banand"))
res4: Int = -1
```

c) Note that the method `search` assumes the collection is sorted in ascending order. When we reverse the order, `search` mostly cannot find what we look for, since it will search the wrong half of the collection.

```
scala> val odd = (1 to 1000000 by 2).toVector
scala> import scala.collection.Searching._
scala> odd.search(odd.last)
res18: collection.Searching.SearchResult = Found(499999)
//Search finds the last element at position 499999 in the collection.

scala> odd.search(odd.last + 1)
res19: collection.Searching.SearchResult = InsertionPoint(500000)
//Search cannot find odd.last + 1 because it does not exist in the collection
//and thus returns an object of type InsertionPoint with value 500000.
//Our element odd.last + 1 would have been at position 500000 if it existed.

scala> odd.reverse.search(odd(0))
res20: collection.Searching.SearchResult = InsertionPoint(0)
//As explained before, search assumes the list is sorted ascending,
//so it cannot find element odd(0) = 1 when the list is reversed.

scala> def lin(x: Int, xs: Seq[Int]) = xs.indexOf(x)
```

```
scala> def bin(x: Int, xs: Seq[Int]) = xs.search(x) match
  case Found(i) => i
  case InsertionPoint(i) => -i

//Defines a method bin using the search method on a sequence.
//With pattern matching it returns only the position i,
//not an object of Found or InsertionPoint.

scala> timed{ lin(odd.last, odd) }
time: 42.294821 ms
res22: (Int, Long) = (499999,42294821)
//Finding odd.last = 499999 with linear search took ~42ms.

scala> timed{ bin(odd.last, odd) }
time: 0.147314 ms
res23: (Int, Long) = (499999,147314)
//Binary search to find 499999 took much less time.
//Because each step halves the search space.
//This is an extreme case since we search for the last element.
//Linear search for the first element would have been as fast as binary search.
```

d) It requires $log_2(n)$ comparisons. This is because at each step, the number of elements left to check is halved. After the first comparison, we have $\frac{n}{2}$ elements remaining. After the second, $\frac{n}{2^2}$, etc. When only one element remains, we have made b comparisons, where:

$$\frac{n}{2^b} = 1$$

Using logarithm laws:

$$\log_2(n) = b$$

Solution Task 3. *Search among LTH's courses with linear search.*

a) The first line contains the column names `Kurskod KursSve KursEng Hskpoang Niva`. Then each row contains a course with data according to the column names.

b) The code downloads data and creates a vector with instances of case class `Course` using the method `fromLine`. Since variable `lth` is declared **`lazy`**, `download()` is not called until `lth` is first referenced. The number of courses is:

```
scala> val n = courses.lth.length
n: Int = 1104
```

c)

```
scala> def isCS(s: String) = s.startsWith("EDA") || s.startsWith("ETS")
scala> val x = courses.lth.find(c => isCS(c.code) && c.level == "G2")
x: Option[courses.Course] = Some(Course(EDAF05,Algoritmer, datastrukturer och
   komplexitet,Algorithms, Data Structures and Complexity,5.0,G2))
```

d)

```
def linearSearch[T](xs: Seq[T])(p: T => Boolean): Int =
   var i = 0
   while(i < xs.length && !p(xs(i))) i += 1
   if (i < xs.length) i else -1
```

e)

```
def rndCode: String =
   //randomizes from 0 to n (inclusive)
   def rnd(n: Int) = (math.random() * (n + 1)).toInt

   def letter = (rnd('Z' - 'A') + 'A').toChar
   def dig = ('0' + rnd(9)).toChar
   Seq(letter, letter, letter, letter, dig, dig).mkString
```

f)

```
val xs = Vector.fill(500000)(rndCode)
val(ixs, elapsedLin) =
  timed { xs.map(x => linearSearch(courses.lth)(_.code == x)) }
val found = ixs.filterNot(_== -1).size
```

g)

```
def linearSearch[T](xs: Seq[T])(p: T => Boolean): Int = xs.indexWhere(p)
```

Solution Task 4. *Search among LTH's courses with binary search.*

a) —

b)

```
def binarySearch(xs: Seq[String], key: String): Int =
  var (low, high) = (0, xs.size - 1)
  var found = false
  var mid = -1

  while (low <= high && !found) do
    mid = (low + high) / 2
    if xs(mid) == key then found = true
    else if xs(mid) < key then low = mid + 1
    else high = mid - 1
  end while
  if found then  mid else -(low + 1)
```

c) On an i7-3770K @ 3.50GHz the searches took:

- Binary search: `time: 142.6 ms`
- Linear search: `time: 3316.5 ms`

On an i7-8700T @ 2.40GHz they took:

- Binary search: `time: 81.5 ms`
- Linear search: `time: 5138.6 ms`

d) Binary search was about 23 times faster on the i7-3770K @ 3.50GHz and about 63 times faster on the i7-8700T CPU @ 2.40GHz.

Solution Task 5. *Insertion sort.*

a) —

b)

```
def insertionSort(xs: Seq[Int]): Seq[Int] =
  val result = scala.collection.mutable.ArrayBuffer.empty[Int]
  for e <- xs do
    var pos = 0
    while pos < result.size && result(pos) < e do pos += 1
    result.insert(pos,e)
  end for
  result.toVector
```

Solution Task 6. *In-place sorting.*

```
def selectionSortInPlace(xs: Array[String]): Unit =
  def indexOfMin(startFrom: Int): Int =
    var minPos = startFrom
    var i = startFrom + 1
    while (i < xs.size) do
      if (xs(i) < xs(minPos)) minPos = i
      i += 1
    end while
    minPos
  end indexOfMin

  def swapIndex(i1: Int, i2: Int): Unit =
    val temp = xs(i1)
    xs(i1) = xs(i2)
    xs(i2) = temp
  end swapIndex

  for i <- 0 to xs.size - 1 do swapIndex(i, indexOfMin(i))
end selectionSortInPlace
```

Solution Task 7. *Check if a sequence is sorted.*

a) The worst case takes $O(n \log n)$ comparisons for Timsort on n elements. Then it requires n comparisons between the sorted and unsorted list. The total comparisons in worst case is at most $n + n \log n$. For 10^6 elements this is about 10^7 comparisons.

```
scala> val n = 1E6
val n: Double = 1000000.0

scala> def worstCase(n: Double) = n + n * math.log(n)
def worstCase(n: Double): Double

scala> println(s"in worst case with n=$n there are ${worstCase(n)} comparisons"
in worst case with n=1000000.0 there are 1.4815510557964273E7 comparisons
```

b) A more efficient version of `isSorted` that stops searching when unsorted elements are detected:

```
def isSorted(xs: Vector[Int]): Boolean =
  if xs.length > 1 then
    var i = 0
    var result = true
    while i < xs.length-1 && result do
      if xs(i) > xs(i+1) then result = false
      i += 1
    end while
    result
  else true
end isSorted
```

c) In worst case one does $n-1$ pairwise comparisons if all elements are sorted except for the last.

d) The 2-tuple is of type `(Int, Int)`.

```
def isSorted(xs: Vector[Int]): Boolean =
  xs.zip(xs.tail).forall(x => x._1 <= x._2)
```

Solution Task 8. *In-place insertion sort.*

```
def insertionSort(xs: Array[Int]): Unit =
  for elem <- 1 until xs.length if xs.length > 0 do
    var pos = elem
    while pos > 0 && xs(pos) < xs(pos - 1) do
      val temp = xs(pos -1)
      xs(pos -1) = xs(pos)
      xs(pos) = temp
      pos -= 1
    end while
  end for
end insertionSort
```

Solution Task 9. *Sorting to new sequence with selection sort.*

```
def selectionSort(xs: Seq[String]): Seq[String] =
  def indexOfMin(xs: Seq[String]): Int = xs.indexOf(xs.min)
  val unsorted = xs.toBuffer
  val result = scala.collection.mutable.ArrayBuffer.empty[String]
  while !unsorted.isEmpty do
    val minPos = indexOfMin(unsorted)
    val elem = unsorted.remove(minPos)
    result.append(elem)
  end while
  result.toVector
end selectionSort
```

L.12.2 Tasks on threads and concurrent execution

Solution Task 10. *Threads.*

a) -

b) `java.lang.IllegalThreadStateException`. You cannot start a thread more than once. Therefore, you can't restart a thread when it has already executed.

c) When `start` is called, the code in `run` executes in parallel. Therefore, `Cucumber` and `Tomato` are printed interleaved. If `run` is called directly, there is no concurrent execution; `Cucumber` is printed 100 times, then `Tomato` 100 times.

d) `Thread.sleep` does not pause the thread exactly as specified. So it will usually print `zzz snark hi!`, but this is not guaranteed.

Solution Task 11. *Concurrent variable update.*

a) In `spendSave` the balance is fetched, changed, and stored — delayed — repeated. If `teddybear` finishes loading, changing, and storing before `ing` does the same, there is a problem, because they compete in a (eng. *race condition*). The problem is that before a thread stores the changed value, the other thread loads the old value. Only one thread 'wins' the race to store its updated value, causing the other's change to be lost. `ing` and `teddybear` thus become upset because not all their withdrawals and deposits are recorded.

Solution Task 12. *Thread-safe `AtomicInteger`.*
Now the `grandma` thread is guaranteed to be able to load the balance, withdraw/change, and store before the `wolf` thread can overwrite the result. In `spendSave` the thread pauses for a millisecond so the `wolf` thread can withdraw before `grandma` deposits again and the balance becomes negative. However, all withdrawals and deposits are recorded because the operations are atomic and the balance will be restored to zero without lost deposits.

Solution Task 13. *Concurrent execution with `scala.concurrent.Future`.*

a) error: Cannot find an implicit ExecutionContext. Future needs an ExecutionContext to run. `f` has type Future[Unit].

b) The function `printLater` returns a Future, meaning when both `printLater` and `println` are called in the loop, they run concurrently. Since starting a Future takes

longer, the `println` prints "all launched!" first, then numbers 1 to 42 print with irregular intervals as the threads sleep varying amounts.

c) `big` is a Future[Int]. The large number has 7,520,383 digits. `r` is of type Try[Int] (see Future documentation if unsure).

d) Since execution blocks until a result is received, you cannot continue typing in REPL while computation is in progress. If you wait too short, you get a TimeOutException and the computation is aborted.

Solution Task 14. *Use `Future` to do multiple things concurrently.*

a) -

b) -

c) Each page is delayed between 2 and 3 seconds (2000-3000 ms). So on average, each page takes 2.5 seconds to load. The vector must be filled before execution proceeds, so all 10 pages load before the first page is visible. So it takes about 2.5 x 10 = 25 seconds.

d) `f` returns a Vector filled with strings where each element is a line of the webpage. Since `f` runs in background, the program can continue while the contents compute. You can print `f` but the computation may not be done yet.

e) Same as above, but each element is itself a vector of strings.

f) Loads data in parallel, so downloads happen simultaneously but at varying speeds because of method `seg`.

g) Since data loads in parallel threads without blocking, they finish in the order threads complete, not sequentially. Eventually all are done and the result vector shows `true` values.

h) Method `happy` is well-defined and throws no exceptions. It prints `:)`. Method `unhappy` throws `java.lang.ArithmeticException` due to division by zero. This exception is caught in the callback and `:(` plus the exception is printed.

L.12.3 Extra Tasks; additional practice

Solution Task 15.

```
def isPrime(n: BigInt): Boolean = n match
  case _ if (n <= 1) => false
  case _ if (n <= 3) => true
  case _ if n % 2 == 0 || n % 3 == 0 => false
  case _ =>
    var i = BigInt(5)
    while i * i < n do
      if (n % i == 0 || n % (i + 2) == 0) false
      i += 6
    end while
    true
end isPrime

import scala.concurrent.*
import ExecutionContext.Implicits.global

val primes = Vector.fill(10)(Future{nextPrime(randomBigInt(16))})
```

```
primes.foreach(_.onSuccess{case i => println(i)})
```

Solution Task 16. *Answer theory questions.*

a) Stackoverflow gives the following explanation:
A thread is an independent set of values for the processor registers (for a single core). Since this includes the Instruction Pointer (aka Program Counter), it controls what executes in what order. It also includes the Stack Pointer, which had better point to a unique area of memory for each thread or else they will interfere with each other.

b)

```
val thread = new Thread(new Runnable{
  def run(){println(''This is a thread'')}
})
```

c) `thread.start`

d) Race conditions can occur in which one thread's resource updates overwrite another's, causing data loss.

e) Thread-safe code means multiple threads can run concurrently without incorrect results. For example, mutable variables modified by multiple threads must be handled carefully.

f) For example, you avoid explicitly creating `Thread` instances by placing code inside a Future, which also handles much under the hood for the programmer.

Solution Task 17. –

Solution Task 18. *Create your own multithreaded web server.*

a) `abbasillen` is printed backwards as `nellisabba`.

b)

c)

d)

e)

f)

g)

h)
Solution proposal:

```
package fibserver.threaded.memcache.whileloop

import java.net.{ServerSocket, Socket}
import java.io.OutputStream
import java.util.Scanner
import scala.util.{Try, Success, Failure}
import scala.concurrent._
import ExecutionContext.Implicits.global

object html:
  def page(body: String): String =  //minimal web page
    s"""<!DOCTYPE html>
       |<html><head><meta charset="UTF-8"><title>Min Sörver</title></head>
       |<body>
```

```
15          |$body
16          |</body>
17          |</html>
18          """.stripMargin
19
20    def header(length: Int): String = //standardized header of reply to client
21      s"HTTP/1.0 200 OK\nContent-length: $length\nContent-type: text/html\n\n"
22
23    def insertBreak(s: String, n: Int = 80): String =
24      if s.size < n then s else s.take(n) + "</br>" + insertBreak(s.drop(n),n)
25
26  object compute:
27    import java.util.concurrent.ConcurrentHashMap
28    val memcache = new ConcurrentHashMap[BigInt, BigInt]
29
30    def fib(n: BigInt): BigInt =
31      if memcache.containsKey(n) then
32        println("CACHE HIT!!! no need to compute: " + n)
33        memcache.get(n)
34      else
35        println("cache miss :( must compute fib:  " + n)
36        val f = superFib(n)
37        memcache.put(n, f)
38        f
39
40    private def superFib(n: BigInt): BigInt =
41      if n <= 0 then 0
42      else if n == 1 || n == 2 then 1
43      else
44        var secondLast: BigInt = 1
45        var last: BigInt = 1
46        var sum: BigInt = secondLast + last
47        var i = 3
48        while i < n do
49          if memcache.containsKey(i) then
50            sum = memcache.get(i)
51          else
52            secondLast = last
53            last = sum
54            sum = secondLast + last
55            memcache.put(i, sum)
56          i += 1
57        sum
58
59
60  object start:
61
62    def fibResponse(num: String) =
63      num.toIntOption match
64        case Some(n) => html.page(s"fib($n) == " + compute.fib(n))
65        case None    => html.page(s"FEL: skriv ett heltal, inte $num")
66
67    def errorResponse(uri:String) = html.page(s"Error: $uri </br> use /fib/heltal")
68
69    def handleRequest(cmd: String, uri: String, socket: Socket): Unit =
70      val os = socket.getOutputStream
71      val afterSlash = uri.toString.drop(1) // skip initial slash
```

```
    println(s"afterSlash:$afterSlash")
    val response: String =
      if afterSlash.startsWith("fib/") then fibResponse(afterSlash.stripPrefix("fib/"))
      else errorResponse(uri)
    os.write(html.header(response.size).getBytes("UTF-8"))
    os.write(response.getBytes("UTF-8"))
    os.close
    socket.close
  end handleRequest

  def serverLoop(server: ServerSocket): Unit =
    println(s"http://localhost:${server.getLocalPort}/hej")
    while true do
      Try {
        var socket = server.accept  // blocks thread until connect
        val scan = new Scanner(socket.getInputStream, "UTF-8")
        val (cmd, uri) = (scan.next, scan.next)
        println(s"Request: $cmd $uri")
        Future { handleRequest(cmd, uri, socket) }.recover {
          case e => println(s"Reqest failed: $e")
        }
      }.recover{ case e: Throwable => s"Connection failed: $e" }

  def main(args: Array[String]) =
    val port = Try{ args(0).toInt }.getOrElse(8089)
    serverLoop(new ServerSocket(port))
```

L.13 Solution Excercise13

Solution Task 1. *Complete your project.* —

Solution Task 2. *Take a final exam.* —

Solution Task 3. *Prepare your project presentation.* –

Solution Task 4. *Create documentation for your project.*–

Solution Task 5. *Review exercises and laboratory work.* –

L.14 Solution `java`

L.14.1 Basic Tasks; preparation for the lab

Solution Task 1. *Translate methods from Java to Scala.*

a)

```
import java.net.URL;
import java.util.ArrayList;
import java.util.{Set => JSet};
import java.util.{HashSet => JHashSet};
import java.util.Scanner;

object Hangman {  // This is Java-like, non-idiomatic Scala code!
    private var hangman: Array[String] = Array[String](
        " ======  ",
        " |/   |  ",
        " |    O  ",
        " |   -|- ",
        " |   / \\ ",
        " |       ",
        " |       ",
        " ==========================   RIP  :(");

    private def renderHangman(n: Int): String = {
        var result: StringBuilder = new StringBuilder();
        for (i: Int <- 0 until n){
            result.append(hangman(i));
            if (i < n - 1) {
                result.append("\n");
            }
        }
        return result.toString();
    }

    private def hideSecret(secret: String,
                           found: JSet[Character]): String = {
        var result: String = "";
        for (i: Int <- 0 until secret.length()) {
            if (found.contains(secret.charAt(i))) {
                result += secret.charAt(i);
            } else {
                result += '_';
            }
        }
        return result;
    }

    private def foundAll(secret: String,
                         found: JSet[Character]): Boolean = {
        var foundMissing: Boolean = false;
        var i: Int = 0;
        while (i < secret.length() && !foundMissing) {
            foundMissing = !found.contains(secret.charAt(i));
            i += 1;
        }
```

```
        return !foundMissing;
    }

    private def makeGuess(): Char = {
        var scan: Scanner = new Scanner(System.in);
        var guess: String = "";
        while ({
           System.out.println("Gissa ett tecken: ");
           guess = scan.next();
           guess.length() > 1;
        }) ()
        return Character.toLowerCase(guess.charAt(0));
    }

    def download(address: String, coding: String): String = {
        var result: String = "lackalänga";
        try {
            var url: URL = new URL(address);
            var words: ArrayList[String] = new ArrayList[String]();
            var scan: Scanner = new Scanner(url.openStream(), coding);
            while (scan.hasNext()) {
                words.add(scan.next());
            }
            var rnd: Int = (Math.random() * words.size()).asInstanceOf[Int];
            result = words.get(rnd);
        } catch { case e: Exception =>
            System.out.println("Error: " + e);
        }
        return result;
    }

    def play(secret: String): Unit = {
        var found: JSet[Character] = new JHashSet[Character]();
        var bad: Int = 0;
        var won: Boolean = false;
        while (bad < hangman.length && !won){
            System.out.println(renderHangman(bad));
            System.out.print("Felgissningar: " + bad + "\t");
            System.out.println(hideSecret(secret, found));
            var guess: Char = makeGuess();
            if (secret.indexOf(guess) >= 0) {
                found.add(guess);
            } else {
              bad += 1;
            }
            won = foundAll(secret, found);
        }
        if (won) {
            System.out.println("BRA! :)");
        } else {
            System.out.println("Hängd! :(");
        }
        System.out.println("Rätt svar: " + secret);
        System.out.println("Antal felgissningar: " + bad);
    }

    def main(args: Array[String] ): Unit = {
```

```
        if (args.length == 0) {
            var runeberg: String =
                "http://runeberg.org/words/ord.ortsnamn.posten";
            play(download(runeberg, "ISO-8859-1"));
        } else {
            var rnd: Int = (Math.random() * args.length).asInstanceOf[Int];
            play(args(rnd));
        }
    }
}
```

b)

```
object hangman:
  val hangman = Vector(
    " ======  ",
    " |/   |  ",
    " |    O  ",
    " |   -|- ",
    " |   / \\",
    " |       ",
    " |       ",
    " ========================   RIP  :(")

  def renderHangman(n: Int): String = hangman.take(n).mkString("\n")

  def hideSecret(secret: String, found: Set[Char]): String =
      secret.map(ch => if found(ch) then ch else '_')

  def makeGuess(): Char =
    val guess = scala.io.StdIn.readLine("Gissa ett tecken: ")
    if guess.length == 1 then guess.toLowerCase.charAt(0)
    else makeGuess()

  def download(address: String, coding: String): Option[String] =
    scala.util.Try {
      import scala.io.Source.fromURL
      val words = fromURL(address, coding).getLines.toVector
      val rnd = (math.random() * words.size).toInt
      words(rnd)
    }.toOption

  def play(secret: String): Unit =
    def loop(found: Set[Char], bad: Int): (Int, Boolean) =
      if secret forall found then (bad, true)
      else if bad >= hangman.length then (bad, false)
      else
        println(renderHangman(bad) + s"\nFelgissningar: $bad\t")
        println(hideSecret(secret, found))
        val guess = makeGuess()
        if secret contains guess then loop(found + guess, bad)
        else loop(found, bad + 1)

    val (badGuesses, won) = loop(Set(),0)
```

```
    val msg = if won then "BRA! :)" else "Hängd! :("
    println(s"$msg\nRätt svar: $secret")
    println(s"Antal felgissningar: $badGuesses")

  def main(args: Array[String] ): Unit =
    if args.length == 0 then
      val runeberg = "http://runeberg.org/words/ord.ortsnamn.posten"
      val secret = download(runeberg, "ISO-8859-1").getOrElse("läckalånga")
      play(secret)
    else play(args((math.random() * args.length).toInt))
```

Solution Task 2. *Translate between classes in Scala and classes in Java.*

a)

```
import java.util.List;
import java.util.ArrayList;

public class JPoint {
    private int x, y;

    public JPoint(int x, int y){
      this.x = x;
      this.y = y;
    }

    public JPoint(int x, int y, boolean save){
        this(x, y);
        if (save) {
            saved.add(0, this);
        }
    }

    public JPoint(){
        this(0, 0);
    }

    public int getX(){
        return x;
    }

    public int getY(){
        return y;
    }

    public double distanceTo(JPoint that) {
      return distanceBetween(this, that);
    }

    @Override public String toString() {
        return "JPoint(" + x + ", " + y + ")";
```

```
    }

    private static List<JPoint> saved = new ArrayList<JPoint>();

    public static Double distanceBetween(JPoint p1, JPoint p2) {
        return Math.hypot(p1.x - p2.x, p1.y - p2.y);
    }

    public static void showSaved() {
        System.out.print("Saved: ");
        for (int i = 0; i < saved.size(); i++){
          System.out.print(saved.get(i));
          if (i < saved.size() - 1) {
            System.out.print(", ");
          }
        }
        System.out.println();
    }
}
```

b) -

c)

```
case class Person(name: String, age: Int = 0)
```

d) p.*TAB* - copy, producArity, ProductIterator, productElement, productPrefix
Person.*TAB* - apply, curried, tupled, unapply

```
scala> p.copy
   def copy(name: String,age: Int): Person

scala> p.copy()
res0: Person = Person(Björn,49)

scala> p.copy(age = p.age + 1)
res1: Person = Person(Björn,50)

scala> Person.unapply(p)
res2: Option[(String, Int)] = Some((Björn,49))
```

Solution Task 3. *Immutable Java class.*

```
// Översätt:  class Point3D(val x: Int, val y: Int, val z: Int = 0)

public class JPoint3D {
  private int x;       // Attributen måste vara privata eftersom
  private int y;       //   val-attribut ej finns i Java.
  private int z;       // Typen skrivs före namnet i deklarationer.

  public JPoint3D(int x, int y, int z){// Konstruktor heter som klassen.
    this.x = x;                        // Satser måste sluta med ;
    this.y = y;                        // this behövs p.g.a. skuggning.
```

```
    this.z = z;
  }

  public JPoint3D(int x, int y){  // I stället för default-argument.
    this(x, y, 0);                // Anropa konstruktor i konstruktor.
  }
  public int getX(){    // I Java brukar man ha get i namnet på getter.
    return x;           // Metoder som ej är procedurer måste ha return.
  }

  public int getY(){    // Metoder måste ha parenteser, även getters.
    return y;
  }

  public int getZ(){   // Synlighet public måste anges explicit.
    return z;
  }
}
```

```
> code JPoint3D.java
> javac JPoint3D.java
> ls
JPoint3D.class  JPoint3D.java
> scala

scala> val p = new JPoint3D(1,2)
val p: JPoint3D = JPoint3D@53b1a3f8

scala> p.x
1 |p.x
  |^^^
  |value x is not a member of JPoint3D

scala> p.getX
val res0: Int = 1
```

Solution Task 4. *Mutable Java class.*

```
//Översätt class MutablePoint3D(var x: Int, var y: Int, var z: Int = 0)

public class JMutablePoint3D {
  private int x;  // Attributen brukar vara privata även i föränngsbara
  private int y;  //   klasser eftersom enhetlig access och syntax för
  private int z;  //   setter med tilldelning ej finns i Java.

  public JMutablePoint3D(int x, int y, int z){
    this.x = x;          // Satser måste sluta med ;
    this.y = y;          // this behövs p.g.a. skuggning.
    this.z = z;
  }

  public JMutablePoint3D(int x, int y){  // Motsv. default-argument.
```

```java
    this(x, y, 0);      // Anropa konstruktor i konstruktor.
  }
  public int getX(){    // I Java brukar man ha get i namnet på getter.
    return x;           // Metoder som ej är procedurer måste ha return.
  }

  public int getY(){    // Metoder måste ha parenteser, även getters.
    return y;
  }

  public int getZ(){   // Synlighet public måste anges explicit.
    return z;
  }

  public void setX(int x){ // I Java brukar man ha set i namnet på setter.
    this.x = x;
  }

  public void setY(int y){ // Nyckelordet void liknar Unit i Scala, men
    this.y = y;            //   det finns inget äkta tomt värde som ()
  }

  public void setZ(int zz){ // Om namn ej krockar behövs inte this, men
    z = zz;                 // man brukar ha samma parameternamn som
  }                         // attributnamn i setters trots skuggning så
}                           // att man slipper hitta på nytt namn.
```

```
> code JMutablePoint3D.java
> javac JMutablePoint3D.java
> ls
JMutablePoint3D.class  JMutablePoint3D.java
> scala

scala> val p = new JMutablePoint3D(1,2)
val p: JMutablePoint3D = JMutablePoint3D@625b215b

scala> p.x
1 |p.x
  |^^^
  |value x is not a member of JMutablePoint3D

scala> p.getZ
val res0: Int = 0

scala> p.setZ(3)

scala> p.getZ
val res1: Int = 3
```

Solution Task 5. *Comparing strings in Java.*

a)

```
String = java.lang.String
Boolean = true
Int = 0
```

b) Examples of 3 different expressions to test `compareTo`:

1. Hej comes first since `H < h`.

```
"hej".compareTo("Hej")
res: Int = 32
```

2. These are equivalent, so `compareTo` returns 0.

```
"hej".compareTo("hej")
res: Int = 0
```

3. *h* comes before *ö*.

```
"hej".compareTo("ö")
res: Int = -142
```

c) Examples of 3 different expressions to test `compareToIgnoreCase`:

1.

```
"hej".compareToIgnoreCase("HEj")
res: Int = 0
```

2.

```
"hej".compareToIgnoreCase("Ö")
res: Int = -142
```

3. Same as above, since Ö is converted to ö before comparison.

```
"hej".compareToIgnoreCase("ö") \\ res: Int = -142
```

d)

```
false
true
0
```

Solution Task 6. *Linear search in Java.*

a)

```java
public static boolean isYatzy(int[] dice){
    int col = 1;
    boolean allSimilar = true;
    while(col < dice.length && allSimilar){
        allSimilar = (dice[0] == dice[col]);
        col++; // this line was missing
    }
    return allSimilar;
}
```

b)

```
public static int findFirstYatzyRow(int[][] m){
    int row = 0;
    int result = -1;
    while(row < m.length){
        if(isYatzy(m[row])){
           result = row;
           break;
        }
        row++;
    }
    return result;
}
```

Solution Task 7. *Comparison support in Java.*

a)

b)

```
val teamComparator = new Comparator[Team]{
  def compare(o1: Team, o2: Team) = o2.rank - o1.rank
}
```

c)

d)

e)

```
case class Point(x: Int, y: Int) extends Comparable[Point] {
  def distanceFromOrigin: Double = math.hypot(x, y)
  def compareTo(that: Point): Int =
    (distanceFromOrigin - that.distanceFromOrigin).round.toInt
}
```

Solution Task 8. *`java.util.Arrays.binarySearch`*

Solution Task 9. *Auto(un)boxing.*

a) -

b) Cell has the type java.lang.Integer. When extracting the value with `c.value` the primitive type `int` is extracted.

c) Using autoboxing 42 is converted to `Integer` and can thereby be compared with another `Integer`.

d) i.compareTo(42) works because of autoboxing. The JVM automatically boxes the primitive type int into an Integer object.

e)

```
0 10 20 30 40 50 60 ... 390 400 410

[0]: 0
[42]: 0
```

```
NOT EQUAL
```

f)

```java
import java.util.ArrayList;

public class Autoboxing2 {
    public static void main(String[] args) {
        ArrayList<Integer> xs = new ArrayList<Integer>();
        for (int i = 0; i < 42; i++) {
            xs.add(i);
        }
        for (int x: xs) {
            int y = x * 10;
            System.out.print(y + " ");
        }
        int pos = xs.size();
        xs.add(pos, 0);
        System.out.println("\n\n[0]: " + xs.get(0));
        System.out.println("[" + pos + "]: " + xs.get(pos));
        if (xs.get(0).equals(xs.get(pos))) {
            System.out.println("EQUAL");
        } else {
            System.out.println("NOT EQUAL");
        }
    }
}
```

g) 42 will be inserted at the front of the list instead of the back, because autounboxing will convert Integer(0) to 0 and vice versa with variable `pos`.

h) If you want to check whether two int variables are equal, use ==, but if variables are of type Integer, you must use `equals`.
The JVM will not warn if you swap `Integer` and `int`, as in `xs.add(0, pos)`.

Solution Task 10. *CollectionConverters.*

a)
Vector[Int] -> java.util.List[Int]
Set[Char] -> java.util.Set[Char]
Map[String, Int] -> java.util.Map[String, Int]

b)
ArrayList[Int] -> scala.collection.mutable.Buffer[Int]
HashSet[Char] -> scala.collection.mutable.Set[Char]
Both become mutable equivalents. This is shown by their membership in `scala.collection.mutable` and as both `ArrayList` and `HashSet` are mutable in Java.

c) `scala.collection.immutable.Set`

d) `sm.asJava.asScala` gives type `scala.collection.mutable.Map[String,Int]`
`sm.asJava.asScala.toMap` gives type `scala.collection.immutable.Map[String,Int]`

e) -

Solution Task 11. *How does a* `switch` *statement in Java (and many other languages) work?*

a) Depending on the first letter of your favorite vegetable you get different responses such as *cucumber is good!* ("cucumber is tasty!") if the first letter is *g*.
Java's `switch` statement tests the first letter of the favorite vegetable by sequentially comparing it with the `case` expressions. If the first letter `firstChar` matches the letter after a `case`, the code after the colon is run until the end of the `switch` statement or until a `break` interrupts the `switch`.
If `firstChar` doesn't match any `case`, there is also a `default`, which is run regardless of the first letter, a general case.

b) If the `case 't'` is run, both *tomato is good!* ("tomato is tasty!") and *broccoli is good!* ("broccoli is tasty!") are printed; this is called "fall through". Without the `break` statement in Java, code in subsequent `case`s runs until a `break` interrupts the execution or the `switch` ends.

Solution Task 12. *Catch exceptions in Java with a* `try-catch` *statement.*

a)

1. Since the first argument is not the string *safe*, an unprotected division of 42 by 42 is done and the result 1 is shown.
2. Since the first argument is not the string *safe*, an unprotected division of 42 by 0 is done which results in an `ArithmeticException` because a number cannot be divided by zero.
3. Since the first argument is the string *safe*, a protected division of 42 by 42 is done and the result 1 is shown.
4. Since the first argument is the string *safe*, a protected division of 42 by 0 is done. This time the `ArithmeticException` is caught by the `try-catch` which replaces the previous division with a safe division by 1 where the result 42 is shown.
5. Since no arguments are given, an `ArrayIndexOutOfBoundsException` is thrown when the program tries to call the `equals` method on a string that does not exist. This could also be checked with a `try-catch` statement.

b)

```
TryCatch.java:16: error: variable input might not have been initialized
```

A compile-time error occurs due to the risk that `input` was not assigned when dividing.

Solution Task 13. *Matrices with arrays in Java.*

a) At initialization, all elements of `xss` are filled with the standard value for the type, `0` in the case of `int`. The outer `for` loop in `showMatrix()` iterates over the rows of `xss`. The inner `for` loop iterates through the elements of the current row and prints row, column, and content. After each row, a newline is printed so that a row in the output corresponds to a row in the matrix.
Examples of differences between usage of matrices in Scala and Java:

- access: `minArray(row)(column)` vs `minArray[row][column]`
- type names: `Array[Array[elementType]]` vs `elementType[][]`

- allocation: `Array.ofDim[typ](xDim,yDim)` vs `new typ[xDim][yDim]`

b)

```java
public class JavaArrayTest {

  public static void showMatrix(int[][] m){
    System.out.println("\n--- showMatrix ---");
    for (int row = 0; row < m.length; row++){
      for (int col = 0; col < m[row].length; col++) {
        System.out.print("[" + row + "]");
        System.out.print("[" + col + "] = ");
        System.out.print(m[row][col] + ";");
      } System.out.println();
    }
  }

  public static void fillRnd(int[][] m, int n){
    for (int row = 0; row < m.length; row++){
      for (int col = 0; col < m[row].length; col++) {
        m[row][col] = (int) (Math.random() * n + 1);
      }
    }
  }

  public static void main(String[] args) {
    System.out.println("Hello JavaArrayTest!");
    int[][] xss = new int[10][5];
    fillRnd(xss, 6);
    showMatrix(xss);
  }
}
```

Solution Task 14. *Translate from Java to Scala.*

```scala
object showInt {
  def show(obj: Any, msg: String = ""): Unit = println(msg + obj)

  def repeatChar(ch: Char, n: Int): String = ch.toString * n

  def showInt(i: Int): Unit = {
    val leading = Integer.numberOfLeadingZeros(i)
    val binaryString = repeatChar('0', leading) + i.toBinaryString
    show(i,                "Heltal : ")
    show(i.asInstanceOf[Char],         "Tecken : ")
    show(binaryString,    "Binärt : ")
    show(i.toHexString,   "Hex    : ")
    show(i.toOctalString, "Oktal  : ")
  }


```

```
  import scala.io.StdIn.readLine
  import scala.util.{Try,Success,Failure}

  def loop: Unit =
    Try { readLine("Heltal annars pang: ").toInt } match {
      case Failure(e) => show(e); show("PANG!")
      case Success(i) => showInt(i); loop
    }

  def main(args: Array[String]): Unit =
    if(args.length > 0) args.foreach(i => showInt(i.toInt))
    else loop
}
```

Solution Task 15. *Content equality and reference equality in Java.*

Solution Task 16. *Implement content equality in Java.*

Solution Task 17. *Arrays and* `for` *loops in Java.*

a) The program simulates 10000 dice rolls (with random seed 42) and prints the occurrence of each die face.

```
Rolling the dice 10000 times with seed 42
Number of 1's: 1654
Number of 2's: 1715
Number of 3's: 1677
Number of 4's: 1629
Number of 5's: 1643
Number of 6's: 1682
```

b) In Java, square brackets are used while Scala has "normal" parentheses. An array in Scala is declared like this:
`val scalaArray = Array.ofDim[Int](6)`
which in Java corresponds to: `int[] javaArray = new int[6];`
`for` loop in Scala is written: `for(i <- 0 to n) {...}` whereas in Java it is written: `for (int i = 0; i < n; i++) { ... }`.
In Java, semicolons must be written after each statement and the type must be explicitly specified at every variable declaration.
In Scala, semicolons are not needed (except to separate statements on the same line) and types can often be inferred by the compiler and do not need to be explicitly written.

c) Add `System.out.println(i);` in the for loops

d)

```
// DiceReg2.java
import java.util.Random;
public class DiceReg2{
  public static int[] diceReg = new int[6];
  private static Random rnd = new Random();

  public static int parseArguments(String[] args){
    int n = 100;
```

```
    if(args.length > 0) {
      n = Integer.parseInt(args[0]);
    }
    if(args.length > 1) {
      int seed = Integer.parseInt(args[1]);
      rnd.setSeed(seed);
    }
    return n;
  }

  public static void registerPips(int n) {
    for(int i = 0; i<n; i++) {
      int pips = rnd.nextInt(6);
      diceReg[pips]++;
    }
  }

  public static void main(String[] args) {
    int n = parseArguments(args);
    registerPips(n);
    printReg();
  }
}
```

e)

```
  // Prints the occurrences of 1000 dice rolls with seed 42.
Number of 1's: 165
Number of 2's: 163
Number of 3's: 178
Number of 4's: 183
Number of 5's: 156
Number of 6's: 155

  // Prints the diceReg attribute
res1: Array[Int] = Array(165, 163, 178, 183, 156, 155)

  // Prints diceReg attribute after 1000 rolls.
res2: Array[Int] = Array(329, 325, 349, 360, 324, 313)

  // Prints diceReg attribute after another 1000 rolls.
res3: Array[Int] = Array(498, 484, 531, 513, 485, 489)

  // It results in a compilation error because the attribute rnd is private
<console>:11: error: value rnd is not a member of object DiceReg2
  DiceReg2.rnd
           ^
```

Solution Task 18. *Read in a sequence of numbers with `Scanner` in Java.*

a)

`hasNextInt()` checks if another number exists and returns **`true`** or **`false`**. `nextInt()` reads the next number.

See https://docs.oracle.com/javase/8/docs/api/java/util/Scanner.html#hasNextInt%28%29 and https://docs.oracle.com/javase/8/docs/api/java/util/Scanner.html#nextInt%28%29.

b)

```
import java.util.Random;
import java.util.Scanner;

public class DiceScanBuggy {
  public static int[] diceReg = new int[6];
  public static Scanner scan = new Scanner(System.in);

  public static void registerPips() {
    System.out.println("Enter pips separated by blanks: ");
    System.out.println("End with -1 and <Enter>.");
    boolean isPips = true;
    while(isPips && scan.hasNextInt()){
      int pips = scan.nextInt();
      if(pips >= 1 && pips <= 6) {
        diceReg[pips-1]++;
      } else {
        isPips = false;
      }
    }
  }

  public static void printReg(){
    for(int i = 1; i<7; i++) {
    System.out.println("Number of " + i + "'s: " + diceReg[i-1]);
    }
  }

  public static void main(String[] args) {
    registerPips();
    printReg();
  }
}
```